

Beamforming

Théorie, algorithmes et mise en œuvre

Joachim Nadal

21 mai 2026

Préface

Le beamforming est l'art de transformer un réseau d'antennes en un instrument géométrique : un dispositif capable d'écouter sélectivement certaines directions de l'espace et d'en ignorer d'autres. Au croisement de la physique des ondes, de l'algèbre linéaire complexe et du traitement statistique du signal, il est devenu un outil central des systèmes radar, des communications mobiles, du Wi-Fi récent, du *massive* MIMO, de l'acoustique des réseaux de microphones et de la radioastronomie.

Ambition de ce document

Ce livre vise à donner une compréhension progressive des modèles, algorithmes et limites pratiques du beamforming moderne. Il ne s'agit ni d'un formulaire d'équations ni d'un catalogue d'algorithmes : chaque chapitre relie systématiquement trois niveaux indissociables,

$$\text{physique} \longrightarrow \text{mathématiques} \longrightarrow \text{implémentation.}$$

La physique explique pourquoi les antennes reçoivent des phases différentes. Les mathématiques modélisent cette structure sous forme de vecteurs, de matrices, de covariances et de sous-espaces. L'implémentation transforme enfin les modèles en code exécutable. Sans la première, les formules ne sont que des manipulations symboliques ; sans la deuxième, les intuitions restent vagues ; sans la troisième, la théorie reste spéculative.

Les fondements classiques de l'array processing sont traités en détail dans les ouvrages de Van Trees [1] et de Johnson et Dudgeon [2]. Le présent texte en reprend les idées centrales avec une orientation plus pédagogique et numérique.

À qui s'adresse ce livre

Le lecteur visé est ingénieur, chercheur ou étudiant en traitement du signal, en télécommunications, en radar, en SDR ou en acoustique. Il est supposé connaître les bases de l'algèbre linéaire (espaces vectoriels, valeurs propres, projections), du traitement du signal classique (transformée de Fourier, échantillonnage de Shannon) et des probabilités (espérance, covariance, processus stationnaires). Le reste est construit progressivement.

Organisation

L'ouvrage est structuré en sept parties qui suivent la logique du sujet.

Partie I — Fondements physiques et mathématiques.

Propagation des ondes, retards, phases, modèle bande étroite, géométrie des réseaux d'antennes, échantillonnage spatial, et émergence du *steering vector* comme signature spatiale d'une direction d'arrivée.

Partie II — Beamforming déterministe.

Beam pattern, Delay-and-Sum et Null Steering. Ces méthodes, sans modèle statistique de l'environnement, fournissent les briques géométriques sur lesquelles reposent toutes les approches plus sophistiquées.

Partie III — Beamforming statistique.

Covariance spatiale, MVDR / Capon, LCMV et structure GSC. L'environnement RF devient un objet aléatoire dont on apprend la structure pour adapter automatiquement les poids du beamformer.

Partie IV — Adaptation itérative.

Filtre de Wiener, LMS, NLMS et RLS. On y traite la mise à jour récursive des poids dans des environnements non stationnaires.

Partie V — Estimation de direction d'arrivée.

Décomposition en sous-espaces signal et bruit, MUSIC, ESPRIT, *bornes de Cramér-Rao*, méthodes parcimonieuses et bayésiennes. On y dépasse la limite de résolution du beamforming conventionnel.

Partie VI — Extensions.

Beamforming *large bande*, *MIMO* et *massive MIMO*, géométries 2D/3D, champ proche, polarisation. On y sort des hypothèses simplificatrices de la Partie I.

Partie VII — Mise en œuvre pratique.

Calibration d'un réseau multi-antennes, robustesse aux erreurs de modèle et chaîne intégrée d'anti-brouillage. C'est dans cette partie que la théorie rencontre la réalité matérielle.

Les trois annexes regroupent les rappels d'algèbre linéaire complexe, l'analyse de perturbations sur valeurs et vecteurs propres, et les extraits de code Python qui jalonnent le livre.

Code et matériel complémentaire

L'intégralité du code Python qui sous-tend les figures, les simulations et les exemples numériques est disponible sur :

<https://github.com/JoachimNadal/Adaptive-Beamforming-Simulation>

Les listings reproduits dans le livre sont volontairement succincts et visent la pédagogie ; la version exécutable, plus complète, se trouve dans le dépôt.

Conventions

Une convention de phase unique — énoncée au chapitre des notations qui suit cette préface — est utilisée dans tout l'ouvrage. Une convention différente n'est pas fautive en soi, mais elle exige une cohérence totale dans les expressions de *steering vector*, de *beam pattern*, de pseudo-spectre MUSIC et de phases ESPRIT. Cet ouvrage adopte

$$a_m(\theta) = e^{-jmkd \sin \theta}$$

et s'y tient dans les développements qui suivent.

Table des matières

Préface	i
Conventions et notations	v
I Fondements physiques et mathématiques	1
1 Introduction au beamforming	2
2 Ondes électromagnétiques et propagation	7
3 Réseaux d'antennes et échantillonnage spatial	13
4 Modèle bande étroite et représentation IQ	18
5 Steering vector et array manifold	22
II Beamforming déterministe	30
6 Beam pattern et fenêtrage spatial	31
7 Delay-and-Sum	38
8 Null Steering : imposer des zéros spatiaux	43
III Beamforming statistique	48
9 Covariance spatiale	49
10 MVDR / Capon	55
11 LCMV et structure GSC	63
IV Adaptation itérative	70
12 Wiener, LMS et NLMS	71
13 RLS et facteur d'oubli	77

V	Estimation de direction d'arrivée	82
14	Décomposition en sous-espaces et détection d'ordre	83
15	MUSIC et Root-MUSIC	88
16	ESPRIT et variantes	96
17	Bornes de Cramér-Rao pour la DOA	101
18	DOA parcimonieuse et bayésienne	107
VI	Extensions	114
	Introduction aux extensions	115
19	Beamforming large bande	116
20	Beamforming MIMO et massive MIMO	122
21	Géométries avancées	128
VII	Mise en œuvre pratique	135
22	Calibration d'un réseau multi-antennes	136
23	Robustesse aux erreurs de modèle	142
24	Chaîne intégrée d'anti-brouillage	150
	Conclusion générale	159
A	Rappels d'algèbre linéaire complexe	162
B	Analyse de perturbations	165
C	Code Python : module de référence	167
	Bibliographie commentée	174

Conventions et notations

Scalars, vecteurs et matrices

Les scalaires sont notés en italique : x , t , f , θ , λ . Les vecteurs sont notés en gras minuscule ($\mathbf{x}$, $\mathbf{w}$, $\mathbf{a}(\theta)$) et les matrices en gras majuscule ($\mathbf{R}$, $\mathbf{A}$, $\mathbf{I}$). Tous les vecteurs sont des vecteurs colonnes. Pour un réseau de N antennes, un *snapshot* spatial est ainsi

$$\mathbf{x}(n) = \begin{bmatrix} x_0(n) & x_1(n) & \cdots & x_{N-1}(n) \end{bmatrix}^T \in \mathbb{C}^N.$$

Opérations complexes

Pour un scalaire $z \in \mathbb{C}$ la conjugaison est notée z^* ; pour un vecteur ou une matrice on emploie la transposée $(\cdot)^T$ et la transposée conjuguée (hermitienne) $(\cdot)^H$. L'énergie d'un vecteur complexe est $\|\mathbf{x}\|^2 = \mathbf{x}^H \mathbf{x}$. La partie réelle et la partie imaginaire d'un scalaire complexe sont notées $\text{Re}\{z\}$ et $\text{Im}\{z\}$.

Convention de phase

Les signaux sont manipulés sous leur forme en bande de base IQ. La convention temporelle adoptée est $e^{j2\pi f_0 t}$, ce qui produit, pour un réseau linéaire uniforme, le *steering vector*

$$\mathbf{a}(\theta) = \begin{bmatrix} 1 \\ e^{-jkd \sin \theta} \\ e^{-j2kd \sin \theta} \\ \vdots \\ e^{-j(N-1)kd \sin \theta} \end{bmatrix}, \quad a_m(\theta) = e^{-jmkd \sin \theta}, \quad m = 0, \dots, N-1.$$

Le nombre d'onde est $k = 2\pi/\lambda$, où $\lambda = c/f_0$ est la longueur d'onde et $c \approx 3 \cdot 10^8 \text{ m} \cdot \text{s}^{-1}$ la vitesse de la lumière. La convention de signe *négatif* dans l'exponentielle est conservée **sans exception** dans tout l'ouvrage : un changement de signe inverserait les angles estimés par ESPRIT, produirait un beam pattern miroir et casserait la cohérence des formules de MVDR et MUSIC.

Géométrie du réseau

Sauf mention contraire, le réseau est un *Uniform Linear Array* (ULA) de N antennes alignées le long d'un axe, séparées par un espacement constant d :

$$p_m = md, \quad m = 0, \dots, N-1.$$

L'antenne $m = 0$ sert de référence : tous les retards et déphasages sont mesurés relativement à elle. L'angle d'arrivée θ est mesuré par rapport à la normale au réseau (le *broadside*), avec

$\theta \in [-90^\circ, +90^\circ]$ pour la *zone visible*. L'espacement standard adopté dans toutes les simulations est

$$d = \lambda/2.$$

Beamforming et beam pattern

La sortie d'un beamformer est définie par

$$y(n) = \mathbf{w}^H \mathbf{x}(n),$$

et le *beam pattern* associé au vecteur de poids $\mathbf{w}$ est

$$B(\theta) = \mathbf{w}^H \mathbf{a}(\theta).$$

Le gain spatial en puissance $|B(\theta)|^2$ est généralement tracé en décibels après normalisation au maximum : $B_{\text{dB}}(\theta) = 20 \log_{10}(|B(\theta)| / \max_{\theta} |B(\theta)|)$.

Covariance spatiale

La covariance spatiale théorique d'un processus snapshot $\mathbf{x}(n)$ stationnaire au sens large est

$$\mathbf{R} = \mathbb{E}[\mathbf{x}(n)\mathbf{x}^H(n)] \in \mathbb{C}^{N \times N}.$$

Son estimée à partir de K snapshots empilés dans $\mathbf{X} = [\mathbf{x}(1), \mathbf{x}(2), \dots, \mathbf{x}(K)]$ est

$$\hat{\mathbf{R}} = \frac{1}{K} \sum_{n=1}^K \mathbf{x}(n)\mathbf{x}^H(n) = \frac{1}{K} \mathbf{X} \mathbf{X}^H.$$

Discipline de notation

Pour éviter les collisions dans les chapitres avancés, les conventions suivantes sont maintenues autant que possible :

- N désigne le nombre d'antennes, pas le nombre de snapshots.
- K désigne le nombre de snapshots.
- L désigne le nombre de sources ou de contraintes liées aux directions, selon le contexte explicitement annoncé.
- M est réservé aux tailles auxiliaires : sous-réseau en spatial smoothing, taille de grille ou taille FFT. En large bande, on note souvent P le nombre de sous-bandes pour éviter de confondre avec M .
- d désigne l'espacement inter-éléments et $D = (N - 1)d$ l'ouverture physique.
- $k = 2\pi/\lambda$ est le nombre d'onde dans les formules. Dans le code, les indices de boucle sont locaux ; lorsqu'une confusion est possible, on préfère `idx`, `ell` ou `kk`.
- Les formules et le code utilisent θ en radians. Les figures et les exemples numériques affichent souvent les angles en degrés.
- $\mathbf{R}$ désigne la covariance théorique, $\hat{\mathbf{R}}$ son estimée, et $\hat{\mathbf{R}}_{\alpha} = \hat{\mathbf{R}} + \alpha \mathbf{I}$ la covariance chargée.

Tableau récapitulatif des notations

Notation	Signification	Dimension / unité
N	Nombre d'antennes du réseau	scalaire entier
m	Indice d'une antenne, $m = 0, \dots, N - 1$	scalaire entier
d	Espacement entre deux antennes adjacentes	m ou λ
$D = (N - 1)d$	Ouverture du réseau	m ou λ
λ, f_0, c	Longueur d'onde, fréquence porteuse, célérité	m, Hz, m/s
$k = 2\pi/\lambda$	Nombre d'onde	rad/m
θ	Angle d'arrivée mesuré depuis le broadside	rad dans le code, deg en figures
$u = \sin \theta$	Variable spatiale réduite, $u \in [-1, 1]$	sans dimension
K	Nombre de <i>snapshots</i>	scalaire entier
L	Nombre de sources ou de contraintes angulaires	scalaire entier
M, P	Taille auxiliaire / nombre de sous-bandes selon contexte	scalaire entier
$\mathbf{a}(\theta)$	Steering vector associé à la direction θ	$N \times 1$
$\mathbf{x}(n)$	Snapshot spatial reçu à l'instant n	$N \times 1$
$\mathbf{X}$	Matrice de snapshots $[\mathbf{x}(1), \dots, \mathbf{x}(K)]$	$N \times K$
$\mathbf{w}$	Vecteur de poids du beamformer	$N \times 1$
$y(n)$	Sortie du beamformer	scalaire complexe
$B(\theta)$	Beam pattern $\mathbf{w}^H \mathbf{a}(\theta)$	scalaire complexe
$\mathbf{R}$	Covariance spatiale théorique	$N \times N$
$\hat{\mathbf{R}}$	Estimée empirique de $\mathbf{R}$	$N \times N$
$\hat{\mathbf{R}}_\alpha$	Covariance chargée $\hat{\mathbf{R}} + \alpha \mathbf{I}_N$	$N \times N$
$\mathbf{R}_{i+n}$	Covariance interférences plus bruit	$N \times N$
$\mathbf{R}_n$	Covariance du bruit seul	$N \times N$
$\mathbf{R}_s$	Covariance du signal utile ou du sous-espace signal	$N \times N$
$\mathbf{C}$	Matrice de contraintes LCMV	$N \times L_c$
$\mathbf{f}$	Réponses imposées des contraintes	$L_c \times 1$
σ^2	Variance du bruit additif	puissance scalaire
$\mathbf{I}_N$	Matrice identité de taille N	$N \times N$
$\mathbf{0}, \mathbf{1}$	Vecteur ou matrice nul / vecteur de uns	selon contexte
SNR, SINR	Rapport signal/bruit, signal/(brouillage+bruit)	linéaire ou dB
WNG	White Noise Gain $1/\ \mathbf{w}\ ^2$ sous gain unitaire	linéaire ou dB
DOA	Direction Of Arrival	angle

Première partie

Fondements physiques et
mathématiques

Chapitre 1

Introduction au beamforming

Avant d’entrer dans les définitions techniques, il faut prendre le temps de comprendre *pourquoi* le beamforming est devenu, en quelques décennies, un sujet aussi central. La réponse tient en une phrase : c’est le cadre central dans lequel l’information *directionnelle* d’une onde devient exploitable par traitement du signal. Les systèmes qui exploitent des ondes électromagnétiques ou acoustiques — radar, 5G, Wi-Fi, satellitaire, radioastronomie, acoustique — mobilisent fréquemment, parfois sans le dire, des principes de beamforming. Ce chapitre brosse le tableau d’ensemble, situe les idées les unes par rapport aux autres, et annonce le parcours que suivra le reste du livre.

1.1 Du traitement temporel au traitement spatial

Le traitement du signal classique s’intéresse à *ce que contient* un signal : son spectre, son énergie, sa modulation, son rapport signal-sur-bruit. Cette approche est puissante mais aveugle à une dimension entière du problème : la *géométrie* de la réception. Deux ondes identiques arrivant de directions différentes sur une antenne isolée y produisent localement le même signal. Une antenne seule peut dire qu’un signal existe, mais ne suffit généralement pas à en déduire la direction.

Cette limitation n’est pas accessoire : elle est fondamentale. Un récepteur monocanal ne dispose que d’un seul échantillon spatial du champ électromagnétique, et toute l’information directionnelle s’y trouve écrasée par projection. C’est l’équivalent spatial d’un instantané pris avec un appareil photo monoculaire : on ne sait pas *où* se trouve l’objet dans la profondeur. Tout ce qu’on peut faire ensuite, par le traitement temporel le plus sophistiqué qui soit, ne ressuscitera pas l’information perdue à l’acquisition.

Le passage au réseau d’antennes change radicalement la donne. Chaque antenne reçoit une version légèrement décalée de la même onde, un retard de quelques fractions de nanoseconde, un déphasage, une corrélation spatiale particulière avec ses voisines. Ces écarts, en apparence négligeables, encodent intégralement la géométrie de propagation. Tout le beamforming peut se lire comme l’art de les exploiter.

L’analogie avec l’optique est éclairante. Un œil unique permet de voir la lumière, deux yeux permettent d’en déduire la profondeur par parallaxe. Un télescope mono-objectif a une résolution limitée par son diamètre, alors qu’un *interféromètre* composé de plusieurs miroirs distants peut atteindre une résolution équivalente à celle d’un télescope géant. Les radioastronomes le savent depuis longtemps : le réseau VLBI (*Very Long Baseline Interferometry*) synthétise une résolution équivalente à celle d’un télescope de la taille de la Terre, en exploitant précisément les déphasages entre antennes distantes de milliers de kilomètres. Le beamforming, dans sa version moderne, est l’héritier direct de cette intuition.

Formellement, le signal observé ne dépend plus uniquement du temps mais aussi de la position

du capteur :

$$x(t) \mapsto x(\mathbf{p}, t).$$

Pour un réseau de N antennes, l'observation devient un vecteur

$$\mathbf{x}(t) = \begin{bmatrix} x_0(t) & x_1(t) & \cdots & x_{N-1}(t) \end{bmatrix}^T \in \mathbb{C}^N,$$

chaque composante étant la mesure complexe d'une antenne. Le beamforming travaille donc sur un objet *vectoriel* là où le traitement temporel travaille sur un objet *scalaire*. Toutes les opérations familières — filtrage, détection, estimation, séparation — y prennent une version spatiale. Filtrer dans le temps sélectionne des fréquences ; filtrer dans l'espace sélectionne des directions.

Une remarque importante avant d'aller plus loin : le passage du scalaire au vecteur n'est pas qu'un changement de complexité calculatoire. Il introduit une géométrie. Le vecteur d'observation $\mathbf{x}(t)$ vit dans un espace complexe de dimension N dans lequel chaque *direction* correspond à une combinaison particulière des antennes, et chaque *onde plane* venant d'un angle d'arrivée particulier excite une direction spécifique de cet espace. Cette correspondance entre angles physiques et directions de $\mathbb{C}^N$ — incarnée par le *steering vector* qu'on construira au chapitre 5 — est l'objet central du livre. Une bonne part de la puissance du beamforming vient de ce que les opérations « physiques » (pointer un faisceau, annuler une direction, séparer deux sources) deviennent des opérations *géométriques* dans cet espace complexe : projections, orthogonalisations, décompositions en sous-espaces.

1.2 Le problème fondamental

Le problème central du beamforming s'énonce simplement :

Comment combiner les signaux reçus par plusieurs antennes pour renforcer certaines directions de l'espace et en atténuer d'autres ?

Trois faits physiques fondent la réponse, et reviendront partout dans la suite.

Propagation finie. Une onde électromagnétique se propage à vitesse finie. Pour une onde plane arrivant à un angle θ sur deux antennes adjacentes séparées d'une distance d , le retard différentiel vaut

$$\tau = \frac{d \sin \theta}{c}.$$

Ce retard est minuscule — moins d'une nanoseconde aux fréquences microondes — mais devient mesurable une fois converti en phase.

Retard $\longleftrightarrow$ phase. Sous l'hypothèse bande étroite (la largeur spectrale du signal est petite devant sa fréquence porteuse), un retard temporel τ se traduit, après démodulation IQ, par une multiplication par

$$e^{-j2\pi f_0 \tau}.$$

Tout le beamforming bande étroite se réduit ainsi à de l'algèbre linéaire complexe, sans manipulation explicite de retards sub-échantillon.

Signature spatiale. En combinant les deux relations précédentes, la phase reçue sur la m -ième antenne d'un ULA s'écrit

$$\varphi_m(\theta) = -mkd \sin \theta, \quad m = 0, \dots, N-1.$$

Cette progression de phase ne dépend que de la direction d'arrivée : chaque angle θ laisse sur le réseau une *signature de phase* reconnaissable, le *steering vector* $\mathbf{a}(\theta)$. Si l'on connaît la signature

d’une direction cible, on peut appliquer des poids complexes aux antennes pour réaligner les contributions issues de cette direction. Les phases s’additionnent constructivement, tandis que les signaux d’autres directions, restés désalignés, sont naturellement atténués.

La beauté de cette construction tient à ce qu’elle se replie sur elle-même : la signature qui sert à *reconnaître* une direction sert aussi à la *créer*. Les mêmes facteurs $e^{-jmkd \sin \theta}$ qui décrivent comment une onde plane se présente sur le réseau servent de poids pour synthétiser un faisceau pointé dans cette direction. Le beamforming est, en ce sens, fondamentalement *réversible* géométriquement : recevoir et émettre sont deux faces de la même manipulation algébrique. Cette symétrie sera exploitée explicitement en MIMO (chapitre 20), où le précodage à l’émission utilise les mêmes vecteurs que le beamforming à la réception.

1.3 L’idée du faisceau spatial

Le terme « beamforming » signifie littéralement *formation de faisceau*. Ce faisceau n’a généralement pas d’existence matérielle : il décrit la *sensibilité directionnelle* du réseau. Lorsque les signaux des antennes sont combinés avec un jeu de poids complexes $\mathbf{w} = [w_0, w_1, \dots, w_{N-1}]^T$, la sortie scalaire

$$y(t) = \mathbf{w}^H \mathbf{x}(t)$$

amplifie certaines directions et en atténue d’autres. C’est l’objet appelé *beam pattern*

$$B(\theta) = \mathbf{w}^H \mathbf{a}(\theta)$$

qui décrit cette sensibilité, et qui fera l’objet d’une étude détaillée au chapitre 6. On y retrouvera systématiquement trois éléments :

- un *lobe principal* dans la direction privilégiée, dont la largeur conditionne la résolution angulaire ;
- des *lobes secondaires*, par lesquels peuvent fuir des interférences indésirables ;
- des *zéros spatiaux*, ou *nulls*, qui annulent fortement certaines directions parasites.

Lire un beam pattern, c’est lire d’un coup d’œil les forces et les limites d’un beamformer.

1.4 Une progression d’idées

Le beamforming n’est pas un algorithme unique mais une famille de techniques que l’on peut hiérarchiser selon le type et la quantité d’information qu’elles exploitent. Plus on en sait sur l’environnement ou sur la cible, plus on peut concevoir un beamformer sélectif — mais aussi plus on devient sensible aux erreurs sur cette information. Le fil rouge des sept parties du livre tient dans cette gradation : chaque chapitre ajoute une brique d’information et étudie ce qu’on en fait. La liste ci-dessous donne une cartographie rapide ; elle peut servir de boussole pour situer chaque chapitre dans l’ensemble.

Beamforming conventionnel (Delay-and-Sum). N’utilise que la géométrie et la direction visée. Ses poids ne dépendent pas des données : robuste mais aveugle à l’environnement RF.

Null Steering. Premier contrôle spatial explicite : on impose un gain unitaire dans la direction cible et un gain nul dans certaines directions d’interférence. C’est la première forme de *contrainte linéaire* sur le vecteur de poids.

MVDR et beamforming adaptatif statistique. On exploite la *covariance spatiale* des signaux reçus pour adapter les poids à l'environnement. Le critère MVDR (*Minimum Variance Distortionless Response*) minimise la puissance reçue tout en conservant un gain unitaire dans la direction cible : le beamformer place automatiquement ses zéros là où l'énergie indésirable se trouve.

LCMV. Généralise MVDR à plusieurs contraintes linéaires, et englobe le Null Steering comme cas particulier sans covariance utile.

LMS et RLS. Au lieu d'une formule fermée fondée sur la covariance, les poids sont ajustés progressivement à partir d'une erreur observée. LMS procède par gradient stochastique ; RLS utilise une estimation réursive plus fine et converge plus rapidement au prix d'une complexité accrue.

MUSIC, ESPRIT. On ne filtre plus spatialement, on *estime* les directions d'arrivée elles-mêmes. La covariance spatiale possède une structure de sous-espaces signal/bruit ; ces deux méthodes exploitent cette structure pour atteindre des résolutions angulaires bien supérieures à celles du beamforming conventionnel.

Méthodes parcimonieuses et bayésiennes. Lorsque l'on dispose de très peu de snapshots ou de sources cohérentes, on reformule l'estimation comme un problème de représentation parcimonieuse (ℓ_1 -SVD) ou de modèle hiérarchique bayésien (*Sparse Bayesian Learning*).

Au-delà du bande étroite. Les signaux radar et certaines liaisons modernes violent l'hypothèse bande étroite. Le beamforming *large bande* (*true time delay*, sous-bandes, FIB) est alors nécessaire. À l'autre extrême, les systèmes *massive MIMO* et 5G exploitent des réseaux à plusieurs dizaines voire centaines d'éléments, avec des architectures hybrides analogique/numérique.

Ces familles ne sont pas en concurrence : elles correspondent à des points d'équilibre différents entre information, performance et robustesse. En général, un système opérationnel combine plusieurs d'entre elles. Une chaîne anti-brouillage typique commencera par un estimateur de DOA (MUSIC ou ESPRIT) pour identifier les brouilleurs, construira ensuite des contraintes de Null Steering sur les directions détectées, et confiera l'adaptation fine à un MVDR ou un LCMV. C'est l'architecture qu'on assemblera explicitement au chapitre 24.

1.5 Pourquoi la calibration est indispensable

Le modèle idéal qui sous-tend les développements précédents suppose des antennes identiques, positionnées sans erreur, connectées à des chaînes RF rigoureusement cohérentes. Cette hypothèse est rarement vérifiée exactement. Câbles de longueurs inégales, gains et phases différents, oscillateurs locaux malphasés, couplage mutuel, dérives thermiques : toutes ces imperfections rompent la correspondance théorique entre direction d'arrivée et structure de phase mesurée.

L'impact est sévère. Le Delay-and-Sum pointe à côté de la cible ; le Null Steering place ses zéros au mauvais endroit ; MVDR atténue une partie du signal utile en croyant atténuer du bruit (*signal cancellation*) ; MUSIC produit des pics biaisés ; ESPRIT, qui repose sur une invariance algébrique exacte, est particulièrement fragile. Un traitement sérieux du beamforming ne peut s'arrêter aux équations idéales : calibration, robustesse et techniques de régularisation (comme le *diagonal loading*) font partie intégrante du sujet. La Partie VII y est entièrement consacrée.

Pour donner une idée de l'ordre de grandeur, dans une simulation représentative mais non universelle : sur un réseau très bien calibré, un MVDR peut produire des nulls proches de -60 dB ; avec quelques degrés d'erreur de phase par voie, ces mêmes nulls peuvent remonter autour de -25 dB ; sans calibration, ils restent souvent beaucoup moins profonds. Le facteur entre

théorie idéale et pratique non calibrée peut donc atteindre plusieurs dizaines de dB de capacité anti-brouillage. Dans ce régime, aucun raffinement algorithmique ne compense complètement un défaut de calibration de cet ordre.

1.6 Applications

Les capacités du beamforming expliquent sa diffusion dans de nombreux systèmes modernes exploitant des ondes :

- radars (détection, poursuite, imagerie SAR) ;
- télécommunications mobiles 4G, 5G et *massive MIMO* ;
- Wi-Fi récent, où le beamforming explicite est désormais standard ;
- liaisons satellitaires et constellations basse orbite ;
- radioastronomie, où des réseaux de plusieurs kilomètres synthétisent des télescopes virtuels géants ;
- acoustique et réseaux de microphones pour la séparation de sources sonores ;
- sonars ;
- guerre électronique et systèmes anti-brouillage ;
- plateformes SDR multi-canaux, qui rendent ces techniques accessibles à un coût modeste.

Dans tous ces cas, une même idée s'impose : *l'espace devient une dimension de traitement du signal au même titre que le temps ou la fréquence.*

1.7 Philosophie du livre

Chaque chapitre suivra autant que possible la progression :

1. intuition physique ;
2. modèle mathématique ;
3. interprétation géométrique ;
4. conséquences algorithmiques ;
5. implémentation numérique ;
6. analyse des limites pratiques.

Théorie et pratique ne sont pas séparées artificiellement : en beamforming, une équation n'a de sens que si elle traduit un phénomène physique, et une implémentation n'est fiable que si ses hypothèses mathématiques sont comprises.

Le beamforming peut se lire comme une chaîne d'idées de plus en plus riches. Au départ, il n'y a qu'un ensemble d'antennes et une onde qui les atteint avec des retards différents. En comprenant que ces retards deviennent des phases, on construit le steering vector. En combinant les antennes avec des poids complexes, on obtient un beamformer. En observant la réponse spatiale, on découvre lobes, nulls et résolution. En introduisant la covariance, on rend le système adaptatif. En exploitant les sous-espaces, on transforme le réseau en instrument d'estimation angulaire de haute résolution. En relâchant les hypothèses de bande étroite et de propagation simple, on entre dans le monde des systèmes réels.

Le chapitre suivant pose les bases physiques — ondes, phases, retards, modèle d'onde plane — sur lesquelles tout le reste reposera. Il s'attarde sur quelques relations simples mais structurantes : comment une onde monochromatique se représente comme une exponentielle complexe, pourquoi le nombre d'onde $k = 2\pi/\lambda$ joue en espace le rôle que la pulsation joue en temps, et comment un retard temporel se mue en un facteur de phase sous l'hypothèse bande étroite. C'est de ces briques élémentaires que naîtra, à la fin du chapitre 5, le *steering vector* qui fera office d'interface entre la physique et tous les algorithmes du livre.

Chapitre 2

Ondes électromagnétiques et propagation

Le beamforming exploite le fait qu’une onde n’atteint pas toutes les antennes d’un réseau simultanément. Ce décalage temporel, même extrêmement faible, se traduit en phase pour un signal bande étroite. Toute la théorie qui suit en découle. Ce chapitre construit avec soin le modèle physique sous-jacent : représentation complexe d’une onde, notion de nombre d’onde, modèle d’onde plane, et conversion d’un retard temporel en déphasage.

Les notions présentées ici relèvent largement du cours élémentaire d’électromagnétisme. On pourrait être tenté de les survoler. Ce serait une erreur : la moindre approximation mal comprise se propage en quelques pages sous forme de biais d’estimation, d’inversions de signe, ou de dégradations silencieuses des performances algorithmiques. Une convention de phase changée dans la mauvaise direction transforme un faisceau pointé en $+30^\circ$ en un faisceau pointé en -30° ; une approximation bande étroite non vérifiée fait perdre plusieurs décibels de gain de réseau sans le moindre message d’alerte. La rigueur initiale est donc payante : elle évite des heures de débogage plus loin dans la chaîne de traitement.

2.1 Représentation complexe d’une onde

2.1.1 Du cosinus réel à l’exponentielle complexe

Une onde harmonique réelle observée en un point fixe s’écrit

$$x(t) = A \cos(2\pi f_0 t + \phi).$$

Cette écriture est intuitive mais peu commode dès qu’on empile plusieurs ondes, plusieurs antennes et plusieurs retards : la trigonométrie devient vite ingérable. On lui préfère la *représentation complexe*

$$\tilde{x}(t) = A e^{j(2\pi f_0 t + \phi)}, \quad x(t) = \operatorname{Re}\{\tilde{x}(t)\}.$$

Les déphasages deviennent alors de simples multiplications par un nombre complexe, et les combinaisons d’antennes se ramènent à de l’algèbre linéaire.

Cette représentation n’est pas un artifice notationnel. Tout signal réel admet un *signal analytique* $x_a(t) = x(t) + j\hat{x}(t)$, où $\hat{x}$ est la transformée de Hilbert de x , et le signal analytique d’un cosinus pur est exactement l’exponentielle complexe $e^{j(2\pi f_0 t + \phi)}$. Dans une chaîne RF moderne, la démodulation IQ fournit directement ce signal complexe : la composante en phase I est la partie réelle, la composante en quadrature Q la partie imaginaire.

Une conséquence importante : tout au long du livre, lorsque nous écrirons « signal $x(t)$ », nous désignerons en réalité son enveloppe complexe IQ, échantillonnée à une cadence largement inférieure à la fréquence porteuse. C’est ce que manipule un FPGA ou un DSP en pratique. La

porteuse RF ne réapparaît qu’au moment de relier les phases à un retard physique, via la relation $\Delta\varphi = -2\pi f_0\tau$; partout ailleurs, elle est invisible dans le formalisme. Cette distinction est une source classique de confusion : un débutant en SDR cherche souvent en vain la porteuse dans les échantillons — elle a disparu à la démodulation, et c’est voulu.

2.1.2 Formule d’Euler et géométrie de la phase

La formule d’Euler

$$e^{j\alpha} = \cos \alpha + j \sin \alpha$$

montre que l’exponentielle complexe encode simultanément amplitude (module) et phase (argument). Multiplier un signal par $e^{j\alpha}$ revient à le faire tourner de l’angle α dans le plan complexe, sans modifier son module. Toutes les opérations nécessaires au beamforming — aligner des antennes, créer un faisceau, imposer un null — se ramènent ainsi à des rotations dans $\mathbb{C}$, donc à de simples multiplications complexes. C’est cette propriété qui rend le beamforming bande étroite à la fois élégant et efficace en pratique : un beamformer à N antennes n’est qu’un produit scalaire complexe à N termes, exécutable en quelques nanosecondes sur un FPGA moderne.

2.2 Fréquence, période et longueur d’onde

Dans le vide, une onde électromagnétique se propage à $c \approx 3 \cdot 10^8 \text{ m} \cdot \text{s}^{-1}$. Pour une fréquence f_0 , la période temporelle vaut $T_0 = 1/f_0$, et la distance parcourue en une période est la *longueur d’onde*

$$\lambda = cT_0 = \frac{c}{f_0}.$$

La longueur d’onde est l’échelle spatiale naturelle du beamforming. Les distances significatives — espacement inter-antennes, ouverture du réseau, distance de validité du modèle d’onde plane — se mesurent toujours en fractions ou multiples de λ . Cette habitude n’est pas anodine : raisonner en mètres masquerait le fait qu’un même réseau, opéré à deux fréquences différentes, produit deux « géométries spatiales » radicalement différentes. Ce qui compte pour le beamformer, ce ne sont jamais les centimètres, mais les pas de phase qu’ils représentent à la fréquence considérée.

TABLE 2.1 – Longueurs d’onde et espacement $\lambda/2$ typiques.

Application	f_0	λ	$\lambda/2$
GSM (bande 900)	900 MHz	33.3 cm	16.7 cm
GPS L1	1.575 GHz	19.0 cm	9.5 cm
Wi-Fi 2.4 GHz	2.4 GHz	12.5 cm	6.25 cm
Wi-Fi 5 GHz	5 GHz	6.0 cm	3.0 cm
Radar bande X	10 GHz	3.0 cm	1.5 cm
5G mmWave	28 GHz	1.07 cm	5.4 mm
Liaison V-band	60 GHz	5.0 mm	2.5 mm

2.3 Nombre d’onde et vecteur d’onde

La phase d’une onde évolue à la fois dans le temps (à la pulsation $\omega_0 = 2\pi f_0$) et dans l’espace. On introduit le *nombre d’onde*

$$k = \frac{2\pi}{\lambda}$$

qui représente la phase accumulée par unité de distance : sur un trajet de longueur r , la phase varie de kr . L'analogie avec la pulsation temporelle est complète,

$$\omega_0 = \text{phase par unité de temps}, \quad k = \text{phase par unité de distance}.$$

En toute généralité, la propagation se fait dans une direction quelconque $\hat{\mathbf{u}}$ de l'espace, et l'on remplace k par le *vecteur d'onde*

$$\mathbf{k} = k \hat{\mathbf{u}}, \quad \|\mathbf{k}\| = k = \frac{2\pi}{\lambda}.$$

La phase accumulée le long d'un trajet $\mathbf{r}$ vaut alors le produit scalaire $\mathbf{k}^T \mathbf{r}$.

Remarque 2.1. Avec $d = \lambda/2$ on a $kd = \pi$, soit exactement un demi-tour de phase par espacement inter-antennes à l'endfire. Cette valeur critique est la racine de la condition d'échantillonnage spatial démontrée au chapitre suivant.

On peut développer un peu cette analogie temps/espace, car elle revient constamment dans la suite. Un signal temporel $x(t) = e^{j\omega_0 t}$ oscille à la pulsation ω_0 : la phase avance de $\omega_0 \delta t$ en un temps δt . Un signal spatial $x(\mathbf{r}) = e^{-j\mathbf{k}^T \mathbf{r}}$ oscille de la même manière dans l'espace : la phase avance de $-\mathbf{k}^T \delta \mathbf{r}$ pour un déplacement $\delta \mathbf{r}$. Le *signe* négatif dans la convention spatiale est dicté par la direction de propagation : une onde qui « avance » voit sa phase « reculer » dans le repère spatial. Ce détail est crucial : changer ce signe conduit à des steering vectors $e^{+jmkd \sin \theta}$ qui correspondent à des angles miroirs, et c'est exactement le genre de bug discret qu'on ne diagnostique qu'après avoir vu son faisceau pointer du mauvais côté.

2.4 Onde plane monochromatique

Le modèle d'onde plane monochromatique s'écrit

$$\tilde{x}(\mathbf{r}, t) = A e^{j(2\pi f_0 t - \mathbf{k}^T \mathbf{r} + \phi)}.$$

Le terme $2\pi f_0 t$ décrit l'évolution temporelle ; le terme $-\mathbf{k}^T \mathbf{r}$ décrit l'évolution spatiale. Le signe négatif n'est pas anecdotique : avec la convention temporelle $e^{j2\pi f_0 t}$, il signifie que l'onde se propage *dans le sens* de $\mathbf{k}$. Pour s'en convaincre, suivons un point à phase constante : $2\pi f_0 t - \mathbf{k}^T \mathbf{r} = \text{cste}$ impose $\mathbf{r}$ parallèle à $\mathbf{k}$ et déplacement à la vitesse $\omega_0/k = f_0 \lambda = c$.

Fronts d'onde. À instant fixé, les points où la phase prend une valeur donnée forment un plan orthogonal à $\mathbf{k}$. Ces *fronts d'onde* se déplacent à la vitesse de la lumière. C'est précisément la planéité locale de ces fronts qui justifiera l'approximation d'onde plane pour une source lointaine.

Effet d'un retard. Un signal retardé de τ s'écrit

$$\tilde{x}(t - \tau) = A e^{j2\pi f_0 t} e^{-j2\pi f_0 \tau}.$$

Un retard positif produit donc le facteur multiplicatif

$$e^{-j2\pi f_0 \tau}.$$

Cette convention de signe sera **conservée sans exception** dans l'ouvrage. Elle conduit, comme on le verra au chapitre 5, au steering vector $a_m(\theta) = e^{-jmkd \sin \theta}$. Tout changement de signe dans cette expression doit s'accompagner d'un changement cohérent dans *toutes* les formules de Delay-and-Sum, Null Steering, MVDR, MUSIC et ESPRIT.

2.5 Approximation d'onde plane

2.5.1 Source lointaine et zone de Fraunhofer

Une source ponctuelle rayonne en réalité une onde *sphérique*,

$$\tilde{x}(r, t) = \frac{A}{r} e^{j(2\pi f_0 t - kr)}.$$

Le facteur $1/r$ traduit l'atténuation géométrique de l'amplitude. Le modèle d'onde plane consiste à supposer que, à l'échelle du réseau, les fronts d'onde sont localement plans. Cette approximation est valable lorsque la source est suffisamment éloignée. Le critère classique est la *condition de Fraunhofer*

$$R \gg \frac{2D^2}{\lambda},$$

où D est l'ouverture du réseau (la distance entre les deux antennes les plus éloignées) et R la distance source-réseau. Cette condition provient d'une analyse géométrique de l'erreur de phase entre trajet sphérique exact et trajet plan approximé.

Exemple 2.2 (Ordre de grandeur). ULA de $N = 16$ antennes espacées de $\lambda/2$ à 2.4 GHz : $D = 15 \cdot 6.25 \text{ cm} \simeq 0.94 \text{ m}$, donc $R_F = 2D^2/\lambda \simeq 14 \text{ m}$. Toute source à plus de quelques dizaines de mètres peut être considérée comme une onde plane.

À l'inverse, pour un radar courte portée ou une imagerie acoustique champ proche, l'approximation tombe. Il faut alors recourir au *beamforming en champ proche*, où le steering vector dépend non seulement de la direction mais aussi de la distance à la source. Nous y reviendrons brièvement au chapitre 21.

2.5.2 Hypothèse de polarisation

Une onde électromagnétique réelle est un champ vectoriel, caractérisé par sa polarisation. Dans tout l'ouvrage on suppose la polarisation commune à toutes les antennes et constante dans le temps, ce qui autorise à traiter les signaux comme des scalaires complexes. Les extensions à polarisation duale (de plus en plus fréquentes en 5G et satellitaire) ajoutent une composante au steering vector mais ne changent pas la structure des algorithmes.

2.6 Géométrie de l'ULA et angle d'arrivée

L'ULA est constitué de N antennes alignées sur l'axe x , aux positions $p_m = md$, $m = 0, \dots, N-1$. L'antenne de référence est $m = 0$. L'angle d'arrivée θ est mesuré par rapport à la normale au réseau :

- $\theta = 0^\circ$: direction *broadside*, transversale au réseau ;
- $\theta = \pm 90^\circ$: direction *endfire*, parallèle au réseau ;
- $\theta \in [-90^\circ, +90^\circ]$: *zone visible*, espace des angles physiquement observables.

Ambiguïté avant/arrière. Un ULA idéal présente une ambiguïté de cône : deux ondes arrivant à θ et à $180^\circ - \theta$ produisent exactement la même projection sur l'axe du réseau, donc la même signature de phase. Cette ambiguïté n'est levée que par des antennes directionnelles ou par une géométrie 2D (planaire, circulaire).

2.7 Retard de propagation entre antennes

Considérons une onde plane arrivant à un angle θ sur deux antennes séparées de d . La différence de marche est la projection de d sur la direction de propagation,

$$\Delta\ell = d \sin \theta,$$

et le retard correspondant

$$\tau = \frac{d \sin \theta}{c}.$$

Ce retard est la quantité physique fondamentale du problème : il ne dépend ni de l'algorithme ni de la représentation choisie. Il sera ensuite *interprété* comme une phase, comme un facteur complexe, comme une composante d'un steering vector, mais sa nature reste celle d'un délai entre deux instants d'arrivée.

Cas particuliers.

- $\theta = 0$ (broadside) : $\tau = 0$, aucune différence de phase entre les antennes ;
- $\theta = \pm 90^\circ$ (endfire) : $\tau = \pm d/c$, retard différentiel maximal ;
- θ quelconque : retard intermédiaire proportionnel à $\sin \theta$.

La direction d'arrivée est intégralement encodée dans le signe et l'amplitude du retard.

2.8 Du retard temporel au déphasage

Pour un signal bande étroite (condition formalisée au chapitre 4), un retard τ produit

$$\tilde{x}(t - \tau) = \tilde{x}(t) e^{-j2\pi f_0 \tau},$$

soit le déphasage

$$\Delta\varphi = -2\pi f_0 \tau = -\frac{2\pi}{\lambda} d \sin \theta = -kd \sin \theta.$$

On obtient ainsi l'équation maîtresse

$$\Delta\varphi(\theta) = -kd \sin \theta.$$

Cette formule, bien que démontrée en quelques lignes, est l'ossature de tout le livre. Elle est *linéaire en $\sin \theta$* et non en θ : les beamformers seront naturellement mieux résolus au voisinage du broadside (où $\sin \theta$ varie rapidement) qu'au voisinage de l'endfire.

Exemple 2.3 (Cas typique). À $f_0 = 2.4$ GHz, $d = \lambda/2$, $\theta = 30^\circ$: $\Delta\varphi = -\pi \sin 30^\circ = -\pi/2$. Une onde à 30° du broadside produit un quart de tour de phase entre antennes adjacentes.

C'est cette quantité élémentaire — un quart de tour de phase — qu'on devra mesurer pour distinguer une direction de 30° d'une direction de, disons, 25° (qui produit $\Delta\varphi = -\pi \sin 25^\circ \simeq -76^\circ$, soit 14° d'écart). C'est dérisoire en absolu, et pourtant c'est suffisant pour qu'un réseau de quelques antennes les sépare sans ambiguïté : la phase s'accumule sur toute l'ouverture du réseau, et ce sont les degrés cumulés sur les antennes extrêmes qui décident de la séparation. On retrouvera cette idée systématiquement quand on discutera de résolution angulaire (chapitre 3) et de borne de Cramér-Rao (chapitre 17).

2.9 Ordres de grandeur

Les retards exploités par un réseau sont extrêmement faibles. À 1 GHz avec $d = \lambda/2 = 15$ cm, le retard maximal endfire vaut $\tau_{\max} = d/c = 0.5$ ns, soit exactement *une demi-période* du signal — ce qui n'est pas un hasard mais la conséquence du choix $d = \lambda/2$.

Plus la fréquence augmente, plus ces retards deviennent infimes, mais en proportion identique de la période (tableau 2.2).

TABLE 2.2 – Retards endfire à $d = \lambda/2$ en fonction de la fréquence.

f_0	T_0	$d = \lambda/2$	$\tau_{\max}$
900 MHz	1.11 ns	16.7 cm	555 ps
2.4 GHz	417 ps	6.25 cm	208 ps
5 GHz	200 ps	3.0 cm	100 ps
10 GHz	100 ps	1.5 cm	50 ps
28 GHz	35.7 ps	5.4 mm	17.8 ps
60 GHz	16.7 ps	2.5 mm	8.3 ps

Conséquence pour le matériel. Une erreur de quelques picosecondes entre voies ADC — imperceptible dans un système monocanal — correspond déjà à plusieurs degrés de phase à 10 GHz et à plusieurs dizaines de degrés à 60 GHz. C’est cette exigence de synchronisation qui rend non triviale la réalisation d’un réseau multi-antennes, et qui justifie le chapitre 22 dédié à la calibration.

2.10 Synthèse

Une onde se propage à vitesse finie. Une source lointaine produit, à l’échelle d’un réseau d’antennes, un front d’onde localement plan (condition de Fraunhofer $R \gg 2D^2/\lambda$). Pour un ULA d’espacement d et une onde à l’angle θ , le retard différentiel entre l’antenne m et la référence vaut

$$\tau_m = \frac{md \sin \theta}{c},$$

qui se traduit sous l’hypothèse bande étroite par le déphasage

$$\Delta\varphi_m(\theta) = -mkd \sin \theta.$$

Le chapitre suivant montre comment cette progression de phase est *échantillonnée* le long du réseau, et pourquoi l’espacement $d = \lambda/2$ s’impose comme l’analogue spatial du critère de Nyquist–Shannon. On y verra émerger la notion de *fréquence spatiale*, qui établit un parallèle systématique entre traitement temporel et traitement spatial : aliasing, résolution, fenêtrage, spectre, tout se transpose. Cette analogie est l’un des outils mentaux les plus puissants pour qui apprend le beamforming en venant du traitement du signal classique : ce qu’on sait déjà sur l’analyse spectrale temporelle s’applique mutatis mutandis à l’analyse spatiale, et il suffit souvent de « changer de tête » pour voir tomber la solution d’un problème spatial qu’on aurait résolu sans hésiter dans le temps.

Chapitre 3

Réseaux d'antennes et échantillonnage spatial

Le chapitre précédent a établi que l'angle d'arrivée d'une onde se traduit, sur deux antennes adjacentes, par un déphasage $\Delta\varphi = -kd \sin \theta$. Cette relation est *locale*. Un réseau réel comporte plusieurs antennes, et il ne mesure pas seulement un déphasage isolé : il mesure une *structure spatiale* complète. Ce chapitre étudie cette structure dans son ensemble. Nous montrons qu'un réseau d'antennes agit comme un instrument d'*échantillonnage spatial*, et que cette analogie avec l'échantillonnage temporel guide rigoureusement le choix de l'espacement d , du nombre d'antennes N et de l'ouverture $D = (N - 1)d$.

Ce chapitre joue un rôle pivot dans le livre. Toutes les questions de *dimensionnement* y trouvent leur réponse : combien d'antennes faut-il pour atteindre telle résolution angulaire ? quel est l'espacement maximal admissible avant que des ambiguïtés apparaissent ? quel est le compromis entre coût matériel et performance ? Les réponses sont déterministes : elles découlent de la condition de Nyquist spatial et du calcul du facteur de réseau d'un ULA. Une fois ces formules acquises, dimensionner un système de beamforming devient un exercice de quelques minutes.

3.1 Le réseau comme échantillonneur spatial

Un signal continu $x(t)$ est converti en suite discrète en l'observant à des instants $t_n = nT_s$. Si T_s est trop grand, des fréquences différentes deviennent indiscernables : c'est l'aliasing temporel.

Un réseau d'antennes réalise une opération analogue dans l'espace. Le champ électromagnétique $x(\mathbf{p}, t)$ existe en tout point de l'espace, mais le réseau ne l'observe qu'aux positions $p_m = md$. Le rôle joué par T_s en temps est joué par d en espace. Si d est trop grand, deux directions différentes produisent la même phase échantillonnée sur les antennes, et le réseau ne peut plus les distinguer.

Temporel		Spatial
t	$\longleftrightarrow$	p
T_s	$\longleftrightarrow$	d
$\omega = 2\pi f$	$\longleftrightarrow$	$\mu = kd \sin \theta$
fréquence f	$\longleftrightarrow$	fréquence spatiale $\nu = (d/\lambda) \sin \theta$
aliasing fréquentiel	$\longleftrightarrow$	aliasing spatial
condition de Nyquist	$\longleftrightarrow$	condition $d \leq \lambda/2$

La variable angulaire naturelle n'est pas θ mais $u = \sin \theta$, parce que la différence de marche est proportionnelle à $\sin \theta$. Beaucoup de résultats prennent une forme plus simple en u . C'est aussi en u que le beam pattern d'un ULA pondéré uniformément prend exactement la forme d'un

noyau de Dirichlet classique, identique à celui qu'on rencontre en analyse de Fourier discrète temporelle. Travailler en u revient donc à mettre le problème spatial dans son langage le plus naturel ; on ne reviendra à θ qu'au moment d'afficher des angles physiques au lecteur ou de spécifier des directions cibles aux opérateurs.

3.2 Fréquence spatiale et démonstration de l'aliasing

À une onde plane d'angle θ correspond la suite spatiale

$$a_m(\theta) = e^{-jmkd \sin \theta} = e^{-j2\pi m \nu(\theta)}, \quad \nu(\theta) = \frac{d}{\lambda} \sin \theta.$$

On reconnaît l'analogie spatiale d'une sinusoïde discrète temporelle de pulsation numérique $2\pi\nu$. La quantité $\nu(\theta)$ est la *fréquence spatiale normalisée* associée à la direction θ .

Deux directions θ_1 et θ_2 produisent exactement la même suite échantillonnée si et seulement si

$$e^{-j2\pi m \nu(\theta_1)} = e^{-j2\pi m \nu(\theta_2)} \quad \forall m \in \mathbb{Z},$$

c'est-à-dire si $\nu(\theta_1) - \nu(\theta_2) \in \mathbb{Z}$. Les fréquences spatiales sont donc observables *modulo* 1 seulement : c'est la racine mathématique de l'aliasing spatial.

3.3 Condition de non-aliasing : $d \leq \lambda/2$

Pour la zone visible $\theta \in [-90^\circ, +90^\circ]$, on a $\sin \theta \in [-1, 1]$, donc

$$\nu(\theta) \in \left[-\frac{d}{\lambda}, +\frac{d}{\lambda} \right].$$

La condition de Nyquist spatial impose

$$\left[-\frac{d}{\lambda}, +\frac{d}{\lambda} \right] \subset \left[-\frac{1}{2}, +\frac{1}{2} \right],$$

soit

$$\boxed{d \leq \frac{\lambda}{2}}.$$

Interprétation.

- À $d = \lambda/2$, la zone visible remplit exactement l'intervalle de Nyquist : le réseau exploite toute sa bande spatiale sans repliement. C'est le cas optimal d'un ULA classique.
- À $d < \lambda/2$, l'échantillonnage est plus fin que nécessaire. Pas d'aliasing, mais plus d'antennes pour une ouverture donnée.
- À $d > \lambda/2$, l'intervalle de Nyquist est dépassé : des *lobes de réseau* apparaissent. Ces lobes ne sont pas de simples artefacts, ce sont de vraies ambiguïtés : une interférence dans un lobe parasite peut être amplifiée autant que la cible.

Exemple 3.1 (Lobes de réseau à $d = \lambda$). Pour $d = \lambda$, $\lambda/d = 1$, et les directions ambiguës sont $\sin \theta_2 = \sin \theta_1 + q$ avec $q \in \mathbb{Z}$. Pointer vers $\theta_1 = 0$ produit alors un lobe parasite à $\theta_2 = \pm 90^\circ$.

Cet exemple n'est pas une curiosité de laboratoire. Les réseaux à $d > \lambda/2$ apparaissent naturellement quand on veut maximiser la résolution sous contrainte de coût matériel : en doublant d et en conservant N , on double l'ouverture D , donc on divise la largeur du lobe principal par deux. La contrepartie — les lobes de réseau — peut être tolérée si l'on dispose d'une information *a priori* sur la zone d'arrivée des cibles, ou si l'on combine plusieurs sous-réseaux d'espacements différents pour lever l'ambiguïté (*coprime arrays* et architectures apparentées, abordées au chapitre 21).

3.4 Nombre d'antennes et résolution angulaire

L'espacement d contrôle l'absence d'aliasing. La capacité à *séparer* deux directions proches dépend d'une autre grandeur : l'ouverture totale $D = (N - 1)d$. Plus le réseau est étendu dans l'espace, plus de petites différences d'angle accumulent des différences de phase mesurables entre les deux extrémités.

Phase accumulée à l'extrémité. Pour deux directions θ_1 et θ_2 , la différence de phase à l'antenne extrême $m = N - 1$ vaut

$$|\Delta\varphi_{\text{bord}}| = kD |\sin \theta_2 - \sin \theta_1| = kD |\Delta u|.$$

Une séparation devient significative lorsque $|\Delta\varphi_{\text{bord}}| \sim 2\pi$, d'où l'ordre de grandeur

$$\Delta u_{\text{res}} \sim \frac{\lambda}{D}.$$

On retrouve, en variable spatiale, l'équivalent exact du principe selon lequel la résolution fréquentielle s'améliore avec la durée d'observation.

Cas $d = \lambda/2$. Avec $D = (N - 1)\lambda/2$, on obtient $\Delta u_{\text{res}} \sim 2/(N - 1) \approx 2/N$ pour N grand. Doubler le nombre d'antennes divise la largeur du lobe principal par deux.

3.5 Facteur de réseau d'un ULA uniforme

Pour un ULA uniformément pondéré $w_m = 1/N$, la sortie en réponse à une onde plane d'angle θ s'écrit

$$B(\theta) = \frac{1}{N} \sum_{m=0}^{N-1} e^{-jm\psi}, \quad \psi = kd \sin \theta.$$

La somme géométrique se met sous la forme close

$$\sum_{m=0}^{N-1} e^{-jm\psi} = \frac{1 - e^{-jN\psi}}{1 - e^{-j\psi}} = e^{-j(N-1)\psi/2} \frac{\sin(N\psi/2)}{\sin(\psi/2)},$$

d'où le *facteur de réseau normalisé*

$$|B(\theta)| = \frac{1}{N} \left| \frac{\sin(N\psi/2)}{\sin(\psi/2)} \right|, \quad \psi = kd \sin \theta.$$

Cette fonction, parfois appelée *noyau de Dirichlet*, joue en beamforming le rôle que joue la fenêtre rectangulaire en analyse de Fourier.

Largeur du lobe principal. Les premiers zéros vérifient $\sin(N\psi/2) = 0$, soit $\psi = \pm 2\pi/N$. En remontant à θ , le premier zéro est à

$$\sin \theta_{\text{zéro}} = \pm \frac{\lambda}{Nd},$$

et la largeur null-to-null vaut, pour $d = \lambda/2$,

$$\Delta\theta_{\text{null-null}} \simeq \frac{4}{N} \text{ rad.}$$

3.6 Compromis N , d , D

Les trois grandeurs N , d , D ne sont pas indépendantes : $D = (N - 1)d$. Chaque choix implique un compromis.

- **Augmenter N à d fixé** : ouverture plus grande, lobe principal plus étroit, mais plus de voies RF, plus d'ADC, plus de synchronisation, plus de calcul, plus de calibration.
- **Augmenter d à N fixé** : ouverture plus grande mais risque d'aliasing si $d > \lambda/2$, donc des lobes de réseau potentiellement dangereux.
- **Diminuer d** : pas d'aliasing, mais ouverture réduite à N fixé, donc lobe principal plus large.

Exemple 3.2 (Wi-Fi 2.4 GHz). $\lambda \simeq 12.5$ cm, $d = \lambda/2 \simeq 6.25$ cm.

$$N = 8 \implies D \simeq 43.75 \text{ cm}, \quad \Delta\theta_{\text{null-null}} \simeq 28.6^\circ.$$

$$N = 16 \implies D \simeq 93.75 \text{ cm}, \quad \Delta\theta_{\text{null-null}} \simeq 14.3^\circ.$$

Doubler N double l'ouverture et divise quasiment la résolution angulaire par deux, mais double le coût matériel.

Le coût matériel n'est pas qu'un facteur deux sur le nombre de chaînes RF : il faut aussi doubler le nombre d'ADC, doubler le débit agrégé de données, fournir une horloge commune à toutes les voies, maintenir la cohérence de phase au cours du temps, et calibrer l'ensemble. Au-delà de quelques dizaines d'antennes, ces contraintes deviennent dominantes et orientent vers des architectures hybrides analogique/numérique (chapitre 20) où une partie du beamforming est effectuée par des déphaseurs RF, avant la conversion numérique. C'est précisément la voie suivie par la 5G mmWave avec ses panneaux à 64 ou 128 éléments.

3.7 Réseaux uniformes et non uniformes

L'ULA est le modèle de référence. Sa structure géométrique régulière permet d'obtenir des expressions fermées pour le steering vector, le beam pattern et les algorithmes haute résolution. En particulier :

- ESPRIT exige une *invariance par translation*, propriété naturelle de l'ULA ;
- MUSIC bénéficie de la régularité pour l'extension Root-MUSIC ;
- le beam pattern se calcule comme une transformée de Fourier discrète spatiale, avec toutes les propriétés afférentes.

D'autres géométries existent : réseaux non uniformes, *sparse arrays*, *co-prime arrays*, géométries circulaires, planaires (URA), volumétriques. Pour un réseau quelconque de positions $\mathbf{p}_m$, le steering vector se généralise par

$$a_m(\hat{\mathbf{u}}) = e^{-j\mathbf{k}^T \mathbf{p}_m}, \quad \mathbf{k} = k \hat{\mathbf{u}}.$$

La structure géométrique simple disparaît, mais l'on gagne parfois en flexibilité : réduction des lobes, allongement de l'ouverture effective, réduction du nombre d'antennes pour une résolution donnée. Le chapitre 21 approfondit ces architectures.

3.8 Du réseau idéal au réseau réel

Tous les modèles précédents supposent implicitement que chaque antenne fournit une mesure idéale du champ. En pratique, un réseau est couplé à une chaîne matérielle complète,

antenne $\rightarrow$ filtre $\rightarrow$ LNA $\rightarrow$ mélangeur $\rightarrow$ ADC $\rightarrow$ traitement numérique.

Chaque voie possède ses imperfections : gains g_m différents, phases parasites ϕ_m , retards de câble, dérives d'oscillateur, couplage mutuel entre éléments rayonnants. Le modèle réaliste devient

$$\mathbf{x}(t) = \mathbf{G} \mathbf{a}(\theta) s(t) + \mathbf{n}(t), \quad \mathbf{G} = \text{diag}(g_0 e^{j\phi_0}, \dots, g_{N-1} e^{j\phi_{N-1}}).$$

Le steering vector effectif est alors $\mathbf{a}_{\text{réel}}(\theta) = \mathbf{G} \mathbf{a}(\theta)$, et c'est précisément lui que mesurera l'algorithme.

Une erreur de phase de seulement 10° entre voies, sur un réseau de 8 ou 16 antennes, suffit à dégrader sensiblement un beam pattern : dans des simulations de null steering ou MVDR, un null théorique très profond peut remonter vers -25 dB ou -20 dB, ce qui change complètement la capacité anti-brouillage. La calibration n'est donc pas un raffinement optionnel : c'est ce qui permet aux algorithmes idéaux de fonctionner sur un réseau matériel.

3.9 Synthèse : ce que contrôle la géométrie

1. **Espacement** d contrôle l'échantillonnage spatial. Choix canonique $d = \lambda/2$ pour éviter l'aliasing tout en maximisant l'ouverture par antenne.
2. **Ouverture** $D = (N - 1)d$ contrôle la résolution angulaire : $\Delta u_{\text{res}} \sim \lambda/D$.
3. **Nombre d'antennes** N contrôle le gain de réseau (gain N en SNR pour un Delay-and-Sum sur cible cohérente) et la complexité matérielle.
4. **Géométrie 1D / 2D / 3D** contrôle les directions observables (azimut seul, azimut + élévation, espace complet).
5. **Imperfections matérielles** dégradent le modèle théorique : calibration et robustesse deviennent indispensables.

Le chapitre suivant formalise l'hypothèse *bande étroite* et construit la représentation IQ qui transforme le retard temporel en simple multiplication complexe, faisant émerger le modèle vectoriel $\mathbf{x}(t) = \mathbf{a}(\theta)s(t) + \mathbf{n}(t)$ utilisé dans tout le reste du livre. C'est cette équation, en apparence anodine, qui condense toutes les hypothèses physiques de la chaîne : champ lointain, bande étroite, polarisation unique, antennes identiques, bruit spatialement blanc. Chaque algorithme étudié à partir de la Partie III prendra cette équation pour acquise et opérera dans le monde « propre » de l'algèbre linéaire complexe. La Partie VII reviendra sur ce qui se passe quand l'une des hypothèses sous-jacentes s'effrite.

Chapitre 4

Modèle bande étroite et représentation IQ

Les deux chapitres précédents ont établi qu’une onde plane arrivant sur un ULA produit des retards différentiels $\tau_m = md \sin \theta / c$, et que ces retards se traduisent en déphasages dès que le signal peut être considéré comme bande étroite. Le présent chapitre formalise ce passage. Il construit avec soin le modèle vectoriel

$$\mathbf{x}(t) = \mathbf{a}(\theta) s(t) + \mathbf{n}(t)$$

qui sera l’équation centrale de tous les algorithmes étudiés par la suite. Cette équation paraît anodine ; elle condense en réalité plusieurs hypothèses physiques, électroniques et mathématiques qu’il importe de séparer clairement.

Une bonne manière d’aborder ce chapitre est de garder à l’esprit qu’il fait office de *passerelle* entre deux mondes. D’un côté, la physique réelle : un signal radio occupant une bande passante B autour d’une porteuse f_0 , propagé entre une source et N antennes, échantillonné à des cadences variées par des chaînes RF hétérogènes. De l’autre, le formalisme mathématique : un vecteur complexe $\mathbf{x}(n) \in \mathbb{C}^N$ supposé issu de la somme d’un signal cohérent $\mathbf{a}(\theta)s(n)$ et d’un bruit gaussien circulaire. Les chapitres suivants vivront entièrement dans le second monde ; le présent chapitre est l’endroit où l’on justifie qu’on peut s’y installer.

4.1 Du signal RF au signal en bande de base

Un signal modulé autour d’une porteuse de fréquence f_0 s’écrit

$$x_{\text{RF}}(t) = A(t) \cos(2\pi f_0 t + \phi(t)).$$

Cette expression réelle est exacte mais peu commode : elle mélange dans un seul cosinus l’amplitude et la phase, et porte sur une fréquence souvent élevée (centaines de MHz à plusieurs GHz). Il serait inefficace de la manipuler directement quand l’information utile, portée par $A(t)$ et $\phi(t)$, varie sur des échelles bien plus lentes.

4.1.1 Enveloppe complexe

On introduit l’*enveloppe complexe* (ou *signal en bande de base*)

$$s(t) = A(t) e^{j\phi(t)},$$

de sorte que

$$x_{\text{RF}}(t) = \text{Re} \left\{ s(t) e^{j2\pi f_0 t} \right\}.$$

L'enveloppe $s(t)$ contient toute l'information utile (modulation, contenu numérique, variations d'amplitude), tandis que la porteuse $e^{j2\pi f_0 t}$ n'est qu'un support fréquentiel qui place le signal autour de f_0 .

4.1.2 Démodulation IQ

En pratique, la chaîne RF mélange le signal reçu avec deux signaux en quadrature, $\cos(2\pi f_0 t)$ et $-\sin(2\pi f_0 t)$, puis filtre les sorties passe-bas. Les deux composantes $I(t)$ et $Q(t)$ ainsi obtenues satisfont

$$s(t) = I(t) + jQ(t).$$

Le signal numérique manipulé est donc *nativement complexe*. L'enveloppe $s(t)$ est échantillonnée à une cadence $F_s \geq B$ liée à la largeur spectrale du signal et non à f_0 . Le beamforming bande étroite manipulera systématiquement le signal sous cette forme IQ.

Cette conversion représente un gain de bande passante numérique considérable. À 2.4 GHz avec un signal Wi-Fi de bande $B = 20$ MHz, on échantillonne typiquement à $F_s = 20$ à 40 MHz plutôt qu'aux ~ 5 GHz que demanderait un échantillonnage direct à la fréquence de Nyquist du signal RF. Cette économie d'un facteur 100 sur la cadence ADC est précisément ce qui rend les systèmes SDR multi-canaux abordables. Sans démodulation IQ analogique, un réseau massive MIMO à 64 antennes nécessiterait 64 ADC à plusieurs GHz — prohibitif en coût et en consommation.

4.2 Effet d'un retard sur la représentation IQ

Si l'onde est retardée de τ , le signal RF retardé vaut

$$x_{\text{RF}}(t - \tau) = \text{Re}\left\{s(t - \tau) e^{j2\pi f_0(t - \tau)}\right\} = \text{Re}\left\{s(t - \tau) e^{j2\pi f_0 t} e^{-j2\pi f_0 \tau}\right\}.$$

Deux effets coexistent :

- un *vrai retard* appliqué à l'enveloppe, $s(t - \tau)$;
- une rotation complexe constante, $e^{-j2\pi f_0 \tau}$.

4.3 Hypothèse bande étroite

L'*hypothèse bande étroite* consiste à négliger le décalage $s(t - \tau)$ devant $s(t)$. Plus précisément, on suppose que sur l'intervalle $[t - \tau_{\text{max}}, t]$ correspondant à la traversée du réseau, l'enveloppe reste essentiellement constante. On peut alors écrire

$$s(t - \tau_m) \approx s(t),$$

et le retard se réduit, dans l'enveloppe complexe, à la simple multiplication par

$$e^{-j2\pi f_0 \tau_m} = e^{-jmkd \sin \theta}.$$

C'est sous cette hypothèse que le modèle vectoriel

$$\boxed{\mathbf{x}(t) = \mathbf{a}(\theta) s(t)}$$

remplace l'écriture spatio-temporelle exacte qui ferait apparaître des versions retardées différentes $s(t - \tau_m)$ sur chaque antenne. Le passage de la dépendance spatio-temporelle à une dépendance purement spatiale (à un facteur scalaire $s(t)$ près) est l'élément qui rend possible toute la machinerie algébrique du beamforming bande étroite.

Il faut bien mesurer le saut conceptuel que représente cette factorisation. Sans elle, on devrait raisonner sur une suite « spatio-temporelle » $\{x_m(t - \tau_m)\}$ qui mélange la géométrie et la modulation, et tout le formalisme matriciel deviendrait inopérant : on ne pourrait plus écrire $\mathbb{E}[\mathbf{x}\mathbf{x}^H]$ comme une simple matrice de Toeplitz, on ne pourrait plus décomposer en sous-espaces signal/bruit, on ne pourrait plus diagonaliser. La factorisation $\mathbf{x}(t) = \mathbf{a}(\theta)s(t)$ est donc l'*enabler* algébrique de tout le livre. Quand elle tombe (régime large bande), il faut soit la restaurer artificiellement par décomposition en sous-bandes, soit changer de formalisme et travailler sur des matrices spatio-fréquentielles. Le chapitre 19 traite ces deux options en détail.

4.3.1 Condition de validité

L'approximation n'est pas toujours valable. Elle dépend de la vitesse de variation de $s(t)$ comparée à $\tau_{\max} \approx D/c$, où D est l'ouverture du réseau. Si le signal a une bande passante B , son enveloppe varie sur un temps caractéristique $T_s \sim 1/B$, et la condition d'approximation $\tau_{\max} \ll T_s$ s'écrit

$$B\tau_{\max} \ll 1 \iff \frac{BD}{c} \ll 1.$$

On peut aussi l'exprimer en fraction de bande :

$$\frac{BD}{c} = \frac{B}{f_0} \cdot \frac{D}{\lambda},$$

ce qui montre que l'hypothèse est facile à satisfaire pour des signaux à faible fraction de bande ou des réseaux petits en multiples de λ , et difficile pour des signaux radar large bande sur de grandes ouvertures.

Exemple 4.1 (Wi-Fi). $B = 20$ MHz, $D = 0.94$ m : $BD/c \simeq 6.3 \cdot 10^{-2} \ll 1$. L'hypothèse bande étroite est largement satisfaite.

Exemple 4.2 (Radar large bande). $B = 500$ MHz, $D = 0.94$ m : $BD/c \simeq 1.6$. L'hypothèse est nettement violée. Deux antennes éloignées du réseau ne voient plus la même enveloppe au même instant, et un beamformer bande étroite produit une dégradation significative du gain de réseau. Le chapitre 19 traite spécifiquement ce régime.

4.4 Bruit et interférences

Le modèle bande étroite serait incomplet sans le bruit additif des voies de réception. Chaque antenne m reçoit un bruit thermique $n_m(t)$ supposé gaussien complexe circulaire, indépendant d'une voie à l'autre, de variance σ^2 . En empilant, on obtient le vecteur de bruit $\mathbf{n}(t) \in \mathbb{C}^N$ tel que

$$\mathbb{E}[\mathbf{n}(t)] = \mathbf{0}, \quad \mathbb{E}[\mathbf{n}(t)\mathbf{n}^H(t)] = \sigma^2 \mathbf{I}_N.$$

On dit alors que le bruit est *spatialement blanc*. Pour plusieurs sources L arrivant simultanément avec des enveloppes $s_\ell(t)$ et des angles θ_ℓ , le modèle vectoriel se généralise en

$$\mathbf{x}(t) = \sum_{\ell=1}^L \mathbf{a}(\theta_\ell) s_\ell(t) + \mathbf{n}(t) = \mathbf{A}(\boldsymbol{\theta}) \mathbf{s}(t) + \mathbf{n}(t),$$

où $\mathbf{A}(\boldsymbol{\theta}) = [\mathbf{a}(\theta_1), \dots, \mathbf{a}(\theta_L)] \in \mathbb{C}^{N \times L}$ est la *matrice de steering* et $\mathbf{s}(t) = [s_1(t), \dots, s_L(t)]^T \in \mathbb{C}^L$ le vecteur des enveloppes des sources.

4.5 Échantillonnage et snapshots

Tous les algorithmes opèrent sur des observations discrètes. On note $\mathbf{x}(n) = \mathbf{x}(nT_s)$ le *snapshot spatial* à l'instant n , c'est-à-dire le vecteur des N échantillons IQ simultanés des antennes. Un « snapshot » est donc une coupe spatiale instantanée du champ électromagnétique reçu, qui sera traitée comme un point de $\mathbb{C}^N$. La collection de K snapshots $\{\mathbf{x}(n)\}_{n=1}^K$ forme la donnée brute du traitement, qu'elle soit utilisée pour estimer une covariance, mettre à jour des poids adaptatifs ou alimenter une procédure d'estimation de DOA.

4.6 Quand l'hypothèse bande étroite se dégrade

L'hypothèse bande étroite étant l'ossature de tout le formalisme, il est utile de récapituler ses conséquences *quand elle est violée* :

- le gain de réseau du Delay-and-Sum diminue car les enveloppes ne coïncident plus exactement après compensation de phase ;
- le beam pattern devient *fréquence-dépendant* : il n'est plus la même fonction pour les composantes basses et hautes de la bande ;
- MVDR sous-estime la covariance utile et atténue partiellement le signal lui-même ;
- MUSIC et ESPRIT produisent des pics et des angles biaisés.

On y répond, selon le degré de violation, par deux familles de techniques :

- *true time delay beamforming*, où l'on compense par de véritables retards (lignes à retard analogiques ou filtres FIR fractionnaires en numérique) ;
- *sous-bandes*, où le signal est divisé en sous-bandes suffisamment étroites pour que l'hypothèse soit retrouvée à l'intérieur de chacune, puis recombinié.

Le chapitre 19 décrit ces approches en détail.

4.7 Synthèse : le modèle vectoriel

À l'issue de cette Partie I, l'équation maîtresse du beamforming bande étroite se présente sous trois formes équivalentes selon le nombre de sources.

Source unique.

$$\mathbf{x}(t) = \mathbf{a}(\theta_0) s(t) + \mathbf{n}(t).$$

Plusieurs sources.

$$\mathbf{x}(t) = \mathbf{A}(\boldsymbol{\theta}) \mathbf{s}(t) + \mathbf{n}(t).$$

Sortie d'un beamformer.

$$y(t) = \mathbf{w}^H \mathbf{x}(t) = \mathbf{w}^H \mathbf{A}(\boldsymbol{\theta}) \mathbf{s}(t) + \mathbf{w}^H \mathbf{n}(t).$$

Le chapitre suivant définit rigoureusement le *steering vector* $\mathbf{a}(\theta)$, étudie sa structure géométrique et le rôle qu'il joue comme interface entre le monde physique et le monde algorithmique. Ce sera la dernière brique avant d'entrer dans les algorithmes proprement dits. À l'issue du chapitre 5, le lecteur disposera de tout le vocabulaire et de toutes les notations nécessaires pour aborder les beamformers déterministes de la Partie II, puis les beamformers statistiques et adaptatifs des parties suivantes. La Partie I est, en ce sens, un sas : tout le reste du livre y puise ses fondations sans jamais y revenir.

Chapitre 5

Steering vector et array manifold

Le *steering vector* est l'objet mathématique central du beamforming. Tous les algorithmes étudiés dans la suite, sans exception, le manipulent comme une référence : Delay-and-Sum y aligne ses poids, Null Steering y impose un gain nul, MVDR l'utilise comme contrainte, MUSIC mesure son orthogonalité au sous-espace bruit. Ce chapitre étudie sa définition, sa structure géométrique, ses propriétés algébriques, et son interprétation comme *filtre adapté spatial*.

On peut résumer son rôle d'une formule : le steering vector est la *fonction de transfert spatiale* qui transforme une direction d'arrivée en une signature complexe mesurable. À chaque angle θ il associe un vecteur $\mathbf{a}(\theta) \in \mathbb{C}^N$, et le problème inverse — retrouver θ à partir des données — est précisément l'objet de toute la Partie V. À l'autre bout du processus, le problème direct — amplifier une direction connue — est l'objet des Parties II à IV. Tout le livre tourne donc autour de cet objet géométrique, et le maîtriser à fond paie largement l'investissement de ce chapitre relativement court.

5.1 Définition

Pour un réseau d'antennes de positions $\{\mathbf{p}_m\}_{m=0}^{N-1}$ et une onde plane de vecteur d'onde $\mathbf{k} = k \hat{\mathbf{u}}$, le steering vector est le vecteur de réponses complexes des N antennes,

$$\mathbf{a}(\hat{\mathbf{u}}) = \begin{bmatrix} e^{-j\mathbf{k}^T \mathbf{p}_0} \\ e^{-j\mathbf{k}^T \mathbf{p}_1} \\ \vdots \\ e^{-j\mathbf{k}^T \mathbf{p}_{N-1}} \end{bmatrix} \in \mathbb{C}^N.$$

La composante m contient la phase relative reçue par l'antenne m , prise relativement à l'origine (généralement $\mathbf{p}_0 = \mathbf{0}$).

Pour un ULA orienté le long de l'axe x , avec un angle d'arrivée θ mesuré depuis le broadside, $\mathbf{k}^T \mathbf{p}_m = m k d \sin \theta$ et l'expression prend la forme canonique

$$\mathbf{a}(\theta) = \begin{bmatrix} 1 \\ e^{-jkd \sin \theta} \\ e^{-j2kd \sin \theta} \\ \vdots \\ e^{-j(N-1)kd \sin \theta} \end{bmatrix}, \quad a_m(\theta) = e^{-jm k d \sin \theta}.$$

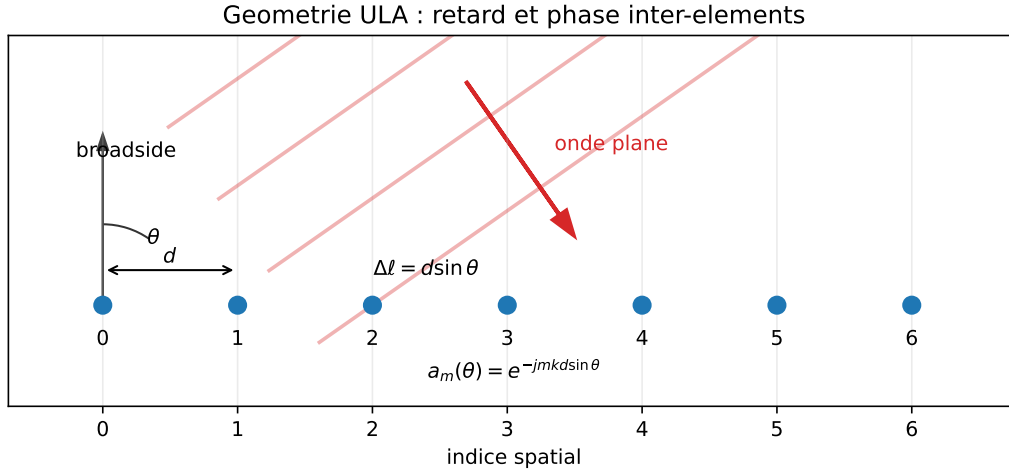


FIGURE 5.1 – Géométrie ULA illustrée ($N = 7$, angle indicatif $\theta = 35^\circ$) et convention de phase : l'écart de marche entre deux antennes adjacentes vaut $d \sin \theta$, d'où la progression $e^{-jmkd \sin \theta}$.

5.2 Structure géométrique

5.2.1 Vecteur de Vandermonde

Posons $z(\theta) = e^{-jkd \sin \theta}$. On voit immédiatement que

$$\mathbf{a}(\theta) = [1, z(\theta), z^2(\theta), \dots, z^{N-1}(\theta)]^T.$$

$\mathbf{a}(\theta)$ est donc une *colonne d'une matrice de Vandermonde* associée au point $z(\theta)$ du cercle unité de $\mathbb{C}$. Cette propriété est cruciale : elle est la racine algébrique d'ESPRIT, qui exploite la *translation* $z \mapsto z$ entre deux sous-réseaux décalés d'une antenne.

5.2.2 Norme et sphère

Pour un ULA, toutes les composantes ont module 1 :

$$\|\mathbf{a}(\theta)\|^2 = \sum_{m=0}^{N-1} |a_m(\theta)|^2 = N.$$

La norme est donc *constante*, indépendante de θ : l'*array manifold* $\mathcal{M} = \{\mathbf{a}(\theta) : \theta \in [-90^\circ, 90^\circ]\}$ est contenu dans la sphère de rayon $\sqrt{N}$ de $\mathbb{C}^N$. C'est la source du facteur $\sqrt{N}$ ou N omniprésent dans les expressions de gain de réseau.

5.2.3 Lien avec la transformée de Fourier discrète

À $d = \lambda/2$, on peut écrire $a_m(\theta) = e^{-j\pi m \sin \theta}$. En posant $u = \sin \theta$ et en discrétisant u aux points $u_k = 2k/N$, on retrouve exactement les atomes de la DFT à N points :

$$a_m(u_k) = e^{-j2\pi mk/N}.$$

Cette parenté n'est pas fortuite : le Delay-and-Sum sur un ULA à $\lambda/2$ se calcule comme une DFT spatiale, et toute la machinerie de l'analyse de Fourier (résolution, fuite spectrale, fenêtrage) s'applique directement aux beam patterns.

Cette observation a une portée algorithmique majeure. Là où on calculerait *naïvement* le beam pattern en $\mathcal{O}(N^2)$ pour N angles, l'usage de la FFT permet de le faire en $\mathcal{O}(N \log N)$. Pour un réseau massive MIMO à 256 antennes, on passe d'environ $6 \cdot 10^4$ opérations par snapshot

à environ $2 \cdot 10^3$: un facteur 30 sur la complexité, et donc sur la consommation électrique et la latence. Toutes les implémentations performantes de Delay-and-Sum en radar et acoustique reposent sur cette équivalence FFT.

5.3 Produit scalaire entre steering vectors

Le produit scalaire $\mathbf{a}^H(\theta_1)\mathbf{a}(\theta_2)$ joue un rôle clé dans la résolution du beamforming. Pour un ULA :

$$\mathbf{a}^H(\theta_1)\mathbf{a}(\theta_2) = \sum_{m=0}^{N-1} e^{jmkd(\sin \theta_1 - \sin \theta_2)} = \sum_{m=0}^{N-1} e^{jm\alpha}$$

avec $\alpha = kd(\sin \theta_1 - \sin \theta_2)$. La somme géométrique donne

$$\mathbf{a}^H(\theta_1)\mathbf{a}(\theta_2) = e^{j(N-1)\alpha/2} \frac{\sin(N\alpha/2)}{\sin(\alpha/2)}.$$

Le module est le *noyau de Dirichlet*

$$\left| \mathbf{a}^H(\theta_1)\mathbf{a}(\theta_2) \right| = \left| \frac{\sin(N\alpha/2)}{\sin(\alpha/2)} \right|.$$

Cette quantité vaut N pour $\alpha = 0$ (directions confondues) et s'annule pour $\alpha = 2\pi q/N$ avec $q \in \mathbb{Z}^*$. C'est exactement la fonction qui détermine la *résolution angulaire* et la forme du lobe principal d'un beamformer conventionnel.

5.4 Steering vector et beamformer

Le beamforming consiste à appliquer un vecteur de poids $\mathbf{w} \in \mathbb{C}^N$ à l'observation, donnant la sortie

$$y(t) = \mathbf{w}^H \mathbf{x}(t).$$

Pour un signal idéal d'une seule source à θ_0 , $\mathbf{x}(t) = \mathbf{a}(\theta_0)s(t)$, on obtient

$$y(t) = \underbrace{\mathbf{w}^H \mathbf{a}(\theta_0)}_{=G(\theta_0)} s(t).$$

Le scalaire complexe $G(\theta) = \mathbf{w}^H \mathbf{a}(\theta)$ est le *gain complexe* du beamformer dans la direction θ , et le *beam pattern* est l'application $\theta \mapsto G(\theta)$, étudié en détail au chapitre 6.

Filtre adapté spatial. À norme de $\mathbf{w}$ fixée, le produit scalaire $\mathbf{w}^H \mathbf{a}(\theta_0)$ est maximisé lorsque $\mathbf{w}$ est colinéaire à $\mathbf{a}(\theta_0)$; on choisit alors

$$\mathbf{w}_{\text{DAS}} = \frac{1}{N} \mathbf{a}(\theta_0),$$

qui n'est rien d'autre que le *filtre adapté spatial* pour la direction θ_0 , analogue du filtre adapté temporel classique. Ce choix maximise le SNR de sortie en présence d'un bruit spatialement blanc. Lorsque le bruit cesse d'être blanc — en présence d'interférences directionnelles —, ce choix n'est plus optimal et MVDR (chapitre 10) prend le relais.

Cette analogie avec le filtre adapté temporel mérite d'être prolongée : en traitement temporel, le filtre adapté maximise le SNR pour la détection d'une forme d'onde connue dans un bruit blanc additif. Le filtre est, à un retard près, le conjugué temporel de la forme à détecter. En traitement spatial, le « signal » à détecter n'est pas une forme d'onde mais une *signature de phase* ; le bruit « blanc » n'est pas dans le temps mais dans l'espace ; et le « filtre » est un vecteur de pondération complexe. Le théorème sous-jacent est exactement le même — l'inégalité de Cauchy-Schwarz — et l'optimalité aussi. C'est pourquoi le DAS n'est pas qu'un beamformer « simple » : il est le beamformer *optimal* en bruit blanc sous modèle idéal, et tous les autres ne s'en éloignent que pour traiter des situations où le bruit est structuré.

5.5 Plusieurs sources et matrice de steering

Pour L sources de directions θ_ℓ , on empile les steering vectors dans la *matrice de steering*

$$\mathbf{A}(\boldsymbol{\theta}) = \begin{bmatrix} \mathbf{a}(\theta_1) & \mathbf{a}(\theta_2) & \cdots & \mathbf{a}(\theta_L) \end{bmatrix} \in \mathbb{C}^{N \times L}.$$

Le modèle vectoriel s'écrit

$$\mathbf{x}(t) = \mathbf{A}(\boldsymbol{\theta}) \mathbf{s}(t) + \mathbf{n}(t), \quad \mathbf{s}(t) = [s_1(t), \dots, s_L(t)]^T.$$

Rang plein des steering vectors. Sur un ULA, $\mathbf{A}$ est de rang plein $\min(N, L)$ dès que les angles θ_ℓ sont distincts deux à deux et restent dans la zone visible. Cette propriété est utilisée par MUSIC et ESPRIT, qui supposent $L < N$ (donc $\mathbf{A}$ de rang plein colonne).

5.6 Réseaux 2D et 3D

Pour un réseau planaire de positions $\mathbf{p}_m = (x_m, y_m, 0)^T$, le steering vector dépend de deux angles, azimut φ et élévation ϑ , via le vecteur unitaire de propagation

$$\hat{\mathbf{u}}(\varphi, \vartheta) = (\cos \vartheta \sin \varphi, \cos \vartheta \cos \varphi, \sin \vartheta)^T.$$

On a alors

$$a_m(\varphi, \vartheta) = e^{-jk(x_m u_x + y_m u_y)}.$$

La structure algébrique reste identique à celle de l'ULA : seul le produit $\mathbf{k}^T \mathbf{p}_m$ change. La plupart des résultats que nous établirons sur l'ULA — beam pattern, condition d'échantillonnage, résolution, sous-espaces — se transposent immédiatement.

5.7 Steering vector réel vs steering vector théorique

Le steering vector théorique repose sur des antennes idéalisées, identiques, et positionnées sans erreur. Le steering vector réel

$$\mathbf{a}_{\text{réel}}(\theta) = \mathbf{G} \mathbf{a}(\theta) + \mathbf{e}(\theta)$$

intègre :

- un gain et une phase par voie, $\mathbf{G} = \text{diag}(g_m e^{j\phi_m})$;
- des erreurs résiduelles $\mathbf{e}(\theta)$ dues au couplage mutuel, aux erreurs de position, aux dérives ;
- éventuellement, une réponse directionnelle individuelle de chaque antenne (différente d'une réponse isotrope).

La quantité fondamentale n'est pas la valeur absolue du steering vector, mais sa *cohérence* d'une voie à l'autre. C'est cette cohérence que la calibration (chapitre 22) cherche à restaurer.

Effet d'une erreur de phase aléatoire. Si chaque antenne subit une phase aléatoire $\delta\phi_m \sim \mathcal{N}(0, \sigma_\phi^2)$, le gain effectif d'un Delay-and-Sum sur la cible devient

$$\mathbb{E} \left[\left| \mathbf{w}_{\text{DAS}}^H \mathbf{a}_{\text{réel}} \right|^2 \right] \approx N \cdot e^{-\sigma_\phi^2},$$

qui se dégrade exponentiellement avec la variance de phase. Une $\sigma_\phi = 30^\circ \simeq 0.52 \text{ rad}$ réduit déjà le gain effectif de $\sim 1.2 \text{ dB}$; au-delà de 60° , la perte dépasse 5 dB et le beamforming perd l'essentiel de son intérêt.

5.8 Erreurs fréquentes de convention

Le steering vector est simple à écrire, mais les erreurs de convention sont parmi les causes les plus fréquentes de résultats faux. Le danger vient rarement d'une convention différente : il vient d'une convention utilisée de manière incohérente entre les chapitres, le code et les figures.

5.8.1 Confondre $\mathbf{a}(\theta)$ et $\mathbf{a}^*(\theta)$

Avec la convention de ce document :

$$\mathbf{a}_m(\theta) = e^{-jmkd \sin \theta}.$$

La sortie d'un beamformer est :

$$y(n) = \mathbf{w}^H \mathbf{x}(n).$$

Pour un Delay-and-Sum pointé vers θ_0 , on choisit :

$$\mathbf{w}_{\text{DAS}} = \frac{1}{N} \mathbf{a}(\theta_0),$$

car alors :

$$\mathbf{w}_{\text{DAS}}^H \mathbf{a}(\theta_0) = \frac{1}{N} \mathbf{a}^H(\theta_0) \mathbf{a}(\theta_0) = 1.$$

Si l'on utilise par erreur $\mathbf{a}^*(\theta_0)$ comme poids avec la même définition de sortie, on obtient :

$$\frac{1}{N} \mathbf{a}^T(\theta_0) \mathbf{a}(\theta_0),$$

qui n'est généralement pas égal à 1. Le faisceau pointe alors vers une direction symétrique ou produit un beam pattern incohérent.

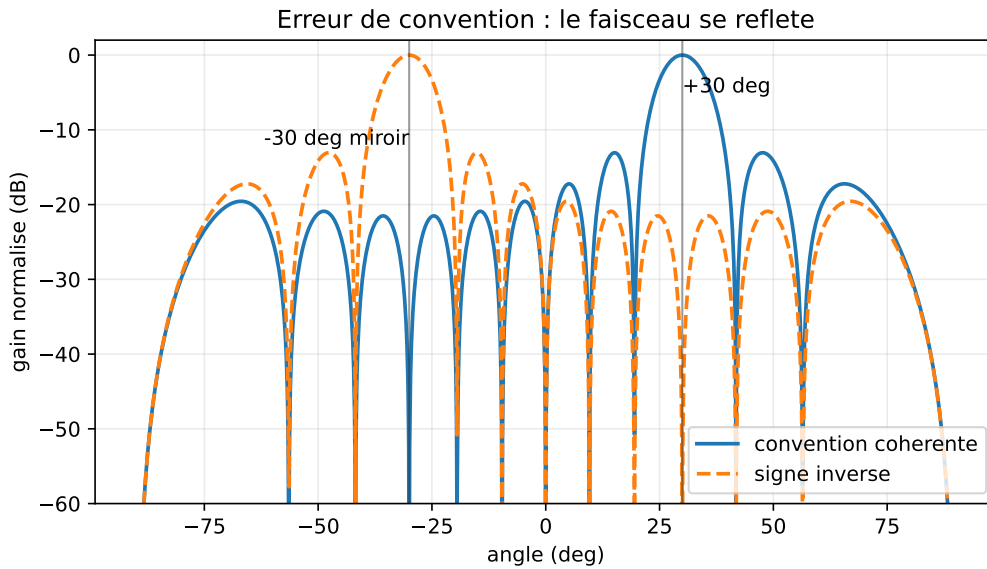


FIGURE 5.2 – Effet typique d'une convention incohérente (DAS, $N = 12$, $d = \lambda/2$) : un faisceau censé pointer vers $+30^\circ$ se retrouve miroir vers -30° .

5.8.2 Réception et émission

En réception, les poids apparaissent sous la forme :

$$y = \mathbf{w}^H \mathbf{x}.$$

En émission, on construit plutôt un signal transmis :

$$\mathbf{x}_{\text{tx}} = \mathbf{w}_{\text{tx}} s.$$

Selon la convention de phase, le vecteur utilisé en émission peut être le conjugué de celui utilisé en réception. Ce n'est pas une contradiction : c'est la conséquence de la réciprocité entre former une somme cohérente à la réception et imposer une phase cohérente à l'émission. Le piège est de mélanger les deux dans le même code.

5.8.3 Confondre $\mathbf{w}^H \mathbf{x}$ et $\mathbf{w}^T \mathbf{x}$

Les signaux IQ sont complexes. Le produit scalaire physique est hermitien :

$$\mathbf{w}^H \mathbf{x}.$$

Utiliser $\mathbf{w}^T \mathbf{x}$ supprime la conjugaison. Le résultat peut sembler plausible pour des vecteurs réels, mais il est faux dès que les phases portent l'information spatiale. Cette erreur casse en particulier la normalisation MVDR, le beam pattern et les tests de null.

5.8.4 Inverser le signe de phase

Certains ouvrages utilisent :

$$a_m(\theta) = e^{+jmkd \sin \theta}.$$

Cette convention est valable si elle est appliquée partout. Mais si le beam pattern est calculé avec $e^{-jmkd \sin \theta}$ et ESPRIT convertit les valeurs propres comme si le signe était positif, les angles estimés sont inversés :

$$\theta \longrightarrow -\theta$$

dans le cas ULA broadside. Le signe du steering vector, le signe dans MUSIC, le signe dans Capon et la formule ESPRIT doivent former un bloc cohérent.

5.8.5 Angle depuis la normale ou depuis l'axe

Dans ce document, $\theta = 0$ désigne le broadside, c'est-à-dire la normale au réseau. Certains textes mesurent l'angle ψ depuis l'axe du réseau. Les deux sont reliés par :

$$\sin \theta = \cos \psi.$$

Une formule écrite avec $\sin \theta$ devient donc une formule avec $\cos \psi$. Mélanger ces deux définitions donne des lobes placés au mauvais endroit, même si le signe de phase est correct.

5.8.6 Incohérence entre beam pattern, MUSIC et ESPRIT

Un test simple devrait être vérifié :

$$B(\theta_0) = \mathbf{w}_{\text{DAS}}^H \mathbf{a}(\theta_0) = 1,$$

le pseudo-spectre MUSIC doit produire un pic au même θ_0 , et ESPRIT doit convertir

$$z = e^{-jkd \sin \theta_0}$$

en :

$$\theta_0 = \arcsin\left(-\frac{\arg z}{kd}\right).$$

Si l'un des trois tests échoue, ce n'est probablement pas un problème d'algorithme : c'est une convention incohérente.

5.9 Vision géométrique : l'array manifold

Lorsque θ varie continûment dans la zone visible, $\mathbf{a}(\theta)$ décrit une courbe lisse $\mathcal{M}$ dans la sphère de rayon $\sqrt{N}$ de $\mathbb{C}^N$. Cette courbe, l'*array manifold*, est l'objet géométrique fondamental du beamforming. Toute la richesse algorithmique du domaine se lit comme une exploration de ses propriétés :

- sa *longueur* et sa *courbure* déterminent la résolution locale du réseau ;
- ses *intersections* avec des sous-espaces décident des performances de MUSIC et de la décomposition signal/bruit ;
- ses *auto-intersections* signalent les ambiguïtés angulaires des réseaux non uniformes ou mal échantillonnés.

Le beamforming consiste, à chaque instant, à choisir un vecteur $\mathbf{w} \in \mathbb{C}^N$ qui a :

- un grand recouvrement avec $\mathbf{a}(\theta_0)$ (gain sur la cible) ;
- un faible recouvrement avec $\mathbf{a}(\theta_i)$ pour les directions d'interférences à rejeter ;
- un faible recouvrement avec les directions porteuses de bruit.

Cette vision géométrique guide toute la suite. Elle apparaîtra explicitement dans la formulation MVDR, où $\mathbf{w}$ est projeté orthogonalement à un sous-espace d'interférences, et dans MUSIC, où la distance $\mathbf{a}(\theta) \rightarrow$ sous-espace bruit devient le critère d'estimation.

5.10 Implémentation numérique

Listing 5.1 – Construction d'un steering vector ULA

```

1  import numpy as np
2
3  def steering_vector(theta, N, d_over_lambda=0.5):
4      """
5      Steering vector d'un ULA à N antennes, espacement d / lambda.
6      theta : angle en radians, mesuré depuis le broadside.
7      """
8      m = np.arange(N)
9      phi = -2.0 * np.pi * d_over_lambda * np.sin(theta) * m
10     return np.exp(1j * phi)
11
12 def steering_matrix(thetas, N, d_over_lambda=0.5):
13     """
14     Matrice de steering A(theta) pour L angles.
15     thetas : array de L angles en radians.
16     Retour : matrice N x L.
17     """
18     return np.column_stack([steering_vector(th, N, d_over_lambda) for th in thetas])

```

Ces deux fonctions sont les briques de base de tous les algorithmes des chapitres suivants. On les retrouvera dans le calcul des beam patterns, des poids MVDR, des pseudo-spectres MUSIC et des estimées ESPRIT.

5.11 Synthèse

À l'issue de la Partie I, on dispose des objets suivants :

- **Modèle vectoriel** : $\mathbf{x}(t) = \mathbf{A}(\boldsymbol{\theta}) \mathbf{s}(t) + \mathbf{n}(t)$.
- **Steering vector ULA** : $a_m(\theta) = e^{-jmkd \sin \theta}$, de norme $\sqrt{N}$.

- **Beamformer** : $y(t) = \mathbf{w}^H \mathbf{x}(t)$, beam pattern $B(\theta) = \mathbf{w}^H \mathbf{a}(\theta)$.
- **Hypothèses sous-jacentes** : champ lointain, bande étroite, antennes identiques, bruit spatialement blanc (sauf indication contraire).

La Partie II construit, à partir de ces objets, les premiers beamformers *déterministes* : ceux dont les poids ne dépendent pas des données reçues mais uniquement de la géométrie du problème et des directions considérées comme connues. C'est l'occasion d'introduire le beam pattern de manière systématique avant de passer à l'adaptation statistique en Partie III.

Une lecture rétrospective de la Partie I peut être utile à ce stade. On y a accompli un trajet précis : d'un signal physique RF arrivant sur des antennes réelles, on a abouti à un vecteur complexe vivant dans $\mathbb{C}^N$ et à un objet géométrique — l'array manifold — qui paramètre les directions par des points d'une sphère complexe. Toute la physique des chapitres 2 et 3 est maintenant absorbée dans la notation $\mathbf{a}(\theta)$. À partir d'ici, les retards τ_m , la porteuse $e^{j2\pi f_0 t}$ et la condition de Fraunhofer resteront en arrière-plan : on manipulera surtout des vecteurs et des matrices. La physique n'a pas disparu ; elle est condensée dans l'algèbre linéaire.

Deuxième partie

Beamforming déterministe

Chapitre 6

Beam pattern et fenêtrage spatial

Au chapitre précédent, on a vu qu'un beamformer transforme un snapshot spatial en un scalaire $y(t) = \mathbf{w}^H \mathbf{x}(t)$. La question naturelle est : à *quelles directions ce système est-il sensible ?* La réponse est le *beam pattern*. Il joue, pour un beamformer, le rôle que la réponse en fréquence joue pour un filtre temporel : il révèle d'un coup d'œil la résolution, la sélectivité spatiale, les fuites latérales et les rejets directionnels du système.

Concevoir un beamformer, c'est essentiellement modeler la forme de son beam pattern. Le beam pattern est l'*interface visuelle* entre l'algorithmicien et la physique du problème : pour un radariste, c'est le diagramme qui dit « ici je vois bien, là je suis aveugle » ; pour un ingénieur télécom, c'est la signature spatiale qui dit « ici je transmets, ici je rejette ». Toute la suite du livre, à chaque fois qu'elle introduit un nouveau beamformer, finit par tracer son beam pattern et commenter ses lobes principal, secondaires et nulls. Maîtriser leur lecture est donc une compétence transversale.

6.1 Définition

Définition 6.1 (Beam pattern). Pour un vecteur de poids $\mathbf{w} \in \mathbb{C}^N$ et un steering vector $\mathbf{a}(\theta)$, le *beam pattern* est la fonction

$$B(\theta) = \mathbf{w}^H \mathbf{a}(\theta).$$

Cette quantité est complexe. Son module $|B(\theta)|$ mesure l'amplitude de la réponse dans la direction θ , et sa phase décrit la phase globale du signal en sortie. Le gain en puissance est $|B(\theta)|^2$. On le représente presque toujours en décibels après normalisation au maximum :

$$B_{\text{dB}}(\theta) = 20 \log_{10} \left(\frac{|B(\theta)|}{\max_{\theta} |B(\theta)|} \right).$$

La normalisation place le lobe principal à 0 dB et exprime les autres directions en atténuation relative.

Remarque 6.2 (Choix de $20 \log_{10}$ ou $10 \log_{10}$). On utilise $20 \log_{10}$ lorsque $B(\theta)$ est interprété comme une amplitude. Si l'on représente directement une puissance ou un spectre (par exemple le pseudo-spectre de Capon $\mathbf{a}^H \mathbf{R}^{-1} \mathbf{a}$), on emploie $10 \log_{10}$ pour garder une lecture en dB cohérente.

6.2 Pourquoi un beam pattern

Le beam pattern ne décrit pas seulement la sensibilité aux ondes incidentes : il décrit aussi la *réjection des directions parasites*. À un beam pattern donné on associe quatre indicateurs fondamentaux.

- Le *lobe principal* : région autour de la direction de pointage où $|B(\theta)|$ est maximal. Sa largeur, mesurée à -3 dB ou entre les premiers zéros, est la résolution angulaire du système.

- Les *lobes secondaires* (*side lobes*) : maxima locaux en dehors du lobe principal. Leur niveau, exprimé en dB sous le maximum, conditionne la quantité d'énergie parasite captée des directions non visées.
- Les *zéros spatiaux* (*nulls*) : directions où $|B(\theta)| = 0$, qui rejettent complètement les interférences correspondantes.
- Le *gain de réseau* : valeur maximale de $|B(\theta_0)|$ sur la cible, à comparer au comportement sur le bruit.

6.3 Lecture spectrale

Pour un ULA d'espacement d , posons $u = \sin \theta$ et $\nu = (d/\lambda)u$. Le beam pattern s'écrit

$$B(u) = \sum_{m=0}^{N-1} w_m^* e^{-j2\pi m\nu} = \mathcal{F}\{w_m^*\}(\nu),$$

où $\mathcal{F}$ désigne la transformée de Fourier discrète spatiale. *Le beam pattern n'est rien d'autre que la transformée de Fourier du conjugué des poids.* Tout le langage de l'analyse de Fourier — résolution, fenêtrage, fuite spectrale — s'applique. Cette observation fonde toute la suite du chapitre.

Le tableau de correspondance temps/espace prend alors toute sa substance :

Filtre temporel	Beamformer spatial
réponse impulsionnelle $h[n]$	poids w_m^*
réponse en fréquence $H(f)$	beam pattern $B(\theta)$
bande passante	largeur du lobe principal
sélectivité fréquentielle	sélectivité angulaire
ripple, lobes secondaires	lobes secondaires
zéros du filtre	nulls spatiaux
filtre RIF / RII	poids fixes / poids adaptatifs

Quiconque a passé du temps à concevoir un filtre numérique trouvera la plupart des techniques de cette partie familières. Les outils changent de nom — on parle de « fenêtrage spatial » là où l'on parlait de « fenêtrage temporel » — mais les principes sont identiques à la lettre. Quand on doute, traduire le problème spatial en problème temporel et inversement est presque toujours fécond.

6.4 ULA à poids uniformes

Pour $w_m = 1/N$ (Delay-and-Sum pointé broadside), on a déjà vu au chapitre 3 :

$$|B(\theta)| = \frac{1}{N} \left| \frac{\sin(N\psi/2)}{\sin(\psi/2)} \right|, \quad \psi = kd \sin \theta.$$

C'est l'analogie spatiale du noyau de Dirichlet associé à une fenêtre rectangulaire dans le temps. On en déduit immédiatement les caractéristiques :

- maximum unitaire à $\theta = 0$;
- premier zéro à $\sin \theta = \pm \lambda/(Nd)$;
- premier lobe secondaire à environ -13.3 dB (limite classique de la fenêtre rectangulaire) ;
- décroissance asymptotique en $1/\theta$.

Pointage vers θ_0 . Le beamformer Delay-and-Sum pointé vers θ_0 est obtenu en choisissant $\mathbf{w} = \mathbf{a}(\theta_0)/N$. Le beam pattern devient

$$B(\theta) = \frac{1}{N} \mathbf{a}^H(\theta_0) \mathbf{a}(\theta) = \frac{1}{N} e^{-j(N-1)\beta/2} \frac{\sin(N\beta/2)}{\sin(\beta/2)},$$

avec $\beta = kd(\sin \theta - \sin \theta_0)$. Le maximum se déplace en $\theta = \theta_0$, mais le *lobe principal s'élargit* lorsque θ_0 s'éloigne du broadside : c'est l'effet du facteur $\sin \theta$ dans la variable conjuguée.

6.5 Influence des paramètres

6.5.1 Nombre d'antennes N

Plus N est grand, plus l'ouverture $D = (N - 1)d$ s'allonge, plus le lobe principal se rétrécit. La largeur entre premiers zéros vaut, en broadside, $\Delta\theta_{n-n} \simeq 4/N$ rad pour $d = \lambda/2$. Le niveau des lobes secondaires reste, lui, fixé à ~ -13.3 dB : ce n'est pas une fonction de N mais de la *forme* de la pondération.

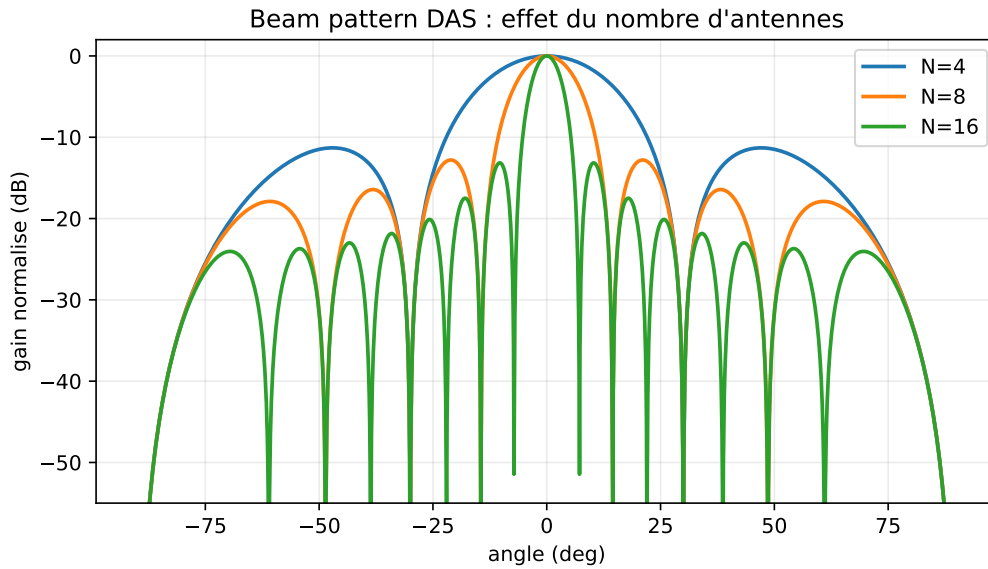


FIGURE 6.1 – Effet du nombre d'antennes sur un DAS broadside ($N = 4, 8, 16$, $d = \lambda/2$) : le lobe principal se resserre lorsque N augmente, tandis que le niveau du premier lobe secondaire reste lié à la pondération uniforme.

6.5.2 Espacement d

- $d < \lambda/2$: lobe principal plus large, pas d'aliasing ;
- $d = \lambda/2$: cas standard, ouverture maximale sans aliasing ;
- $d > \lambda/2$: apparition de *lobes de réseau* (*grating lobes*) qui atteignent le même niveau que le lobe principal. Ces lobes ne sont pas des artefacts numériques mais de véritables ambiguïtés angulaires.

6.5.3 Pointage

Une erreur de pointage $\delta\theta_0$ déplace le lobe principal à côté de la cible. Le gain perçu sur la cible est alors

$$|B(\theta_{\text{cible}})|^2 = \frac{1}{N^2} \frac{\sin^2(N\beta/2)}{\sin^2(\beta/2)}, \quad \beta = kd(\sin \theta_{\text{cible}} - \sin \theta_0).$$

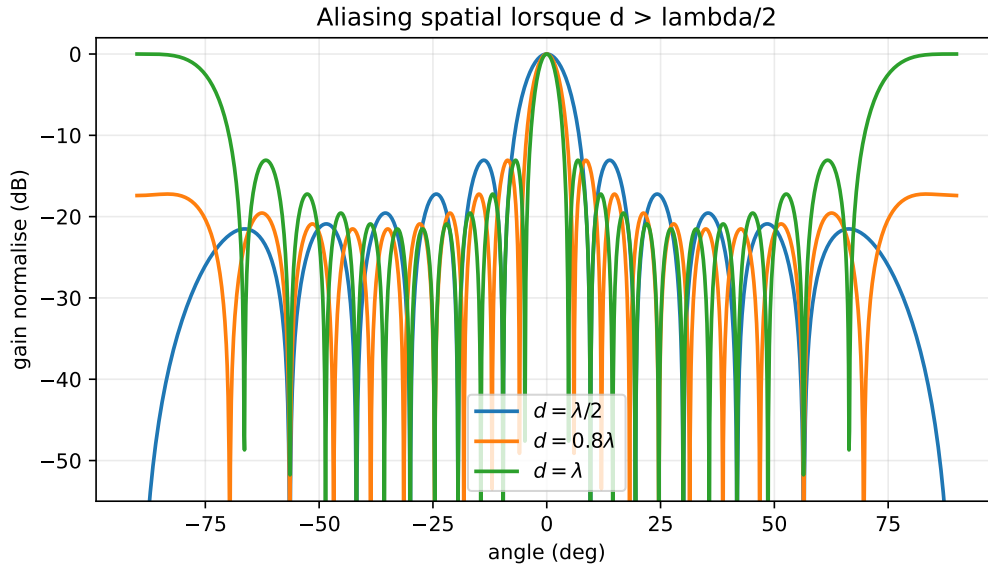


FIGURE 6.2 – Aliasing spatial (DAS broadside, $N = 12$, $d/\lambda \in \{0.5, 0.8, 1.0\}$) : quand d dépasse $\lambda/2$, des lobes de réseau apparaissent dans la zone visible et créent des directions ambiguës.

Une erreur d’une fraction de largeur de lobe principal coûte déjà plusieurs dB. C’est l’une des fragilités du Delay-and-Sum, et l’une des motivations historiques du *spatial tapering*.

6.6 Fenêtrage spatial : compromis lobe principal / lobes secondaires

La pondération uniforme produit un lobe principal étroit mais des lobes secondaires élevés. Pour réduire ces lobes, on applique aux poids un *fenêtrage spatial*, exactement comme on applique un fenêtrage (Hann, Hamming, Kaiser...) en analyse de Fourier temporelle.

Pour un beamformer pointé en θ_0 , les poids fenêtrés s’écrivent

$$\mathbf{w}_W = \mathbf{W} \mathbf{a}(\theta_0)/N, \quad \mathbf{W} = \text{diag}(w_0^W, w_1^W, \dots, w_{N-1}^W),$$

où les coefficients $w_m^W \in \mathbb{R}^+$ proviennent d’une fenêtre classique :

Fenêtre	1 ^{er} lobe secondaire	élargissement lobe principal	décroissance
Rectangulaire	−13.3 dB	$\times 1$	$1/\theta$
Hann	−31.5 dB	$\times 1.5$	$1/\theta^3$
Hamming	−42.7 dB	$\times 1.5$	$1/\theta$
Blackman	−58 dB	$\times 1.7$	$1/\theta^3$
Chebyshev	ajustable	ajustable	uniforme
Taylor	ajustable	ajustable	uniforme

Remarque 6.3 (Chebyshev et Taylor). Les fenêtres de Dolph–Chebyshev produisent des lobes secondaires strictement uniformes au niveau choisi (e.g. −30 dB, −40 dB), optimaux au sens du compromis lobe principal / niveau secondaire. Les fenêtres de Taylor en sont une variante régularisée. Toutes deux sont courantes en radar.

Conséquence sur le gain. Le fenêtrage améliore la sélectivité au prix de deux dégradations :

- le lobe principal s’élargit (résolution dégradée) ;

— le gain de réseau sur la cible diminue, car les antennes extrêmes contribuent moins.
La perte de gain par rapport au Delay-and-Sum uniforme se mesure par le *taper efficiency*

$$\eta_T = \frac{|\sum_m w_m^W|^2}{N \sum_m |w_m^W|^2} \in [0, 1].$$

Pour une fenêtre de Hamming, $\eta_T \simeq 0.73$ (perte de ~ 1.3 dB de gain de réseau) ; pour Blackman, $\eta_T \simeq 0.50$.

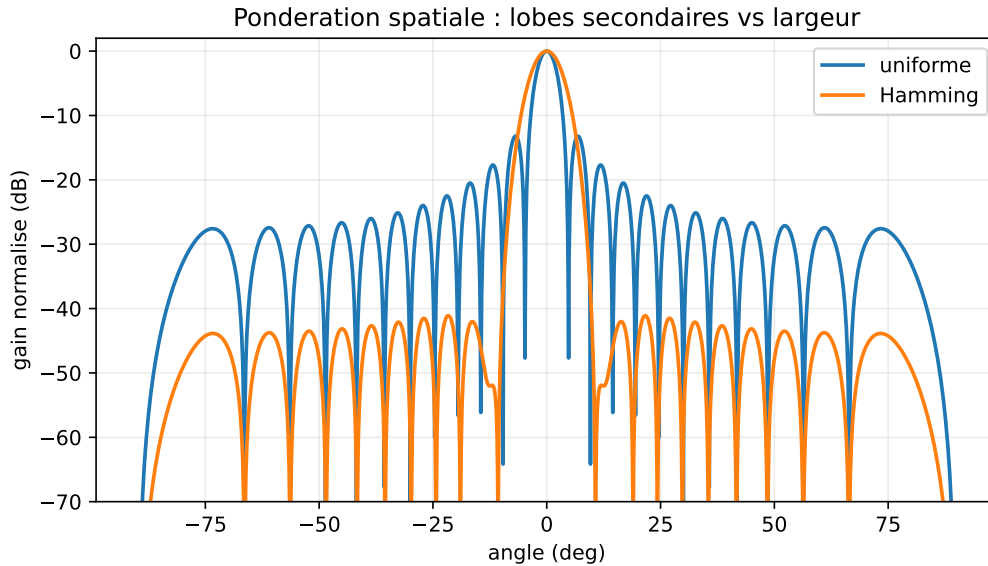


FIGURE 6.3 – Pondération uniforme et Hamming ($N = 24$, $d = \lambda/2$) : Hamming réduit nettement les lobes secondaires, au prix d'un lobe principal plus large et d'un gain de réseau légèrement plus faible.

6.7 Beam pattern en angle et en variable spatiale

On peut tracer le beam pattern en fonction de θ (échelle angulaire physique) ou en fonction de $u = \sin \theta$ (variable spatiale réduite). Cette dernière représentation a l'avantage de rendre le beam pattern indépendant de la direction de pointage : pour un ULA pointé en θ_0 , $B(u)$ est simplement translatée par $u_0 = \sin \theta_0$. Elle révèle aussi clairement l'aliasing spatial (repliements à ± 2 pour $d = \lambda/2$).

La représentation en θ est, en revanche, l'image *physique* de la scène. Elle révèle l'élargissement apparent du lobe principal près de l'endfire, où une variation Δu correspond à une variation $\Delta \theta = \Delta u / \cos \theta$ qui diverge.

6.8 Représentations cartésienne et polaire

La représentation cartésienne $\theta \mapsto B_{\text{dB}}(\theta)$ est préférée pour quantifier précisément niveaux de lobes secondaires et largeurs de lobe principal. La représentation polaire est plus parlante visuellement : elle dessine $|B(\theta)|^2$ dans le plan azimutal en coordonnées polaires, et donne une intuition spatiale directe du faisceau formé.

6.9 Synthèse de beam pattern : approche par optimisation

Plutôt que de partir d'une fenêtre standard, on peut formuler la conception comme un problème d'optimisation :

$$\min_w \max_{\theta \in \mathcal{S}} |B(\theta)|^2 \quad \text{sous} \quad B(\theta_0) = 1,$$

où $\mathcal{S}$ est l'ensemble des directions à atténuer. Sous contraintes linéaires de gain, le problème devient un programme convexe (en pratique, second-order cone program ou linear program) que les solveurs modernes traitent efficacement. Cette approche est à la base des beam patterns optimisés pour antennes radar et pour le *shaped beam* en télécoms.

6.10 Implémentation numérique

Listing 6.1 – Calcul et tracé d'un beam pattern

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def beam_pattern(w, N, d_over_lambda=0.5, n_theta=721):
5     theta = np.linspace(-np.pi/2, np.pi/2, n_theta)
6     m = np.arange(N)
7     # Matrice N x n_theta des steering vectors a(theta)
8     A = np.exp(-1j * 2*np.pi * d_over_lambda * np.sin(theta)[None, :] * m[:, None])
9     B = w.conj() @ A
10    B = np.abs(B)
11    B_dB = 20*np.log10(B / B.max() + 1e-12)
12    return theta, B_dB
13
14 # Exemple : DAS pointé à 0° + fenêtre Hamming
15 N = 16
16 theta0 = 0.0
17 m = np.arange(N)
18 a0 = np.exp(-1j * np.pi * np.sin(theta0) * m)
19 w_uni = a0 / N
20 w_ham = np.hamming(N) * a0
21 w_ham /= np.sum(np.hamming(N))
22
23 theta, B_uni = beam_pattern(w_uni, N)
24 _, B_ham = beam_pattern(w_ham, N)
25
26 plt.plot(np.rad2deg(theta), B_uni, label='Uniforme')
27 plt.plot(np.rad2deg(theta), B_ham, label='Hamming')
28 plt.xlabel(r'$\theta$ (deg)'); plt.ylabel('dB'); plt.legend()
29 plt.ylim(-60, 1); plt.grid(True)

```

Ce code servira de référence dans tout le livre. On l'enrichira au fil des chapitres pour intégrer les poids MVDR, LCMV, ou adaptatifs.

6.11 Synthèse

Le beam pattern est la signature spatiale d'un beamformer. Il se calcule comme une simple transformée de Fourier des poids, ce qui place toute la théorie du fenêtrage classique à sa disposition. Sa lecture donne en un coup d'œil :

— la *résolution* (largeur du lobe principal) ;

- la *sélectivité* (niveau des lobes secondaires) ;
- la *réjection* (profondeur des zéros) ;
- le *gain de réseau* (valeur sur la cible).

Le chapitre suivant étudie en détail le premier — et le plus simple — des beamformers : le Delay-and-Sum, dont le beam pattern uniforme sert de référence à tous les autres. C'est par lui qu'on commence historiquement le sujet, et c'est par lui qu'on continue à le commencer pédagogiquement : sa simplicité permet de fixer les idées de gain de réseau, de filtre adapté spatial, et de robustesse, qui resteront valables — avec des nuances — pour tous les beamformers plus sophistiqués.

Chapitre 7

Delay-and-Sum

Le Delay-and-Sum (DAS) est le beamformer le plus simple. Il consiste, comme son nom l'indique, à retarder les signaux des antennes pour les aligner sur une direction cible, puis à les sommer. En version bande étroite, ces « retards » deviennent de simples multiplications complexes. Le DAS sert de *référence* dans tout le livre : c'est le beamformer de plus faible complexité, le plus robuste numériquement, et celui qui maximise le SNR en bruit blanc. Tous les beamformers adaptatifs étudiés ensuite cherchent à faire mieux *en présence d'interférences* — jamais à faire mieux en bruit blanc pur.

Historiquement, le DAS est aussi le premier beamformer à avoir été exploité industriellement, dès la Seconde Guerre mondiale dans les radars à balayage électronique, puis dans les sonars passifs des années 1950. Sa simplicité le rendait compatible avec les moyens analogiques de l'époque : des lignes à retard de longueurs réglables ou des déphaseurs commandés en tension. Soixante-dix ans plus tard, le même principe est implanté dans les antennes 5G mmWave, mais à des échelles considérablement plus grandes et avec une commande numérique. La permanence du DAS dans toutes les générations de technologie illustre une vérité simple : *quand l'environnement est dominé par le bruit thermique, il n'y a rien de mieux à faire que sommer en phase.*

7.1 Construction

Pour aligner les contributions issues d'une direction cible θ_0 sur les N antennes, il faut compenser le déphasage $-mkd \sin \theta_0$ subi par chaque antenne. On choisit donc les poids

$$w_m = \frac{1}{N} e^{-jmkd \sin \theta_0},$$

soit

$$\mathbf{w}_{\text{DAS}}(\theta_0) = \frac{1}{N} \mathbf{a}(\theta_0).$$

Le facteur $1/N$ normalise la sortie : on aura un gain unitaire dans la direction cible.

7.1.1 Sortie du beamformer

Pour une onde plane d'amplitude $s(t)$ arrivant à $\theta = \theta_0$:

$$\mathbf{x}(t) = \mathbf{a}(\theta_0) s(t) + \mathbf{n}(t),$$

la sortie vaut

$$y(t) = \mathbf{w}_{\text{DAS}}^H \mathbf{x}(t) = \frac{1}{N} \mathbf{a}^H(\theta_0) \mathbf{a}(\theta_0) s(t) + \mathbf{w}_{\text{DAS}}^H \mathbf{n}(t).$$

Comme $\mathbf{a}^H(\theta_0) \mathbf{a}(\theta_0) = N$, le terme signal vaut exactement $s(t)$: le gain sur la cible est unitaire.

7.1.2 Composante par composante

Si l'on développe en sommes :

$$y(t) = \frac{1}{N} \sum_{m=0}^{N-1} e^{jmkd \sin \theta_0} x_m(t).$$

Chaque antenne reçoit $x_m(t) = e^{-jmkd \sin \theta_0} s(t) + n_m(t)$, donc l'expression ci-dessus :

- multiplie chaque $x_m(t)$ par un coefficient de norme 1 qui *compense* exactement le déphasage géométrique attendu de la cible ;
- somme les N contributions devenues en phase ;
- normalise par N .

C'est précisément l'analogie spatiale du « filtre adapté ».

7.2 Beam pattern

Le beam pattern du DAS pointé en θ_0 est

$$B(\theta) = \mathbf{w}^H \mathbf{a}(\theta) = \frac{1}{N} \mathbf{a}^H(\theta_0) \mathbf{a}(\theta) = \frac{1}{N} e^{-j(N-1)\beta/2} \frac{\sin(N\beta/2)}{\sin(\beta/2)},$$

avec $\beta = kd(\sin \theta - \sin \theta_0)$. Le module est le noyau de Dirichlet déjà rencontré au chapitre 3. Ses caractéristiques principales :

- gain unitaire en $\theta = \theta_0$;
- premier zéro à $\sin \theta - \sin \theta_0 = \pm \lambda/(Nd)$;
- premier lobe secondaire à ~ -13.3 dB ;
- décroissance asymptotique en $1/\beta$.

7.3 Gain de réseau

7.3.1 Sortie en bruit blanc

Si le bruit additif est spatialement blanc de variance σ^2 , le bruit de sortie a pour variance

$$\sigma_y^2 = \mathbb{E}[|\mathbf{w}^H \mathbf{n}|^2] = \sigma^2 \|\mathbf{w}\|^2 = \sigma^2 \cdot \frac{N}{N^2} = \frac{\sigma^2}{N}.$$

7.3.2 Gain SNR

Pour le DAS, le signal de sortie a pour variance $|s|^2$ et le bruit a pour variance σ^2/N . Le rapport

$$\text{SNR}_{\text{out}} = \frac{|s|^2}{\sigma^2/N} = N \cdot \frac{|s|^2}{\sigma^2} = N \cdot \text{SNR}_{\text{in}}.$$

On gagne donc un facteur N , soit $10 \log_{10} N$ en dB. C'est le *gain de réseau* (*array gain*) du beamformer Delay-and-Sum : 8 antennes donnent +9 dB, 16 antennes donnent +12 dB, 32 antennes +15 dB.

Théorème 7.1 (Optimalité en bruit blanc). *En présence d'un bruit gaussien complexe circulaire spatialement blanc (de covariance $\sigma^2 \mathbf{I}_N$) et d'une seule source à θ_0 , le beamformer $\mathbf{w}_{\text{DAS}} = \mathbf{a}(\theta_0)/N$ maximise le SNR de sortie parmi tous les vecteurs de poids de norme normalisée par $\mathbf{w}^H \mathbf{a}(\theta_0) = 1$.*

Démonstration. Maximiser le SNR à contrainte de gain unitaire revient à

$$\min_{\mathbf{w}} \mathbf{w}^H \mathbf{R}_n \mathbf{w} \quad \text{sous} \quad \mathbf{w}^H \mathbf{a}(\theta_0) = 1.$$

Pour $\mathbf{R}_n = \sigma^2 \mathbf{I}_N$, le lagrangien donne $\mathbf{w}^* = \mathbf{I}^{-1} \mathbf{a}(\theta_0) / (\mathbf{a}^H \mathbf{I}^{-1} \mathbf{a}) = \mathbf{a}(\theta_0) / N$. $\square$

C'est précisément la borne que le DAS atteint, et c'est précisément celle que MVDR généralisera lorsque $\mathbf{R}_n$ cesse d'être proportionnelle à l'identité.

7.4 Pourquoi le DAS n'est pas adaptatif

Le DAS est entièrement *déterministe* : ses poids dépendent uniquement de θ_0 et de la géométrie. Aucune statistique des données n'est utilisée. Cette propriété a deux conséquences opposées.

Avantage : robustesse. Le DAS n'a pas besoin de covariance estimée. Il n'introduit aucun biais lié à un nombre fini de snapshots, aucun risque de *signal cancellation* (annulation involontaire du signal utile), aucune sensibilité à un mauvais conditionnement de matrice.

Inconvénient : aveugle aux interférences. À titre de borne simple, si une interférence puissante arrive à θ_i par un lobe secondaire à -13.3 dB, elle apparaît en sortie atténuée de seulement 13 dB. Un brouilleur 30 dB au-dessus du signal utile reste donc 17 dB au-dessus en sortie : le DAS ne peut pas le rejeter. C'est précisément ce que MVDR ou LCMV résoudront en plaçant un *null* adaptatif sur θ_i .

Exemple 7.2 (Interférence dans un lobe secondaire). ULA $N = 16$, $d = \lambda/2$, cible à 0° , brouilleur à 30° . Le DAS donne en 30° un gain de ~ -15 dB (lobe secondaire profond, pour ce cas précis). Un brouilleur à $+40$ dB au-dessus du signal donne en sortie $+25$ dB au-dessus du signal utile : la sortie est dominée par le brouilleur.

Cette défaillance n'est pas marginale : c'est l'une des deux motivations historiques majeures pour développer du beamforming adaptatif (l'autre étant l'estimation de DOA). À chaque fois qu'un système doit fonctionner dans un environnement contenant des sources RF puissantes non coopératives (radars militaires en zone congestionnée, récepteurs GPS sous brouillage, communications urbaines saturées), le DAS seul ne suffit pas, et il faut compléter par MVDR, LCMV, ou des techniques de Null Steering. La hiérarchie de la Partie III prend toute sa cohérence à partir de ce constat.

7.5 Erreurs et fragilités

7.5.1 Erreur de pointage

Une erreur $\delta\theta_0$ sur la direction supposée de la cible décale le lobe principal. Le gain effectif sur la cible chute selon le noyau de Dirichlet :

$$G_{\text{eff}} = \frac{1}{N^2} \frac{\sin^2(N\beta/2)}{\sin^2(\beta/2)} \quad \text{avec} \quad \beta = kd(\sin \theta_{\text{cible}} - \sin \theta_0).$$

Une erreur d'une fraction significative de la largeur de lobe principal suffit à perdre plusieurs dB. Pour un ULA de 16 antennes à $\lambda/2$, une simulation broadside donne environ 3 dB de perte pour quelques degrés d'erreur et une perte beaucoup plus forte lorsque l'erreur approche la largeur du lobe principal.

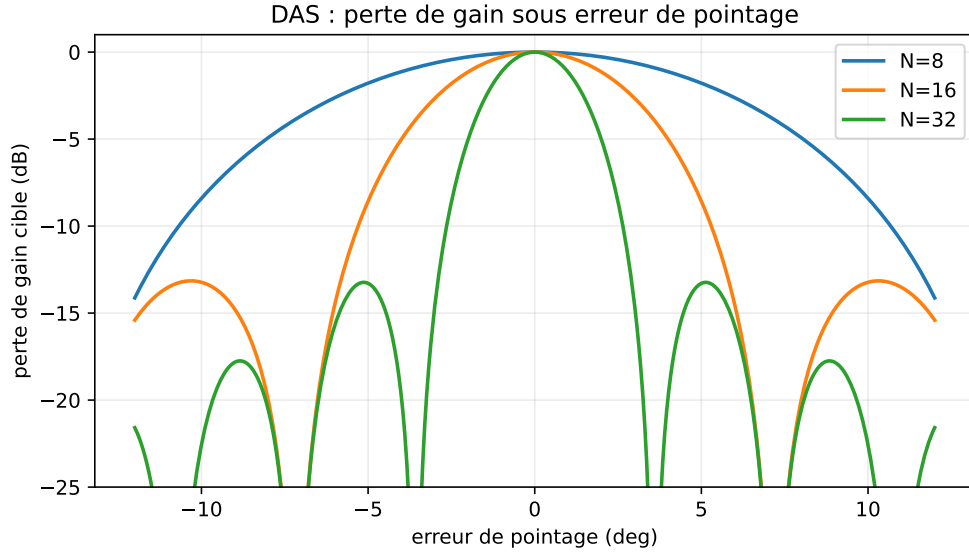


FIGURE 7.1 – Simulation DAS broadside ($N = 8, 16, 32$, $d = \lambda/2$) : la même erreur angulaire coûte plus cher quand N augmente, car le lobe principal devient plus étroit.

7.5.2 Erreurs de phase et de gain

Si chaque voie subit une phase aléatoire $\delta\phi_m \sim \mathcal{N}(0, \sigma_\phi^2)$, le gain effectif moyen sur la cible vaut

$$\mathbb{E}[G_{\text{eff}}] \approx N e^{-\sigma_\phi^2}.$$

Dans ce modèle indépendant par voie, une dispersion $\sigma_\phi = 30^\circ$ coûte ~ 1.2 dB et $\sigma_\phi = 60^\circ$ coûte environ 5 dB ; au-delà, le beamforming perd rapidement l'essentiel de son intérêt. La calibration (chapitre 22) vise donc à ramener σ_ϕ à quelques degrés lorsque le matériel le permet.

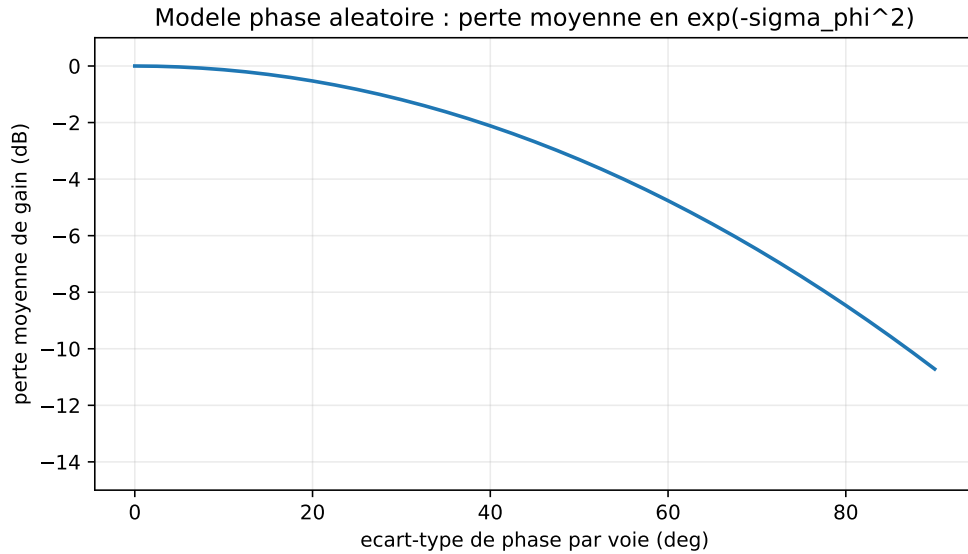


FIGURE 7.2 – Modèle indicatif d'erreurs de phase indépendantes par voie (erreurs gaussiennes i.i.d., gain normalisé) : la perte moyenne suit $e^{-\sigma_\phi^2}$ lorsque σ_ϕ est exprimé en radians.

7.5.3 Validité bande étroite

Le DAS « pure phase » suppose l'approximation bande étroite. Pour un signal large bande, le retard ne se réduit plus à une phase constante et le gain dépend de la fréquence. La parade est le *true time delay* ou le beamforming par sous-bandes (chapitre 19).

7.6 Implémentation numérique

Listing 7.1 – Beamformer Delay-and-Sum

```

1 import numpy as np
2
3 def das_weights(theta0, N, d_over_lambda=0.5):
4     """Poids Delay-and-Sum pointés en theta0 (rad)."""
5     m = np.arange(N)
6     a0 = np.exp(-1j * 2*np.pi * d_over_lambda * np.sin(theta0) * m)
7     return a0 / N
8
9 def das_output(X, theta0, d_over_lambda=0.5):
10    """X : matrice (N, K) de snapshots. Renvoie y de longueur K."""
11    N = X.shape[0]
12    w = das_weights(theta0, N, d_over_lambda)
13    return w.conj() @ X

```

7.7 Le DAS comme référence universelle

Le Delay-and-Sum joue dans la suite trois rôles indispensables.

1. **Borne supérieure de robustesse.** Aucun beamformer ne sera plus robuste numériquement que le DAS. C'est ce qu'on retrouvera quand on introduira la régularisation par *diagonal loading* : plus on charge la covariance, plus le MVDR « se rapproche » du DAS.
2. **Limite inférieure de performance avec interférences.** Toute méthode adaptative doit, sur un environnement contenant interférences, faire *mieux* que le DAS. Si ce n'est pas le cas, elle introduit plus de biais qu'elle ne corrige.
3. **Beam pattern uniforme.** La fonction de Dirichlet $|\sin(N\beta/2)/\sin(\beta/2)|/N$ est la référence de tous les fenêtrages spatiaux. Les autres pondérations s'y comparent en *taper efficiency* et en *niveau de lobes secondaires*.

Le chapitre suivant introduit la première méthode déterministe qui améliore le DAS : le *Null Steering*, qui impose explicitement des zéros spatiaux dans les directions d'interférences connues. C'est aussi l'occasion d'introduire le formalisme des contraintes linéaires, qui sera ensuite généralisé en MVDR et LCMV. Le Null Steering est encore purement déterministe (il ne regarde pas les données reçues, seulement les angles supposés connus), mais il franchit un premier seuil qualitatif : on passe d'un beamformer « aveugle » à un beamformer *informé*, ce qui ouvre la voie à toutes les approches plus sophistiquées des parties suivantes.

Chapitre 8

Null Steering : imposer des zéros spatiaux

Le Delay-and-Sum traite l'environnement comme un bruit isotrope : il maximise le gain dans la direction visée sans tenir compte des interférences directionnelles. C'est précisément ce qui le rend inopérant face à un brouilleur puissant entrant par un lobe secondaire. Le *Null Steering* introduit la première forme de contrôle spatial explicite : on impose, par construction, des *zéros* dans les directions d'interférences. C'est aussi l'occasion de poser le formalisme des *contraintes linéaires*, qui sera généralisé en MVDR (chapitre 10) puis en LCMV (chapitre 11).

L'intuition est presque triviale : si l'on sait précisément d'où vient un brouilleur, il suffit de demander au beamformer de ne rien recevoir de cette direction. Toute la subtilité tient à formuler cette demande comme un problème mathématiquement bien posé — ce qui est l'objet de la première partie du chapitre — puis à comprendre ce qu'on *perd* en imposant cette contrainte : le coût en degrés de liberté, l'amplification du bruit, la sensibilité aux erreurs d'angle. C'est la deuxième partie du chapitre. Le Null Steering est en cela un bon laboratoire pour comprendre les compromis qui réapparaîtront dans MVDR et LCMV.

8.1 Formulation

8.1.1 Contraintes de gain

Soit θ_s la direction de la source utile et $\theta_{i,1}, \dots, \theta_{i,L}$ celles de L interférences supposées connues. On souhaite que le beamformer satisfasse simultanément

$$\mathbf{w}^H \mathbf{a}(\theta_s) = 1, \quad \mathbf{w}^H \mathbf{a}(\theta_{i,\ell}) = 0, \quad \ell = 1, \dots, L.$$

La première équation garantit un gain unitaire sur la cible (signal préservé sans distorsion), les L autres imposent des nulls sur les brouilleurs.

8.1.2 Forme matricielle

On rassemble les contraintes en une matrice $\mathbf{C} = [\mathbf{a}(\theta_s), \mathbf{a}(\theta_{i,1}), \dots, \mathbf{a}(\theta_{i,L})] \in \mathbb{C}^{N \times (L+1)}$ et un vecteur de réponse $\mathbf{f} = [1, 0, \dots, 0]^T \in \mathbb{C}^{L+1}$. Le problème s'écrit alors

$$\boxed{\mathbf{C}^H \mathbf{w} = \mathbf{f}.}$$

C'est un système linéaire en $\mathbf{w}$.

8.2 Solution de norme minimale

Si $L + 1 < N$, le système est sous-déterminé : il existe en général une infinité de $\mathbf{w}$ satisfaisant les contraintes. Le choix canonique est celui de *norme minimale*, qui maximise la robustesse au bruit (un grand $\|\mathbf{w}\|$ amplifie le bruit en sortie).

Théorème 8.1 (Solution Null Steering de norme minimale). *La solution unique du problème*

$$\min_{\mathbf{w}} \|\mathbf{w}\|^2 \quad \text{sous} \quad \mathbf{C}^H \mathbf{w} = \mathbf{f},$$

lorsque $\mathbf{C}$ est de rang plein colonne ($L + 1 \leq N$), est

$$\mathbf{w}_{NS} = \mathbf{C} (\mathbf{C}^H \mathbf{C})^{-1} \mathbf{f}.$$

Démonstration. Méthode des multiplicateurs de Lagrange. On forme

$$\mathcal{L}(\mathbf{w}, \boldsymbol{\mu}) = \mathbf{w}^H \mathbf{w} + \boldsymbol{\mu}^H (\mathbf{C}^H \mathbf{w} - \mathbf{f}) + (\mathbf{C}^H \mathbf{w} - \mathbf{f})^H \boldsymbol{\mu}.$$

La condition de stationnarité par rapport à $\mathbf{w}$ donne $\mathbf{w} = -\mathbf{C} \boldsymbol{\mu}$ (à un signe près qui sera absorbé). En injectant dans la contrainte $\mathbf{C}^H \mathbf{w} = \mathbf{f}$: $-\mathbf{C}^H \mathbf{C} \boldsymbol{\mu} = \mathbf{f}$, soit $\boldsymbol{\mu} = -(\mathbf{C}^H \mathbf{C})^{-1} \mathbf{f}$, et $\mathbf{w} = \mathbf{C} (\mathbf{C}^H \mathbf{C})^{-1} \mathbf{f}$. L'unicité provient de la stricte convexité de $\|\mathbf{w}\|^2$ et de la régularité de $\mathbf{C}^H \mathbf{C}$. $\square$

La matrice $\mathbf{C}^{+\dagger} = \mathbf{C} (\mathbf{C}^H \mathbf{C})^{-1}$ est appelée *pseudo-inverse de Moore–Penrose* de $\mathbf{C}^H$.

8.3 Lecture géométrique : projection orthogonale

Considérons la décomposition $\mathbb{C}^N = \text{Im}(\mathbf{C}) \oplus \text{Ker}(\mathbf{C}^H)$. La solution de norme minimale est entièrement dans $\text{Im}(\mathbf{C})$: elle n'a pas de composante « inutile » dans le complément orthogonal. Elle est donc la projection orthogonale (au sens hermitien) du vecteur idéal $\mathbf{a}(\theta_s)/N$ sur le sous-espace des contraintes.

Plus intuitivement : pour imposer un null en θ_i , il faut prendre une composante de $\mathbf{w}$ orthogonale à $\mathbf{a}(\theta_i)$. À chaque contrainte ajoutée, le beamformer « consomme » un degré de liberté dans $\mathbb{C}^N$.

8.4 Degrés de liberté

Proposition 8.2. *Un ULA à N antennes possède N degrés de liberté complexes. Il peut imposer simultanément :*

- *un gain unitaire dans la direction cible (1 contrainte) ;*
- *L nulls dans des directions distinctes (L contraintes).*

La condition est $1 + L \leq N$, soit $L \leq N - 1$.

Pratiquement, on évite de saturer les degrés de liberté ($L \ll N$ est plus sûr) car :

- le conditionnement de $\mathbf{C}^H \mathbf{C}$ se dégrade lorsque les contraintes deviennent presque colinéaires ;
- la norme $\|\mathbf{w}\|^2$ explose, amplifiant le bruit ;
- la robustesse aux erreurs angulaires diminue.

8.5 Effet sur le beam pattern

L'application des poids $\mathbf{w}_{\text{NS}}$ produit un beam pattern qui :

- conserve un lobe principal proche du Delay-and-Sum centré sur θ_s ;
- présente des *zéros théoriques* aux $\theta_{i,\ell}$ (à précision numérique près) ;
- modifie la forme des lobes secondaires par rapport au DAS, dans des directions souvent imprévisibles.

Exemple 8.3 (ULA 8 antennes, 2 brouilleurs). $N = 8$, $\theta_s = 0^\circ$, brouilleurs à $\pm 30^\circ$. Le DAS donne en 30° un gain de -13.5 dB ; le Null Steering y impose $-\infty$ (limité par la précision numérique à ~ -300 dB dans un calcul double précision idéal). Le lobe principal s'élargit légèrement ; dans cette simulation, la perte de gain de réseau reste de l'ordre du demi-dB.

8.6 Profondeur effective d'un null

En arithmétique infinie le null est rigoureusement nul. En pratique :

- le steering vector utilisé pour calculer $\mathbf{w}$ est $\mathbf{a}(\theta_{i,\ell})$ théorique, qui diffère du steering vector *réel* de l'antenne $\mathbf{a}_{\text{réel}}(\theta_{i,\ell})$;
- le brouilleur n'arrive pas exactement à $\theta_{i,\ell}$ mais à $\theta_{i,\ell} + \delta\theta$;
- les erreurs de calibration produisent un décalage résiduel.

La conséquence est qu'un null théorique de $-\infty$ dB se mesure souvent, selon la calibration et la stabilité angulaire, dans une plage beaucoup moins spectaculaire (par exemple -30 à -50 dB sur un banc bien calibré). Cette dégradation est étudiée en détail dans le chapitre 23 (contraintes dérivatives, nulls élargis).

8.7 Effet sur le bruit : amplification

Le bruit en sortie a pour variance $\sigma_y^2 = \sigma^2 \|\mathbf{w}_{\text{NS}}\|^2$. Le Null Steering n'optimise pas $\|\mathbf{w}\|^2$: il le minimise sous contrainte. Mais lorsque le brouilleur s'approche de la cible (i.e. $\theta_i \rightarrow \theta_s$), $\mathbf{a}(\theta_i)$ devient presque colinéaire à $\mathbf{a}(\theta_s)$, $\mathbf{C}^H \mathbf{C}$ devient mal conditionnée, et $\|\mathbf{w}\|^2$ explose.

White Noise Gain (WNG). On définit le gain en bruit blanc

$$\text{WNG} = \frac{|\mathbf{w}^H \mathbf{a}(\theta_s)|^2}{\|\mathbf{w}\|^2} = \frac{1}{\|\mathbf{w}\|^2} \quad (\text{avec contrainte } \mathbf{w}^H \mathbf{a}(\theta_s) = 1).$$

Pour le DAS, $\text{WNG} = N$. Pour le Null Steering, $\text{WNG} < N$, et la perte mesure le coût en SNR des contraintes imposées.

8.8 Régularisation simple

Lorsque les contraintes sont mal conditionnées, on régularise par

$$\mathbf{w}_{\text{NS}}^{\text{reg}} = \mathbf{C} (\mathbf{C}^H \mathbf{C} + \alpha \mathbf{I})^{-1} \mathbf{f},$$

avec $\alpha > 0$ petit. Le coût est une légère dégradation de la profondeur des nulls, mais on retrouve une norme $\|\mathbf{w}\|$ contrôlée. Cette régularisation préfigure exactement le *diagonal loading* qui sera systématisé en MVDR.

8.9 Différences avec MVDR

Trois différences fondamentales séparent le Null Steering de MVDR.

1. **Information utilisée.** Null Steering utilise les *positions angulaires connues* des interférences. MVDR utilise une *covariance estimée* sur les données reçues.
2. **Profondeur du null.** Null Steering produit des nulls à $-\infty$ dB en théorie. MVDR produit des nulls profonds mais dont la profondeur dépend du rapport interférence/bruit (un brouilleur très puissant produit un null très profond ; un brouilleur faible produit un null peu profond).
3. **Adaptabilité.** Null Steering exige de connaître les $\theta_{i,\ell}$ *a priori*. MVDR les apprend des données.

En pratique on combine souvent les deux : Null Steering pour les directions *connues* (ami, satellite, balise), MVDR ou LCMV pour les directions *inconnues* (brouilleurs adversaires).

8.10 Implémentation numérique

Listing 8.1 – Null Steering générique

```

1 import numpy as np
2
3 def null_steering(theta_s, theta_i, N, d_over_lambda=0.5, alpha=0.0):
4     """
5     theta_s : direction utile (rad)
6     theta_i : liste/array des directions d'interférences (rad)
7     alpha   : régularisation (>=0)
8     """
9     m = np.arange(N)
10    def a(theta):
11        return np.exp(-1j * 2*np.pi * d_over_lambda * np.sin(theta) * m)
12
13    angles = [theta_s] + list(theta_i)
14    C = np.column_stack([a(th) for th in angles])          # N x (L+1)
15    f = np.zeros(len(angles), dtype=complex)
16    f[0] = 1.0
17    G = C.conj().T @ C + alpha * np.eye(len(angles))
18    return C @ np.linalg.solve(G, f)

```

8.11 Quand utiliser le Null Steering ?

Adapté.

- Brouilleurs aux directions connues précisément (balise GPS adverse repérée, satellite voisin, source RF stationnaire).
- Études analytiques où l'on souhaite imposer un beam pattern théorique précis.
- Initialisation d'un beamformer adaptatif.

Inadapté.

- Directions d'interférences inconnues ou variables.
- Environnements à trajets multiples (les directions effectives ne sont pas celles des sources physiques).
- Systèmes sans calibration soignée : un null sur un faux angle ne sert à rien.

8.12 Conclusion

Le Null Steering franchit un premier pas vers l'adaptation : il remplace le DAS aveugle aux interférences par un beamformer *informé* de leurs directions. Mais cette information doit être fournie de l'extérieur, et le coût en degrés de liberté est explicite.

La prochaine étape consiste à *extraire* cette information des données reçues elles-mêmes. La grandeur statistique qui le permet est la *covariance spatiale*, objet central de la Partie III. Le chapitre 9 introduit cette matrice, étudie ses propriétés, sa structure et son estimation. C'est elle qui ouvrira la voie à MVDR puis à LCMV.

La transition Partie II $\rightarrow$ Partie III correspond exactement à un changement de paradigme : on quitte le monde des beamformers *data-independent* pour celui des beamformers *data-dependent*. Les premiers sont fragiles aux interférences mais robustes au calcul ; les seconds optimisent leur réponse à l'environnement observé mais paient cette optimisation par une sensibilité accrue aux artefacts d'estimation et aux erreurs de modèle. La Partie VII reviendra sur ces fragilités et fournira les outils (diagonal loading, contraintes dérivatives) qui permettent de les contenir.

Troisième partie

Beamforming statistique

Chapitre 9

Covariance spatiale

La Partie II a construit des beamformers dont les poids ne dépendent que de la géométrie et des angles supposés connus. La Partie III inverse la perspective : on suppose les angles *inconnus* (ou seulement la cible connue) et on apprend la structure de l'environnement à partir des données. L'outil statistique central de ce changement de paradigme est la *covariance spatiale*. Elle contient toute l'information de second ordre sur les corrélations inter-antennes ; elle code implicitement les directions, les puissances et les corrélations des sources présentes. Le présent chapitre l'introduit, étudie sa structure et son estimation.

Il est utile, avant de plonger dans la définition formelle, de saisir pourquoi la covariance est l'objet pertinent — et pas, par exemple, la moyenne ou la corrélation simple. Une source RF arrive sur les antennes avec une amplitude et une phase qui dépendent du temps de manière essentiellement aléatoire (modulation, fluctuations de puissance, etc.). Si l'on calcule simplement $\mathbb{E}[\mathbf{x}(n)]$, on obtient zéro (en hypothèse de moyenne nulle), ce qui ne dit rien sur la structure spatiale. En revanche, $\mathbb{E}[\mathbf{x}(n)\mathbf{x}^H(n)]$ capture les *relations de phase moyennes* entre antennes, indépendamment des fluctuations d'amplitude. C'est cette stabilité statistique de second ordre qui rend la covariance exploitable, et qui en fait le pivot de tous les algorithmes adaptatifs et de sous-espaces qui suivent.

9.1 Définition

Définition 9.1 (Covariance spatiale). Pour un processus snapshot $\mathbf{x}(n) \in \mathbb{C}^N$ stationnaire au sens large et centré, la *covariance spatiale* est la matrice

$$\mathbf{R} = \mathbb{E}[\mathbf{x}(n)\mathbf{x}^H(n)] \in \mathbb{C}^{N \times N}.$$

Le coefficient $R_{ij} = \mathbb{E}[x_i(n)x_j^*(n)]$ mesure la corrélation complexe entre les signaux des antennes i et j . La diagonale $R_{mm} = \mathbb{E}[|x_m(n)|^2]$ est la *puissance moyenne* sur l'antenne m . Les éléments hors diagonale capturent les déphasages moyens d'une voie à l'autre.

Pourquoi le conjugué ? Sans conjugué, on aurait $\mathbb{E}[x_i x_j]$. Pour un signal complexe à phase fluctuante, x_j peut être noyé dans une exponentielle complexe rapide qui ne porte pas d'information utile : $\mathbb{E}[x_i x_j]$ serait nul même pour deux voies parfaitement corrélées. Le conjugué « rebobine » la phase et fait apparaître la corrélation de structure. C'est pourquoi $\mathbb{E}[x_i x_j^*]$, et plus généralement la covariance *hermitienne*, est la bonne quantité pour des signaux complexes circulaires.

9.2 Source unique et modèle de rang 1

Pour une source à θ_0 avec enveloppe $s(n)$ et sans bruit : $\mathbf{x}(n) = \mathbf{a}(\theta_0)s(n)$. Alors

$$\mathbf{R} = \mathbb{E}[s(n)s^*(n)] \mathbf{a}(\theta_0) \mathbf{a}^H(\theta_0) = p_0 \mathbf{a}(\theta_0) \mathbf{a}^H(\theta_0),$$

avec $p_0 = \mathbb{E}[|s(n)|^2]$ la puissance de la source. La matrice $\mathbf{R}$ est ici de *rang 1*, et son unique vecteur propre non nul est proportionnel à $\mathbf{a}(\theta_0)$ avec valeur propre Np_0 .

Proposition 9.2 (Source unique sans bruit). *Pour une seule source en θ_0 , $\mathbf{R}$ a une unique valeur propre non nulle $\lambda_1 = Np_0$ associée au vecteur propre $\mathbf{a}(\theta_0)/\sqrt{N}$. Les $N - 1$ autres valeurs propres sont nulles.*

Cette propriété est la racine algébrique des méthodes de sous-espaces (Partie V). On peut la formuler autrement : une source unique « remplit » une seule direction de $\mathbb{C}^N$, celle du steering vector $\mathbf{a}(\theta_0)$, et le reste de l'espace est strictement inutilisé. C'est ce « reste » (le complément orthogonal) que MUSIC exploitera comme *sous-espace bruit* pour localiser exactement θ_0 . La même logique vaudra pour L sources : le sous-espace signal est de dimension L , et son complément orthogonal — de dimension $N - L$ — contient toute l'information sur l'absence de sources dans les directions « non occupées ».

9.3 Avec bruit blanc

Ajoutons un bruit blanc spatialement de variance σ^2 : $\mathbf{x}(n) = \mathbf{a}(\theta_0)s(n) + \mathbf{n}(n)$, avec $\mathbb{E}[\mathbf{n}\mathbf{n}^H] = \sigma^2 \mathbf{I}_N$ et $\mathbf{n}$ décorrélié du signal. Par bilinéarité,

$$\mathbf{R} = p_0 \mathbf{a}(\theta_0) \mathbf{a}^H(\theta_0) + \sigma^2 \mathbf{I}_N.$$

Les valeurs propres sont :

- $\lambda_1 = Np_0 + \sigma^2$ avec vecteur propre $\mathbf{a}(\theta_0)/\sqrt{N}$ (sous-espace signal) ;
- $\lambda_2 = \dots = \lambda_N = \sigma^2$, avec un sous-espace propre orthogonal de dimension $N - 1$ (sous-espace bruit).

Cette structure — une valeur propre forte, $N - 1$ valeurs propres au plancher de bruit — est l'image-type d'une source unique en bruit. Elle reste lisible jusqu'à des SNR modérés.

9.4 Plusieurs sources

Pour L sources d'angles distincts et de puissances p_ℓ :

$$\mathbf{x}(n) = \mathbf{A} \mathbf{s}(n) + \mathbf{n}(n), \quad \mathbf{A} = [\mathbf{a}(\theta_1), \dots, \mathbf{a}(\theta_L)].$$

En supposant les sources *décorrélées* entre elles ($\mathbb{E}[\mathbf{s}\mathbf{s}^H] = \text{diag}(p_1, \dots, p_L) = \mathbf{P}_s$) et décorréliées du bruit, on obtient

$$\mathbf{R} = \mathbf{A} \mathbf{P}_s \mathbf{A}^H + \sigma^2 \mathbf{I}_N.$$

Structure des valeurs propres. Si $\mathbf{A}$ est de rang plein colonne $L < N$, alors $\mathbf{A} \mathbf{P}_s \mathbf{A}^H$ est de rang L . Les valeurs propres de $\mathbf{R}$ se séparent en :

- L valeurs propres *signal* : $\lambda_1, \dots, \lambda_L$, toutes supérieures à σ^2 ;
- $N - L$ valeurs propres *bruit* : toutes égales à σ^2 .

Cette séparation en deux groupes est la base de la décomposition signal/bruit utilisée par MUSIC et ESPRIT.

Sources cohérentes. Si deux sources sont parfaitement corrélées (par exemple un trajet direct et un écho), $\mathbf{P}_s$ devient elle-même de rang inférieur à L , et $\mathbf{A}\mathbf{P}_s\mathbf{A}^H$ peut être de rang $L' < L$. Le nombre de *valeurs propres signal* est alors L' , et MUSIC sous-estime le nombre de sources. La parade classique est le *spatial smoothing* (chapitre 15).

9.5 Propriétés mathématiques

Théorème 9.3 (Propriétés de la covariance). $\mathbf{R}$ est :

- Hermitienne : $\mathbf{R}^H = \mathbf{R}$. En effet, $(\mathbb{E}[\mathbf{x}\mathbf{x}^H])^H = \mathbb{E}[\mathbf{x}\mathbf{x}^H]$.
- Semi-définie positive : $\mathbf{w}^H \mathbf{R} \mathbf{w} = \mathbb{E}[|\mathbf{w}^H \mathbf{x}|^2] \geq 0$ pour tout $\mathbf{w} \in \mathbb{C}^N$.
- Diagonalisable en base orthonormée, à valeurs propres réelles positives (ou nulles).

Conséquence pratique. Tous les algorithmes manipulant $\mathbf{R}$ exploitent ces propriétés : décomposition propre en base orthonormée, factorisations de Cholesky, conditionnement du problème d'inversion. Une matrice estimée numériquement n'est jamais exactement hermitienne (erreurs d'arrondi) : on la symétrise en pratique via $\hat{\mathbf{R}} \leftarrow (\hat{\mathbf{R}} + \hat{\mathbf{R}}^H)/2$.

9.6 Estimation à partir des données

9.6.1 Sample Covariance Matrix

La covariance théorique $\mathbf{R}$ est inconnue : on l'estime à partir de K snapshots,

$$\hat{\mathbf{R}} = \frac{1}{K} \sum_{n=1}^K \mathbf{x}(n)\mathbf{x}^H(n) = \frac{1}{K} \mathbf{X}\mathbf{X}^H,$$

où $\mathbf{X} = [\mathbf{x}(1), \dots, \mathbf{x}(K)] \in \mathbb{C}^{N \times K}$. $\hat{\mathbf{R}}$ est appelée *Sample Covariance Matrix* (SCM). Elle est sans biais : $\mathbb{E}[\hat{\mathbf{R}}] = \mathbf{R}$.

9.6.2 Nombre de snapshots et qualité d'estimation

La qualité de l'estimée dépend du rapport K/N . On retient :

- $K < N$: $\hat{\mathbf{R}}$ est singulière (rang $\leq K < N$), non exploitable telle quelle pour MVDR sans régularisation. Cas typique des systèmes « snapshot-limited » : radar à courte durée d'illumination, signaux non stationnaires.
- $K \approx N$: $\hat{\mathbf{R}}$ est mal conditionnée, MVDR sous-performe.
- $K \geq 2N$: repère empirique *Reed–Mallett–Brennan* (dans le modèle SMI gaussien idéal, $K \simeq 2N$ correspond à une perte moyenne d'environ 3 dB par rapport à la covariance théorique).
- $K \gg N$: $\hat{\mathbf{R}}$ converge vers $\mathbf{R}$ avec erreur $\mathcal{O}(N/K)$ dans une norme matricielle appropriée.

Théorème 9.4 (Reed–Mallett–Brennan [6]). Pour une covariance gaussienne complexe et un beamformer Sample Matrix Inversion (SMI), la perte moyenne de SINR par rapport à $\mathbf{R}$ vaut

$$\mathbb{E} \left[\frac{\text{SINR}_{\text{SMI}}}{\text{SINR}_{\text{opt}}} \right] = \frac{K - N + 2}{K + 1}.$$

Pour $K = 2N$, le rapport vaut ~ 0.5 , soit une perte de 3 dB.

Ce résultat, connu sous le nom de *règle RMB*, gouverne tout le dimensionnement pratique, mais il doit être lu comme un résultat moyen dans un cadre gaussien idéal. Il sera affiné pour MVDR régularisé au chapitre 23.

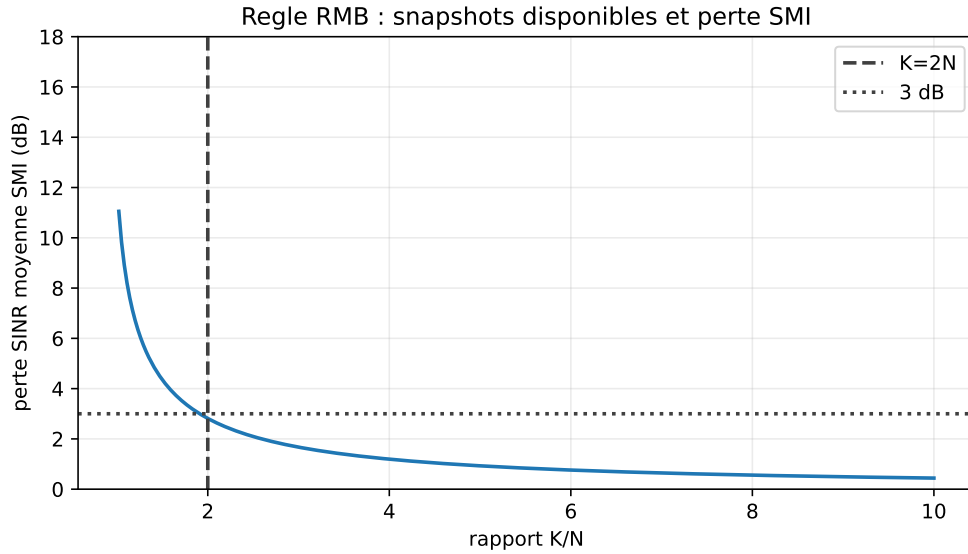


FIGURE 9.1 – Lecture de la règle de Reed–Mallett–Brennan ($N = 32$, modèle SMI gaussien idéal) : à $K = 2N$, la perte moyenne SMI est de l'ordre de 3 dB ; en dessous, la covariance estimée devient très fragile.

9.6.3 Stationnarité

L'estimation par moyenne exige la stationnarité du processus sur la fenêtre des K snapshots. Lorsque les sources se déplacent ou que les puissances varient, on doit :

- raccourcir la fenêtre (au prix de la qualité d'estimation) ;
- utiliser un *oubli exponentiel* (cf. chapitre 13) ;
- segmenter et fusionner par moyenne mobile.

9.6.4 Forward–Backward averaging

Pour un ULA et des sources cohérentes ou faiblement corrélées, on augmente artificiellement le nombre de snapshots utiles en exploitant la *symétrie persymétrique* de $\mathbf{R}$. La version « forward–backward » de l'estimateur est

$$\hat{\mathbf{R}}_{FB} = \frac{1}{2} (\hat{\mathbf{R}} + \mathbf{J} \hat{\mathbf{R}}^* \mathbf{J}),$$

où $\mathbf{J}$ est la matrice de permutation « flip ». Cette estimée est plus précise pour les petits K et reste persymétrique.

9.7 Inversibilité et conditionnement

9.7.1 Cas singulier

Lorsque $K < N$ ou que des sources sont parfaitement cohérentes, $\hat{\mathbf{R}}$ peut être singulière ou très mal conditionnée. Les algorithmes qui requièrent l'inversion de $\mathbf{R}$ (MVDR, LCMV, RLS) deviennent alors instables ou impossibles.

9.7.2 Conditionnement

Le *nombre de conditionnement* $\kappa(\mathbf{R}) = \lambda_{\max}/\lambda_{\min}$ mesure la difficulté numérique d'inversion. Pour un brouilleur 30 dB au-dessus du bruit sur un ULA $N = 16$, $\lambda_1 \sim 10^3 \sigma^2$ et $\lambda_N = \sigma^2$, donc $\kappa \sim 10^3$. Avec un brouilleur 60 dB, κ atteint 10^6 : l'inversion en double précision (~ 16 chiffres) commence à perdre 6 chiffres de précision.

9.7.3 Régularisation par diagonal loading

La solution canonique est de remplacer $\hat{\mathbf{R}}$ par

$$\hat{\mathbf{R}}_\alpha = \hat{\mathbf{R}} + \alpha \mathbf{I}_N,$$

avec $\alpha > 0$. Toutes les valeurs propres sont relevées de α , le conditionnement devient $\kappa = (\lambda_{\max} + \alpha)/(\lambda_{\min} + \alpha)$, et l'inversion redevient stable. Le coût est une légère perte de performance asymptotique (cf. chapitres 10 et 23 pour le choix optimal de α).

9.8 Interprétation physique

La covariance contient implicitement toute la *cartographie spatiale* des sources et du bruit. Plus concrètement :

- la *trace* $\text{tr}(\mathbf{R}) = \sum_m \mathbb{E}[|x_m|^2]$ est la puissance totale reçue par le réseau ;
- le *déterminant* $\det(\mathbf{R})$ mesure le volume d'incertitude du snapshot ;
- la quantité $\mathbf{a}^H(\theta) \mathbf{R} \mathbf{a}(\theta)$ est la puissance qui sortirait du beamformer DAS pointé en θ , et constitue le *Bartlett spectrum* ;
- la quantité $\mathbf{a}^H(\theta) \mathbf{R}^{-1} \mathbf{a}(\theta)$ joue un rôle dual et sera le pseudo-spectre de Capon, étudié au chapitre suivant.

9.9 Visualisation

On représente couramment $|\mathbf{R}|$ comme une image (module en décibels) et la phase comme une seconde image. Pour un ULA en broadside, les éléments hors diagonale sont approximativement constants le long des diagonales (structure *Toeplitz*) : la matrice de covariance est alors quasi-Toeplitz, propriété exploitée dans les estimateurs Toeplitz-régularisés.

9.10 Limites de la covariance

Présence du signal utile dans $\mathbf{R}$. En pratique, la covariance estimée contient à la fois les interférences et le signal utile. Si l'on construit un MVDR à partir de cette $\hat{\mathbf{R}}$, l'algorithme tente d'*atténuer le signal utile* en croyant atténuer du bruit (*signal self-cancellation*, cf. chapitre 23). La parade est, lorsque c'est possible, d'estimer une covariance *interférence + bruit seule*, en utilisant des intervalles de temps où le signal est absent.

Sensibilité à la calibration. Une erreur de phase entre voies entraîne une covariance dont la structure ne correspond plus aux steering vectors théoriques. Les algorithmes « pensent voir » des sources qui n'existent pas. Le chapitre 22 traite ce problème en profondeur.

9.11 Implémentation numérique

Listing 9.1 – Estimation de covariance et diagonal loading

```

1 import numpy as np
2
3 def sample_covariance(X):
4     """X : matrice (N, K) de snapshots. Renvoie R (N,N)."""
5     N, K = X.shape
6     assert K > 0
7     R = (X @ X.conj().T) / K

```

```

8     return 0.5 * (R + R.conj().T)    # symétrisation hermitienne
9
10 def diagonal_loading(R, alpha):
11     N = R.shape[0]
12     assert R.shape == (N, N)
13     assert alpha >= 0.0
14     R_alpha = R + alpha * np.eye(N)
15     return 0.5 * (R_alpha + R_alpha.conj().T)
16
17 def forward_backward(R):
18     N = R.shape[0]
19     assert R.shape == (N, N)
20     J = np.flip(np.eye(N), axis=0)
21     return 0.5 * (R + J @ R.conj() @ J)

```

9.12 Synthèse

La covariance spatiale est l'objet pivot du beamforming statistique :

- elle code la structure spatiale des sources et du bruit ;
- sa décomposition propre sépare un *sous-espace signal* (de dimension L pour L sources) et un *sous-espace bruit* (de dimension $N - L$) ;
- son estimation empirique exige souvent K de l'ordre de quelques N pour rester satisfaisante ;
- son inversion exige une régularisation par diagonal loading dès qu'elle devient mal conditionnée ou singulière.

À partir de cette matrice, le chapitre suivant construit le beamformer *Minimum Variance Distortionless Response*, qui place automatiquement ses nulls sur l'énergie indésirable détectée dans $\mathbf{R}$. C'est l'archétype du beamformer adaptatif : ses poids dépendent entièrement de la covariance, qui elle-même dépend entièrement des données. L'opération devient ainsi totalement tournée vers la scène RF observée, sans hypothèse explicite sur les directions des brouilleurs.

Chapitre 10

MVDR / Capon

Le Null Steering exige de connaître les directions des interférences. Si elles sont inconnues, ou s'il en arrive de nouvelles, il ne sait plus quoi faire. Le *Minimum Variance Distortionless Response* (MVDR), aussi appelé *beamformer de Capon* [5], résout ce problème en exploitant directement la covariance spatiale. Il minimise la puissance totale en sortie tout en imposant un gain unitaire dans la direction cible : automatiquement, ses zéros se placent là où l'énergie reçue est la plus forte, donc où sont les brouilleurs — sans qu'on les nomme.

L'idée a une élégance particulière : on demande au beamformer de *faire le moins de bruit possible*, sous la seule contrainte de ne pas toucher au signal utile. Tout ce qui n'est pas le signal utile est, par construction, de « l'énergie indésirable », et MVDR l'élimine sans avoir besoin de la nommer. C'est une forme de beamforming « par soustraction », par opposition au Null Steering qui est « par contrainte explicite ». Ce changement de perspective a deux conséquences pratiques majeures : MVDR n'a pas besoin de connaître les directions des brouilleurs, et il s'adapte automatiquement quand celles-ci changent. Sa contrepartie, qu'on verra émerger plus loin, est une sensibilité accrue aux erreurs sur la direction cible et sur l'estimation de covariance — les deux objets sur lesquels il s'appuie pour fonctionner.

10.1 Formulation

Définition 10.1 (Problème MVDR). Pour une covariance $\mathbf{R}$ et une direction cible θ_s , le beamformer MVDR est la solution de

$$\min_{\mathbf{w}} \mathbf{w}^H \mathbf{R} \mathbf{w} \quad \text{sous} \quad \mathbf{w}^H \mathbf{a}(\theta_s) = 1.$$

L'objectif $\mathbf{w}^H \mathbf{R} \mathbf{w}$ est exactement la *puissance moyenne en sortie* ; $\mathbb{E}[|y(n)|^2] = \mathbf{w}^H \mathbb{E}[\mathbf{x}\mathbf{x}^H] \mathbf{w} = \mathbf{w}^H \mathbf{R} \mathbf{w}$. La contrainte exige que la cible passe sans distorsion. Minimiser l'une sous l'autre revient à n'atténuer que ce qui n'est pas le signal utile.

10.2 Solution

Théorème 10.2 (Solution de Capon). Si $\mathbf{R}$ est inversible, le problème MVDR admet pour solution unique

$$\mathbf{w}_{\text{MVDR}} = \frac{\mathbf{R}^{-1} \mathbf{a}(\theta_s)}{\mathbf{a}^H(\theta_s) \mathbf{R}^{-1} \mathbf{a}(\theta_s)}.$$

Démonstration. On applique les multiplicateurs de Lagrange complexes. Pour alléger l'écriture, posons :

$$\mathbf{a}_s = \mathbf{a}(\theta_s).$$

Le problème est :

$$\min_{\mathbf{w}} \mathbf{w}^H \mathbf{R} \mathbf{w} \quad \text{sous} \quad \mathbf{w}^H \mathbf{a}_s = 1.$$

Comme la contrainte complexe impose à la fois une partie réelle et une partie imaginaire, on écrit un lagrangien réel sous la forme :

$$\mathcal{L}(\mathbf{w}, \mu) = \mathbf{w}^H \mathbf{R} \mathbf{w} + \mu^* (\mathbf{w}^H \mathbf{a}_s - 1) + \mu (\mathbf{a}_s^H \mathbf{w} - 1).$$

La dérivée de Wirtinger par rapport à $\mathbf{w}^*$ donne :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}^*} = \mathbf{R} \mathbf{w} + \mu^* \mathbf{a}_s = \mathbf{0}.$$

Donc :

$$\mathbf{R} \mathbf{w} = -\mu^* \mathbf{a}_s.$$

Si $\mathbf{R}$ est inversible :

$$\mathbf{w} = -\mu^* \mathbf{R}^{-1} \mathbf{a}_s.$$

Le vecteur optimal est donc nécessairement proportionnel à $\mathbf{R}^{-1} \mathbf{a}_s$. Il reste seulement à choisir la constante pour respecter la contrainte distortionless.

En remplaçant dans $\mathbf{w}^H \mathbf{a}_s = 1$:

$$(-\mu^* \mathbf{R}^{-1} \mathbf{a}_s)^H \mathbf{a}_s = 1.$$

Comme $\mathbf{R}$ est hermitienne, $\mathbf{R}^{-1}$ l'est aussi :

$$-\mu \mathbf{a}_s^H \mathbf{R}^{-1} \mathbf{a}_s = 1.$$

Ainsi :

$$-\mu = \frac{1}{\mathbf{a}_s^H \mathbf{R}^{-1} \mathbf{a}_s}.$$

On obtient finalement :

$$\mathbf{w}_{\text{MVDR}} = \frac{\mathbf{R}^{-1} \mathbf{a}_s}{\mathbf{a}_s^H \mathbf{R}^{-1} \mathbf{a}_s}.$$

La hessienne $\mathbf{R}$ étant semi-définie positive et la contrainte étant linéaire, le point stationnaire est un minimum global. $\square$

10.3 MVDR en pratique : éviter l'inverse explicite de $\mathbf{R}$

La formule contient $\mathbf{R}^{-1}$, mais une implémentation sérieuse ne forme pas l'inverse explicite. On calcule d'abord

$$\mathbf{R} \mathbf{u} = \mathbf{a}_s,$$

puis on normalise :

$$\mathbf{w}_{\text{MVDR}} = \frac{\mathbf{u}}{\mathbf{a}_s^H \mathbf{u}}.$$

Numériquement, cela signifie utiliser une résolution linéaire :

$$\mathbf{u} = \text{solve}(\mathbf{R}, \mathbf{a}_s),$$

et non :

$$\mathbf{u} = \mathbf{R}^{-1} \mathbf{a}_s.$$

Cette différence est importante :

- calculer l'inverse explicite coûte plus cher ;
- l'inversion amplifie davantage les erreurs d'arrondi ;
- la résolution linéaire exploite mieux la structure hermitienne positive de $\mathbf{R}$, par exemple avec Cholesky ;
- avec diagonal loading, on résout simplement $(\hat{\mathbf{R}} + \alpha \mathbf{I}) \mathbf{u} = \mathbf{a}_s$.

Forme « scaled ». En pratique on cherche d'abord le vecteur $\mathbf{u}$ qui vérifie $\mathbf{R}\mathbf{u} = \mathbf{a}(\theta_s)$, puis on normalise $\mathbf{w} = \mathbf{u}/(\mathbf{a}^H(\theta_s)\mathbf{u})$. L'inversion $\mathbf{R}^{-1}$ n'est pas réalisée explicitement : on résout $\mathbf{R}\mathbf{u} = \mathbf{a}(\theta_s)$ par Cholesky ou décomposition LU, sensiblement moins coûteux et plus stable.

10.4 Pseudo-spectre de Capon

En appliquant $\mathbf{w}_{\text{MVDR}}$ à la covariance, la puissance de sortie vaut

$$P_{\text{MVDR}}(\theta_s) = \mathbf{w}^H \mathbf{R} \mathbf{w} = \frac{1}{\mathbf{a}^H(\theta_s) \mathbf{R}^{-1} \mathbf{a}(\theta_s)}.$$

En faisant varier θ_s , cette expression définit le *pseudo-spectre de Capon*

$$P_{\text{Capon}}(\theta) = \frac{1}{\mathbf{a}^H(\theta) \mathbf{R}^{-1} \mathbf{a}(\theta)}.$$

Ses maxima localisent les sources avec une résolution bien meilleure que le *Bartlett spectrum* $P_{\text{Bartlett}}(\theta) = \mathbf{a}^H(\theta) \mathbf{R} \mathbf{a}(\theta)/N^2$ (qui n'est rien d'autre que le DAS appliqué à $\mathbf{R}$).

Comparaison qualitative.

- Bartlett a la résolution du Delay-and-Sum (limite de Fourier $\sim \lambda/D$), c'est-à-dire la borne de Rayleigh.
- Capon dépasse cette limite, avec une résolution qui s'améliore quand le SNR augmente.
- MUSIC (chapitre 15) ira encore plus loin en exploitant la structure de sous-espaces de $\mathbf{R}$.

10.5 Cas particulier : bruit blanc seul

Si $\mathbf{R} = \sigma^2 \mathbf{I}_N$ (pas d'interférence directionnelle), $\mathbf{R}^{-1} = \mathbf{I}_N/\sigma^2$ et

$$\mathbf{w}_{\text{MVDR}} = \frac{\mathbf{a}(\theta_s)/\sigma^2}{N/\sigma^2} = \frac{\mathbf{a}(\theta_s)}{N} = \mathbf{w}_{\text{DAS}}.$$

En bruit blanc pur, MVDR coïncide exactement avec le Delay-and-Sum. C'est une confirmation rassurante : on n'a rien gagné — mais rien perdu — là où le DAS est déjà optimal.

Cette équivalence n'est pas anecdotique : elle dit que MVDR généralise le DAS sans le supplanter. Là où l'environnement est bénin, les deux sont identiques ; là où il devient hostile, MVDR développe des nulls là où le DAS reste inactif. On peut donc voir MVDR comme un DAS « augmenté » qui ne se distingue de son cousin que dans les scénarios où c'est nécessaire. C'est précisément ce qui le rend attractif comme remplaçant systématique du DAS : on ne perd rien quand il n'y a rien à gagner, et on gagne énormément quand il y a quelque chose à rejeter.

10.6 Cas avec une interférence puissante

Supposons une interférence forte à θ_i , donc $\mathbf{R} = p_i \mathbf{a}_i \mathbf{a}_i^H + \sigma^2 \mathbf{I}_N$ avec $\mathbf{a}_i = \mathbf{a}(\theta_i)$ et $p_i \gg \sigma^2$. Par la formule de Sherman–Morrison :

$$\mathbf{R}^{-1} = \frac{1}{\sigma^2} \mathbf{I}_N - \frac{p_i/\sigma^4}{1 + (p_i/\sigma^2)N} \mathbf{a}_i \mathbf{a}_i^H.$$

Pour $p_i/\sigma^2 \gg 1/N$ (brouilleur fort), cette formule approche un *projecteur orthogonal au sous-espace de l'interférence* :

$$\mathbf{R}^{-1} \rightarrow \frac{1}{\sigma^2} \left(\mathbf{I}_N - \frac{\mathbf{a}_i \mathbf{a}_i^H}{N} \right).$$

Le beamformer MVDR devient alors approximativement

$$\mathbf{w}_{\text{MVDR}} \approx \frac{1}{N} \left(\mathbf{a}(\theta_s) - \frac{\mathbf{a}_i^H \mathbf{a}(\theta_s)}{N} \mathbf{a}_i \right).$$

On lit immédiatement la structure : MVDR *soustrait* à $\mathbf{a}(\theta_s)$ sa composante le long de $\mathbf{a}_i$, exactement comme un Null Steering sur θ_i . Mais ici, θ_i n'a jamais été donné : il a été *appris* via $\mathbf{R}$.

10.7 Interprétation : Null Steering implicite

La leçon précédente se généralise. MVDR place ses zéros sur les directions qui contribuent le plus à $\mathbf{R}$, c'est-à-dire celles des interférences les plus puissantes. La profondeur du null dépend de la puissance relative :

$$|B(\theta_i)|^2 \sim \frac{\sigma^2}{N p_i} \quad (\text{brouilleur fort}).$$

Plus le brouilleur est fort, plus le null est profond. Inversement, une interférence faible (proche du niveau de bruit) produit un null peu profond : MVDR ne dépense pas de degrés de liberté pour rien.

10.8 SINR de sortie

Une métrique plus complète que le SNR est le *SINR* (Signal to Interference + Noise Ratio). Pour une covariance interférence+bruit $\mathbf{R}_{i+n}$ et une cible $\mathbf{a}_s = \mathbf{a}(\theta_s)$:

$$\text{SINR}_{\text{out}} = \frac{p_s |\mathbf{w}^H \mathbf{a}_s|^2}{\mathbf{w}^H \mathbf{R}_{i+n} \mathbf{w}}.$$

Théorème 10.3 (Optimalité SINR de MVDR). *Le beamformer qui maximise SINR_{out} sous la contrainte $\mathbf{w}^H \mathbf{a}_s = 1$ est exactement*

$$\mathbf{w}^* = \frac{\mathbf{R}_{i+n}^{-1} \mathbf{a}_s}{\mathbf{a}_s^H \mathbf{R}_{i+n}^{-1} \mathbf{a}_s},$$

et le *SINR* optimal vaut

$$\text{SINR}^* = p_s \mathbf{a}_s^H \mathbf{R}_{i+n}^{-1} \mathbf{a}_s.$$

L'objet pertinent est donc $\mathbf{R}_{i+n}$ (interférence + bruit *sans* signal), et non $\mathbf{R}$ (qui contient aussi le signal). Quand on utilise $\mathbf{R}$ à la place de $\mathbf{R}_{i+n}$, on est dans le régime *Sample Matrix Inversion (SMI)*, légèrement sous-optimal mais bien plus simple à mettre en œuvre (pas besoin d'identifier les intervalles « signal absent »).

10.9 Estimation pratique : SMI

En pratique on remplace $\mathbf{R}$ par sa version échantillon $\hat{\mathbf{R}}$ obtenue sur K snapshots (chapitre 9) :

$$\hat{\mathbf{w}}_{\text{MVDR}} = \frac{\hat{\mathbf{R}}^{-1} \mathbf{a}(\theta_s)}{\mathbf{a}^H(\theta_s) \hat{\mathbf{R}}^{-1} \mathbf{a}(\theta_s)}.$$

La règle de Reed–Mallett–Brennan (théorème 9.4) dit qu'avec $K \simeq 2N$, la perte moyenne de SINR par rapport au cas idéal est de l'ordre de 3 dB dans le modèle SMI gaussien.

10.10 Diagonal loading : la régularisation indispensable

10.10.1 Pourquoi

Trois situations imposent une régularisation :

1. $K < N$: $\hat{\mathbf{R}}$ est singulière et non inversible ;
2. $K \sim N$: $\hat{\mathbf{R}}$ est mal conditionnée ;
3. *steering vector mismatch* : la cible n'est pas exactement à θ_s , et MVDR atténue partiellement le signal utile.

10.10.2 Effet sur les valeurs propres

On remplace $\hat{\mathbf{R}}$ par

$$\hat{\mathbf{R}}_\alpha = \hat{\mathbf{R}} + \alpha \mathbf{I}_N,$$

qui relève toutes les valeurs propres de α . Conditionnement $\kappa = (\lambda_{\max} + \alpha)/(\lambda_{\min} + \alpha)$; inversion stable même quand $\lambda_{\min} \rightarrow 0$.

10.10.3 Limite $\alpha \rightarrow \infty$

Quand α est grand devant toutes les valeurs propres de $\hat{\mathbf{R}}$, $\hat{\mathbf{R}}_\alpha \approx \alpha \mathbf{I}$, et MVDR redevient le DAS. Le diagonal loading est donc un *glissement continu* entre MVDR (faible α , sélectif mais fragile) et DAS (grand α , robuste mais peu sélectif). Le bon α vit quelque part au milieu.

10.10.4 Choix de α

Plusieurs heuristiques :

- $\alpha = c \cdot \text{tr}(\hat{\mathbf{R}})/N$ avec $c \in [10^{-3}, 10^{-1}]$: robuste et simple.
- $\alpha = c \sigma^2$ avec σ^2 estimé par les plus petites valeurs propres de $\hat{\mathbf{R}}$.
- Méthodes « worst-case » [8] : sphère d'incertitude autour de $\mathbf{a}(\theta_s)$, résolution par SOCP.
- *Robust Capon Beamforming* [7] : optimisation explicite sur la sphère d'incertitude, lien analytique avec un α optimal.

Dans tous les cas, le calcul numérique suit la même logique : résoudre $(\hat{\mathbf{R}} + \alpha \mathbf{I})\mathbf{z} = \mathbf{a}_s$, puis normaliser $\mathbf{w} = \mathbf{z}/(\mathbf{a}_s^H \mathbf{z})$, sans former l'inverse matriciel. Le chapitre 23 approfondit ces méthodes.

10.11 Steering vector mismatch

Si la cible est en réalité à $\theta_s + \delta\theta$, le vecteur $\mathbf{a}(\theta_s)$ utilisé par MVDR ne correspond pas exactement à la signature du signal utile. Le signal apparaît alors comme une « fausse interférence » dans $\hat{\mathbf{R}}$, et MVDR le traite comme telle : c'est la *signal self-cancellation*. Le résultat peut être catastrophique : dans des simulations sévères où la cible est présente dans la covariance, la perte sur la cible peut monter à plusieurs dizaines de dB pour un mismatch de quelques fractions de largeur de lobe principal.

Les parades sont :

- le diagonal loading ;
- l'utilisation d'une covariance *interférence + bruit seule* $\mathbf{R}_{i+n}$;
- les contraintes dérivatives (chapitre 23), qui imposent en plus $\partial_\theta B(\theta_s) = 0$, élargissant la zone de gain unitaire ;
- les approches *robustes* (*Robust Capon*).

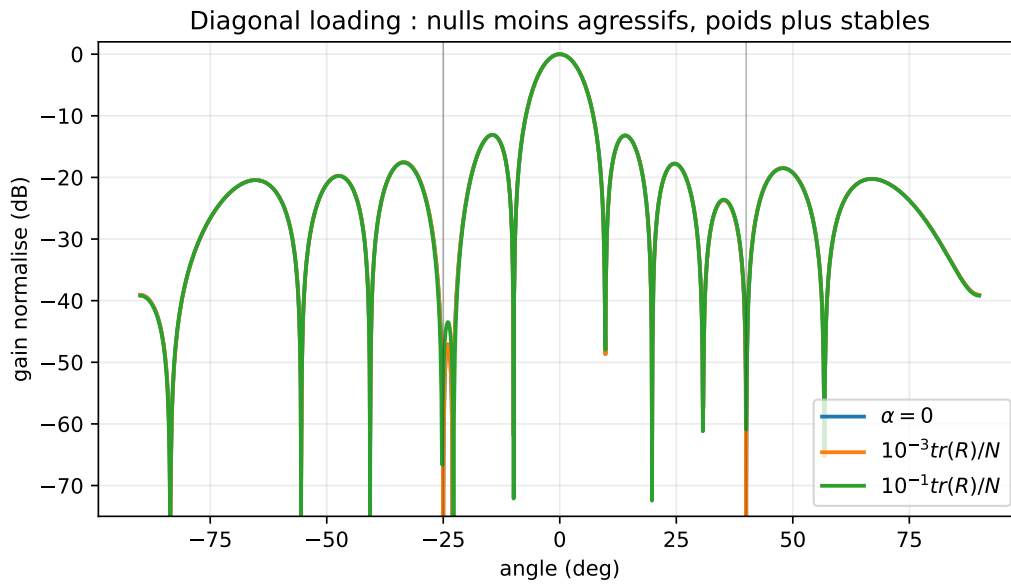


FIGURE 10.1 – Effet du diagonal loading ($N = 12$, cible 0° , brouilleurs -25° et 40° , INR 40 dB) : un chargement plus fort rend les nulls moins agressifs mais stabilise les poids et réduit la sensibilité aux erreurs de modèle.

10.12 Beam pattern et nulls adaptatifs

Sur des données réelles, le beam pattern MVDR a typiquement les caractéristiques suivantes :

- lobe principal proche du DAS sur la cible ;
- *nulls profonds* aux directions des interférences puissantes, souvent dans la plage -30 à -60 dB lorsque la covariance et la calibration sont favorables ;
- *lobes secondaires* légèrement remontés par rapport au DAS là où il n’y a pas d’énergie (le critère est seulement quadratique en $\mathbf{w}$, pas en niveau de lobes) ;
- *forme dépendante des données* : deux données différentes produisent deux beam patterns différents.

10.13 Comparaison Delay-and-Sum, Null Steering, MVDR

Critère	DAS	Null Steering	MVDR
Information requise	θ_s	θ_s, θ_i	$\theta_s, \mathbf{R}$
Adaptation	non	non	oui (via $\mathbf{R}$)
Nulls sur interférences	non	explicites	implicites
Gain SNR (bruit blanc)	N (optimal)	$\leq N$	N (équiv. DAS)
SINR optimal	non	non	oui (asyp.)
Robustesse	élevée	moyenne	faible sans reg.
Complexité	$\mathcal{O}(N)$	$\mathcal{O}(NL^2)$	$\mathcal{O}(N^3)$

10.14 Implémentation numérique

Listing 10.1 – MVDR avec diagonal loading

```

1 import numpy as np
2

```

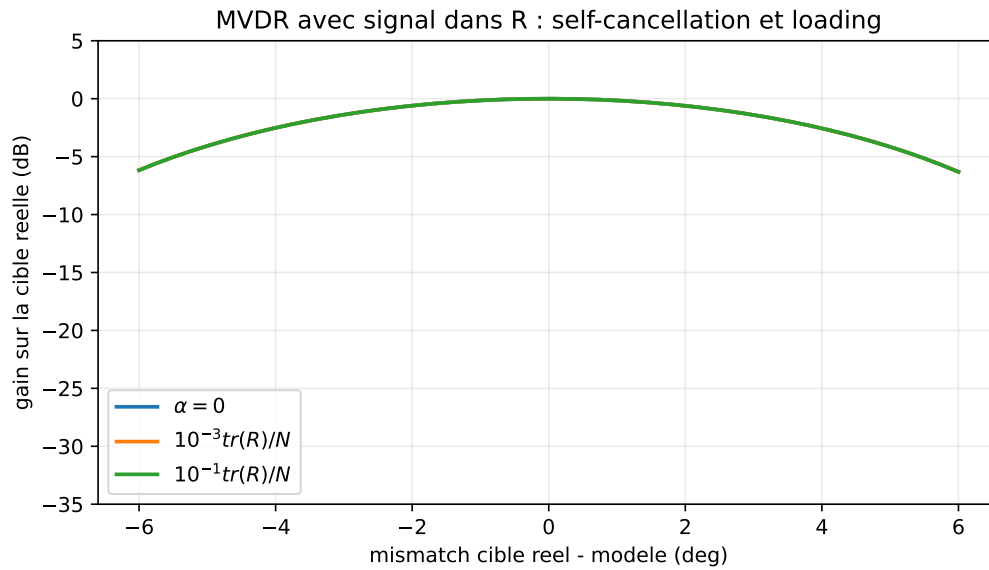


FIGURE 10.2 – Simulation sévère ($N = 12$, cible 0° , brouilleurs -25° et 40° , signal utile présent dans la covariance) : un MVDR non chargé peut atténuer la cible dès que le steering vector est légèrement faux.

```

3 def mvdr_weights(R, a_s, alpha=0.0):
4     """
5     R : matrice de covariance (N,N), hermitienne SDP
6     a_s : steering vector cible (N,)
7     alpha : diagonal loading (>=0)
8     """
9     N = R.shape[0]
10    assert R.shape == (N, N)
11    assert a_s.shape == (N,)
12    assert alpha >= 0.0
13    R_alpha = R + alpha * np.eye(N)
14    R_alpha = 0.5*(R_alpha + R_alpha.conj().T)
15    if np.linalg.cond(R_alpha) > 1e12:
16        raise np.linalg.LinAlgError("R_alpha mal conditionnee")
17    u = np.linalg.solve(R_alpha, a_s)
18    den = a_s.conj() @ u
19    if abs(den) < np.finfo(float).eps:
20        raise np.linalg.LinAlgError("normalisation MVDR instable")
21    return u / den
22
23 def capon_spectrum(R, thetas, N, d_over_lambda=0.5, alpha=0.0):
24    assert R.shape == (N, N)
25    assert alpha >= 0.0
26    R_alpha = R + alpha * np.eye(N)
27    R_alpha = 0.5*(R_alpha + R_alpha.conj().T)
28    if np.linalg.cond(R_alpha) > 1e12:
29        raise np.linalg.LinAlgError("R_alpha mal conditionnee")
30    m = np.arange(N)
31    spec = np.zeros(len(thetas))
32    eps = np.finfo(float).eps
33    for idx, th in enumerate(thetas):
34        a = np.exp(-1j * 2*np.pi * d_over_lambda * np.sin(th) * m)
35        u = np.linalg.solve(R_alpha, a)
36        den = max(np.real(a.conj() @ u), eps)

```

```

37         spec[idx] = 1.0 / den
38     return spec

```

10.15 Synthèse

MVDR transforme le beamforming en un problème d'optimisation *informé par les données* :

- il généralise le DAS : en bruit blanc, ils coïncident ;
- il généralise le Null Steering : ses nulls se placent automatiquement là où l'énergie est concentrée ;
- il maximise le SINR à contrainte de distorsion nulle ;
- il exige une covariance bien estimée et régularisée.

À retenir

- **Formule centrale** : $\mathbf{w} = \mathbf{R}^{-1}\mathbf{a}/(\mathbf{a}^H\mathbf{R}^{-1}\mathbf{a})$.
- **Hypothèse principale** : covariance fiable et steering vector cible correct.
- **Avantage** : nulls adaptatifs sans connaître les directions des brouilleurs.
- **Limite** : sensibilité au mismatch et à une covariance mal conditionnée.
- **Piège pratique** : éviter de calculer explicitement $\mathbf{R}^{-1}$; résoudre un système linéaire.
- **Suite** : LCMV généralise MVDR à plusieurs contraintes.

Le chapitre suivant le généralise en LCMV, qui combine plusieurs contraintes linéaires (gain unitaire + zéros + dérivées) dans un seul cadre d'optimisation. À l'issue du chapitre 11, on disposera d'un cadre unifié englobant DAS, Null Steering et MVDR — et qui admettra une décomposition en deux branches (GSC) particulièrement commode pour les implémentations adaptatives qui font l'objet de la Partie IV.

Chapitre 11

LCMV et structure GSC

Le Null Steering impose des contraintes linéaires sans modèle statistique. MVDR impose une seule contrainte (gain unitaire) mais exploite la covariance. Le *Linearly Constrained Minimum Variance* (LCMV) unifie les deux : il minimise la puissance de sortie sous *plusieurs* contraintes linéaires. Il englobe MVDR (1 contrainte) et Null Steering (covariance scalaire $\sigma^2 \mathbf{I}$) comme cas particuliers. La *Generalized Sidelobe Canceller* (GSC) est sa reformulation en deux branches qui ouvre la voie à l'implémentation adaptative de la Partie IV.

LCMV peut paraître redondant après MVDR : pourquoi formaliser un cadre plus général alors que MVDR suffit déjà à minimiser la variance ? Trois raisons motivent ce chapitre. D'abord, LCMV permet de combiner la puissance statistique de MVDR avec la précision déterministe du Null Steering : on impose à la fois « je veux ce signal » et « je ne veux pas ces interférences-là ». Ensuite, il permet d'imposer des contraintes plus subtiles comme la *dérivée nulle* du beam pattern (pour la robustesse aux erreurs angulaires sur la cible), qui sera centrale au chapitre 23. Enfin, sa reformulation en GSC fournit une décomposition fondamentale qui sépare la partie « contraintes » (à calculer une fois) de la partie « adaptation » (à mettre à jour en continu) — ce qui rend possible l'implantation efficace sur des plateformes contraintes en calcul.

11.1 Formulation LCMV

Définition 11.1 (Problème LCMV). Soit $\mathbf{R} \in \mathbb{C}^{N \times N}$ hermitienne définie positive, $\mathbf{C} \in \mathbb{C}^{N \times L_c}$ une matrice de contraintes de rang plein colonne ($L_c \leq N$), et $\mathbf{f} \in \mathbb{C}^{L_c}$ un vecteur de réponses imposées. Le beamformer LCMV résout

$$\min_{\mathbf{w}} \mathbf{w}^H \mathbf{R} \mathbf{w} \quad \text{sous} \quad \mathbf{C}^H \mathbf{w} = \mathbf{f}.$$

11.2 Solution analytique

Théorème 11.2 (Solution LCMV). Si $\mathbf{R}$ est définie positive et $\mathbf{C}^H \mathbf{R}^{-1} \mathbf{C}$ est inversible, la solution unique est

$$\mathbf{w}_{\text{LCMV}} = \mathbf{R}^{-1} \mathbf{C} (\mathbf{C}^H \mathbf{R}^{-1} \mathbf{C})^{-1} \mathbf{f}.$$

Démonstration. Lagrangien complexe : $\mathcal{L} = \mathbf{w}^H \mathbf{R} \mathbf{w} + (\mathbf{C}^H \mathbf{w} - \mathbf{f})^H \boldsymbol{\mu} + \boldsymbol{\mu}^H (\mathbf{C}^H \mathbf{w} - \mathbf{f})$. Stationnarité : $\mathbf{R} \mathbf{w} + \mathbf{C} \boldsymbol{\mu} = \mathbf{0}$, soit $\mathbf{w} = -\mathbf{R}^{-1} \mathbf{C} \boldsymbol{\mu}$. Contrainte : $\mathbf{C}^H \mathbf{w} = -\mathbf{C}^H \mathbf{R}^{-1} \mathbf{C} \boldsymbol{\mu} = \mathbf{f}$, d'où $\boldsymbol{\mu} = -(\mathbf{C}^H \mathbf{R}^{-1} \mathbf{C})^{-1} \mathbf{f}$. On en déduit la formule. $\square$

11.3 MVDR comme cas particulier

Avec $\mathbf{C} = \mathbf{a}(\theta_s) \in \mathbb{C}^{N \times 1}$ et $\mathbf{f} = 1$, on retrouve

$$\mathbf{w}_{\text{LCMV}} = \mathbf{R}^{-1} \mathbf{a}(\theta_s) (\mathbf{a}^H(\theta_s) \mathbf{R}^{-1} \mathbf{a}(\theta_s))^{-1} = \mathbf{w}_{\text{MVDR}}.$$

11.4 Null Steering comme cas particulier

Avec $\mathbf{R} = \sigma^2 \mathbf{I}_N$ (pas d'interférence statistique exploitée),

$$\mathbf{w}_{\text{LCMV}} = \mathbf{C} (\mathbf{C}^H \mathbf{C})^{-1} \mathbf{f} = \mathbf{w}_{\text{NS}}.$$

LCMV est donc strictement plus général : c'est le « Null Steering informé par la statistique ».

11.5 Contraintes typiques

11.5.1 Exemple complet : une cible et deux brouilleurs

Supposons une cible à θ_s et deux brouilleurs connus à $\theta_{i,1}$ et $\theta_{i,2}$. On construit :

$$\mathbf{C} = \begin{bmatrix} \mathbf{a}(\theta_s) & \mathbf{a}(\theta_{i,1}) & \mathbf{a}(\theta_{i,2}) \end{bmatrix}$$

et :

$$\mathbf{f} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}.$$

La contrainte LCMV :

$$\mathbf{C}^H \mathbf{w} = \mathbf{f}$$

signifie explicitement :

$$\begin{cases} \mathbf{w}^H \mathbf{a}(\theta_s) = 1, \\ \mathbf{w}^H \mathbf{a}(\theta_{i,1}) = 0, \\ \mathbf{w}^H \mathbf{a}(\theta_{i,2}) = 0. \end{cases}$$

Le beamformer conserve donc la cible, annule deux directions imposées, et utilise les degrés de liberté restants pour minimiser la puissance de sortie selon $\mathbf{R}$. La solution est :

$$\mathbf{w}_{\text{LCMV}} = \mathbf{R}^{-1} \mathbf{C} (\mathbf{C}^H \mathbf{R}^{-1} \mathbf{C})^{-1} \mathbf{f}.$$

Cette écriture montre exactement comment Null Steering et MVDR se combinent : les contraintes fixent ce qui doit être garanti, la covariance optimise tout le reste.

1. Gain unitaire + nulls explicites. $\mathbf{C} = [\mathbf{a}(\theta_s), \mathbf{a}(\theta_{i,1}), \dots, \mathbf{a}(\theta_{i,L})]$, $\mathbf{f} = [1, 0, \dots, 0]^T$. Le beamformer maintient le gain sur la cible, place des nulls sur les L interférences *connues*, et utilise sa puissance statistique résiduelle pour rejeter les interférences *inconnues*.

2. Multi-faisceau. Pour suivre plusieurs cibles simultanément à des gains spécifiés : $\mathbf{C} = [\mathbf{a}(\theta_1), \dots, \mathbf{a}(\theta_M)]$, $\mathbf{f} = [1, 1, \dots, 1]^T$. C'est l'équivalent d'un « multi-beam » statistiquement optimisé.

3. Contraintes dérivatives (élargissement de robustesse). Outre $\mathbf{a}(\theta_s)$ on impose $\partial_{\theta} \mathbf{a}(\theta_s)$ comme deuxième colonne avec réponse imposée à 0. Cela force la dérivée du beam pattern à s'annuler en θ_s , élargissant la zone plate du lobe principal et améliorant la robustesse aux erreurs angulaires.

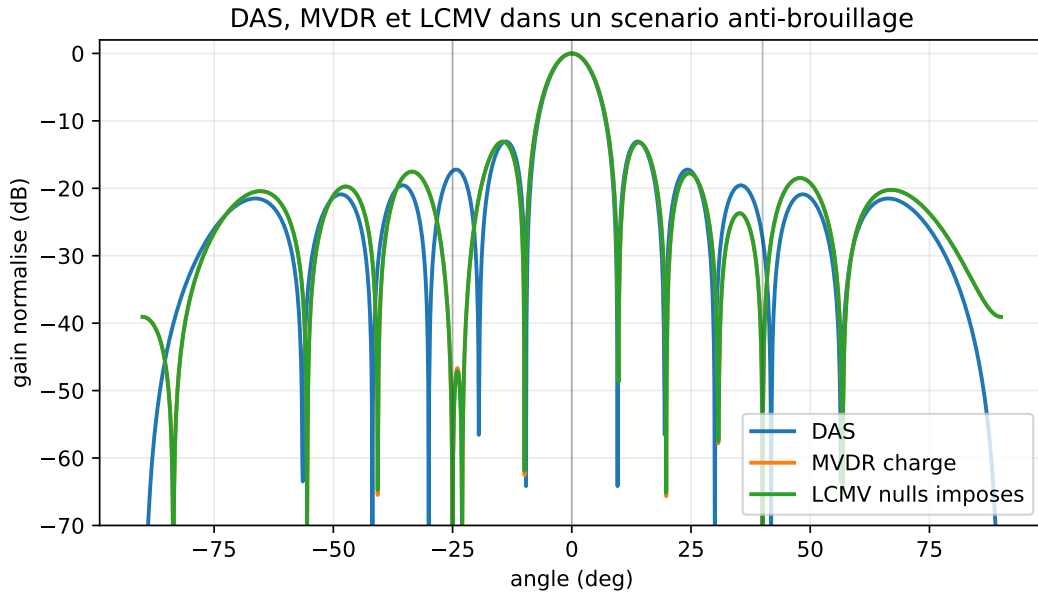


FIGURE 11.1 – Comparaison qualitative entre DAS, MVDR chargé et LCMV ($N = 12$, cible 0° , brouilleurs -25° et 40° , INR 35 dB). DAS ne sait pas placer de nulls, MVDR les adapte à la covariance, LCMV les impose explicitement aux directions choisies.

4. Nulls élargis. Pour rejeter une interférence avec une incertitude angulaire $\pm\delta$, on remplace la contrainte unique $\mathbf{a}(\theta_i)$ par plusieurs contraintes $\mathbf{a}(\theta_i - \delta)$, $\mathbf{a}(\theta_i)$, $\mathbf{a}(\theta_i + \delta)$. Le *null* devient une zone d'atténuation plutôt qu'un point.

Une autre façon d'élargir le null consiste à imposer une dérivée nulle :

$$B(\theta_i) = 0, \quad B'(\theta_i) = 0.$$

Comme

$$B(\theta) = \mathbf{w}^H \mathbf{a}(\theta),$$

cela revient à ajouter :

$$\mathbf{w}^H \dot{\mathbf{a}}(\theta_i) = 0.$$

On obtient alors un null moins ponctuel : il reste profond lorsque l'interférence se déplace légèrement autour de θ_i . En pratique, cette contrainte est souvent plus réaliste qu'un null infiniment fin sur une DOA estimée avec incertitude.

11.6 Degrés de liberté

Comme pour le Null Steering, le coût en degrés de liberté est explicite : L_c contraintes consomment L_c degrés de liberté sur N disponibles. Il reste $N - L_c$ degrés pour rejeter statistiquement les interférences *inconnues* via la minimisation $\mathbf{w}^H \mathbf{R} \mathbf{w}$.

Pratique. On évite de saturer ($L_c \ll N$). Avec $L_c = N$, le beamformer est entièrement contraint et il n'y a plus d'adaptation statistique : LCMV redevient un simple système linéaire $\mathbf{C}^H \mathbf{w} = \mathbf{f}$.

11.6.1 Nombre maximal de nulls

Avec N antennes, le vecteur $\mathbf{w} \in \mathbb{C}^N$ possède N degrés de liberté complexes. Une contrainte de gain utile consomme un degré :

$$\mathbf{w}^H \mathbf{a}(\theta_s) = 1.$$

Chaque null explicite consomme un degré supplémentaire :

$$\mathbf{w}^H \mathbf{a}(\theta_{i,\ell}) = 0.$$

Le nombre théorique maximal de nulls ponctuels est donc :

$$L_{\max} = N - 1.$$

Mais cette limite est rarement utilisable en pratique. Si l'on impose $N - 1$ nulls, il ne reste aucun degré de liberté pour :

- réduire le bruit ;
- compenser une erreur de DOA ;
- limiter la norme de $\mathbf{w}$;
- s'adapter à une interférence inconnue dans $\mathbf{R}$.

Une règle d'ingénierie prudente est de garder plusieurs degrés de liberté libres :

$$L_{\text{nulls}} \leq N - 3 \quad \text{ou} \quad N - 4$$

si l'environnement est incertain. Les derniers degrés de liberté valent souvent plus cher que les premiers : ce sont eux qui absorbent les erreurs de calibration, les DOA imparfaites et le bruit de covariance.

11.7 Reformulation GSC (Generalized Sidelobe Canceller)

LCMV se reformule en une décomposition à deux branches qui simplifie l'implémentation adaptative et clarifie la structure du problème. L'idée : séparer le beamformer en une partie *fixe* qui satisfait les contraintes, et une partie *adaptative* qui ne peut pas les violer.

11.7.1 Construction

Soit $\mathbf{w}_q = \mathbf{C}(\mathbf{C}^H \mathbf{C})^{-1} \mathbf{f}$ la *partie fixe* (la solution Null Steering de norme minimale, qui satisfait $\mathbf{C}^H \mathbf{w}_q = \mathbf{f}$). Soit $\mathbf{B} \in \mathbb{C}^{N \times (N-L_c)}$ une *matrice de blocage* dont les colonnes engendrent l'espace $\text{Ker}(\mathbf{C}^H)$, c'est-à-dire $\mathbf{C}^H \mathbf{B} = \mathbf{0}$. Tout vecteur de la forme

$$\mathbf{w} = \mathbf{w}_q - \mathbf{B} \mathbf{w}_a, \quad \mathbf{w}_a \in \mathbb{C}^{N-L_c},$$

satisfait automatiquement $\mathbf{C}^H \mathbf{w} = \mathbf{f}$. La contrainte est « absorbée » dans la structure.

11.7.2 Problème équivalent sur $\mathbf{w}_a$

Substituant dans le critère :

$$\mathbf{w}^H \mathbf{R} \mathbf{w} = (\mathbf{w}_q - \mathbf{B} \mathbf{w}_a)^H \mathbf{R} (\mathbf{w}_q - \mathbf{B} \mathbf{w}_a).$$

Minimiser sur $\mathbf{w}_a$ donne

$$\mathbf{w}_a^* = (\mathbf{B}^H \mathbf{R} \mathbf{B})^{-1} \mathbf{B}^H \mathbf{R} \mathbf{w}_q.$$

11.7.3 Interprétation : deux branches

- **Branche supérieure** : $y_q(n) = \mathbf{w}_q^H \mathbf{x}(n)$, sortie d'un beamformer *fixe* satisfaisant les contraintes (par exemple Null Steering ou DAS constraint).
- **Branche inférieure** : $\mathbf{z}(n) = \mathbf{B}^H \mathbf{x}(n) \in \mathbb{C}^{N-L_c}$, « référence de bruit » qui contient l'environnement projeté hors du sous-espace des contraintes.
- **Soustraction adaptative** : $y(n) = y_q(n) - \mathbf{w}_a^H \mathbf{z}(n)$, où $\mathbf{w}_a$ est ajusté pour minimiser $\mathbb{E}[|y(n)|^2]$.

Le problème est alors un *filtrage de Wiener sans contraintes* : $\mathbf{w}_a$ s'obtient comme la solution Wiener du problème « minimiser $|y_q - \mathbf{w}_a^H \mathbf{z}|^2$ ». C'est cette forme qui sera attaquée par LMS et RLS en Partie IV.

Avantages.

- l'adaptation porte sur un sous-problème *non contraint* ;
- la structure deux-branches isole clairement la partie « contraintes » de la partie « adaptation » ;
- la dimension du problème adaptatif est $N - L_c$, souvent bien plus petite que N pour des contraintes peu nombreuses.

Choix de $\mathbf{B}$. Une matrice de blocage de qualité a des colonnes orthonormées et bien conditionnées. Méthodes courantes :

- complément orthogonal de $\mathbf{C}$ via QR ;
- décomposition en valeurs singulières de $\mathbf{I} - \mathbf{C}(\mathbf{C}^H \mathbf{C})^{-1} \mathbf{C}^H$;
- construction explicite (matrice de Householder) pour $\mathbf{C}$ Toeplitz.

11.8 Conditionnement de $\mathbf{C}$

Lorsque les angles imposés par les contraintes sont proches, $\mathbf{C}^H \mathbf{C}$ devient mal conditionnée, et la norme du beamformer LCMV explose. Trois remèdes classiques :

- supprimer les contraintes presque colinéaires ;
- régulariser : $\mathbf{C}^H \mathbf{R}^{-1} \mathbf{C} \rightarrow (\mathbf{C}^H \mathbf{R}^{-1} \mathbf{C} + \beta \mathbf{I})$;
- passer en formulation *soft constraints*, où la contrainte devient une pénalité quadratique.

11.9 Diagonal loading sur LCMV

Comme pour MVDR, on remplace $\mathbf{R}$ par $\mathbf{R} + \alpha \mathbf{I}_N$. La structure LCMV est préservée, le conditionnement amélioré, et l'on borne la norme $\|\mathbf{w}_{\text{LCMV}}\|^2$.

11.10 Implémentation numérique

Listing 11.1 – LCMV générique

```

1 import numpy as np
2
3 def lcmv_weights(R, C, f, alpha=0.0):
4     """
5     R      : (N,N) covariance hermitienne SDP
6     C      : (N, Lc) matrice de contraintes (rang plein colonne)

```

```

7      f      : (Lc,) réponses imposées
8      alpha : diagonal loading
9      """
10     N = R.shape[0]
11     assert R.shape == (N, N)
12     assert C.shape[0] == N
13     assert f.shape == (C.shape[1],)
14     assert alpha >= 0.0
15     R_alpha = R + alpha * np.eye(N)
16     R_alpha = 0.5*(R_alpha + R_alpha.conj().T)
17     if np.linalg.cond(R_alpha) > 1e12:
18         raise np.linalg.LinAlgError("R_alpha mal conditionnee")
19     Rinv_C = np.linalg.solve(R_alpha, C)          # (N, Lc)
20     M = C.conj().T @ Rinv_C                       # (Lc, Lc)
21     if np.linalg.cond(M) > 1e12:
22         raise np.linalg.LinAlgError("contraintes LCMV mal conditionnees")
23     return Rinv_C @ np.linalg.solve(M, f)

```

11.11 Quand utiliser LCMV plutôt que MVDR

Adapté.

- Plusieurs cibles à maintenir simultanément ;
- Brouilleurs connus dont on veut imposer la rejection *en plus* de l'adaptation MVDR ;
- Robustesse renforcée par contraintes dérivatives ;
- Architectures GSC avec partie adaptative séparée.

Inadapté.

- Lorsque N est petit et qu'on a déjà besoin de tous les degrés de liberté pour les interférences statistiques ;
- Lorsque la calibration ne permet pas d'imposer des contraintes angulaires précises.

11.12 Comparaison synoptique

	DAS	NS	MVDR	LCMV
Information requise	θ_s	θ_s, θ_i	$\theta_s, \mathbf{R}$	$\theta_s, \theta_i, \mathbf{R}$
Statistique exploitée	non	non	oui	oui
Contraintes	1 gain	gain + nulls	1 gain	multiples
Nulls	non	imposés	adaptatifs	imposés et adaptatifs
Coût en degrés libres	faible	explicite	implicite	explicite
Robustesse	élevée	moyenne	faible sans loading	réglable

11.13 Synthèse

LCMV unifie en un seul formalisme tous les beamformers étudiés en Parties II et III. Sa structure GSC en deux branches sépare clairement les contraintes (résolues une fois pour toutes) de l'adaptation (qui peut être faite récursivement). Cette dernière, en pratique, n'est presque jamais réalisée par inversion directe : elle est confiée à des algorithmes *itératifs* — LMS, NLMS, RLS — qui font l'objet de la Partie IV.

À retenir

- **Formule centrale** : $\mathbf{w} = \mathbf{R}^{-1}\mathbf{C}(\mathbf{C}^H\mathbf{R}^{-1}\mathbf{C})^{-1}\mathbf{f}$.
- **Hypothèse principale** : contraintes indépendantes et covariance suffisamment conditionnée.
- **Avantage** : combine gain utile, nulls explicites, robustesse angulaire et minimisation statistique.
- **Limite** : chaque contrainte consomme un degré de liberté.
- **Piège pratique** : trop de nulls donne des poids de grande norme et un WNG faible.
- **Suite** : la structure GSC rend l'adaptation LMS/RLS possible sans violer les contraintes.

À l'issue de cette Partie III, le lecteur dispose d'une chaîne théorique complète :

$$\text{Données} \rightarrow \hat{\mathbf{R}} \rightarrow \mathbf{R}_{\text{DL}}^{-1} \rightarrow \mathbf{w}_{\text{LCMV}} \rightarrow y(n) = \mathbf{w}^H \mathbf{x}(n).$$

Cette chaîne suppose néanmoins qu'on peut, à chaque mise à jour des poids, recalculer une covariance et l'inverser. Ce coût est acceptable quand l'environnement varie lentement (mises à jour à la seconde ou à la minute), mais devient prohibitif quand on doit suivre des brouilleurs mobiles à la milliseconde ou poursuivre des cibles dynamiques. La Partie IV substitue à cette mise à jour « en bloc » une mise à jour *réursive*, où les poids évoluent à chaque nouvel échantillon.

Le chapitre suivant pose les bases de l'adaptation par minimisation itérative : filtre de Wiener, descente de gradient, algorithme LMS.

Quatrième partie

Adaptation itérative

Chapitre 12

Wiener, LMS et NLMS

Les chapitres précédents ont fourni des *formules fermées* pour les poids des beamformers : DAS, Null Steering, MVDR, LCMV. Ces formules supposent qu'on dispose d'une covariance $\mathbf{R}$ et qu'on peut l'inverser. C'est une hypothèse confortable au plan théorique, mais qui se heurte à deux réalités opérationnelles : la covariance n'est jamais parfaitement connue (elle est estimée sur un nombre fini de snapshots), et l'inverser à chaque mise à jour coûte $\mathcal{O}(N^3)$, ce qui devient prohibitif sur les grands réseaux ou en temps réel rapide. La Partie IV répond à ces deux problèmes par des approches *itératives* qui mettent à jour les poids à chaque échantillon sans jamais former ni inverser explicitement la covariance.

En pratique, deux situations exigent cette approche différente.

1. L'environnement est *non stationnaire* : les directions et puissances des sources varient au cours du temps. Recalculer $\hat{\mathbf{R}}$ et inverser à chaque instant est trop coûteux et risqué (la fenêtre d'estimation devient trop courte).
2. On dispose d'un *signal de référence* $d(n)$ (un pilote, un préambule, un signal connu) : on peut alors poser le problème comme une minimisation directe d'erreur, sans estimer explicitement la covariance.

Le présent chapitre construit le *filtre de Wiener*, point de départ théorique, puis introduit le *Least Mean Squares* (LMS) et sa variante normalisée NLMS.

12.1 Beamformer avec référence : structure

On suppose un signal de référence $d(n)$ disponible, corrélé au signal utile. L'objectif est de choisir $\mathbf{w}$ pour que la sortie $y(n) = \mathbf{w}^H \mathbf{x}(n)$ approche $d(n)$. L'erreur est

$$e(n) = d(n) - y(n) = d(n) - \mathbf{w}^H \mathbf{x}(n).$$

Le critère MMSE (*Minimum Mean Square Error*) est

$$J(\mathbf{w}) = \mathbb{E}[|e(n)|^2] = \mathbb{E}[|d|^2] - \mathbf{w}^H \mathbf{r}_{xd} - \mathbf{r}_{xd}^H \mathbf{w} + \mathbf{w}^H \mathbf{R} \mathbf{w},$$

avec $\mathbf{R} = \mathbb{E}[\mathbf{x}\mathbf{x}^H]$ et $\mathbf{r}_{xd} = \mathbb{E}[\mathbf{x}(n) d^*(n)]$.

12.2 Solution de Wiener

Théorème 12.1 (Filtre de Wiener). *Si $\mathbf{R}$ est définie positive, le minimum de J est atteint en*

$$\mathbf{w}_\star = \mathbf{R}^{-1} \mathbf{r}_{xd}.$$

Démonstration. $\nabla_{\mathbf{w}^*} J = -\mathbf{r}_{xd} + \mathbf{R}\mathbf{w} = \mathbf{0}$ donne $\mathbf{w} = \mathbf{R}^{-1}\mathbf{r}_{xd}$. La hessienne est $\mathbf{R}$, semi-définie positive : c'est un minimum. $\square$

L'erreur minimale est

$$J_{\min} = \mathbb{E}[|d|^2] - \mathbf{r}_{xd}^H \mathbf{R}^{-1} \mathbf{r}_{xd}.$$

Lien avec MVDR. Le filtre de Wiener et MVDR ont la même structure $\mathbf{R}^{-1}(\cdot)$: MVDR est un cas particulier où la « référence » est le steering vector $\mathbf{a}(\theta_s)$, multiplié par un facteur de normalisation. La famille des beamformers optimaux est donc unifiée par ce schéma. Cette unification n'est pas qu'une jolie convergence formelle : elle dit que tout ce qu'on sait sur le filtre de Wiener (théorie du signal aléatoire, principe d'orthogonalité, prédiction linéaire) s'applique directement au beamforming. En particulier, les techniques d'accélération de convergence et de *tracking* développées en filtrage adaptatif classique se transposent telles quelles au beamforming adaptatif. La littérature des deux domaines forme un continuum bien plus qu'on ne le perçoit en les apprenant séparément.

12.3 Descente de gradient déterministe

Si l'on n'inverse pas $\mathbf{R}$, on minimise itérativement $J(\mathbf{w})$ par descente de gradient :

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \nabla_{\mathbf{w}^*} J(\mathbf{w}(n)) = \mathbf{w}(n) + \mu (\mathbf{r}_{xd} - \mathbf{R}\mathbf{w}(n)).$$

Cette récurrence converge vers $\mathbf{w}_\star = \mathbf{R}^{-1}\mathbf{r}_{xd}$ si $0 < \mu < 2/\lambda_{\max}(\mathbf{R})$.

12.4 Gradient instantané et LMS

Les espérances $\mathbf{R}$ et $\mathbf{r}_{xd}$ sont inconnues. L'idée du LMS est de les remplacer par leurs *réalisations instantanées* : $\mathbf{R} \approx \mathbf{x}(n)\mathbf{x}^H(n)$ et $\mathbf{r}_{xd} \approx \mathbf{x}(n)d^*(n)$. Le gradient instantané devient

$$\hat{\nabla} J(n) = -\mathbf{x}(n)d^*(n) + \mathbf{x}(n)\mathbf{x}^H(n)\mathbf{w}(n) = -\mathbf{x}(n)e^*(n),$$

en utilisant $e(n) = d(n) - \mathbf{w}^H(n)\mathbf{x}(n)$.

Définition 12.2 (Algorithme LMS).

$$\begin{aligned} y(n) &= \mathbf{w}^H(n)\mathbf{x}(n), \\ e(n) &= d(n) - y(n), \\ \mathbf{w}(n+1) &= \mathbf{w}(n) + \mu \mathbf{x}(n)e^*(n). \end{aligned}$$

Trois lignes par snapshot : c'est l'un des algorithmes les plus simples du traitement du signal adaptatif. Sa complexité est $\mathcal{O}(N)$ par échantillon, sans inversion matricielle, sans mémoire de covariance.

12.5 Exemple complet : beamforming supervisé par pilote

Considérons une liaison où la source utile transmet une séquence pilote connue $d(n)$. Le réseau reçoit :

$$\mathbf{x}(n) = \mathbf{a}(\theta_s)s(n) + \mathbf{a}(\theta_i)i(n) + \mathbf{n}(n),$$

avec $d(n) = s(n)$ pendant la phase d'apprentissage. Le beamformer produit :

$$y(n) = \mathbf{w}^H(n)\mathbf{x}(n),$$

puis compare cette sortie au pilote :

$$e(n) = d(n) - y(n).$$

La mise à jour :

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu \mathbf{x}(n) e^*(n)$$

a une lecture physique directe :

- si $|e(n)|$ est grand, la correction est grande ;
- si une composante de $\mathbf{x}(n)$ est forte, elle influence plus fortement la correction ;
- si μ est trop grand, les poids sur-réagissent et divergent ;
- si μ est trop petit, la convergence est stable mais lente.

LMS ne connaît pas explicitement θ_s ni θ_i . Il apprend une combinaison spatiale qui rend la sortie proche du pilote. Si le brouilleur est décorrélié du pilote, il contribue à l'erreur et l'algorithme apprend à le réduire. Si la référence est contaminée par le brouilleur, LMS apprend au contraire une mauvaise solution : la qualité de $d(n)$ est donc aussi importante que le choix de μ .

12.6 Convergence et stabilité

12.6.1 Espérance de la trajectoire

En espérance, $\mathbb{E}[\mathbf{w}(n+1)] = (\mathbf{I} - \mu \mathbf{R})\mathbb{E}[\mathbf{w}(n)] + \mu \mathbf{r}_{xd}$. La récurrence est stable si toutes les valeurs propres de $\mathbf{I} - \mu \mathbf{R}$ sont de module < 1 , soit

$$0 < \mu < \frac{2}{\lambda_{\max}(\mathbf{R})}.$$

En pratique, on borne par la trace : $\lambda_{\max} \leq \text{tr}(\mathbf{R})$, d'où la condition pratique $\mu < 2/\text{tr}(\mathbf{R})$ ou plus prudemment $\mu < 1/\text{tr}(\mathbf{R})$.

12.6.2 Constante de temps

La k -ième composante propre converge à un taux $|1 - \mu \lambda_k|^n$; la convergence globale est dominée par la plus petite valeur propre. Si $\lambda_{\min}/\lambda_{\max}$ est petit (covariance mal conditionnée), la convergence est lente. C'est la limitation majeure du LMS.

12.6.3 Misadjustment

À l'équilibre, $\mathbf{w}(n)$ fluctue autour de $\mathbf{w}_\star$ avec une variance non nulle. La perte de performance se mesure par le *misadjustment*

$$\mathcal{M} = \frac{\mathbb{E}[J(\mathbf{w}(\infty))] - J_{\min}}{J_{\min}} \approx \frac{\mu}{2} \text{tr}(\mathbf{R}).$$

On a donc un compromis : μ grand $\Rightarrow$ convergence rapide mais $\mathcal{M}$ élevé ; μ petit $\Rightarrow$ précision asymptotique mais convergence lente.

12.7 NLMS : pas adaptatif

Pour rendre μ indépendant de l'amplitude des données, on normalise par la puissance instantanée du snapshot :

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\tilde{\mu}}{\epsilon + \|\mathbf{x}(n)\|^2} \mathbf{x}(n) e^*(n),$$

avec $0 < \tilde{\mu} < 2$ (pratique : $\tilde{\mu} \approx 0.5$) et $\epsilon > 0$ petit pour éviter la division par zéro. Le NLMS est robuste aux variations d'échelle du signal, sans surcoût significatif.

12.7.1 Pourquoi la normalisation stabilise

Dans LMS, la taille réelle de la correction est :

$$\|\mu \mathbf{x}(n) e^*(n)\|.$$

Si la puissance d'entrée augmente brutalement, la correction augmente aussi, même si μ n'a pas changé. C'est dangereux dans les systèmes RF, où un brouilleur peut faire varier $\|\mathbf{x}(n)\|^2$ de plusieurs dizaines de dB.

NLMS remplace le pas fixe par un pas effectif :

$$\mu_{\text{eff}}(n) = \frac{\tilde{\mu}}{\epsilon + \|\mathbf{x}(n)\|^2}.$$

Ainsi :

$$\|\mathbf{x}(n)\|^2 \text{ grand} \Rightarrow \mu_{\text{eff}}(n) \text{ petit}.$$

La correction reste de taille contrôlée. C'est pourquoi NLMS est souvent préféré au LMS pur lorsque la puissance d'entrée varie fortement.

12.8 Application au beamforming

12.8.1 Mode supervisé

Si l'on dispose d'un *pilote* ou *préambule* connu $d(n)$, LMS peut entraîner directement le beamformer pour pointer le signal utile et rejeter le reste. Architecture classique en télécommunications.

12.8.2 Mode constant-modulus

Sans pilote, on peut imposer $|y(n)| = 1$ (*Constant Modulus Algorithm*, CMA) pour des modulations à enveloppe constante (PSK). Critère : $J_{\text{CMA}}(\mathbf{w}) = \mathbb{E}[(|y(n)|^2 - 1)^2]$.

12.8.3 Mode LCMV adaptatif via GSC

L'architecture GSC (chapitre 11) sépare $\mathbf{w}_q$ (fixe) et $\mathbf{w}_a$ (adaptatif). Le signal d'erreur devient $e(n) = y_q(n) - \mathbf{w}_a^H(n) \mathbf{z}(n)$, où $\mathbf{z}(n) = \mathbf{B}^H \mathbf{x}(n)$ est la sortie de la matrice de blocage. LMS s'applique sur $\mathbf{w}_a$ sans contraintes, et l'ensemble réalise un LCMV adaptatif.

12.9 Limites du LMS

Convergence lente sur covariance mal conditionnée. Un brouilleur très au-dessus du bruit peut créer un rapport propre $\lambda_{\text{max}}/\lambda_{\text{min}}$ de plusieurs ordres de grandeur : LMS peut alors demander un nombre d'itérations du même ordre que ce conditionnement. RLS (chapitre suivant) corrige largement ce défaut.

Choix de μ . Sensible au régime de signal. NLMS atténue le problème mais ne l'élimine pas.

Sensibilité aux signaux corrélés. Lorsque $d(n)$ est fortement corrélé à des composantes de $\mathbf{x}(n)$, le gradient instantané est très bruité.

Pas de modèle structurel. Contrairement à MVDR / LCMV, LMS n'incorpore pas de contraintes angulaires. Le *Constrained LMS* ou la structure GSC sont nécessaires pour des contraintes explicites.

12.10 Algorithmes apparentés

Affine Projection Algorithm (APA). Compromis entre LMS et RLS : on utilise les L derniers snapshots pour calculer un gradient moins bruité que LMS, à coût intermédiaire entre $\mathcal{O}(N)$ et $\mathcal{O}(N^2)$.

Leaky LMS. Ajoute un terme de régularisation $-\mu\gamma\mathbf{w}(n)$ à la mise à jour, ce qui empêche $\mathbf{w}$ de dériver dans le noyau de $\mathbf{R}$ (utile en régime *persistent excitation* faible).

Sign-LMS / Sign-Error LMS. Remplace $e(n)$ par $\text{sgn}(e(n))$ pour économiser des multiplications en FPGA.

Block LMS. Mise à jour à partir d'un bloc de L échantillons — la sortie est calculée par FFT, ce qui devient efficace pour de grandes fenêtres temporelles.

12.11 Implémentation numérique

Listing 12.1 – LMS et NLMS pour beamforming supervisé

```

1  import numpy as np
2
3  def lms(X, d, mu, w0=None):
4      """
5      X : (N, K) snapshots
6      d : (K,) référence (complexe)
7      mu : pas
8      """
9      N, K = X.shape
10     w = np.zeros(N, dtype=complex) if w0 is None else w0.astype(complex).copy()
11     err = np.zeros(K, dtype=complex)
12     for n in range(K):
13         x = X[:, n]
14         y = w.conj() @ x
15         e = d[n] - y
16         w += mu * x * np.conj(e)
17         err[n] = e
18     return w, err
19
20 def nlms(X, d, mu_tilde=0.5, eps=1e-6, w0=None):
21     N, K = X.shape
22     w = np.zeros(N, dtype=complex) if w0 is None else w0.astype(complex).copy()
23     err = np.zeros(K, dtype=complex)
24     for n in range(K):
25         x = X[:, n]

```

```

26     y = w.conj() @ x
27     e = d[n] - y
28     norm = eps + (x.conj() @ x).real
29     w += (mu_tilde / norm) * x * np.conj(e)
30     err[n] = e
31     return w, err

```

12.12 Synthèse

LMS et NLMS sont les algorithmes adaptatifs de beamforming les plus simples et les plus utilisés :

- complexité $\mathcal{O}(N)$ par snapshot ;
- pas de covariance à estimer ni à inverser ;
- convergence garantie (en moyenne) si $\mu < 2/\lambda_{\max}$;
- compromis μ / vitesse de convergence / misadjustment.

Leur principale limitation est la convergence lente sur des environnements à grande disparité de puissances. Le chapitre suivant introduit le RLS, qui converge souvent en un nombre de snapshots de l'ordre de quelques N lorsque les hypothèses numériques sont favorables, au prix d'une complexité $\mathcal{O}(N^2)$.

Le compromis LMS / RLS / SMI résume bien la triade de tout traitement adaptatif : on peut être *simple*, *rapide*, *précis*, mais généralement seulement deux des trois. LMS est simple et rapide (par échantillon) mais peu précis (convergence lente). SMI (Sample Matrix Inversion, vu en Partie III) est précis et rapide en convergence lorsque K est de l'ordre de quelques N , mais coûteux par mise à jour. RLS est précis et rapide en convergence, au prix d'une complexité intermédiaire. Le choix entre les trois dépend des contraintes matérielles et dynamiques de l'application visée.

Chapitre 13

RLS et facteur d'oubli

LMS converge mais lentement, et sa lenteur est dictée par le conditionnement de la covariance. Quand l'environnement contient un brouilleur 40 ou 60 dB au-dessus du bruit, $\kappa(\mathbf{R})$ peut devenir très grand et rendre LMS trop lent pour l'application : on peut attendre des milliers, voire des dizaines de milliers d'itérations pour converger, ce qui dépasse parfois la durée pendant laquelle l'environnement reste stationnaire. Le *Recursive Least Squares* (RLS) réduit fortement ce problème : dans des conditions favorables, il converge souvent en un nombre de snapshots de l'ordre de quelques N , avec une dépendance beaucoup plus faible au conditionnement que LMS. Le prix à payer est une complexité $\mathcal{O}(N^2)$ par snapshot et une mémoire plus importante.

RLS est, à bien des égards, un *LMS avec mémoire structurée*. Au lieu de moyennner les gradients via un pas μ très petit pour ne pas sur-réagir au bruit, il garde explicitement la trace d'une covariance inverse pondérée par un facteur d'oubli λ . La descente n'est plus aveugle : elle est conditionnée par la *forme* de la covariance, ce qui permet d'avancer plus vite dans les directions où la variance est élevée. C'est l'équivalent stochastique d'une descente de Newton (qui utilise la hessienne) par opposition à une descente de gradient (qui ne l'utilise pas) — et c'est précisément pourquoi RLS réduit fortement l'effet du conditionnement sur la vitesse de convergence.

13.1 Critère des moindres carrés récurrents

Plutôt qu'une espérance, RLS minimise une somme empirique pondérée des erreurs passées :

$$J(n) = \sum_{i=1}^n \lambda^{n-i} |e(i)|^2 = \sum_{i=1}^n \lambda^{n-i} |d(i) - \mathbf{w}^H \mathbf{x}(i)|^2,$$

où $\lambda \in (0, 1]$ est le *facteur d'oubli* (*forgetting factor*). Pour $\lambda = 1$, toutes les erreurs ont le même poids : on minimise globalement les moindres carrés. Pour $\lambda < 1$, les échantillons anciens sont oubliés exponentiellement, ce qui permet de suivre des environnements non stationnaires.

13.2 Forme matricielle

Définissons

$$\mathbf{R}(n) = \sum_{i=1}^n \lambda^{n-i} \mathbf{x}(i) \mathbf{x}^H(i), \quad \mathbf{r}(n) = \sum_{i=1}^n \lambda^{n-i} \mathbf{x}(i) d^*(i).$$

Le minimum est

$$\mathbf{w}(n) = \mathbf{R}^{-1}(n) \mathbf{r}(n).$$

On retrouve la structure de Wiener, mais sur des estimations *pondérées dans le temps*.

13.3 Récurrence des grandeurs

L'estimateur se construit récursivement :

$$\mathbf{R}(n) = \lambda \mathbf{R}(n-1) + \mathbf{x}(n)\mathbf{x}^H(n), \quad \mathbf{r}(n) = \lambda \mathbf{r}(n-1) + \mathbf{x}(n) d^*(n).$$

Calculer $\mathbf{R}^{-1}(n)$ à chaque pas coûte $\mathcal{O}(N^3)$: trop cher pour une boucle temps réel. L'astuce est d'utiliser le *lemme d'inversion matricielle*.

13.4 Lemme d'inversion matricielle

Lemme 13.1 (Sherman–Morrison). *Pour $\mathbf{A}$ inversible et des vecteurs $\mathbf{u}, \mathbf{v}$,*

$$(\mathbf{A} + \mathbf{u}\mathbf{v}^H)^{-1} = \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1}\mathbf{u}\mathbf{v}^H\mathbf{A}^{-1}}{1 + \mathbf{v}^H\mathbf{A}^{-1}\mathbf{u}}.$$

Posons $\mathbf{P}(n) = \mathbf{R}^{-1}(n)$ et $\mathbf{A} = \lambda \mathbf{R}(n-1)$, $\mathbf{u} = \mathbf{v} = \mathbf{x}(n)$. Le lemme donne directement

$$\mathbf{P}(n) = \frac{1}{\lambda} \left(\mathbf{P}(n-1) - \frac{\mathbf{P}(n-1)\mathbf{x}(n)\mathbf{x}^H(n)\mathbf{P}(n-1)}{\lambda + \mathbf{x}^H(n)\mathbf{P}(n-1)\mathbf{x}(n)} \right).$$

13.5 Gain RLS et mise à jour des poids

On définit le *gain RLS*

$$\mathbf{k}(n) = \frac{\mathbf{P}(n-1)\mathbf{x}(n)}{\lambda + \mathbf{x}^H(n)\mathbf{P}(n-1)\mathbf{x}(n)}.$$

On peut alors écrire la mise à jour de manière compacte :

$\begin{aligned} \alpha(n) &= d(n) - \mathbf{w}^H(n-1)\mathbf{x}(n) \quad (\text{erreur a priori}) \\ \mathbf{w}(n) &= \mathbf{w}(n-1) + \mathbf{k}(n)\alpha^*(n) \\ \mathbf{P}(n) &= \frac{1}{\lambda} \left(\mathbf{P}(n-1) - \mathbf{k}(n)\mathbf{x}^H(n)\mathbf{P}(n-1) \right). \end{aligned}$
--

13.6 Algorithme RLS complet

1. Initialisation : $\mathbf{w}(0) = \mathbf{0}$, $\mathbf{P}(0) = \delta^{-1}\mathbf{I}_N$ avec $\delta > 0$ petit (typiquement $\delta = 10^{-2}\sigma_x^2$).
2. Pour chaque snapshot $n = 1, 2, \dots$:
 - (a) calculer $\mathbf{u}(n) = \mathbf{P}(n-1)\mathbf{x}(n)$;
 - (b) calculer $\mathbf{k}(n) = \mathbf{u}(n)/(\lambda + \mathbf{x}^H(n)\mathbf{u}(n))$;
 - (c) calculer l'erreur a priori $\alpha(n) = d(n) - \mathbf{w}^H(n-1)\mathbf{x}(n)$;
 - (d) mettre à jour $\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{k}(n)\alpha^*(n)$;
 - (e) mettre à jour $\mathbf{P}(n) = (\mathbf{P}(n-1) - \mathbf{k}(n)\mathbf{u}^H(n))/\lambda$.

Complexité : $\mathcal{O}(N^2)$ multiplications par snapshot.

13.7 Rôle du facteur d'oubli

- $\lambda = 1$: pas d'oubli, RLS converge globalement vers la solution moindres carrés. Adapté aux environnements stationnaires.

- λ proche de 1 (typique 0.95–0.999) : oubli lent, RLS suit des variations lentes de l'environnement.
- λ plus bas (0.9 par exemple) : oubli rapide, suivi agressif au prix d'une variance d'estimation plus élevée.

La *mémoire effective* de RLS est $1/(1 - \lambda)$ snapshots. Par exemple $\lambda = 0.98$ donne une mémoire ~ 50 snapshots.

Choix. En pratique on règle λ pour que la mémoire effective soit légèrement supérieure à la durée de stationnarité de l'environnement, mais pas tellement plus, faute de quoi RLS ne suivra pas les évolutions.

13.8 Initialisation de P

Le choix de $P(0) = \delta^{-1} \mathbf{I}_N$ joue le rôle d'un *prior bayésien* : il revient à régulariser le problème par $\delta \|\mathbf{w}\|^2$. Un δ trop petit donne une convergence initiale instable ; trop grand, une convergence lente. Une heuristique courante : $\delta = 0.01 \cdot \text{trace}(\mathbf{R}_x)/N$ ou simplement $\delta = 10^{-2}$ après normalisation des données.

13.9 Comparaison LMS / NLMS / RLS

Critère	LMS	NLMS	RLS
Coût / snapshot	faible	faible	élevé
Complexité	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\mathcal{O}(N^2)$
Mémoire	$\mathcal{O}(N)$	$\mathcal{O}(N)$	$\mathcal{O}(N^2)$
Convergence	lente	moyenne	rapide
Robustesse numérique	bonne	bonne	plus délicate
Réglage	μ	$\tilde{\mu}, \epsilon$	λ, δ
Puissance d'entrée variable	sensible	robuste	robuste mais coûteux
Usage typique	temps réel simple	puissance variable	environnements rapides

En règle générale : LMS pour la simplicité maximale, NLMS lorsque la puissance d'entrée varie fortement, RLS lorsque l'environnement change vite ou lorsque la covariance est trop mal conditionnée pour que LMS converge à temps.

13.10 RLS et MVDR

RLS estime implicitement $\mathbf{R}^{-1}$ via $P(n)$. On peut l'utiliser pour construire *en ligne* un beamformer MVDR :

$$\mathbf{w}_{\text{MVDR}}(n) = \frac{P(n) \mathbf{a}(\theta_s)}{\mathbf{a}^H(\theta_s) P(n) \mathbf{a}(\theta_s)}.$$

Cette version récursive de MVDR est très utilisée en radar adaptatif et en télécommunications : pas de matrice à inverser explicitement, adaptation continue à l'environnement.

13.11 RLS sous contraintes (CRLS)

Pour intégrer LCMV dans une boucle RLS, on utilise la structure GSC (chapitre 11) :

- la partie fixe $\mathbf{w}_q$ satisfait les contraintes une fois pour toutes ;

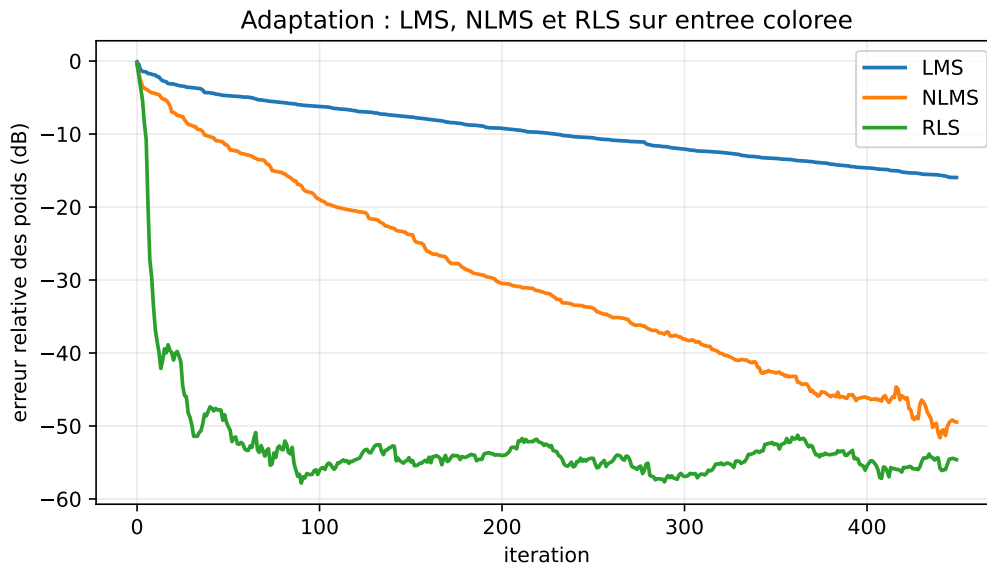


FIGURE 13.1 – Simulation reproductible sur entrée colorée ($N = 8$, 450 itérations, spectre propre de 30 à 1, RLS $\lambda = 0.99$) : LMS progresse lentement, NLMS stabilise le pas, RLS converge plus vite au prix d'un coût et d'une fragilité numérique plus élevés.

— la partie adaptative $\mathbf{w}_a$ minimise $\mathbb{E}[|y_q - \mathbf{w}_a^H \mathbf{z}|^2]$, sans contraintes, par RLS sur $\mathbf{z}(n) = \mathbf{B}^H \mathbf{x}(n)$.

On obtient un *Constrained RLS* (CRLS) sans surcoût important.

13.12 Stabilité numérique

Le lemme d'inversion peut, en arithmétique flottante, perdre la symétrie hermitienne de $\mathbf{P}$ et la définir positivité. Conséquence : divergence numérique pour de longues simulations. Deux remèdes :

Forme symétrisée. À chaque pas, on impose $\mathbf{P}(n) \leftarrow (\mathbf{P}(n) + \mathbf{P}^H(n))/2$. Solution simple mais imparfaite.

Forme racine carrée (square-root RLS). On manipule une factorisation de Cholesky $\mathbf{P}(n) = \mathbf{L}(n)\mathbf{L}^H(n)$ plutôt que $\mathbf{P}$ directement. Stable mais plus complexe à implémenter. Référence essentielle en aéronautique et défense.

QR-RLS. Utilisation de rotations de Givens sur les échantillons : numériquement très stable, parallélisable sur FPGA, mais hors du cadre de ce chapitre.

13.13 Implémentation numérique

Listing 13.1 – RLS pour beamforming supervisé

```

1 import numpy as np
2
3 def rls(X, d, lam=0.99, delta=1e-2):
4     """
5     X      : (N, K) snapshots
6     d      : (K,) référence

```



```

7   lam    : facteur d'oubli (0 < lam <= 1)
8   delta  : initialisation de P
9   """
10  N, K = X.shape
11  w = np.zeros(N, dtype=complex)
12  P = np.eye(N, dtype=complex) / delta
13  err = np.zeros(K, dtype=complex)
14  for n in range(K):
15      x = X[:, n]
16      u = P @ x
17      gain = u / (lam + np.real(x.conj() @ u))
18      alpha = d[n] - w.conj() @ x
19      w = w + gain * np.conj(alpha)
20      P = (P - np.outer(gain, x.conj() @ P)) / lam
21      # symétrisation hermitienne
22      P = 0.5 * (P + P.conj().T)
23      err[n] = alpha
24  return w, err

```

13.14 Synthèse

RLS converge souvent beaucoup plus vite que LMS, typiquement en un nombre de snapshots de l'ordre de quelques N dans les cas bien conditionnés ou correctement régularisés. Le prix est une complexité $\mathcal{O}(N^2)$ et une sensibilité numérique non négligeable. Il est l'outil de choix pour le suivi rapide d'environnements non stationnaires lorsque LMS converge trop lentement.

À ce stade, on dispose de toute la palette des beamformers adaptatifs :

- formules fermées (DAS, MVDR, LCMV) ;
- algorithmes itératifs (LMS, NLMS, RLS).

La Partie V change de problème : on ne cherche plus à former un faisceau mais à *estimer les directions* elles-mêmes, à partir des mêmes données. C'est le passage du *filtrage spatial* à l'*estimation paramétrique spatiale*.

Ce changement d'objectif n'est pas qu'un raffinement supplémentaire : c'est un saut conceptuel. Là où les Parties II-IV cherchaient à *filtrer* (produire un signal aussi propre que possible), la Partie V cherche à *mesurer* (produire un angle aussi précis que possible). Les deux tâches sont liées — on n'aura jamais un bon filtre sans bonne mesure — mais leurs critères de qualité sont différents. Un filtre se juge à son SINR de sortie ; un estimateur de DOA se juge à la variance de l'angle estimé, idéalement comparée à la borne de Cramér-Rao. La Partie V introduira d'ailleurs cette borne comme étalon universel des estimateurs de direction.

Cinquième partie

Estimation de direction d'arrivée

Chapitre 14

Décomposition en sous-espaces et détection d'ordre

Le beamforming consiste à *former* un faisceau. L'estimation de direction d'arrivée (DOA) consiste à *trouver* d'où viennent les sources. La transition entre les deux mondes passe par une observation algébrique remarquable de la covariance spatiale : ses vecteurs propres se séparent naturellement en un sous-espace *signal* et un sous-espace *bruit*. Cette structure, exploitée par MUSIC, ESPRIT et leurs variantes, autorise des résolutions bien supérieures à la limite de Rayleigh du beamforming conventionnel.

Cette idée mérite qu'on s'y arrête. La limite de Rayleigh — $\Delta u_{\text{res}} \sim \lambda/D$ — est une limite *déterministe* fondée sur la largeur du lobe principal d'une ouverture finie. Elle ne dépend ni du SNR ni du nombre de snapshots : même avec un SNR infini, le DAS et MVDR ne séparent pas deux sources plus proches que cette limite. Les méthodes de sous-espaces, elles, ne font pas de cette limite un mur infranchissable. Elles l'exploitent plus subtilement : pour deux sources même très proches, leurs steering vectors restent *distinctes* en tant que vecteurs de $\mathbb{C}^N$, et donc le sous-espace signal a bien dimension 2 (et non 1). La résolution effective dépend alors de la qualité d'estimation de ce sous-espace, qui s'améliore avec K et avec le SNR. C'est ce qui permet, dans le régime favorable, de séparer des sources à un dixième ou un centième de la limite de Rayleigh.

Ce chapitre formalise la décomposition, étudie les sous-espaces associés et discute la détection du nombre de sources (*model order selection*), problème préalable à toute estimation de DOA par sous-espaces.

14.1 Modèle multi-sources

Le modèle de référence est

$$\mathbf{x}(n) = \mathbf{A}(\boldsymbol{\theta}) \mathbf{s}(n) + \mathbf{n}(n),$$

avec :

- $\mathbf{A}(\boldsymbol{\theta}) = [\mathbf{a}(\theta_1), \dots, \mathbf{a}(\theta_L)] \in \mathbb{C}^{N \times L}$ matrice de steering ;
- $\mathbf{s}(n) \in \mathbb{C}^L$ vecteur des enveloppes des sources, de covariance $\mathbf{P}_s = \mathbb{E}[\mathbf{s}\mathbf{s}^H]$;
- $\mathbf{n}(n) \in \mathbb{C}^N$ bruit blanc spatial, de covariance $\sigma^2 \mathbf{I}_N$ et décorrélié du signal.

Hypothèses techniques.

- (H1) Champ lointain : modèle d'onde plane valide.
- (H2) Bande étroite : steering vector indépendant de la fréquence.
- (H3) $L < N$: moins de sources que d'antennes (sinon identifiabilité compromise).
- (H4) $\mathbf{A}$ de rang plein colonne L (sources angulairement distinctes).

(H5) $\mathbf{P}_s$ de rang plein (sources non parfaitement cohérentes ; cas dégénéré traité plus loin).

(H6) Bruit blanc spatial, de variance connue ou estimable.

14.2 Structure de la covariance

Théorème 14.1 (Décomposition signal/bruit). *Sous les hypothèses (H1)–(H6), la covariance*

$$\mathbf{R} = \mathbf{A} \mathbf{P}_s \mathbf{A}^H + \sigma^2 \mathbf{I}_N$$

admet la décomposition spectrale

$$\mathbf{R} = \sum_{k=1}^L \lambda_k \mathbf{u}_k \mathbf{u}_k^H + \sum_{k=L+1}^N \sigma^2 \mathbf{u}_k \mathbf{u}_k^H = \mathbf{U}_s \mathbf{\Lambda}_s \mathbf{U}_s^H + \sigma^2 \mathbf{U}_n \mathbf{U}_n^H,$$

où :

- $\mathbf{U}_s = [\mathbf{u}_1, \dots, \mathbf{u}_L]$ engendre le sous-espace signal, de dimension L ;
- $\mathbf{U}_n = [\mathbf{u}_{L+1}, \dots, \mathbf{u}_N]$ engendre le sous-espace bruit, de dimension $N - L$;
- $\lambda_1 \geq \dots \geq \lambda_L > \sigma^2$;
- $\lambda_{L+1} = \dots = \lambda_N = \sigma^2$.

De plus,

$$\text{Im}(\mathbf{U}_s) = \text{Im}(\mathbf{A}),$$

les colonnes de $\mathbf{A}$ et celles de $\mathbf{U}_s$ engendrent le même sous-espace de $\mathbb{C}^N$.

Démonstration. $\mathbf{A} \mathbf{P}_s \mathbf{A}^H$ est hermitienne semi-définie positive de rang L (composition de rang L). Ses L valeurs propres non nulles μ_k sont positives ; les autres sont nulles. En ajoutant $\sigma^2 \mathbf{I}_N$, chaque valeur propre est décalée de σ^2 : les L premières deviennent $\mu_k + \sigma^2 > \sigma^2$ et les $N - L$ dernières deviennent exactement σ^2 . Les vecteurs propres restent inchangés (ils diagonalisent $\mathbf{I}_N$ aussi). Le sous-espace des L premiers vecteurs propres coïncide avec $\text{Im}(\mathbf{A} \mathbf{P}_s \mathbf{A}^H) = \text{Im}(\mathbf{A})$ par (H4)-(H5). $\square$

14.3 Orthogonalité fondamentale

Corollaire 14.2 (Orthogonalité signal-bruit). *Pour tout $\ell = 1, \dots, L$,*

$$\mathbf{U}_n^H \mathbf{a}(\theta_\ell) = \mathbf{0}.$$

C'est la pierre angulaire de l'algorithme MUSIC. Le steering vector de toute vraie source est orthogonal au sous-espace bruit ; tout candidat θ qui n'est pas une source ne le sera pas. Faire varier θ et chercher les angles minimisant $\|\mathbf{U}_n^H \mathbf{a}(\theta)\|$ revient à localiser les sources.

14.4 Sources cohérentes et rang réduit

L'hypothèse (H5) $\mathbf{P}_s$ de rang plein est essentielle. En trajets multiples, deux sources peuvent être cohérentes : leur $\mathbf{P}_s$ devient singulière, $\mathbf{A} \mathbf{P}_s \mathbf{A}^H$ est de rang $L' < L$, et la décomposition signal/bruit identifie L' sources au lieu de L . MUSIC voit alors moins de pics qu'il n'y a de directions réelles.

Spatial smoothing. L'astuce consiste à diviser le réseau en *sous-réseaux décalés* et à moyenner les covariances. La décorrélation effective entre sources est ainsi rétablie au prix d'une ouverture effective réduite. Cette technique sera détaillée au chapitre 15.

14.5 Estimation pratique

En pratique on ne dispose que de $\hat{\mathbf{R}}$ (estimée sur K snapshots), et la décomposition n'est pas *exactement* celle du théorème 14.1 : les valeurs propres bruit ne sont plus exactement égales à σ^2 , elles fluctuent autour. Les vecteurs propres sont perturbés à hauteur de $\mathcal{O}(1/\sqrt{K})$. On sépare néanmoins les deux sous-espaces en triant les valeurs propres par ordre décroissant et en cherchant un « saut » dans le spectre propre.

14.6 Détection du nombre de sources

Estimer L correctement est crucial : trop faible, on rate des sources ; trop élevé, on ajoute des sources fantômes. Trois familles de critères existent.

14.6.1 Critère de seuil

Le plus simple : on classe les valeurs propres par ordre décroissant, et on compte celles qui dépassent un seuil $\tau\sigma^2$ avec $\tau > 1$ (typiquement $\tau = 2$ ou 3). Robuste mais peu sélectif.

14.6.2 AIC (Akaike Information Criterion)

Pour la détection du nombre de sources en array processing, les formes AIC et MDL utilisées ici suivent la formulation classique de Wax et Kailath [11].

$$\text{AIC}(\ell) = -2K(N - \ell) \log \left(\frac{(\prod_{k=\ell+1}^N \hat{\lambda}_k)^{1/(N-\ell)}}{\frac{1}{N-\ell} \sum_{k=\ell+1}^N \hat{\lambda}_k} \right) + 2\ell(2N - \ell).$$

On choisit $L = \arg \min_{\ell} \text{AIC}(\ell)$. AIC tend à sur-estimer L asymptotiquement.

14.6.3 MDL (Minimum Description Length)

$$\text{MDL}(\ell) = -K(N - \ell) \log \left(\frac{(\prod_{k=\ell+1}^N \hat{\lambda}_k)^{1/(N-\ell)}}{\frac{1}{N-\ell} \sum_{k=\ell+1}^N \hat{\lambda}_k} \right) + \frac{1}{2}\ell(2N - \ell) \log K.$$

MDL est *consistante* : la probabilité d'erreur tend vers 0 quand $K \rightarrow \infty$. Préférée en pratique.

14.6.4 Critères modernes

Pour les petits K/N , les critères classiques échouent. Des extensions à random matrix theory (*Marčenko–Pastur*) ou des techniques de *bootstrap* sur les valeurs propres donnent des estimateurs plus robustes. La théorie des grandes matrices aléatoires fournit des seuils explicites pour distinguer une vraie source d'un artefact d'estimation : c'est le *spike model*.

14.7 Vecteurs propres : interprétation géométrique

Les vecteurs propres du sous-espace signal ne sont pas, en général, proportionnels aux steering vectors individuels $\mathbf{a}(\theta_{\ell})$. Ils sont des *combinaisons linéaires* de ces steering vectors, adaptées à $\mathbf{P}_s$. C'est précisément pour cela que MUSIC ne lit pas les vecteurs propres directement : il cherche les directions *orthogonales* au sous-espace bruit, qui sont nécessairement les $\mathbf{a}(\theta_{\ell})$.

14.8 Compromis du modèle

La structure de sous-espaces dépend lourdement des hypothèses (H1)–(H6). Une violation a des conséquences :

- bruit *non blanc spatialement* : les valeurs propres bruit ne sont plus toutes égales à σ^2 , la séparation se floute. Solution : *prewhitening* par la covariance du bruit estimée sur des intervalles « signal absent ».
- sources *cohérentes* : rang signal réduit. Solution : spatial smoothing.
- nombre de *snapshots* $K \sim N$: sous-espaces estimés très imprécis. Solution : régularisation, fusion de techniques (beamforming + sous-espaces).
- *calibration* imparfaite : steering vectors faussés, sources non orthogonales au sous-espace bruit estimé. Solution : chapitre 22.

14.9 Implémentation numérique

Listing 14.1 – Décomposition signal/bruit et détection d'ordre

```

1 import numpy as np
2
3 def signal_noise_subspace(R, L):
4     """Renvoie U_s, U_n, eigvals (décroissants)."""
5     eigvals, eigvecs = np.linalg.eigh(R)
6     idx = np.argsort(eigvals)[::-1]
7     eigvals = eigvals[idx]
8     eigvecs = eigvecs[:, idx]
9     U_s = eigvecs[:, :L]
10    U_n = eigvecs[:, L:]
11    return U_s, U_n, eigvals
12
13 def estimate_L_mdl(eigvals, K):
14     """Estime L par critère MDL. eigvals décroissants."""
15     N = len(eigvals)
16     mdl = np.zeros(N)
17     for ell in range(N):
18         tail = eigvals[ell:]
19         geo = np.prod(tail) ** (1.0 / (N - ell))
20         ari = np.mean(tail)
21         if geo <= 0 or ari <= 0:
22             mdl[ell] = np.inf
23             continue
24         log_ratio = np.log(ari / geo)
25         mdl[ell] = K * (N - ell) * log_ratio + 0.5 * ell * (2 * N - ell) * np.log(K)
26     return int(np.argmin(mdl))

```

14.10 Synthèse

La décomposition signal/bruit de la covariance fournit deux sous-espaces orthogonaux, dont la structure encode entièrement la position des sources. MUSIC exploite l'orthogonalité au sous-espace bruit ; ESPRIT exploite une invariance de translation à l'intérieur du sous-espace signal. La détection d'ordre par MDL ou AIC est préalable indispensable.

Les chapitres suivants approfondissent successivement MUSIC, ESPRIT, puis les bornes de Cramér-Rao qui fixent les performances ultimes de tout estimateur de DOA. Cette progression est volontairement adossée à l'ordre historique : MUSIC en 1979/86, ESPRIT en 1986, CRB

formalisée pour la DOA par Stoica et Nehorai [13]. Les méthodes parcimonieuses et bayésiennes du chapitre 18 arriveront plus tard (à partir des années 2000) avec l'essor du *compressed sensing*, et compléteront le tableau dans des régimes que MUSIC et ESPRIT ne traitent pas bien.

Chapitre 15

MUSIC et Root-MUSIC

L'algorithme MUSIC (*Multiple Signal Classification*, Schmidt [9]) exploite directement l'orthogonalité entre les steering vectors des sources et le sous-espace bruit (corollaire 14.2). Il a marqué un tournant dans l'estimation de direction d'arrivée : pour la première fois, on dépassait significativement la limite de Rayleigh du beamforming conventionnel sans hypothèse paramétrique forte sur les sources.

L'élégance de MUSIC tient à son utilisation *minimaliste* de l'information disponible. Il n'exploite pas la totalité du sous-espace signal — ce que ferait par exemple le maximum de vraisemblance — mais seulement *l'orthogonalité* à son complément. Cette information est, en un certain sens, la « part propre » de la covariance qui ne dépend pas de la distribution des puissances entre sources. Dans le modèle asymptotique idéal, cela rend MUSIC peu sensible aux puissances relatives entre sources. En données finies, une source faible reste néanmoins plus difficile à extraire : elle réduit l'eigengap, rend le sous-espace bruit moins stable, et demande plus de snapshots ou de SNR.

15.1 Idée fondamentale

Pour tout angle θ correspondant à une vraie source, $\mathbf{a}(\theta) \in \text{Im}(\mathbf{A}) = \text{Im}(\mathbf{U}_s)$, donc

$$\mathbf{U}_n^H \mathbf{a}(\theta) = \mathbf{0} \iff \|\mathbf{U}_n^H \mathbf{a}(\theta)\|^2 = 0.$$

Pour tout autre θ , cette quantité est non nulle. En faisant varier θ sur la zone visible et en cherchant les minima — c'est-à-dire les maxima de l'inverse — on localise les sources.

15.2 Pseudo-spectre

On définit le *pseudo-spectre MUSIC*

$$P_{\text{MUSIC}}(\theta) = \frac{1}{\mathbf{a}^H(\theta) \mathbf{U}_n \mathbf{U}_n^H \mathbf{a}(\theta)}.$$

$P_{\text{MUSIC}}(\theta)$ tend vers l'infini (en théorie) aux angles des sources, et reste fini ailleurs. L'estimation consiste à détecter les L pics les plus prononcés.

Pseudo et non spectre. Ce n'est pas un spectre de puissance au sens physique : la valeur de P_{MUSIC} n'a pas de dimension. C'est un *indicateur de proximité* entre $\mathbf{a}(\theta)$ et le sous-espace signal. On le trace en dB pour mieux distinguer les pics.

15.3 Algorithme

1. Collecter K snapshots $\mathbf{x}(1), \dots, \mathbf{x}(K)$.
2. Estimer $\hat{\mathbf{R}} = \frac{1}{K} \mathbf{X} \mathbf{X}^H$.
3. Décomposer en valeurs et vecteurs propres $\hat{\mathbf{R}} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^H$.
4. Estimer $\hat{L}$ (MDL, AIC, seuil ou connaissance *a priori*).
5. Extraire $\hat{\mathbf{U}}_n$: les $N - \hat{L}$ vecteurs propres associés aux plus petites valeurs propres.
6. Évaluer $P_{\text{MUSIC}}(\theta)$ sur une grille fine.
7. Détecter les $\hat{L}$ pics les plus prononcés.

15.4 Résolution angulaire

MUSIC peut dépasser la limite de Rayleigh lorsque :

- le SNR est suffisamment élevé ;
- le nombre de snapshots est suffisant ;
- la calibration est correcte.

La *résolution effective* de MUSIC dépend de $\sqrt{K \cdot \text{SNR}}$, contrairement à Capon (qui dépend de SNR seul) et au DAS (limite fixe). À très haut SNR ou très grand K , MUSIC peut séparer des sources que Capon ne sépare pas. En revanche, à faible K , faible SNR, mauvais ordre de modèle ou mismatch de steering vector, ses pics peuvent disparaître, se dédoubler ou se biaiser.

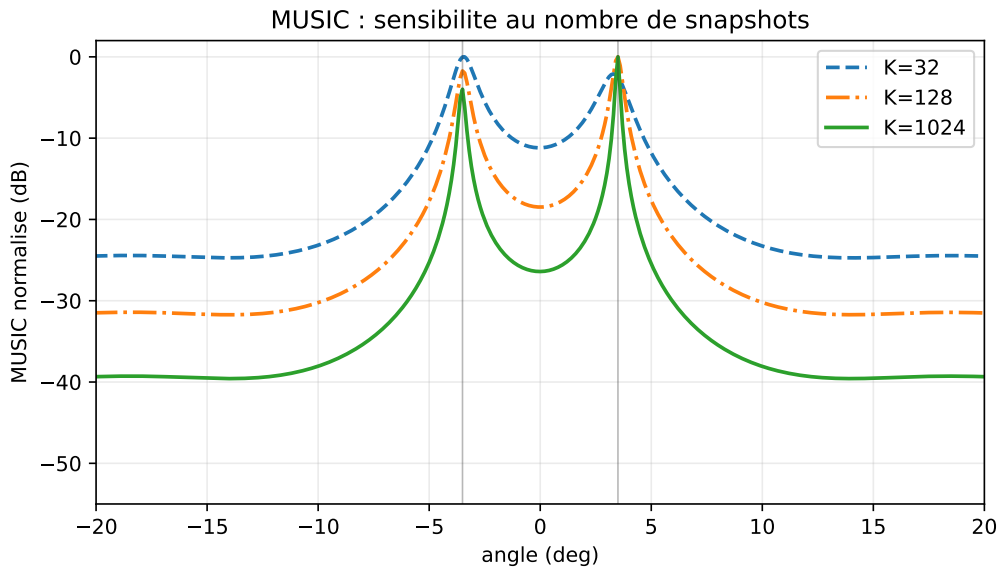


FIGURE 15.1 – Simulation MUSIC ($N = 12$, sources à $\pm 3.5^\circ$, SNR 8 dB, $K = 32, 128, 1024$) : deux sources proches deviennent beaucoup plus lisibles quand le nombre de snapshots augmente. Cette figure illustre une tendance, pas une garantie universelle.

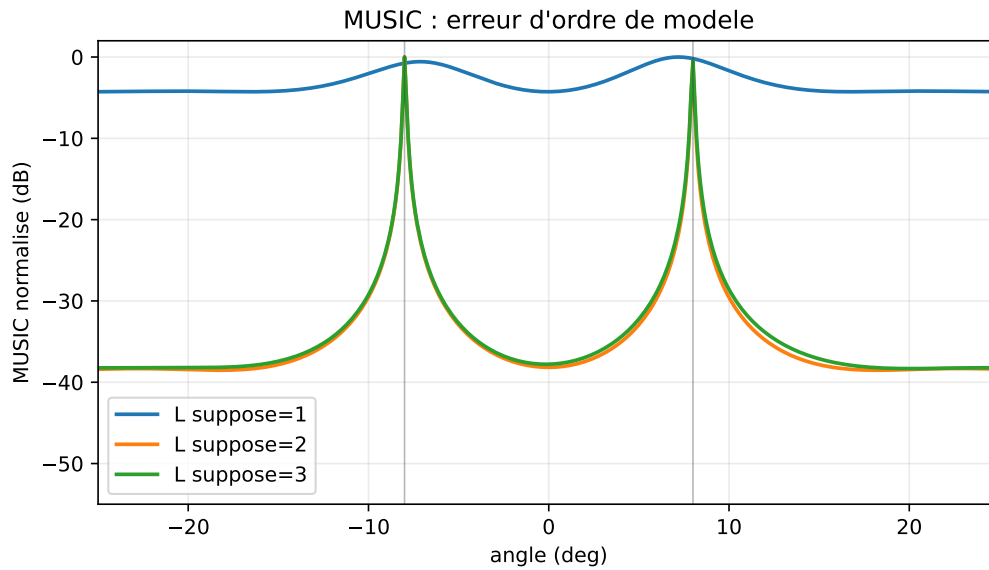


FIGURE 15.2 – MUSIC avec erreur d'ordre de modèle ($N = 12$, deux sources à $\pm 8^\circ$, SNR 12 dB, $K = 512$) : sous-estimer L fusionne les sources, le surestimer introduit des pics parasites ou instables.

15.5 Comparaison Bartlett, Capon, MUSIC

Critère	Bartlett (DAS)	Capon	MUSIC
Information requise	$\mathbf{R}$	$\mathbf{R}^{-1}$	$\mathbf{U}_n$
Coût	$\mathcal{O}(NM)$	$\mathcal{O}(N^3 + NM)$	$\mathcal{O}(N^3 + NM)$
Résolution	$\sim 1/N$	meilleure	haute si hypothèses OK
Sources cohérentes	robuste	sensible	échoue (sans smoothing)
Calibration	robuste	sensible	très sensible

(M : nombre de points de la grille angulaire.)

La figure 15.3 illustre le point essentiel : MUSIC n'est pas seulement un beamformer plus fin. Il utilise une information différente. Bartlett mesure la puissance projetée sur $\mathbf{a}(\theta)$; Capon mesure la puissance minimale compatible avec un gain unitaire ; MUSIC mesure la distance au sous-espace bruit. C'est pourquoi les pics MUSIC peuvent être beaucoup plus étroits.

15.6 Sources cohérentes et spatial smoothing

Pour deux sources cohérentes (typique d'un trajet direct + écho), $\mathbf{P}_s$ devient singulière et le rang signal est $L' < L$. MUSIC sous-estime alors le nombre de sources. La parade canonique est le *forward spatial smoothing* [12].

15.6.1 Exemple : trajet direct et multipath

Supposons deux trajets parfaitement cohérents issus du même signal :

$$\mathbf{x}(n) = \mathbf{a}(\theta_1)s(n) + \alpha\mathbf{a}(\theta_2)s(n) + \mathbf{n}(n).$$

On peut factoriser :

$$\mathbf{x}(n) = [\mathbf{a}(\theta_1) + \alpha\mathbf{a}(\theta_2)]s(n) + \mathbf{n}(n).$$

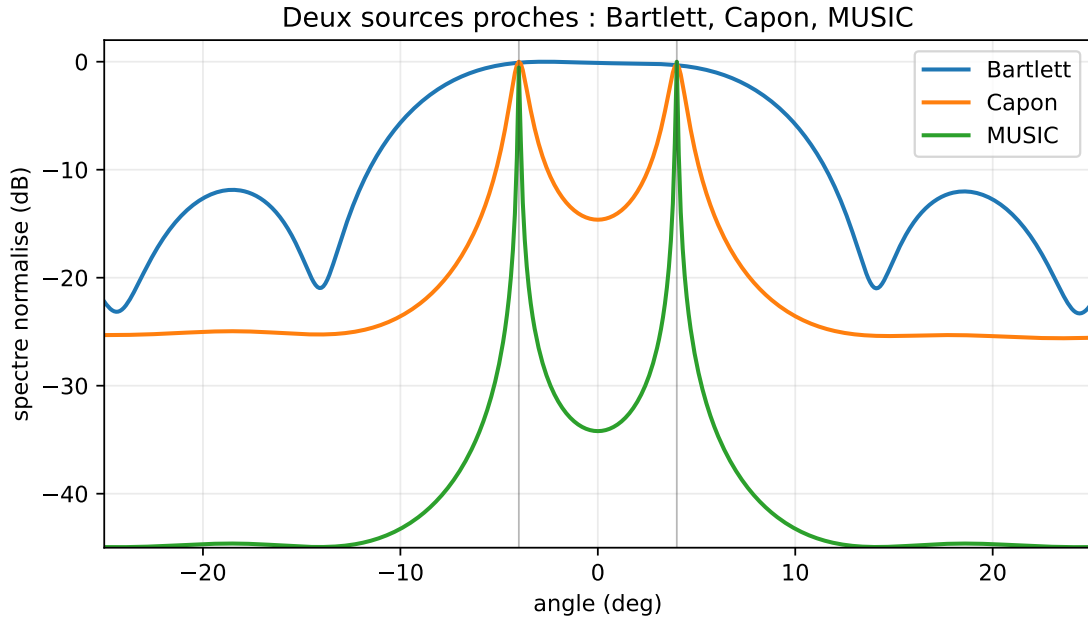


FIGURE 15.3 – Comparaison typique entre Bartlett, Capon et MUSIC ($N = 12$, sources à $\pm 4^\circ$, SNR 15 dB, $K = 512$). Bartlett élargit les pics, Capon les affine, MUSIC produit les pics les plus fins lorsque les hypothèses de sous-espace sont satisfaites.

Même s'il existe deux directions physiques, la partie signal est de rang 1, car elle ne dépend que d'un seul signal temporel $s(n)$. La covariance signal vaut :

$$\mathbf{R}_s = P_s [\mathbf{a}(\theta_1) + \alpha \mathbf{a}(\theta_2)] [\mathbf{a}(\theta_1) + \alpha \mathbf{a}(\theta_2)]^H,$$

donc son rang est 1. MUSIC cherche deux dimensions de sous-espace signal, mais n'en observe qu'une : il peut produire un seul pic, des pics biaisés ou une séparation instable.

Le spatial smoothing casse cette cohérence en exploitant le fait que les deux trajets accumulent des phases différentes d'un sous-réseau à l'autre. On restaure ainsi le rang signal, mais au prix d'une ouverture effective plus petite. La résolution diminue donc : on échange du rang contre de l'ouverture.

15.6.2 Principe

On divise l'ULA de N antennes en J sous-réseaux décalés de taille $M = N - J + 1$. Chaque sous-réseau j ($j = 0, \dots, J - 1$) fournit une covariance partielle

$$\hat{\mathbf{R}}^{(j)} = \frac{1}{K} \mathbf{X}^{(j)} \mathbf{X}^{(j)H},$$

où $\mathbf{X}^{(j)}$ ne contient que les lignes $j, j + 1, \dots, j + M - 1$ de $\mathbf{X}$. La covariance lissée est

$$\hat{\mathbf{R}}_{ss} = \frac{1}{J} \sum_{j=0}^{J-1} \hat{\mathbf{R}}^{(j)} \in \mathbb{C}^{M \times M}.$$

Pour $J \geq L$, $\hat{\mathbf{R}}_{ss}$ a son rang signal restauré à L (à condition que les phases relatives des sources entre sous-réseaux suffisent à décorréler).

15.6.3 Limites

- L'ouverture effective passe de $N - 1$ à $M - 1 < N - 1$: la résolution se dégrade.

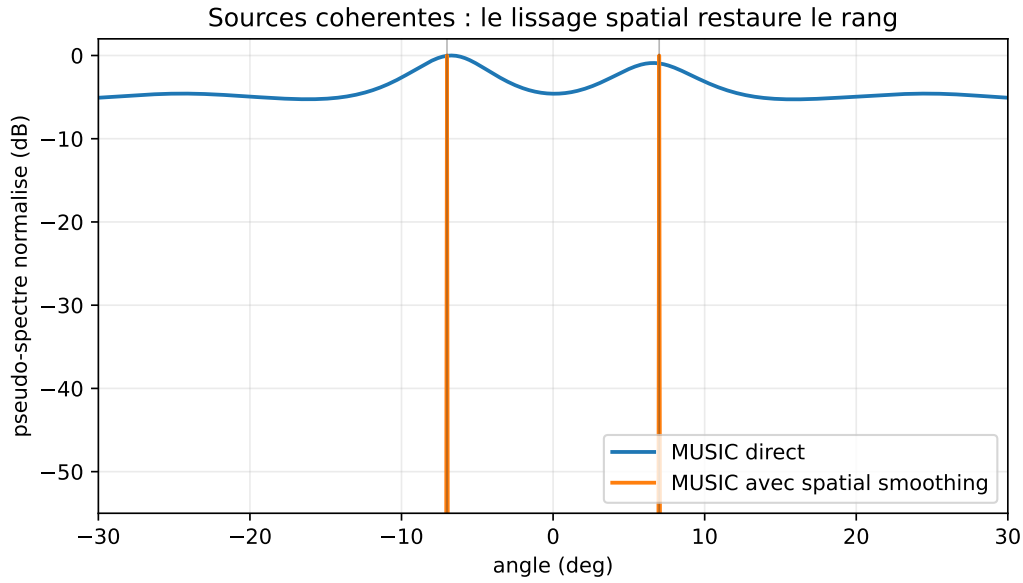


FIGURE 15.4 – Deux sources cohérentes ($N = 12$, sources à $\pm 7^\circ$, sous-réseaux de taille $M = 8$) peuvent se réduire à un sous-espace signal de rang 1. Le spatial smoothing restaure deux pics MUSIC, au prix d’une ouverture effective plus petite.

- Exige un ULA (géométrie régulière).
- Le *forward-backward smoothing* double l’effet en utilisant aussi les sous-réseaux indexés à l’envers.

15.7 Root-MUSIC

Pour un ULA, le scan angulaire est inutile : on peut résoudre l’équation algébriquement.

15.7.1 Polynôme MUSIC

Posons $z = e^{-jkd \sin \theta}$. Le steering vector ULA est $\mathbf{a}(z) = [1, z, z^2, \dots, z^{N-1}]^T$. La fonction

$$f(\theta) = \mathbf{a}^H(\theta) \mathbf{U}_n \mathbf{U}_n^H \mathbf{a}(\theta)$$

peut s’écrire, en posant $\mathbf{C} = \mathbf{U}_n \mathbf{U}_n^H$,

$$f(z) = \sum_{m,n} (\mathbf{C})_{m,n} z^{-(m-n)}.$$

$f(z)$ est ainsi un polynôme en z et z^{-1} , de degré $2(N-1)$.

15.7.2 Algorithme

1. Calculer $\mathbf{U}_n$ comme dans MUSIC standard.
2. Construire le polynôme $p(z) = f(z) \cdot z^{N-1}$ de degré $2(N-1)$.
3. Calculer ses racines $z_1, \dots, z_{2(N-1)}$.
4. Retenir les L racines les plus proches du cercle unité, à l’intérieur ou sur lui. (Les racines viennent en paires $(z_i, 1/z_i^*)$.)
5. Pour chaque racine retenue, l’angle estimé est $\hat{\theta}_\ell = \arcsin(-\arg(z_\ell)/(kd))$.

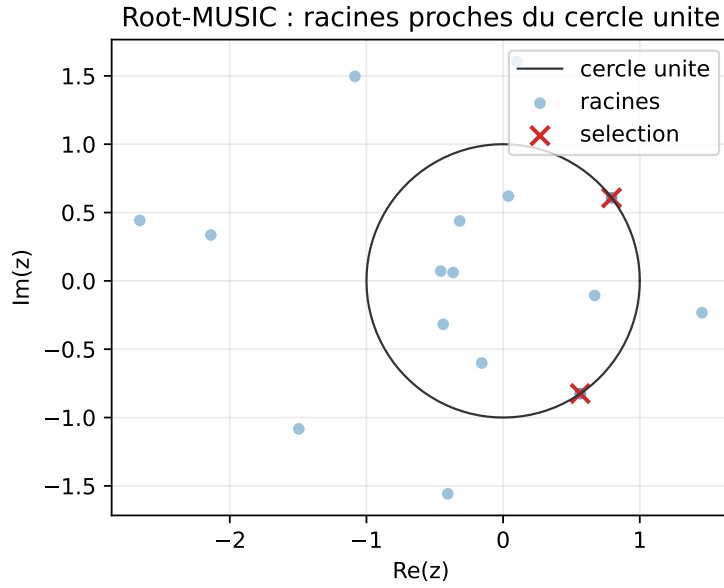


FIGURE 15.5 – Root-MUSIC ($N = 10$, sources -12° et 18° , SNR 20 dB, $K = 2048$) transforme le scan angulaire en sélection de racines proches du cercle unité. Une mauvaise sélection de racines donne immédiatement des angles faux.

15.7.3 Avantages

- Pas de grille angulaire à choisir : pas d'erreur de discrétisation.
- Coût $\mathcal{O}(N^3)$ pour les racines : compétitif avec un scan dense.
- Précision potentiellement améliorée lorsque le modèle ULA et l'ordre L sont fiables.

15.7.4 Limites

- Strictement réservé aux ULA. Pour réseaux 2D ou non uniformes, Root-MUSIC n'a pas d'analogue direct (extensions *Beamspace Root-MUSIC* pour certains cas).
- Sensibilité numérique : les racines proches du cercle unité sont mal localisées à très bas SNR.
- Sélection des racines délicate lorsque les sources sont proches, cohérentes ou lorsque l'ordre L est mal estimé.
- La formule $\arcsin(\cdot)$ doit être protégée numériquement : en pratique on borne son argument dans $[-1, 1]$ avant conversion en angle.

15.8 Variantes

Beamspace MUSIC. On projette $\mathbf{x}$ et $\mathbf{a}(\theta)$ dans un sous-espace de dimension réduite (une matrice de transformation $\mathbf{T}$ fixe) avant d'appliquer MUSIC. Cela diminue le coût et, correctement choisi, améliore la résolution sur une plage angulaire restreinte.

Cyclic MUSIC. Pour des signaux cyclostationnaires, on remplace la covariance par une covariance *cyclique* qui n'inclut que les signaux ayant une fréquence cyclique donnée. Permet de séparer des sources de standards différents (Wi-Fi vs. LTE par exemple).

Maximum likelihood (ML). Recherche directement les angles qui maximisent la vraisemblance. Plus précis que MUSIC à très bas SNR, mais nécessite une optimisation multidimensionnelle (alternating projection, Newton, etc.). On y reviendra brièvement au chapitre suivant lors du calcul des bornes de Cramér-Rao.

15.9 Calibration et MUSIC

MUSIC repose sur la connaissance exacte de $\mathbf{a}(\theta)$. Une erreur de phase entre voies $\delta\phi_m$ produit un steering vector réel $\mathbf{a}_{\text{réel}}(\theta) = \mathbf{G}\mathbf{a}(\theta)$. Le pseudo-spectre calculé avec le $\mathbf{a}$ théorique présente alors :

- des pics *décalés* en angle ;
- des pics *remontés en niveau* (moins discriminants) ;
- parfois des *pics fantômes*.

Le chapitre 22 traite ce problème en détail.

15.10 Implémentation numérique

Listing 15.1 – MUSIC standard et Root-MUSIC

```

1 import numpy as np
2
3 def music_spectrum(R, L, thetas, N, d_over_lambda=0.5):
4     assert R.shape == (N, N)
5     assert 0 <= L < N
6     R = 0.5 * (R + R.conj().T)
7     _, eigvecs = np.linalg.eigh(R)    # croissant
8     Un = eigvecs[:, :N-L]           # plus petites valeurs propres
9     m = np.arange(N)
10    spec = np.zeros(len(thetas))
11    eps = np.finfo(float).eps
12    for idx, th in enumerate(thetas):
13        a = np.exp(-1j * 2*np.pi * d_over_lambda * np.sin(th) * m)
14        v = Un.conj().T @ a
15        den = max((v.conj() @ v).real, eps)
16        spec[idx] = 1.0 / den
17    return spec
18
19 def root_music(R, L, d_over_lambda=0.5):
20     """Retourne les L DOA estimées (rad) par Root-MUSIC."""
21     N = R.shape[0]
22     assert R.shape == (N, N)
23     assert 0 < L < N
24     R = 0.5 * (R + R.conj().T)
25     eigvals, eigvecs = np.linalg.eigh(R)
26     Un = eigvecs[:, :N-L]
27     C = Un @ Un.conj().T            # N x N
28
29     # Coefficients du polynôme MUSIC dans z
30     coeffs = np.zeros(2*N - 1, dtype=complex)
31     for lag in range(-(N-1), N):
32         coeffs[N - 1 + lag] = np.trace(C, offset=lag)
33     roots = np.roots(coeffs[::-1])
34
35     # Racines à l'intérieur ou sur le cercle unité

```

```

36     valid = roots[np.abs(roots) <= 1.0]
37     if len(valid) < L:
38         raise ValueError("Root-MUSIC: pas assez de racines valides")
39     # Trier par distance au cercle unité
40     order = np.argsort(np.abs(np.abs(valid) - 1.0))
41     selected = valid[order][:L]
42     arg = np.angle(selected) / (-2*np.pi*d_over_lambda)
43     angles = np.arcsin(np.clip(arg, -1.0, 1.0))
44     return np.sort(np.real(angles))

```

15.11 Limites pratiques de MUSIC

- exige une covariance bien estimée ($K \geq$ quelques N) ;
- exige une connaissance précise de L ;
- exige des sources non parfaitement cohérentes (ou spatial smoothing) ;
- exige une calibration soignée ;
- ne fournit pas naturellement une mesure de confiance ;
- coût d'un scan dense en angle $\sim \mathcal{O}(NM)$ par évaluation de pseudo-spectre.

15.12 Synthèse

MUSIC est l'algorithme *historique* de l'estimation haute résolution de DOA. Son fondement (orthogonalité signal-bruit) est limpide ; sous hypothèses favorables et en régime asymptotique, ses performances peuvent approcher la borne de Cramér-Rao (qu'on étudiera au chapitre 17). Ses fragilités — sources cohérentes, calibration, choix de L — sont bien comprises et largement traitées.

À retenir

- **Formule centrale** : $P_{\text{MUSIC}}(\theta) = 1/(\mathbf{a}^H \mathbf{U}_n \mathbf{U}_n^H \mathbf{a})$.
- **Hypothèse principale** : séparation nette entre sous-espace signal et sous-espace bruit.
- **Avantage** : haute résolution et géométrie flexible.
- **Limite** : sources cohérentes, choix de L et calibration.
- **Piège pratique** : un pic MUSIC n'est pas une puissance ; c'est une mesure de distance à un sous-espace.
- **Suite** : ESPRIT remplace le scan par une relation d'invariance.

Le chapitre suivant introduit ESPRIT, qui supprime le scan angulaire au profit d'une décomposition algébrique fermée exploitant l'invariance par translation des ULA. Pour une géométrie ULA bien calibrée, ESPRIT est typiquement plus rapide que MUSIC à performance comparable, et plus facile à intégrer sur des plateformes embarquées. Pour des géométries plus générales (planaires, sparse, conformes), MUSIC reste souvent l'outil de choix, sa généralité l'emportant alors sur la spécialisation algébrique d'ESPRIT.

Chapitre 16

ESPRIT et variantes

MUSIC localise les sources en évaluant un pseudo-spectre sur une grille angulaire. La technique fonctionne, mais elle a deux limites : le coût du scan, et la résolution limitée par la grille. ESPRIT (*Estimation of Signal Parameters via Rotational Invariance Techniques*, Roy & Kailath [10]) s'en affranchit en exploitant une propriété purement *algébrique* des ULA : leur invariance par translation. Les DOA s'obtiennent en valeurs propres d'une matrice $L \times L$ — sans scan, sans grille.

L'idée a quelque chose de magique. Au lieu de « chercher » les directions en parcourant l'espace, on les « lit » dans le spectre d'une matrice. L'astuce repose sur le fait qu'un ULA est *auto-similaire* : on peut le découper en deux sous-réseaux qui sont identiques à une translation près. Cette translation induit une relation algébrique précise entre les sous-espaces signal des deux sous-réseaux, dont les angles sont les inconnues. Résoudre cette relation revient à diagonaliser une matrice. Toute la puissance d'ESPRIT vient de cette transformation d'un problème de localisation en un problème spectral.

16.1 Invariance par translation

16.1.1 Sous-réseaux décalés

Soit un ULA de N antennes. Divisons-le en deux sous-réseaux de $M = N - 1$ antennes : le premier ($\mathcal{X}_1$) couvre les indices $0, 1, \dots, N - 2$, le second ($\mathcal{X}_2$) les indices $1, 2, \dots, N - 1$. Les deux sous-réseaux sont identiques à une translation près d'une antenne le long de l'axe du réseau.

Si $\mathbf{a}_M(\theta)$ est le steering vector ULA à M antennes,

$$\mathbf{a}_M(\theta) = [1, z, z^2, \dots, z^{M-1}]^T, \quad z = z(\theta) = e^{-jkd \sin \theta},$$

alors le steering vector du second sous-réseau, mesuré relativement à sa première antenne, est *le même* ; mais relativement à l'antenne de référence du premier sous-réseau, il est multiplié par $z(\theta)$:

$$\mathbf{a}_M^{(2)}(\theta) = z(\theta) \cdot \mathbf{a}_M(\theta).$$

C'est cette propriété — l'application $\mathbf{a}_M \mapsto z(\theta)\mathbf{a}_M$ — qui donne son nom à ESPRIT : la *rotational invariance*.

16.1.2 Matrices de sélection

Formellement, on introduit deux matrices $\mathbf{J}_1, \mathbf{J}_2 \in \mathbb{R}^{M \times N}$ qui sélectionnent respectivement les M premières et les M dernières antennes :

$$\mathbf{J}_1 = [\mathbf{I}_M \ \mathbf{0}], \quad \mathbf{J}_2 = [\mathbf{0} \ \mathbf{I}_M].$$

Concrètement, pour $N = 5$ et $M = 4$:

$$\mathbf{J}_1 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{J}_2 = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

$\mathbf{J}_1$ garde les $N - 1$ premières antennes ; $\mathbf{J}_2$ garde les $N - 1$ dernières. Le second sous-réseau est donc le premier translaté d'un pas d .

Pour la matrice de steering $\mathbf{A}_N = [\mathbf{a}_N(\theta_1), \dots, \mathbf{a}_N(\theta_L)]$,

$$\mathbf{A}_1 \equiv \mathbf{J}_1 \mathbf{A}_N = [\mathbf{a}_M(\theta_1), \dots, \mathbf{a}_M(\theta_L)], \quad \mathbf{A}_2 \equiv \mathbf{J}_2 \mathbf{A}_N = \mathbf{A}_1 \Phi,$$

où

$$\Phi = \text{diag}(e^{-jkd \sin \theta_1}, \dots, e^{-jkd \sin \theta_L}) \in \mathbb{C}^{L \times L}.$$

Les DOA sont entièrement encodées dans les valeurs propres de Φ .

La relation clé peut donc se lire en une seule ligne :

$$\boxed{\mathbf{J}_2 \mathbf{A}_N = \mathbf{J}_1 \mathbf{A}_N \Phi.}$$

Chaque colonne de $\mathbf{A}_N$ est une progression géométrique. Décaler le réseau d'une antenne revient à multiplier cette colonne par :

$$e^{-jkd \sin \theta_\ell}.$$

ESPRIT n'est rien d'autre que l'exploitation de cette multiplication.

16.2 Sous-espace signal et relation d'invariance

Au lieu de $\mathbf{A}_1$ et $\mathbf{A}_2$ directement (inconnues), ESPRIT travaille avec les sous-espaces signal estimés. Soit $\mathbf{U}_s \in \mathbb{C}^{N \times L}$ le sous-espace signal de $\mathbf{R}$. Comme $\text{Im}(\mathbf{U}_s) = \text{Im}(\mathbf{A}_N)$, il existe une matrice $\mathbf{T} \in \mathbb{C}^{L \times L}$ inversible telle que

$$\mathbf{U}_s = \mathbf{A}_N \mathbf{T}.$$

En sélectionnant les sous-réseaux :

$$\mathbf{U}_{s,1} \equiv \mathbf{J}_1 \mathbf{U}_s = \mathbf{A}_1 \mathbf{T}, \quad \mathbf{U}_{s,2} \equiv \mathbf{J}_2 \mathbf{U}_s = \mathbf{A}_2 \mathbf{T} = \mathbf{A}_1 \Phi \mathbf{T}.$$

Donc

$$\mathbf{U}_{s,2} = \mathbf{U}_{s,1} (\mathbf{T}^{-1} \Phi \mathbf{T}) \equiv \mathbf{U}_{s,1} \Psi.$$

Théorème 16.1 (Relation d'invariance ESPRIT). *Il existe une matrice $\Psi \in \mathbb{C}^{L \times L}$ telle que $\mathbf{U}_{s,2} = \mathbf{U}_{s,1} \Psi$, et Ψ est similaire à Φ . Ses valeurs propres sont donc exactement les $z(\theta_\ell) = e^{-jkd \sin \theta_\ell}$.*

16.3 Algorithme

1. Collecter K snapshots et estimer $\hat{\mathbf{R}}$.
2. Décomposer $\hat{\mathbf{R}}$ et choisir L .
3. Extraire $\hat{\mathbf{U}}_s \in \mathbb{C}^{N \times L}$ (sous-espace signal estimé).
4. Construire $\hat{\mathbf{U}}_{s,1} = \mathbf{J}_1 \hat{\mathbf{U}}_s$ et $\hat{\mathbf{U}}_{s,2} = \mathbf{J}_2 \hat{\mathbf{U}}_s$.
5. Résoudre $\hat{\mathbf{U}}_{s,2} = \hat{\mathbf{U}}_{s,1} \hat{\Psi}$ au sens moindres carrés (LS-ESPRIT) ou moindres carrés totaux (TLS-ESPRIT).
6. Diagonaliser $\hat{\Psi}$ et lire ses valeurs propres $\hat{z}_\ell$.
7. Estimer les DOA : $\hat{\theta}_\ell = \arcsin(-\arg(\hat{z}_\ell)/(kd))$.

16.4 LS-ESPRIT et TLS-ESPRIT

16.4.1 LS

Le problème $\widehat{\mathbf{U}}_{s,2} = \widehat{\mathbf{U}}_{s,1} \widehat{\Psi}$ suppose $\widehat{\mathbf{U}}_{s,1}$ exact. On résout par moindres carrés classiques :

$$\widehat{\Psi}_{\text{LS}} = (\widehat{\mathbf{U}}_{s,1}^H \widehat{\mathbf{U}}_{s,1})^{-1} \widehat{\mathbf{U}}_{s,1}^H \widehat{\mathbf{U}}_{s,2}.$$

LS-ESPRIT est simple et rapide, mais introduit un biais quand $\widehat{\mathbf{U}}_{s,1}$ est aussi bruitée que $\widehat{\mathbf{U}}_{s,2}$.

16.4.2 TLS

Le *Total Least Squares* suppose que les deux matrices contiennent du bruit. On forme $\widetilde{\mathbf{U}} = [\widehat{\mathbf{U}}_{s,1}, \widehat{\mathbf{U}}_{s,2}] \in \mathbb{C}^{M \times 2L}$ et on calcule sa SVD. Les vecteurs singuliers associés aux plus petites valeurs singulières donnent l'estimateur TLS qui est typiquement plus précis à bas SNR que LS.

16.4.3 Lecture pratique

LS-ESPRIT répond à la question :

quelle matrice transforme au mieux $\widehat{\mathbf{U}}_{s,1}$ en $\widehat{\mathbf{U}}_{s,2}$?

TLS-ESPRIT répond à une question plus symétrique :

quelle relation d'invariance est la plus proche des deux matrices bruitées ?

Comme les deux sous-espaces sont extraits de la même covariance estimée, ils sont tous les deux bruités. C'est pourquoi TLS est souvent le choix par défaut en traitement radar ou télécom exigeant.

16.5 Cas d'une source unique

Pour $L = 1$, Φ est scalaire et la matrice Ψ aussi. Le résultat se réduit à

$$\widehat{z} = \frac{\widehat{\mathbf{u}}_{s,1}^H \widehat{\mathbf{u}}_{s,2}}{\widehat{\mathbf{u}}_{s,1}^H \widehat{\mathbf{u}}_{s,1}},$$

soit une simple division. L'angle est $\widehat{\theta} = \arcsin(-\arg(\widehat{z})/(kd))$.

16.6 Comparaison MUSIC / ESPRIT

Critère	MUSIC	ESPRIT
Scan angulaire	oui (ou racines)	non
Géométrie	quelconque (sauf Root-MUSIC)	ULA (ou invariance)
Coût	$\mathcal{O}(N^3 + NM)$	$\mathcal{O}(N^3 + NL^2)$
Précision (haut SNR)	$\sim \text{CRB}$	$\sim \text{CRB}$
Précision (bas SNR)	légèrement meilleure	légèrement moins bonne
Sources cohérentes	smoothing requis	smoothing requis
Calibration	sensible	très sensible

Les deux algorithmes ont des performances très proches en asymptotique de SNR ou de K . ESPRIT a un avantage en coût de calcul ; MUSIC a un avantage en généricité géométrique.

16.7 Variantes importantes

Unitary ESPRIT. Reformulation en arithmétique réelle via transformations unitaires sur les données centrées et leurs versions « forward-backward ». Coût ramené à un calcul de SVD réel. Gain substantiel sur FPGA.

Beamspace ESPRIT. Projette préalablement $\widehat{\mathbf{U}}_s$ dans un sous-espace de dimension réduite ; coût moindre, performances comparables sur des zones angulaires restreintes.

2D-ESPRIT. Étend ESPRIT aux réseaux planaires (URA), avec deux matrices Ψ_x et Ψ_y pour azimuth et élévation.

Multiple Invariance ESPRIT (MI-ESPRIT). Exploite plusieurs sous-réseaux décalés simultanément, gain de précision pour un coût légèrement plus élevé.

16.8 Conditions de validité

- **Invariance de translation.** ESPRIT exige une géométrie où une sous-partie du réseau est identique à une autre à une translation près. Vrai pour les ULA, vrai pour les URA séparables, faux pour les réseaux circulaires ou aléatoires.
- $L < N$. Au plus $N - 1$ sources pour un ULA.
- **Calibration géométrique.** Une erreur de position d'antenne casse l'invariance et biaise toutes les estimations.
- **Sources suffisamment décorrélées.** Sinon, smoothing préalable, qui réduit l'ouverture effective.

16.9 Ambiguïtés et espacement

Pour $d > \lambda/2$, $z(\theta) = e^{-jkd \sin \theta}$ n'est plus bijectif sur la zone visible : plusieurs angles peuvent donner le même z . ESPRIT fournit alors des valeurs propres correctes en z , mais on ne peut plus remonter *sans ambiguïté* à θ . La parade : $d \leq \lambda/2$ ou utilisation de techniques de *déballage* (*phase unwrapping*) basées sur plusieurs sous-réseaux d'espacement différent (*coprime arrays*).

16.10 Implémentation numérique

Listing 16.1 – ESPRIT LS et TLS

```

1 import numpy as np
2
3 def esprit(R, L, d_over_lambda=0.5, tls=True):
4     """
5     R : (N,N)
6     L : nombre de sources
7     tls : True pour TLS-ESPRIT, False pour LS-ESPRIT
8     Retour : array de L DOA estimées (rad)
9     """
10    N = R.shape[0]
11    assert R.shape == (N, N)
12    assert 0 < L < N
13    R = 0.5 * (R + R.conj().T)
14    eigvals, eigvecs = np.linalg.eigh(R)

```

```

15     Us = eigvecs[:, -L:]                                # sous-espace signal
16     Us1 = Us[:-1, :]
17     Us2 = Us[1:, :]
18
19     if not tls:
20         Psi = np.linalg.lstsq(Us1, Us2, rcond=None)[0]
21     else:
22         Z = np.hstack([Us1, Us2])                        # M x 2L
23         _, _, Vh = np.linalg.svd(Z, full_matrices=False)
24         V = Vh.conj().T                                   # 2L x 2L
25         V12 = V[:, L, L:]
26         V22 = V[L:, L:]
27         if np.linalg.cond(V22) > 1e12:
28             raise np.linalg.LinAlgError("TLS-ESPRIT mal conditionné")
29         Psi = -np.linalg.solve(V22.T, V12.T).T
30
31     eig_psi, _ = np.linalg.eig(Psi)
32     arg = -np.angle(eig_psi) / (2*np.pi*d_over_lambda)
33     angles = np.arcsin(np.clip(arg, -1.0, 1.0))
34     return np.sort(np.real(angles))

```

16.11 Comparaison Bartlett / Capon / MUSIC / ESPRIT

Critère	Bartlett	Capon	MUSIC	ESPRIT
Principe	puissance projetée	variance minimale	sous-espace bruit	invariance
Scan angulaire	oui	oui	oui	non
Résolution	classique	meilleure	haute	haute
Géométrie	générale	générale	générale	réseaux invariants
Sources cohérentes	visible mais flou	sensible	smoothing requis	smoothing requis
Sensibilité calibration	faible	moyenne	forte	très forte
Sortie	spectre	spectre	pseudo-spectre	angles directs

16.12 Synthèse

ESPRIT exploite une structure algébrique exacte (l'invariance par translation des ULA) pour transformer l'estimation de DOA en un problème de valeurs propres $L \times L$. Pas de scan, pas de grille, pas d'erreur de discrétisation. Pour les ULA bien calibrés, avec un ordre de modèle fiable et des sources non cohérentes, c'est l'un des algorithmes de référence en production.

Le chapitre suivant fixe les performances ultimes de *tout* estimateur de DOA — MUSIC, ESPRIT, ML, et tout estimateur futur — via les *bornes de Cramér-Rao*. C'est une étape souvent négligée dans les présentations introductives, et c'est dommage : sans référence absolue, on ne peut pas juger qu'un estimateur « fait bien ». La CRB fournit cette référence : si un estimateur s'en approche à 0.5 dB asymptotiquement, on sait qu'on tient quelque chose ; s'il en est à 10 dB, on sait qu'il y a de la marge à gagner.

Chapitre 17

Bornes de Cramér-Rao pour la DOA

MUSIC et ESPRIT atteignent des résolutions remarquables, mais quelle est la *limite ultime* de précision pour l'estimation des directions d'arrivée ? Aucun estimateur ne peut faire mieux que la *borne de Cramér-Rao* (CRB) : c'est la variance minimale de tout estimateur non biaisé, dictée uniquement par le modèle statistique. Ce chapitre établit les CRB pour la DOA — déterministe et stochastique — et compare aux performances de MUSIC, ESPRIT et au maximum de vraisemblance.

Le rôle de la CRB en statistique paramétrique est analogue à celui de la borne de Shannon en théorie de l'information : c'est un plafond théorique infranchissable qui structure tout le champ. On y compare les estimateurs comme on compare les modulations à la capacité d'un canal. Un estimateur qui atteint la CRB est dit *efficient* ; c'est le mieux qu'on puisse espérer. Un estimateur qui en est loin a de la marge — soit pour l'améliorer, soit pour relâcher des hypothèses (compléter par des informations *a priori*, etc.). Connaître la CRB est donc, pour qui conçoit un système, une boussole indispensable : elle dit à la fois « on ne fera pas mieux que ça » et « avec assez d'efforts, on peut y arriver ».

17.1 À quoi sert vraiment la CRB ?

La CRB ne donne pas l'erreur d'un algorithme particulier. Elle donne une limite théorique pour tout estimateur non biaisé compatible avec le modèle statistique. Il faut donc l'interpréter correctement.

- Elle ne dit pas que MUSIC ou ESPRIT atteignent toujours cette erreur.
- Elle dit qu'aucun estimateur non biaisé ne peut avoir une variance plus faible.
- Elle sert à comparer les estimateurs en régime asymptotique : MUSIC, ESPRIT, maximum de vraisemblance, estimateurs sparse.
- Elle révèle l'effet de paramètres physiques : SNR, nombre de snapshots, nombre d'antennes, ouverture, séparation angulaire.

Dans un projet d'ingénierie, on l'utilise comme test de maturité. Si un estimateur simulé est très loin de la CRB dans un régime favorable, il y a probablement un problème de mise en œuvre, de convention ou de calibration. S'il s'en approche, améliorer encore l'algorithme rapportera peu : il faut agir sur le système physique, par exemple augmenter N , K , le SNR ou l'ouverture.

17.2 Borne de Cramér-Rao : généralités

17.2.1 Information de Fisher

Soit $\boldsymbol{\theta} \in \mathbb{R}^p$ un vecteur de paramètres à estimer, et $\mathbf{x}$ une observation de densité $p(\mathbf{x}; \boldsymbol{\theta})$. La matrice d'information de Fisher est

$$\mathbf{F}(\boldsymbol{\theta})_{ij} = -\mathbb{E} \left[\frac{\partial^2 \log p(\mathbf{x}; \boldsymbol{\theta})}{\partial \theta_i \partial \theta_j} \right] = \mathbb{E} \left[\frac{\partial \log p}{\partial \theta_i} \frac{\partial \log p}{\partial \theta_j} \right].$$

Théorème 17.1 (Borne de Cramér-Rao). *Pour tout estimateur non biaisé $\hat{\boldsymbol{\theta}}$ de $\boldsymbol{\theta}$,*

$$\text{Cov}(\hat{\boldsymbol{\theta}}) \succeq \mathbf{F}^{-1}(\boldsymbol{\theta}),$$

au sens des matrices semi-définies positives. En particulier la variance de chaque composante est minorée par le terme diagonal correspondant de $\mathbf{F}^{-1}$.

Un estimateur qui atteint cette borne est dit *efficient*. Le maximum de vraisemblance est efficient asymptotiquement (sous conditions de régularité).

17.2.2 Densité gaussienne complexe circulaire

Pour $\mathbf{x} \sim \mathcal{CN}(\boldsymbol{\mu}(\boldsymbol{\theta}), \mathbf{R}(\boldsymbol{\theta}))$ gaussien complexe circulaire,

$$\log p(\mathbf{x}; \boldsymbol{\theta}) = -N \log \pi - \log \det \mathbf{R}(\boldsymbol{\theta}) - (\mathbf{x} - \boldsymbol{\mu})^H \mathbf{R}^{-1}(\boldsymbol{\theta}) (\mathbf{x} - \boldsymbol{\mu}).$$

Si $\boldsymbol{\mu}$ ne dépend pas de $\boldsymbol{\theta}$, on obtient la *formule de Slepian-Bangs*

$$\boxed{\mathbf{F}_{ij} = \text{tr} \left(\mathbf{R}^{-1} \frac{\partial \mathbf{R}}{\partial \theta_i} \mathbf{R}^{-1} \frac{\partial \mathbf{R}}{\partial \theta_j} \right)}.$$

17.3 Modèles déterministe et stochastique

Pour le problème DOA on distingue deux modèles selon que les enveloppes $\mathbf{s}(n)$ sont déterministes ou stochastiques.

17.3.1 Modèle déterministe (conditionnel)

Les enveloppes $\mathbf{s}(1), \dots, \mathbf{s}(K)$ sont des paramètres inconnus à estimer en plus de $\boldsymbol{\theta}$. Le modèle est $\mathbf{x}(n) = \mathbf{A}(\boldsymbol{\theta}) \mathbf{s}(n) + \mathbf{n}(n)$ avec $\mathbf{n} \sim \mathcal{CN}(\mathbf{0}, \sigma^2 \mathbf{I}_N)$.

17.3.2 Modèle stochastique (inconditionnel)

Les enveloppes sont aléatoires : $\mathbf{s}(n) \sim \mathcal{CN}(\mathbf{0}, \mathbf{P}_s)$. Le modèle est $\mathbf{x}(n) \sim \mathcal{CN}(\mathbf{0}, \mathbf{R})$ avec $\mathbf{R} = \mathbf{A} \mathbf{P}_s \mathbf{A}^H + \sigma^2 \mathbf{I}_N$.

Le modèle stochastique est typiquement le plus pertinent en pratique (sources passives, brouilleurs, multipath), tandis que le déterministe modélise mieux les sources connues (pilotes, signaux émis par soi-même).

17.4 Borne CRB déterministe

Théorème 17.2 (CRB déterministe). *Sous le modèle déterministe, la CRB sur $\boldsymbol{\theta} = (\theta_1, \dots, \theta_L)^T$ issue de K snapshots est*

$$\text{CRB}_{\text{det}}(\boldsymbol{\theta}) = \frac{\sigma^2}{2K} \left[\text{Re} \left\{ (\mathbf{D}^H \mathbf{P}_A^\perp \mathbf{D}) \odot \widehat{\mathbf{P}}_s^T \right\} \right]^{-1},$$

où :

- $\mathbf{D} = [\dot{\mathbf{a}}(\theta_1), \dots, \dot{\mathbf{a}}(\theta_L)]$ avec $\dot{\mathbf{a}} = \partial \mathbf{a} / \partial \boldsymbol{\theta}$;
- $\mathbf{P}_A^\perp = \mathbf{I}_N - \mathbf{A}(\mathbf{A}^H \mathbf{A})^{-1} \mathbf{A}^H$ est le projecteur sur le complément orthogonal de $\text{Im}(\mathbf{A})$;
- $\widehat{\mathbf{P}}_s = \frac{1}{K} \sum_n \mathbf{s}(n) \mathbf{s}^H(n)$ est la covariance empirique des sources ;
- $\odot$ est le produit de Hadamard (composante par composante).

17.5 Borne CRB stochastique

Théorème 17.3 (CRB stochastique). *Sous le modèle stochastique, la CRB est*

$$\text{CRB}_{\text{sto}}(\boldsymbol{\theta}) = \frac{\sigma^2}{2K} \left[\text{Re} \left\{ (\mathbf{D}^H \mathbf{P}_A^\perp \mathbf{D}) \odot (\mathbf{P}_s \mathbf{A}^H \mathbf{R}^{-1} \mathbf{A} \mathbf{P}_s)^T \right\} \right]^{-1}.$$

Remarque 17.4 (Relations entre les bornes). On démontre que $\text{CRB}_{\text{sto}} \leq \text{CRB}_{\text{det}}$ asymptotiquement à grand SNR. Le modèle stochastique tire profit d'une hypothèse plus informative (la distribution des sources est connue), ce qui peut conduire à des bornes plus faibles. À bas SNR, les deux bornes coïncident.

17.6 Cas d'une source unique sur ULA

C'est le cas le plus instructif : tous les facteurs se calculent explicitement.

17.6.1 Dérivée du steering vector

Pour un ULA à $d = \lambda/2$,

$$\dot{\mathbf{a}}_m(\theta) = -j\pi m \cos \theta e^{-j\pi m \sin \theta}.$$

La norme au carré de $\dot{\mathbf{a}}(\theta)$ est

$$\|\dot{\mathbf{a}}(\theta)\|^2 = \pi^2 \cos^2 \theta \sum_{m=0}^{N-1} m^2 = \pi^2 \cos^2 \theta \frac{(N-1)N(2N-1)}{6}.$$

17.6.2 CRB explicite

Pour une source unique de puissance p_s et un bruit blanc σ^2 , on a $\mathbf{P}_A^\perp \mathbf{D} = \dot{\mathbf{a}} - (\mathbf{a}^H \dot{\mathbf{a}}/N) \mathbf{a}$. Pour un ULA à $d = \lambda/2$, on a $\mathbf{a}^H \dot{\mathbf{a}} = -j\pi \cos \theta \sum_m m = -j\pi \cos \theta \cdot N(N-1)/2$, et

$$\dot{\mathbf{a}}^H \mathbf{P}_A^\perp \dot{\mathbf{a}} = \pi^2 \cos^2 \theta \frac{N(N^2-1)}{12}.$$

Corollaire 17.5 (CRB ULA, source unique).

$$\text{CRB}_{\text{sto}}(\theta) = \frac{6\sigma^2}{K p_s \pi^2 \cos^2 \theta N(N^2-1)} \frac{1 + N p_s / \sigma^2}{N p_s / \sigma^2}.$$

17.6.3 Comportement asymptotique

À grand SNR ($Np_s/\sigma^2 \gg 1$),

$$\text{CRB}(\theta) \sim \frac{6}{K \pi^2 \cos^2 \theta N^3} \cdot \frac{\sigma^2}{p_s N} = \frac{6}{K \cdot \text{SNR}_{\text{array}} \cdot \pi^2 \cos^2 \theta N^3},$$

où $\text{SNR}_{\text{array}} = Np_s/\sigma^2$ est le SNR *après* beamforming DAS. Trois enseignements :

- décroissance en $1/N^3$: doubler N divise par 8 la variance ;
- décroissance en $1/K$: doubler le nombre de snapshots divise par 2 ;
- décroissance en $1/\text{SNR}$;
- facteur $1/\cos^2 \theta$: la précision se dégrade quand on s'éloigne du broadside (au pôle endfire $\theta = \pm 90^\circ$, $\cos \theta = 0$, la CRB diverge).

Loi N^3 . Le facteur $1/N^3$ s'explique par : (i) un facteur N pour le gain de réseau, (ii) un facteur N^2 pour l'ouverture quadratique — une grande ouverture pèse plus dans une régression de phase. C'est la *loi de Cramér-Rao spatiale* : la précision angulaire varie comme l'inverse du *cube* du nombre d'antennes, contrairement au gain SNR qui ne croît qu'en N .

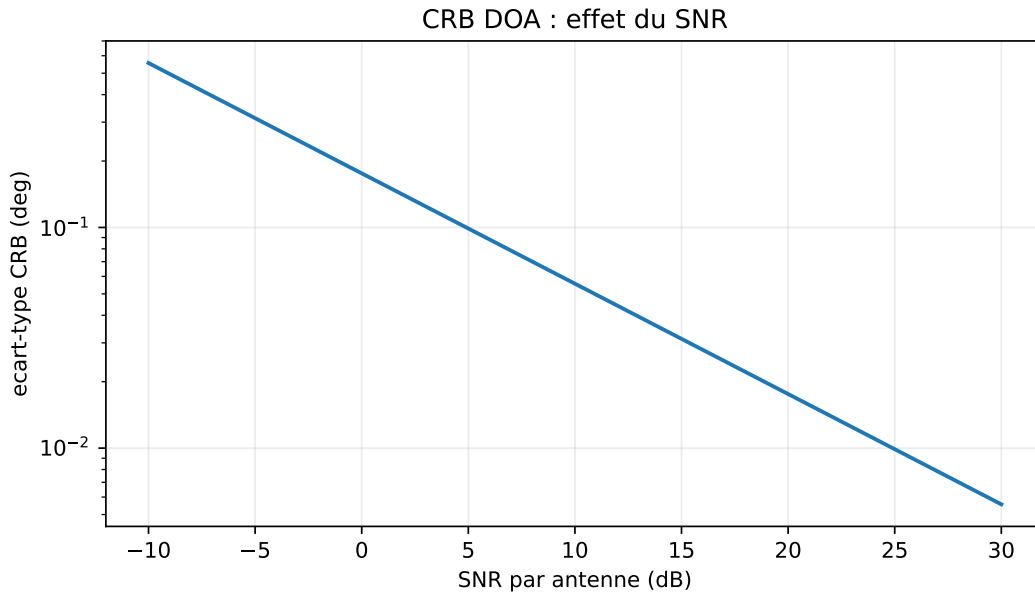


FIGURE 17.1 – Écart-type théorique donné par une CRB simplifiée (source broadside, $N = 8$, $K = 128$, $d = \lambda/2$), en fonction du SNR. L'amélioration avec N est plus rapide qu'un simple gain de puissance, car l'ouverture spatiale augmente aussi.

17.7 Deux sources : effet du couplage

Pour deux sources d'angles θ_1 et θ_2 , la CRB pour θ_1 contient un terme qui dépend de $|\mathbf{a}^H(\theta_1)\mathbf{a}(\theta_2)|^2$. Quand les deux sources sont angulairement proches, leur produit scalaire approche N (les steering vectors sont presque colinéaires), et la CRB explose :

$$\text{CRB}(\theta_1) \sim \frac{1}{1 - |\rho|^2},$$

où $\rho = \mathbf{a}^H(\theta_1)\mathbf{a}(\theta_2)/N$ est le *coefficient de corrélation des steerings*. C'est la formalisation de l'intuition : deux sources très proches sont fondamentalement difficiles à séparer.

17.8 Limite de résolution

Combinant CRB et seuil de détection, on définit la *limite de résolution statistique* (Smith 2005)

$$\Delta\theta_{\min} \sim \sqrt{\frac{2}{\text{CRB}^{-1}(\theta)}} \sim \frac{1}{N^{3/2} \sqrt{K \cdot \text{SNR}} |\cos \theta|}.$$

Comparée à la limite de Rayleigh $\sim 2/N$, on a un *gain en résolution* d'un facteur $\sqrt{N \cdot K \cdot \text{SNR}}$: c'est le « super-resolution » qu'atteignent les méthodes sous-espaces.

17.9 Comparaison aux performances pratiques

17.9.1 MUSIC asymptotique

Pour MUSIC, la variance asymptotique de $\hat{\theta}_\ell$ dans le cadre étudié par Stoica et Nehorai [13] est

$$\text{Var}_{\text{MUSIC}}(\hat{\theta}_\ell) = \frac{\sigma^2}{2K} \frac{(\mathbf{P}_s \mathbf{A}^H \mathbf{R}^{-1} \mathbf{A} \mathbf{P}_s)_{\ell,\ell}^{-1}}{\text{Re}\{(\hat{\mathbf{a}}_\ell^H \mathbf{P}_A^\perp \hat{\mathbf{a}}_\ell)\}}.$$

On démontre que $\text{Var}_{\text{MUSIC}} \geq \text{CRB}_{\text{sto}}$ avec égalité dans deux cas : (i) source unique, (ii) sources de même puissance et orthogonales en steering. Ce résultat dit que *MUSIC n'est pas efficient pour deux sources proches*, mais le reste asymptotiquement à grand SNR.

17.9.2 ESPRIT asymptotique

ESPRIT atteint des performances très proches de MUSIC mais légèrement moins bonnes asymptotiquement : la perte par rapport à la CRB est *plus marquée* pour ESPRIT que pour MUSIC. C'est un argument en faveur de MUSIC pour les configurations exigeantes en précision, malgré son coût plus élevé.

17.9.3 Maximum de vraisemblance

Sous les hypothèses de régularité usuelles, le maximum de vraisemblance (ML) est asymptotiquement efficient : il peut atteindre la CRB lorsque $K \rightarrow \infty$. Son coût de calcul est typiquement prohibitif (optimisation multi-dimensionnelle) mais des méthodes itératives (alternating projection, Newton, EM, IQML) le rendent tractable pour $L \leq 4$ ou 5.

17.10 Implémentation : calcul numérique de la CRB

Listing 17.1 – CRB stochastique pour DOA

```

1 import numpy as np
2
3 def steering_and_derivative(theta, N, d_over_lambda=0.5):
4     m = np.arange(N)
5     phase = -2*np.pi * d_over_lambda * np.sin(theta) * m
6     a = np.exp(1j * phase)
7     da = a * (-1j * 2*np.pi * d_over_lambda * np.cos(theta) * m)
8     return a, da
9
10 def crb_stochastic(thetas, Ps, sigma2, N, K, d_over_lambda=0.5):
11     """
12     thetas : (L,) angles (rad)
13     Ps      : (L,L) covariance des sources

```

```

14     sigma2 : variance du bruit
15     K      : nombre de snapshots
16     """
17     L = len(thetas)
18     assert Ps.shape == (L, L)
19     assert sigma2 > 0.0 and K > 0 and N > L
20     A = np.zeros((N, L), dtype=complex)
21     D = np.zeros((N, L), dtype=complex)
22     for ell, th in enumerate(thetas):
23         a, da = steering_and_derivative(th, N, d_over_lambda)
24         A[:, ell] = a
25         D[:, ell] = da
26
27     R = A @ Ps @ A.conj().T + sigma2 * np.eye(N)
28     R = 0.5 * (R + R.conj().T)
29     G = A.conj().T @ A
30     if np.linalg.cond(G) > 1e12 or np.linalg.cond(R) > 1e12:
31         raise np.linalg.LinAlgError("CRB mal conditionnée")
32     PA_perp = np.eye(N) - A @ np.linalg.solve(G, A.conj().T)
33     M = D.conj().T @ PA_perp @ D # L x L
34     H = Ps @ A.conj().T @ np.linalg.solve(R, A) @ Ps
35     F = 2 * K / sigma2 * np.real(M * H.T)
36     return np.linalg.pinv(F)

```

17.11 Synthèse

Les bornes de Cramér-Rao fixent les performances ultimes de tout estimateur de DOA non biaisé. Pour un ULA à $d = \lambda/2$, la variance de l'estimation d'une source unique varie comme $1/(K \cdot \text{SNR} \cdot N^3 \cdot \cos^2 \theta)$: précision proportionnelle au cube du nombre d'antennes, inverse du SNR et du nombre de snapshots, dégradation en bordure de zone visible. MUSIC et ESPRIT atteignent ces bornes asymptotiquement pour une source unique, mais leur écart à la CRB se creuse pour des sources proches ou bas SNR. Le maximum de vraisemblance est l'estimateur efficient de référence quand le coût le permet.

Le chapitre suivant clôt la Partie V avec les méthodes *parcimonieuses et bayésiennes*, qui répondent à un autre type de problèmes : K très petit, sources cohérentes, off-grid. Ces méthodes plus récentes ne sont pas en concurrence frontale avec MUSIC et ESPRIT : elles couvrent des régimes différents. Quand MUSIC fonctionne (assez de snapshots, sources non cohérentes), c'est lui qu'on emploie ; quand il échoue, sparse ou SBL prennent le relais. Un panorama solide des estimateurs de DOA doit inclure les uns et les autres.

Chapitre 18

DOA parcimonieuse et bayésienne

MUSIC et ESPRIT peuvent approcher la borne de Cramér-Rao en régime asymptotique favorable, mais reposent sur des hypothèses parfois inadéquates : nombre de snapshots suffisant ($K \geq$ quelques N), sources non cohérentes, connaissance préalable de L . Pour les situations où ces hypothèses échouent — snapshot unique, sources cohérentes (trajets multiples), L inconnu — d'autres approches sont nécessaires. Ce chapitre introduit les méthodes *parcimonieuses* (compressed sensing appliqué à la DOA) et *bayésiennes* (*Sparse Bayesian Learning*), qui ont émergé après 2005 et complètent l'arsenal classique.

Ces méthodes représentent un changement de point de vue : on ne cherche plus à diagonaliser une covariance, mais à expliquer un signal observé comme la somme d'un petit nombre de composantes choisies dans un grand dictionnaire de directions candidates. Le problème devient algébriquement très différent : pas de décomposition propre, pas de scan angulaire, mais un programme convexe (ou non-convexe selon la formulation) qui force la parcimonie de la solution. La théorie sous-jacente est celle du *compressed sensing* de Donoho [14] et Candès–Romberg–Tao [15], dont la profondeur dépasse largement le cadre de ce livre ; on en présente ici les versions adaptées à la DOA, qui se révèlent particulièrement robustes dans les régimes où les méthodes classiques échouent.

18.1 Reformulation en problème sparse

18.1.1 Discrétisation de la zone visible

On discrétise la zone visible en G angles candidats $\{\bar{\theta}_g\}_{g=1}^G$ avec $G \gg N$ (souvent $G \sim 10N$ à $100N$). On construit le *dictionnaire* surcomplet

$$\mathbf{D} = [\mathbf{a}(\bar{\theta}_1), \dots, \mathbf{a}(\bar{\theta}_G)] \in \mathbb{C}^{N \times G}.$$

Si les sources sont aux angles $\theta_1, \dots, \theta_L$ et que ces angles tombent (approximativement) sur la grille, on peut écrire

$$\mathbf{x}(n) = \mathbf{D} \mathbf{s}_g(n) + \mathbf{n}(n),$$

où $\mathbf{s}_g(n) \in \mathbb{C}^G$ est un vecteur *sparse* : seules les L composantes correspondant aux vraies sources sont non nulles.

18.1.2 Pourquoi c'est utile

Cette reformulation transforme la DOA en un problème *d'identification du support sparse* de $\mathbf{s}_g(n)$. Avantages :

- pas besoin de connaître L *a priori* ;
- un seul snapshot peut suffire (compressed sensing) ;

- insensible aux sources cohérentes ;
- structure naturelle pour des contraintes complémentaires (régularité temporelle, structure de groupes).

Inconvénient : on est limité par la *résolution de la grille*. Si une source vraie tombe entre deux nœuds de grille, elle est mal représentée. On parle de *basis mismatch* ou de problème *off-grid*. Plusieurs remèdes existent (grille raffinée, optimisation continue, *atomic norm minimization*).

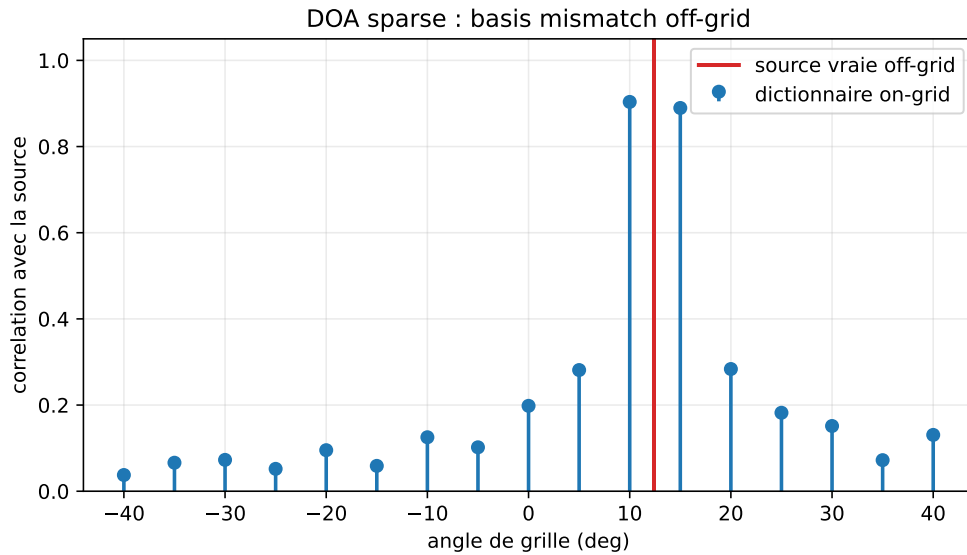


FIGURE 18.1 – Visualisation du basis mismatch ($N = 12$, source vraie à 12.4° , grille tous les 5°) : une source située entre deux points de grille se projette sur plusieurs colonnes voisines du dictionnaire.

18.1.3 Condition d'identifiabilité

Le dictionnaire $\mathbf{D}$ est large : $G \gg N$. Le problème $\mathbf{x} = \mathbf{D}\mathbf{s}_g$ possède donc une infinité de solutions si aucune contrainte n'est ajoutée. La parcimonie rend le problème identifiable lorsque les colonnes actives de $\mathbf{D}$ ne sont pas trop corrélées. Une mesure simple est la cohérence mutuelle :

$$\mu_D = \max_{i \neq j} \frac{|\mathbf{d}_i^H \mathbf{d}_j|}{\|\mathbf{d}_i\|_2 \|\mathbf{d}_j\|_2}.$$

Plus la grille est fine, plus deux colonnes voisines se ressemblent, et plus μ_D augmente. On gagne en résolution nominale, mais on rend le problème numérique plus mal conditionné. C'est un point souvent oublié : raffiner la grille ne rend pas automatiquement l'estimation meilleure.

Une règle de lecture est :

grille grossière $\Rightarrow$ basis mismatch,

grille trop fine $\Rightarrow$ colonnes quasi colinéaires.

Les méthodes parcimonieuses vivent exactement dans ce compromis.

18.2 Régularisation ℓ_1 et LASSO

18.2.1 Snapshot unique

Pour un seul snapshot $\mathbf{x}$, on résout

$$\min_{\mathbf{s}_g} \|\mathbf{x} - \mathbf{D}\mathbf{s}_g\|_2^2 + \mu \|\mathbf{s}_g\|_1.$$

La pénalité ℓ_1 favorise les solutions parcimonieuses. C'est le *LASSO* en estimation statistique, ou *Basis Pursuit Denoising* en signal processing. Tout solveur convexe le résout (coordinate descent, ADMM, FISTA...).

Le choix de μ est délicat. Pratiquement :

- μ petit : peu de parcimonie, des sources fantômes ;
- μ grand : trop parcimonie, on perd des sources réelles.
- Heuristique : $\mu \approx \sigma \sqrt{2 \log G}$ (seuil de détection de Donoho [14]).

18.2.2 Multiples snapshots : $\ell_{2,1}$ MMV

Pour K snapshots empilés en $\mathbf{S}_g \in \mathbb{C}^{G \times K}$, on a $\mathbf{X} = \mathbf{D} \mathbf{S}_g + \mathbf{N}$. On suppose que toutes les colonnes partagent le même support sparse (*Multiple Measurement Vectors*, MMV). Le problème devient

$$\min_{\mathbf{S}_g} \|\mathbf{X} - \mathbf{D} \mathbf{S}_g\|_F^2 + \mu \sum_{g=1}^G \|\mathbf{S}_g[g, :]\|_2,$$

où $\mathbf{S}_g[g, :]$ est la g -ième ligne. La norme $\ell_{2,1}$ pénalise le *nombre de lignes non nulles*, donc le nombre de sources.

18.3 ℓ_1 -SVD

Calculer le LASSO MMV sur K colonnes a un coût lourd. Malioutov, Çetin & Willsky [16] proposent ℓ_1 -SVD : on compresse d'abord les données par SVD, en gardant uniquement les L' premiers vecteurs singuliers (avec L' choisi par MDL ou par défaut $L' = N$), puis on applique $\ell_{2,1}$ -LASSO sur cette projection. Gain de calcul substantiel sans perte d'information signal.

18.4 Sparse Bayesian Learning (SBL)

18.4.1 Modèle hiérarchique

SBL [17, 18] place un prior gaussien hiérarchique sur $\mathbf{s}_g$:

$$\mathbf{s}_g \sim \mathcal{CN}(\mathbf{0}, \mathbf{\Gamma}), \quad \mathbf{\Gamma} = \text{diag}(\gamma_1, \dots, \gamma_G),$$

où chaque γ_g est lui-même un hyperparamètre. Le bruit reste $\mathbf{n} \sim \mathcal{CN}(\mathbf{0}, \sigma^2 \mathbf{I}_N)$.

La distribution marginale de $\mathbf{x}$ est

$$\mathbf{x} \sim \mathcal{CN}(\mathbf{0}, \sigma^2 \mathbf{I}_N + \mathbf{D} \mathbf{\Gamma} \mathbf{D}^H).$$

On apprend les γ_g par maximum de vraisemblance des données ; *la plupart se concentrent à zéro automatiquement*, ne laissant non nuls que ceux correspondant aux vraies sources. C'est la *sélection automatique* de support sparse.

18.4.2 Algorithme

L'algorithme classique alterne :

1. Étape E : on calcule la moyenne et la covariance de $\mathbf{s}_g | \mathbf{x}$.
2. Étape M : on met à jour les γ_g via les statistiques postérieures.

La règle de mise à jour est :

$$\gamma_g^{\text{new}} = \frac{|\mu_g|^2 + \Sigma_{g,g}}{1},$$

où $\boldsymbol{\mu}$ et $\boldsymbol{\Sigma}$ sont les moyenne et covariance postérieures de $\mathbf{s}_g$ étant donné $\mathbf{x}$.

18.4.3 Dérivation des grandeurs postérieures

Avec

$$\mathbf{x} = \mathbf{D}\mathbf{s}_g + \mathbf{n}, \quad \mathbf{s}_g \sim \mathcal{CN}(\mathbf{0}, \mathbf{\Gamma}), \quad \mathbf{n} \sim \mathcal{CN}(\mathbf{0}, \sigma^2 \mathbf{I}),$$

la loi postérieure reste gaussienne :

$$p(\mathbf{s}_g | \mathbf{x}) = \mathcal{CN}(\boldsymbol{\mu}, \boldsymbol{\Sigma}).$$

En complétant le carré dans l'exposant gaussien, on obtient :

$$\boldsymbol{\Sigma} = \left(\sigma^{-2} \mathbf{D}^H \mathbf{D} + \mathbf{\Gamma}^{-1} \right)^{-1}$$

et

$$\boldsymbol{\mu} = \sigma^{-2} \boldsymbol{\Sigma} \mathbf{D}^H \mathbf{x}.$$

La mise à jour

$$\gamma_g \leftarrow |\mu_g|^2 + \Sigma_{g,g}$$

est donc simplement l'énergie postérieure moyenne de la composante g . Si une direction n'explique pas les données, sa moyenne et sa variance postérieures deviennent faibles, puis γ_g se contracte vers zéro.

18.4.4 Cas multi-snapshots

Pour plusieurs snapshots partageant le même support :

$$\mathbf{X} = \mathbf{D}\mathbf{S}_g + \mathbf{N},$$

la même idée donne une variance commune γ_g par ligne de $\mathbf{S}_g$. La mise à jour devient :

$$\gamma_g^{\text{new}} = \frac{1}{K} \sum_{n=1}^K |\mu_{g,n}|^2 + \Sigma_{g,g}.$$

Le terme moyen sur les snapshots stabilise fortement l'estimation. SBL multi-snapshots est souvent plus robuste que LASSO-MMV lorsque les sources sont proches, car il réestime simultanément le support et la puissance des composantes.

18.4.5 Avantages de SBL pour la DOA

- Pas de paramètre de régularisation à régler (contrairement au LASSO et μ).
- Converge vers des solutions plus sparses que le LASSO en pratique.
- Performances supérieures à bas SNR et faible K .
- Estimation simultanée du nombre de sources : les γ_g non négligeables sont automatiquement les sources.

18.4.6 Inconvénients

- Coût $\mathcal{O}(N^3 + N^2 G)$ par itération.
- Pas de garantie globale de convergence (non convexe).
- Toujours soumis au *basis mismatch*.

18.5 Atomic Norm Minimization (off-grid)

Pour s'affranchir de la grille, on remplace l' ℓ_1 discret par sa contrepartie continue : la *norme atomique*

$$\|\mathbf{x}\|_{\mathcal{A}} = \inf \left\{ \sum_{\ell} c_{\ell} : \mathbf{x} = \sum_{\ell} c_{\ell} \mathbf{a}(\theta_{\ell}), c_{\ell} \geq 0 \right\}.$$

Pour un ULA, on démontre notamment dans les travaux off-grid de Tang et al. [19] que cette norme se calcule par un programme *semi-défini positif* (SDP) sur une matrice de Toeplitz hermitienne. L'estimation se fait ensuite en localisant les fréquences spatiales du polynôme dual sur le cercle unité.

Avantage. Estimation *continue* en angle, sans discrétisation, pouvant approcher les limites asymptotiques dans les régimes favorables.

Limite. Restreint aux ULA, coût SDP en $\mathcal{O}(N^{3.5})$.

18.5.1 Forme SDP pour un ULA

Pour un ULA, les steering vectors sont des exponentielles complexes. La norme atomique possède alors une représentation semi-définie. Une forme standard consiste à résoudre :

$$\min_{\mathbf{x}_0, \mathbf{T}, t} \frac{1}{2} \|\mathbf{x} - \mathbf{x}_0\|_2^2 + \tau \left(\frac{1}{2N} \text{tr}(\mathbf{T}) + \frac{1}{2}t \right)$$

sous la contrainte :

$$\begin{bmatrix} \mathbf{T} & \mathbf{x}_0 \\ \mathbf{x}_0^H & t \end{bmatrix} \succeq 0, \quad \mathbf{T} \text{ Toeplitz hermitienne.}$$

La matrice de Toeplitz encode une mesure spectrale continue. Après résolution, les DOA sont extraites par décomposition de Vandermonde de $\mathbf{T}$, ou par localisation des zéros du polynôme dual.

Cette formulation explique la différence profonde avec le LASSO : le LASSO choisit parmi G colonnes fixées ; la norme atomique choisit dans un continuum de colonnes. Elle élimine le basis mismatch, mais paie ce gain par un coût d'optimisation nettement plus élevé et par une dépendance forte à la structure ULA.

18.6 Erreurs fréquentes d'interprétation

Les méthodes sparse ne sont pas une version magique de MUSIC. Elles résolvent un autre problème, avec d'autres fragilités.

- Un pic sparse n'est pas toujours une source : un mauvais μ ou une grille trop corrélée crée des supports artificiels.
- SBL sélectionne automatiquement le support, mais son optimisation reste non convexe ; l'initialisation et l'échelle du bruit comptent.
- L'atomic norm évite la grille, mais suppose un modèle ULA idéal ; une calibration imparfaite réintroduit un mismatch.
- En régime asymptotique favorable (K grand, sources séparées, manifold calibré), MUSIC/ESPRIT restent souvent plus simples et plus prévisibles.

18.7 Méthodes Bayésiennes variationnelles

Pour les versions plus modernes (*variational SBL*, *Bayesian sparse learning* avec modèles structurés), on combine inférence variationnelle et structures de groupe (blocs sparses, group LASSO, sparsité dynamique). On obtient des estimateurs robustes même pour $K = 1$ et L inconnu.

18.8 Comparaison avec MUSIC/ESPRIT

Critère	MUSIC	ℓ_1 -SVD	SBL
$K = 1$ snapshot	échec	possible	possible
Sources cohérentes	nécessite smoothing	ok	ok
L inconnu	via MDL/AIC	automatique	automatique
Off-grid	non concerné	oui (basis mismatch)	oui (idem)
Paramètre à régler	L ou seuil	μ	aucun
Coût	$\mathcal{O}(N^3)$	$\mathcal{O}(N^2G)$	$\mathcal{O}(N^2G)$
Performance à bas SNR	limitée	bonne	supérieure

18.9 Recommandations pratiques

Quand utiliser sparse / bayésien ?

- snapshot unique ou $K < N$;
- sources cohérentes (trajets multiples) sans pouvoir faire de smoothing ;
- L inconnu et difficile à estimer ;
- priors structurels disponibles (continuité temporelle, groupes, parcimonie spatiale).

Quand préférer MUSIC/ESPRIT ?

- K de l'ordre de quelques N ou plus, sources non cohérentes ;
- besoin d'une estimation rapide (temps réel sur FPGA) ;
- performance asymptotique élevée visée (proche de la CRB) sur des sources bien séparées.

18.10 Implémentation : ℓ_1 DOA snapshot unique

Listing 18.1 – LASSO pour DOA snapshot unique (via ADMM simplifié)

```

1 import numpy as np
2
3 def soft_thresh(z, t):
4     return np.sign(z) * np.maximum(np.abs(z) - t, 0.0)
5
6 def lasso_doa(x, D, mu, n_iter=200, rho=1.0):
7     """
8     x : (N,) snapshot complexe
9     D : (N, G) dictionnaire de steerings
10    mu : régularisation
11    Retour : sg (G,) sparse
12    """
13    N, G = D.shape
14    assert x.shape == (N,)
15    assert mu >= 0.0 and rho > 0.0 and n_iter > 0

```



```

16     # ADMM sur problème complexe : on traite Re/Im séparément
17     Dr = np.vstack([np.hstack([D.real, -D.imag]),
18                     np.hstack([D.imag, D.real])])
19     xr = np.concatenate([x.real, x.imag])
20     z = np.zeros(2*G); u = np.zeros(2*G); s = np.zeros(2*G)
21     DtD = Dr.T @ Dr + rho * np.eye(2*G)
22     if np.linalg.cond(DtD) > 1e12:
23         raise np.linalg.LinAlgError("système ADMM mal conditionné")
24     Dtx = Dr.T @ xr
25     for _ in range(n_iter):
26         s = np.linalg.solve(DtD, Dtx + rho*(z - u))
27         z = soft_thresh(s + u, mu / rho)
28         u += s - z
29     return z[:G] + 1j*z[G:]

```

18.11 Synthèse

Les méthodes parcimonieuses et bayésiennes étendent l'estimation de DOA au-delà du régime de validité de MUSIC et ESPRIT :

- snapshot unique et sources cohérentes : oui ;
- nombre de sources inconnu : auto-sélectionné ;
- off-grid via *atomic norm* sur ULA.

Le coût est plus élevé et les garanties asymptotiques moins fortes, mais leur flexibilité les rend précieuses dans les cas pratiques que les méthodes « classiques » ne couvrent pas.

La Partie VI quitte les hypothèses simplificatrices de bande étroite, de réseau linéaire et d'antenne unique : elle étudie le beamforming large bande, MIMO/Massive MIMO et les géométries avancées. C'est dans cette partie qu'on retrouve les systèmes industriels les plus contemporains : 5G, Wi-Fi 6, radars automobiles, antennes satellitaires phased-array de constellations basse-orbite. Les formalismes restent les mêmes que ceux développés dans les parties précédentes ; seules les hypothèses simplificatrices — bande étroite, polarisation unique, antenne unique côté émetteur — sont relâchées une à une.

Sixième partie

Extensions

Introduction aux extensions

Les chapitres suivants étendent le modèle de base construit jusque-là : réseau ULA, bande étroite, champ lointain, polarisation unique et réception mono-utilisateur. Chacune de ces hypothèses est utile pour comprendre les fondations, mais chacune finit par céder dans les systèmes modernes.

Le large bande introduit une dépendance fréquentielle du beamforming. Le MIMO ajoute des antennes à l'émission et transforme le canal en matrice. Les géométries 2D/3D remplacent l'angle unique par l'azimut et l'élévation. Le champ proche remplace le simple pointage angulaire par une focalisation portée-angle. La polarisation ajoute des composantes vectorielles au steering vector.

Ces chapitres ne prétendent pas couvrir exhaustivement chacun de ces domaines. Chacun d'eux pourrait remplir un livre spécialisé. Leur rôle est plus précis : montrer comment les hypothèses initiales se modifient, quelles formules changent, quels algorithmes restent valables et quelles limites pratiques apparaissent.

La question directrice reste donc toujours la même :

qu'est-ce qui change dans le steering vector, la covariance et les contraintes ?

Si cette question est claire, les extensions ne sont plus une collection de cas particuliers. Elles deviennent des variations contrôlées autour du même noyau théorique.

Chapitre 19

Beamforming large bande

L'hypothèse bande étroite (chapitre 4), qui réduit un retard temporel à une simple multiplication par $e^{-j2\pi f_0\tau}$, n'est plus valable lorsque la condition $BD/c \ll 1$ est violée. C'est le cas en radar large bande, en SAR haute résolution, en sonar, en acoustique des microphones et dans certaines liaisons satellitaires. Le présent chapitre adapte le formalisme à ce régime. Il introduit le *true time delay* (TTD), le *filtre FIR par antenne*, et le *beamforming par sous-bandes* ; il étudie le *frequency invariant beamforming* (FIB) et discute les compromis matériels.

Sortir de la bande étroite n'est pas qu'une affaire de calcul plus gourmand : c'est un changement de nature du problème. En bande étroite, un beamformer est entièrement caractérisé par N poids complexes ; en large bande, il faut $N \cdot L_h$ coefficients (un filtre FIR par antenne) ou un traitement par sous-bandes équivalent. La conception devient un problème d'optimisation à beaucoup plus de variables, et le beam pattern lui-même devient une fonction *bidimensionnelle* de l'angle *et* de la fréquence. Tous les algorithmes des parties précédentes admettent une extension large bande, mais cette extension impose des choix architecturaux non triviaux qui font partie intégrante de la conception système.

Critère	Bande étroite	Large bande
Modèle spatial	phase à f_0	retard ou sous-bandes
Poids	N scalaires complexes	NL_h coefficients ou M sous-bandes
Pattern	$B(\theta)$	$B(\theta, f)$
Risque principal	lobes / mismatch	beam squint, désalignement temporel
Implémentation	phaseurs	TTD, FIR, FFT par sous-bandes
Question clé	où pointer ?	où pointer à chaque fréquence ?

19.1 Pourquoi la bande étroite ne suffit plus

Reprenons le retard différentiel $\tau_m = md \sin \theta / c$. Pour un signal bande étroite, le signal reçu sur l'antenne m est $x_m(t) \approx s(t) e^{-j2\pi f_0 \tau_m}$, et la sortie d'un DAS qui compense τ_m par $e^{+j2\pi f_0 \tau_m}$ vaut $s(t)$ à toutes les fréquences — exactement.

Si la bande est large, l'enveloppe $s(t)$ varie significativement entre t et $t - \tau_{\max}$. La compensation pure phase compense la porteuse mais pas l'enveloppe, et le signal aligné sur deux antennes extrêmes du réseau *ne coïncide plus*. Le gain de réseau chute, le beam pattern devient *fréquence-dépendant*, et les algorithmes adaptatifs sous-performent ou divergent.

Quantitativement, la condition de validité bande étroite était

$$\frac{BD}{c} \ll 1.$$

Au-delà, deux familles de techniques s'imposent : les *lignes à retard* (TTD) qui appliquent un vrai retard temporel, et les *sous-bandes* qui transforment le signal large bande en plusieurs sous-signaux bande étroite traités indépendamment.

19.2 Beam squint

Le défaut le plus visible d'un beamformer purement phase en large bande est le *beam squint*. Les poids sont calculés pour une fréquence centrale f_0 :

$$w_m = \frac{1}{N} e^{-j2\pi f_0 m d \sin \theta_0 / c}.$$

À une fréquence $f \neq f_0$, le maximum du faisceau n'apparaît plus à θ_0 , mais à l'angle θ_f qui vérifie approximativement :

$$f \sin \theta_f = f_0 \sin \theta_0.$$

Donc :

$$\sin \theta_f = \frac{f_0}{f} \sin \theta_0.$$

Le lobe principal glisse avec la fréquence. Plus la bande relative B/f_0 est grande et plus θ_0 est éloigné du broadside, plus le dépointage est visible.

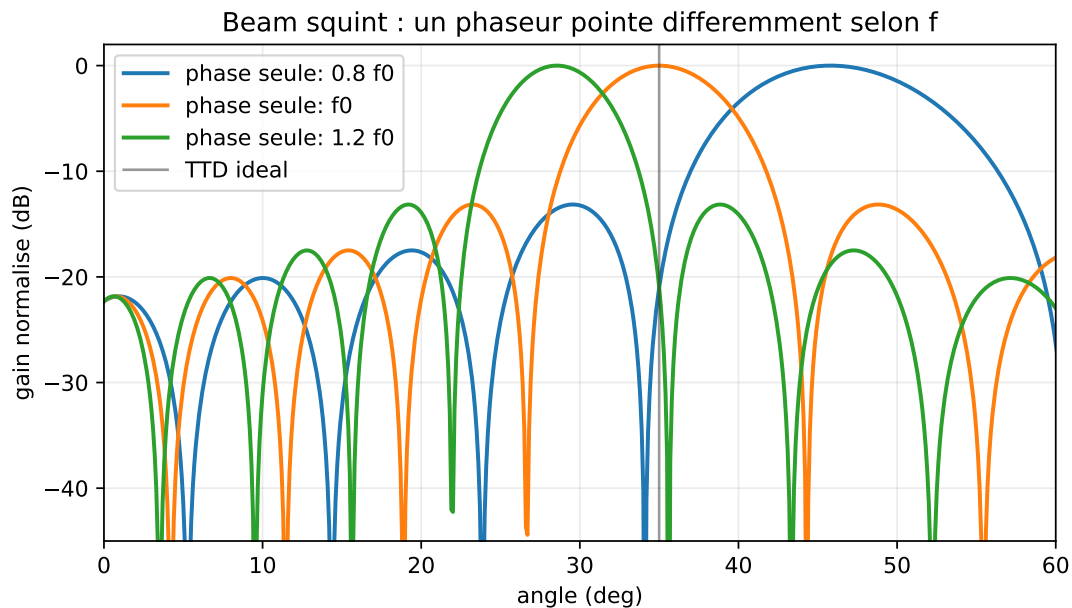


FIGURE 19.1 – Beam squint avec un réseau phase-shifter ($N = 16$, pointage 35° , $d = \lambda_0/2$, fréquences $0.8f_0$, f_0 , $1.2f_0$) : les poids calculés à f_0 pointent correctement à la fréquence centrale, mais le lobe se décale hors fréquence centrale.

Le true time delay corrige ce problème parce qu'il applique un retard temporel réel, dont la phase varie automatiquement avec f :

$$e^{-j2\pi f \tau_m}.$$

Un déphaseur fixe ne peut être exact qu'à une seule fréquence ; une ligne à retard peut rester exacte sur toute la bande.

19.3 True time delay beamforming

19.3.1 Principe

Plutôt que de multiplier par une phase constante, on retarde réellement le signal de l'antenne m de τ_m . La sortie du beamformer est

$$y(t) = \frac{1}{N} \sum_{m=0}^{N-1} x_m(t + \tau_m) \quad \text{avec } \tau_m = -md \sin \theta_0 / c.$$

Le retard appliqué annule *exactement* le retard physique, indépendamment de la fréquence. Le résultat est un *beam pattern fréquence-invariant* dans la direction θ_0 .

19.3.2 Implémentation

Analogique. Lignes à retard variables (LdR). Réalisées historiquement avec des câbles de longueurs variables (radar à balayage électronique) ou des lignes *photoniques* (technologie plus récente). Précision sub-picoseconde réalisable.

Numérique. À une cadence d'échantillonnage F_s , on peut appliquer un retard sous-échantillon par interpolation. Un retard fractionnaire $\tau_m = (k_m + \alpha_m)/F_s$ se décompose en :

- retard entier k_m : décalage d'index ;
- retard fractionnaire α_m : filtrage par un FIR de *filtre Lagrange* ou *filtre sinc fenêtré*.

Coût : un filtre FIR par antenne. Précision : selon l'ordre du filtre et la bande traitée ; des ordres de quelques dizaines à une centaine de taps sont fréquents, mais la résolution effective doit être vérifiée par simulation ou mesure.

19.4 Beamforming en sous-bandes

19.4.1 Décomposition

On filtre $x_m(t)$ par une banque de P filtres passe-bande étroits de largeur B/P . À l'intérieur de chaque sous-bande p , le signal est quasi-bande étroite et on lui applique un beamforming bande étroite classique avec un steering vector dépendant de la fréquence centrale f_p de la sous-bande :

$$a_m^{(p)}(\theta) = e^{-j2\pi f_p m d \sin \theta / c}.$$

19.4.2 Recomposition

Les sorties des P sous-bandes sont sommées (ou filtrées-puis-sommées) pour reconstituer le signal large bande complet. Si la banque de filtres est à reconstruction parfaite (cosine modulated, DFT modulated), on récupère exactement le signal d'origine.

19.4.3 Avantages et limites

Avantages.

- Réutilisation directe de tous les algorithmes bande étroite (MVDR, LCMV, MUSIC) à l'intérieur de chaque sous-bande.
- Implémentation efficace par FFT (DFT *filterbank*).
- Permet une adaptation indépendante par sous-bande.

Limites.

- Coût : P traitements bande étroite par snapshot.
- Bord des sous-bandes : besoin de chevauchements pour éviter les artefacts.
- Choix de P : trop petit, l'hypothèse bande étroite reste violée ; trop grand, on multiplie le coût.

Règle pratique. On choisit P tel que $B/P \cdot D/c \ll 1$, soit $P \gg BD/c$.

19.5 Beamforming par filtre FIR par antenne

Une formulation unifiée, plus générale que le TTD et que les sous-bandes, est le *filter-and-sum beamformer* : on applique à chaque antenne m un filtre FIR $\mathbf{h}_m(t)$ de longueur L_h , puis on somme :

$$y(t) = \sum_{m=0}^{N-1} \sum_{k=0}^{L_h-1} h_m[k] x_m(t - kT_s).$$

On dispose alors de NL_h degrés de liberté — bien plus que les N degrés du beamforming bande étroite. Ce surcroît permet :

- de contrôler le beam pattern à *chaque fréquence* ;
- d'imposer une réponse fréquence-invariante par construction ;
- d'optimiser des critères complexes (LCMV large bande, MVDR large bande).

19.5.1 Formulation matricielle

On empile :

$$\mathbf{x}_t(t) = [x_0(t), x_0(t - T_s), \dots, x_0(t - (L_h - 1)T_s), x_1(t), \dots, x_{N-1}(t - (L_h - 1)T_s)]^T \in \mathbb{C}^{NL_h}.$$

Le vecteur de poids $\mathbf{w} \in \mathbb{C}^{NL_h}$ contient les NL_h coefficients. La sortie est $y(t) = \mathbf{w}^H \mathbf{x}_t(t)$. Les algorithmes (MVDR, LCMV, etc.) s'appliquent identiquement, à la dimension du problème près.

19.5.2 Steering vector large bande

À fréquence f , le steering vector « augmenté » est

$$\mathbf{a}_t(\theta, f) = \mathbf{a}(\theta, f) \otimes \mathbf{d}(f),$$

où $\mathbf{d}(f) = [1, e^{-j2\pi f T_s}, \dots, e^{-j2\pi f (L_h-1)T_s}]^T$ est le vecteur des retards temporels et $\otimes$ le produit de Kronecker.

19.6 Frequency Invariant Beamforming (FIB)

19.6.1 Objectif

Concevoir un beam pattern qui soit *indépendant de la fréquence* sur toute la bande utile : un faisceau aussi propre à 1 GHz qu'à 2 GHz. Indispensable pour les applications acoustiques large bande (téléconférence, contrôle actif) et les radars à très large bande.

19.6.2 Méthodes classiques

Beam pattern défini par construction. On définit une réponse spatiale désirée $B_{\text{désiré}}(\theta)$ *indépendante de f* , et on optimise les poids fréquentiels $\mathbf{w}(f)$ pour l'approximer à chaque f . Le problème devient une famille de problèmes quadratiques indépendants.

Réseaux à éléments « auto-similaires ». En réutilisant la même réponse géométrique à toutes les fréquences — typiquement un *nested array* aux espacements $d, 2d, 4d, \dots$ —, on peut concevoir un beam pattern dont la résolution change avec la fréquence mais dont la forme normalisée reste constante.

Constraint-based design. On impose des contraintes linéaires de gain $B(\theta_q, f_p) = b_{q,p}$ sur une grille (θ, f) , et on résout par LCMV étendu.

19.7 Modèles statistiques pour le bruit large bande

Le bruit large bande est typiquement coloré dans la fréquence (réponse des LNA, plancher thermique non plat). On le caractérise par sa covariance fréquence-dépendante $\sigma^2(f)\mathbf{I}_N$ (bruit spatialement blanc mais coloré en fréquence) ou par une matrice spatio-fréquentielle complète. Les algorithmes MVDR et LCMV en sous-bandes traitent automatiquement le cas où σ^2 varie avec f .

19.8 DOA large bande

Pour estimer les DOA en large bande, plusieurs approches existent.

Sous-bandes + MUSIC. On applique MUSIC dans chaque sous-bande, puis on combine les pseudo-spectres (somme, produit ou moyenne).

Coherent Signal-Subspace Method (CSSM). On focalise les sous-bandes vers une fréquence de référence par une transformation unitaire, puis on applique MUSIC sur la covariance « focalisée ».

Wideband ESPRIT. Extension d'ESPRIT au cas multi-fréquences.

Test Subspace Fitting. Optimisation directe sur la matrice de steering augmentée.

19.9 Implémentation par sous-bandes

Listing 19.1 – Beamforming par sous-bandes via FFT

```

1 import numpy as np
2
3 def subband_beamform(X, theta0, N, fs, fc, d, c=3e8, M_fft=256, hop=128):
4     """
5     X      : (N, T) snapshots à fs, signal large bande
6     theta0 : direction cible (rad)
7     fc     : fréquence centrale
8     fs     : fréquence d'échantillonnage
9     d      : espacement inter-antennes (m)
10    M_fft  : taille de FFT
11    hop    : pas entre FFT successives
12    """
13    T = X.shape[1]
14    n_frames = (T - M_fft) // hop + 1
15    y = np.zeros((n_frames, M_fft), dtype=complex)
16    freqs = fs * np.arange(M_fft) / M_fft - fs/2 # symétrique
17    m = np.arange(N)
18    for fr in range(n_frames):
19        seg = X[:, fr*hop : fr*hop + M_fft]
20        # FFT par antenne
21        S = np.fft.fft(seg, axis=1)
22        for bin_idx in range(M_fft):
23            fk = fc + freqs[bin_idx] # ou directement freqs si déjà en bande de base
24            a = np.exp(-1j * 2*np.pi * fk * d * np.sin(theta0) / c * m)
25            w = a / N
26            y[fr, bin_idx] = w.conj() @ S[:, bin_idx]
27    # IFFT pour revenir au temps

```



```
28     y_time = np.fft.ifft(y, axis=1)
29     return y_time
```

19.10 Synthèse

Le beamforming large bande étend le formalisme pour les signaux qui violent l'hypothèse bande étroite. Trois familles de techniques :

- *true time delay* : analogique ou via FIR fractionnaire par antenne ;
- *sous-bandes* : décomposition par banc de filtres / FFT, beamforming bande étroite indépendant ;
- *filter-and-sum* : NL_h poids, contrôle complet du beam pattern à toutes les fréquences.

Le coût matériel et algorithmique augmente significativement, mais les techniques modernes (banques de filtres rapides, FPGA dédiés, photoniques RF) le rendent praticable.

Le chapitre suivant aborde l'autre grande extension du beamforming moderne : le *MIMO* et le *massive MIMO*, où le beamforming n'est plus seulement réception mais aussi émission, et où des architectures matérielles hybrides analogique/numérique deviennent indispensables. Le contraste est instructif : la Partie I supposait un seul émetteur scalaire et un réseau récepteur ; la 5G inverse souvent le problème en plaçant le réseau du côté émetteur (station de base) et un terminal mono- ou multi-antenne du côté récepteur. Les mêmes outils algébriques — steering vectors, beam patterns, décomposition en sous-espaces — s'appliquent symétriquement aux deux configurations.

Chapitre 20

Beamforming MIMO et massive MIMO

Tous les chapitres précédents considéraient un beamforming en *réception* avec un seul utilisateur (une seule direction cible). Les systèmes modernes — 5G, Wi-Fi récent, satellitaires — exploitent plusieurs antennes *aux deux extrémités* de la liaison (*MIMO*, *Multiple Input Multiple Output*) et un grand nombre d'antennes côté station de base (*massive MIMO*). Le beamforming y devient à la fois un outil de *réception adaptative* et un outil de *précodage à l'émission*. Le présent chapitre adapte les formalismes développés à ce contexte.

Le MIMO ne se contente pas d'ajouter des antennes ; il transforme la notion même de canal de communication. Là où une liaison classique peut être vue comme un seul « tuyau » scalaire, une liaison MIMO est un *ensemble de tuyaux parallèles*, dont le nombre est limité par le rang du canal et donc, en pratique, par le minimum entre nombre d'antennes émettrices et réceptrices. Cette multiplicité ouvre deux usages : le *multiplexage spatial*, où chaque tuyau transporte un flux indépendant ; et le *beamforming de diversité*, où plusieurs tuyaux transportent la même information codée pour résister aux évanouissements. Les systèmes actuels combinent les deux de manière adaptative selon les conditions de propagation.

20.1 Modèle de canal MIMO

20.1.1 Liaison point-à-point

Considérons un émetteur à N_T antennes transmettant à un récepteur à N_R antennes. Le canal est modélisé par une matrice $\mathbf{H} \in \mathbb{C}^{N_R \times N_T}$ tel que

$$\mathbf{y} = \mathbf{H} \mathbf{x} + \mathbf{n},$$

avec $\mathbf{x} \in \mathbb{C}^{N_T}$ le vecteur transmis et $\mathbf{y} \in \mathbb{C}^{N_R}$ le vecteur reçu. La matrice $\mathbf{H}$ contient les coefficients complexes de propagation de chaque antenne d'émission vers chaque antenne de réception. Elle est typiquement constante sur un *bloc de cohérence* (durée pendant laquelle elle peut être considérée stationnaire).

20.1.2 Lien avec le formalisme DOA

Si le canal résulte de quelques trajets directs ou réfléchis, on peut le décomposer en

$$\mathbf{H} = \sum_{\ell=1}^L \alpha_{\ell} \mathbf{a}_R(\theta_{R,\ell}) \mathbf{a}_T^H(\theta_{T,\ell}),$$

où $\mathbf{a}_R$ et $\mathbf{a}_T$ sont les steering vectors des deux réseaux, et α_{ℓ} les amplitudes complexes des trajets. C'est exactement la *même* structure que la matrice de steering multi-sources des chapitres précédents, mais bilatérale.

Cas spécial : un seul trajet. Pour un canal LOS unique, $\mathbf{H} = \alpha \mathbf{a}_R(\theta_R) \mathbf{a}_T^H(\theta_T)$ est de rang 1. Tous les degrés de liberté du canal sont concentrés sur une unique direction.

Cas spécial : canal de Rayleigh i.i.d. En milieu très diffractant, $\mathbf{H}$ a des éléments gaussiens i.i.d. ; ses valeurs singulières sont approximativement uniformément distribuées, et le canal peut transmettre $\min(N_T, N_R)$ flux indépendants simultanément. C'est le principe du *multiplexage spatial*.

20.2 Décomposition en valeurs singulières du canal

Théorème 20.1 (SVD du canal MIMO). *Tout canal $\mathbf{H} \in \mathbb{C}^{N_R \times N_T}$ admet la décomposition*

$$\mathbf{H} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^H,$$

où $\mathbf{U} \in \mathbb{C}^{N_R \times N_R}$, $\mathbf{V} \in \mathbb{C}^{N_T \times N_T}$ sont unitaires et $\mathbf{\Sigma} \in \mathbb{R}^{N_R \times N_T}$ contient les valeurs singulières $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$.

Diagonalisation du canal. Si l'émetteur précode par $\mathbf{x} = \mathbf{V} \tilde{\mathbf{x}}$ et le récepteur applique $\tilde{\mathbf{y}} = \mathbf{U}^H \mathbf{y}$, le canal effectif devient

$$\tilde{\mathbf{y}} = \mathbf{\Sigma} \tilde{\mathbf{x}} + \mathbf{U}^H \mathbf{n},$$

qui est diagonal : $r = \text{rg}(\mathbf{H})$ sous-canaux scalaires parallèles, chacun de gain σ_i . Cette *décomposition spatiale* est la base du multiplexage MIMO.

20.3 Deux objectifs différents : diversité et multiplexage

Le mot MIMO recouvre deux objectifs qu'il faut distinguer.

Diversité spatiale. On transmet la même information à travers plusieurs chemins spatiaux. L'objectif est de réduire la probabilité d'évanouissement profond. Le débit n'augmente pas nécessairement, mais la liaison devient plus fiable.

Multiplexage spatial. On transmet plusieurs flux indépendants en parallèle. Si le canal a le rang r , on peut idéalement transmettre jusqu'à r flux :

$$r = \text{rg}(\mathbf{H}) \leq \min(N_T, N_R).$$

Le débit augmente, mais seulement si les sous-canaux singuliers sont suffisamment puissants.

La SVD permet de voir les deux régimes. Si une seule valeur singulière domine, le système fonctionne surtout en beamforming de diversité. Si plusieurs valeurs singulières sont comparables, le multiplexage spatial devient intéressant.

20.3.1 Capacité point-à-point

Si l'émetteur connaît $\mathbf{H}$ et répartit la puissance sur les modes singuliers, la capacité vaut :

$$C = \max_{\sum_i p_i \leq P} \sum_{i=1}^r \log_2 \left(1 + \frac{p_i \sigma_i^2}{\sigma_n^2} \right).$$

La solution est le *water-filling* :

$$p_i = \left(\nu - \frac{\sigma_n^2}{\sigma_i^2} \right)_+,$$

où ν est choisi pour respecter la puissance totale. Les modes faibles ne reçoivent aucune puissance. Cette formule donne une lecture très concrète : ajouter des antennes ne suffit pas ; il faut créer des modes spatiaux réellement exploitables.

20.4 Précodage en émission

Le beamforming à l'émission consiste à choisir le vecteur $\mathbf{x}$ transmis (à partir d'un signal scalaire s) pour maximiser quelque objectif au récepteur. Les choix classiques :

Matched filter / MRT (Maximum Ratio Transmission). Le récepteur reçoit $\mathbf{y} = \mathbf{H} \mathbf{w}_T s + \mathbf{n}$, avec $\mathbf{w}_T \in \mathbb{C}^{N_T}$. Maximiser $|\mathbf{H} \mathbf{w}_T|^2$ donne $\mathbf{w}_T \propto \mathbf{H}^H \mathbf{u}$ pour un certain $\mathbf{u}$; si le récepteur applique aussi un MRC, le SNR de bout en bout est maximisé par $\mathbf{w}_T \propto \mathbf{v}_1$, premier vecteur singulier droit de $\mathbf{H}$.

Zero-Forcing (ZF). Pour K utilisateurs avec canaux $\mathbf{H}_1, \dots, \mathbf{H}_K$, le ZF impose que chaque utilisateur ne voie pas l'interférence des autres :

$$\mathbf{W}_{\text{ZF}} = \mathbf{H}^H (\mathbf{H} \mathbf{H}^H)^{-1},$$

où $\mathbf{H} = [\mathbf{H}_1^T, \dots, \mathbf{H}_K^T]^T$ empile tous les canaux. Inversion possible si $K \leq N_T$. Inconvénient : amplifie le bruit si le canal est mal conditionné.

Regularized ZF (RZF, MMSE-précodage). Régularise par $\mathbf{W} = \mathbf{H}^H (\mathbf{H} \mathbf{H}^H + \sigma^2/p \cdot \mathbf{I})^{-1}$, analogue du MVDR pour l'émission. Compromis bruit/interférence optimisé.

20.5 Massive MIMO

20.5.1 Régime $N_T \gg K$

Quand $N_T \rightarrow \infty$ (avec K utilisateurs fixe), des phénomènes remarquables apparaissent :

- les canaux $\mathbf{h}_k$ entre utilisateurs distincts deviennent *asymptotiquement orthogonaux* : $\mathbf{h}_i^H \mathbf{h}_j / N_T \rightarrow 0$;
- le bruit « se moyenne » sur les N_T antennes ;
- le précodage matched-filter (MRT) $\mathbf{w}_k \propto \mathbf{h}_k$ suffit : ZF et RZF n'apportent plus rien asymptotiquement ;
- les évanouissements rapides disparaissent (*channel hardening*).

C'est l'argument fondamental de Marzetta (2010) qui a lancé le massive MIMO comme paradigme de 5G.

20.5.2 Démonstration du durcissement de canal

Supposons un canal i.i.d.

$$\mathbf{h}_k \sim \mathcal{CN}(\mathbf{0}, \mathbf{I}_{N_T}).$$

Alors :

$$\frac{1}{N_T} \|\mathbf{h}_k\|^2 = \frac{1}{N_T} \sum_{m=1}^{N_T} |h_{k,m}|^2 \xrightarrow{N_T \rightarrow \infty} 1.$$

Par la loi des grands nombres, la norme du canal devient presque déterministe. De même, pour deux utilisateurs indépendants :

$$\frac{1}{N_T} \mathbf{h}_i^H \mathbf{h}_j \xrightarrow{N_T \rightarrow \infty} 0.$$

Les canaux se rendent orthogonaux par la dimension. C'est ce qui explique pourquoi un simple MRT peut devenir suffisant : le faisceau d'un utilisateur a naturellement peu de projection sur les autres utilisateurs.

Cette démonstration est asymptotique. En pratique, la corrélation spatiale, les trajets communs, les erreurs de calibration et la contamination par pilotes ralentissent la convergence vers ce régime idéal.

20.5.3 Précodage MRT massive MIMO

En présence de K utilisateurs et canaux estimés $\hat{\mathbf{h}}_k$, le MRT (*Maximum Ratio Transmission*) choisit

$$\mathbf{w}_k = \frac{\hat{\mathbf{h}}_k}{\|\hat{\mathbf{h}}_k\|},$$

et le signal transmis est $\mathbf{x} = \sum_k \sqrt{p_k} \mathbf{w}_k s_k$. Coût : $\mathcal{O}(N_T K)$ par symbole. Optimal asymptotiquement, et souvent très proche de l'optimal dans les régimes massive MIMO favorables (N_T grand, canaux suffisamment séparés spatialement).

20.5.4 Estimation du canal et réciprocité TDD

Massive MIMO repose sur la connaissance du canal côté station de base. En FDD (*Frequency Division Duplex*) le *feedback* des $N_T K$ coefficients coûterait prohibitif. En TDD (*Time Division Duplex*), la réciprocité du canal permet d'estimer $\mathbf{H}$ depuis l'uplink (où les utilisateurs envoient des pilotes mutuellement orthogonaux) et de l'utiliser en downlink. C'est pourquoi le massive MIMO est principalement déployé en TDD.

20.5.5 Contamination par pilotes

Limitation fondamentale du massive MIMO : si deux cellules réutilisent les mêmes pilotes, leurs estimations de canal se contaminent mutuellement. L'inter-cell interference subsiste et borne la performance même quand $N_T \rightarrow \infty$. Solutions : pilotes orthogonaux globalement, schémas hybrides, machine learning sur les covariances spatiales.

20.5.6 Lecture beamforming de la contamination

Si l'utilisateur k et un utilisateur d'une cellule voisine utilisent le même pilote, l'estimation devient approximativement :

$$\hat{\mathbf{h}}_k = \mathbf{h}_k + \mathbf{h}_{\text{cont}} + \mathbf{e}.$$

Le MRT construit alors :

$$\mathbf{w}_k \propto \hat{\mathbf{h}}_k.$$

Le faisceau downlink pointe donc partiellement vers l'utilisateur contaminant. Même lorsque N_T augmente, cette composante ne se moyenne pas, car elle est intégrée dans le vecteur de canal estimé. C'est une erreur de modèle, pas seulement du bruit. Elle est donc structurellement plus dangereuse qu'une erreur thermique indépendante.

20.6 Architectures hybrides analogique/numérique

Pour fixer les ordres de grandeur, dès que N_T atteint plusieurs dizaines ou centaines d'antennes, équiper chaque antenne d'une chaîne RF complète (LNA, mélangeur, ADC, DAC) devient prohibitif en coût, en énergie et en encombrement. La parade : architectures *hybrides*, où un précodage analogique grossier (N_T déphaseurs / atténuateurs) est suivi d'un précodage numérique fin sur un nombre réduit N_{RF} de chaînes RF.

20.6.1 Structure

$$\mathbf{x} = \mathbf{F}_{RF} \mathbf{F}_{BB} \mathbf{s},$$

où $\mathbf{F}_{RF} \in \mathbb{C}^{N_T \times N_{RF}}$ est le précodage analogique (à modules unitaires : phaseurs) et $\mathbf{F}_{BB} \in \mathbb{C}^{N_{RF} \times K}$ le précodage numérique. Typiquement $N_{RF} = K$ ou $N_{RF} = 2K$.

20.6.2 Conception

Le problème de conception consiste à approximer un précodage purement numérique optimal $\mathbf{F}_{\text{opt}}$ par le produit $\mathbf{F}_{\text{RF}}\mathbf{F}_{\text{BB}}$, sous la contrainte que $|\mathbf{F}_{\text{RF}}|_{i,j} = 1$ (modules unitaires). Problème non convexe, résolu par :

- algorithmes de *matching pursuit* (OMP) qui sélectionnent les colonnes de $\mathbf{F}_{\text{RF}}$ dans un dictionnaire de steering vectors ;
- descente alternée sur $\mathbf{F}_{\text{RF}}$ et $\mathbf{F}_{\text{BB}}$;
- *manifold optimization* sur la variété des matrices à modules unitaires.

20.6.3 Performance

Pour $N_T \gg K$ et un canal géométrique avec peu de trajets, le précodage hybride peut approcher fortement la capacité du précodage purement numérique, souvent au prix d'une petite perte dans les canaux mmWave favorables. C'est l'architecture déployée en *millimeter-wave 5G* (28 GHz, 39 GHz, 60 GHz) où le coût de N_T chaînes RF complètes serait prohibitif.

20.7 Beamforming et MIMO en émission

Vu du point de vue du beamforming, le précodage MIMO est :

- **MRT** : DAS en émission, optimal en bruit blanc canal unique.
- **ZF** : Null Steering en émission, force le gain à 0 vers les autres utilisateurs.
- **RZF / MMSE** : MVDR en émission, compromis bruit / interférences.
- **Précodage hybride** : structure GSC pour le hardware constraint.

Tous les outils théoriques — contraintes linéaires, optimisation convexe, sous-espaces — s'appliquent identiquement, à la réinterprétation du « snapshot » près.

20.8 Multi-utilisateur et capacité MIMO

Pour K utilisateurs partageant les mêmes ressources, le débit agrégé maximal (*sum rate*) est borné par la capacité du canal multi-utilisateur :

$$C_{\text{MU}} = \max_{\sum_k p_k \leq P} \sum_{k=1}^K \log_2 \left(1 + p_k \frac{|\mathbf{w}_k^H \mathbf{h}_k|^2}{\sigma^2 + \sum_{j \neq k} p_j |\mathbf{w}_k^H \mathbf{h}_j|^2} \right).$$

La solution est typiquement non triviale, obtenue par *water-filling* sur la SVD du canal global. En massive MIMO ($N_T \rightarrow \infty$), la contamination par interférence devient négligeable et $C_{\text{MU}} \rightarrow K \log_2(1 + N_T \text{SNR})$: croissance linéaire en K et logarithmique en N_T .

20.9 Lien avec le beamforming en réception classique

Tous les algorithmes adaptatifs étudiés en Partie III et IV (MVDR, LCMV, LMS, RLS) se transposent directement en réception MIMO : chaque utilisateur dispose d'une covariance interférence-plus-bruit spécifique, et MVDR sur cette covariance fournit le beamformer optimal. La spécificité de MIMO est :

- l'existence d'un *beamforming réciproque* en émission ;
- le multiplexage de *plusieurs flux* en parallèle via la SVD ;
- l'*estimation conjointe* du canal et des données.

20.10 Implémentation : précodage MRT massive MIMO

Listing 20.1 – Précodage MRT et RZF

```

1 import numpy as np
2
3 def precoding_mrt(H, p):
4     """
5     H : (K, NT) canaux estimés
6     p : (K,) puissances par utilisateur
7     Retour : W (NT, K) précodeurs
8     """
9     K, NT = H.shape
10    W = H.conj().T.copy() # (NT, K)
11    norms = np.linalg.norm(W, axis=0)
12    W = W / (norms + 1e-12)
13    W = W * np.sqrt(p)[None, :]
14    return W
15
16 def precoding_rzf(H, sigma2, P_total):
17     """
18     Regularized ZF avec budget de puissance total P_total.
19     """
20    K, NT = H.shape
21    assert K <= NT
22    assert sigma2 >= 0.0 and P_total > 0.0
23    alpha = K * sigma2 / P_total
24    G = H @ H.conj().T + alpha * np.eye(K)
25    G = 0.5 * (G + G.conj().T)
26    if np.linalg.cond(G) > 1e12:
27        raise np.linalg.LinAlgError("précodage RZF mal conditionné")
28    W = np.linalg.solve(G, H).conj().T
29    # Normalisation pour respecter le budget de puissance
30    p = P_total / np.linalg.norm(W, 'fro')**2
31    return np.sqrt(p) * W

```

20.11 Synthèse

Le MIMO et le massive MIMO étendent le beamforming au cas bilatéral émission/réception. Quelques messages clés :

- la SVD du canal diagonalise le problème en sous-canaux indépendants ;
- le précodage MRT est le « DAS en émission », ZF est le « Null Steering », RZF est le « MVDR » ;
- en massive MIMO ($N_T \gg K$), MRT devient souvent compétitif en régime asymptotique et le canal *durcit* ;
- les architectures hybrides répondent aux contraintes matérielles aux fréquences millimétriques ;
- la *contamination par pilotes* et l'estimation de canal restent les principales limites pratiques.

Le chapitre suivant couvre les géométries de réseaux avancées : réseaux planaires, *sparse arrays*, champ proche, polarisation. Ces extensions complètent le panorama des situations où les hypothèses simplificatrices ULA + champ lointain + polarisation identique cessent de tenir. À l'issue du chapitre 21, on aura couvert les principales architectures rencontrées dans l'industrie, depuis les réseaux uniformes 1D des liaisons classiques jusqu'aux configurations cylindriques des antennes embarquées en passant par les *sparse arrays* de radioastronomie.

Chapitre 21

Géométries avancées

L'ULA en champ lointain avec polarisation unique a guidé toute la théorie précédente. La pratique impose souvent de relâcher ces hypothèses. Le présent chapitre couvre les principales extensions : réseaux planaires (URA), *sparse arrays* et *coprime arrays*, champ proche, polarisation double, et réseaux conformes (sur surfaces courbes). Pour chaque cas, on indique comment le steering vector, le beam pattern et les algorithmes étudiés précédemment s'adaptent.

Le message d'ensemble est rassurant : les extensions traitées ici ne remettent pas en cause l'*ossature algébrique* du beamforming. Le modèle vectoriel $\mathbf{x}(n) = \mathbf{A}(\boldsymbol{\theta})\mathbf{s}(n) + \mathbf{n}(n)$ reste valable ; seule la définition de $\mathbf{a}(\theta)$ change selon la géométrie. Tous les algorithmes qui ne supposent rien de spécifique sur la forme du steering vector — DAS, MVDR, LCMV, MUSIC — se transposent avec peu de modifications. Seules ESPRIT (qui exige l'invariance par translation) et Root-MUSIC (qui exige une structure de Vandermonde) imposent une géométrie particulière. Pour le reste, c'est la formule de $\mathbf{a}$ qui change, pas la machinerie qui l'entoure.

21.1 Réseaux planaires (URA)

21.1.1 Géométrie

Un *Uniform Rectangular Array* (URA) est une grille $M_x \times M_y$ d'antennes aux positions $\mathbf{p}_{m,n} = (md_x, nd_y, 0)^T$ avec $m = 0, \dots, M_x - 1$, $n = 0, \dots, M_y - 1$. Total : $N = M_x M_y$ antennes.

21.1.2 Steering vector

La direction d'une source en coordonnées sphériques est repérée par (θ, φ) : élévation θ , azimut φ . Le vecteur unitaire de propagation est

$$\hat{\mathbf{u}} = (\sin \theta \cos \varphi, \sin \theta \sin \varphi, \cos \theta)^T.$$

Le steering vector de l'antenne (m, n) est

$$a_{m,n}(\theta, \varphi) = e^{-jk(md_x \sin \theta \cos \varphi + nd_y \sin \theta \sin \varphi)}.$$

On définit les fréquences spatiales

$$u = \sin \theta \cos \varphi, \quad v = \sin \theta \sin \varphi,$$

et alors

$$a_{m,n}(u, v) = e^{-jk(md_x u + nd_y v)}.$$

21.1.3 Séparabilité

Le steering vector se décompose comme un produit de Kronecker :

$$\mathbf{a}(u, v) = \mathbf{a}_y(v) \otimes \mathbf{a}_x(u),$$

où $\mathbf{a}_x(u) = [1, e^{-jkd_x u}, \dots]^T$ et de même pour $\mathbf{a}_y(v)$. Cette séparabilité simplifie tous les calculs :

- le beam pattern est le produit $B(u, v) = B_x(u) B_y(v)$ pour des poids séparables $\mathbf{w} = \mathbf{w}_y \otimes \mathbf{w}_x$;
- MUSIC en 2D peut être appliqué sur les deux dimensions indépendamment via *2D-MUSIC* ou *2D Root-MUSIC* ;
- ESPRIT en 2D fournit conjointement u et v via *2D-ESPRIT* ou *Unitary 2D-ESPRIT*.

21.1.4 Conséquence : pas d'ambiguïté avant/arrière

L'URA résout l'ambiguïté de cône de l'ULA : pour des angles $\theta < 90^\circ$, l'azimut φ est uniquement déterminé par (u, v) .

21.2 Réseaux non uniformes

21.2.1 Motivation

Pour une ouverture D donnée, un ULA exige $N = 2D/\lambda$ antennes. Pour les très grandes ouvertures (radioastronomie), c'est prohibitif. Les *sparse arrays* placent moins d'antennes mais sur des positions optimisées pour conserver une bonne résolution.

21.2.2 Minimum Redundancy Arrays

Conception classique de Moffet (1968) : on place N antennes sur positions $\{p_0, \dots, p_{N-1}\}$ telles que les différences $\{p_i - p_j\}$ couvrent un intervalle d'entiers le plus large possible. La covariance d'un réseau MRA, sous des hypothèses de sources non corrélées, est équivalente à celle d'un ULA virtuel d'une ouverture nettement plus grande — on peut traiter jusqu'à $\mathcal{O}(N^2)$ sources avec N antennes (*co-array*).

21.2.3 Coprime Arrays

Vaidyanathan & Pal (2010) : deux ULA d'espacements $M\lambda/2$ et $N\lambda/2$ (M, N premiers entre eux) entrelacés. Total $M + N - 1$ antennes ; le *co-array différentiel* a $\mathcal{O}(MN)$ positions distinctes, ce qui permet d'identifier $\mathcal{O}(MN)$ sources — beaucoup plus que $M + N$ classiquement.

21.2.4 Nested Arrays

Pal & Vaidyanathan (2010) : un ULA dense $1, 2, \dots, N_1$ suivi d'un ULA espacé $(N_1 + 1)(N_1 + 1), 2(N_1 + 1)(N_1 + 1), \dots$. Le *co-array* est un ULA virtuel de $\mathcal{O}(N_1 N_2)$ antennes. Permet de détecter $\mathcal{O}(N^2)$ sources avec N antennes physiques.

21.2.5 DOA sur sparse arrays

L'approche standard : *vectorization* de la covariance $\mathbf{r} = \text{vec}(\mathbf{R})$, qui se comporte comme un snapshot d'un « co-array » virtuel. On applique MUSIC sur ce co-array, typiquement après spatial smoothing pour pallier l'unicité de $\mathbf{r}$ (un seul « snapshot »).

21.3 Champ proche

21.3.1 Quand

L'approximation d'onde plane échoue quand $R < 2D^2/\lambda$ (chapitre 2). C'est le régime *near-field*, rencontré en imagerie acoustique, radar courte portée, biomédical, micro-ondes intra-bâtiment.

Critère	Champ lointain	Champ proche
Front d'onde	plan	sphérique
Paramètres	angle θ	angle + distance (θ, R)
Steering	$\mathbf{a}(\theta)$	$\mathbf{a}(R, \theta)$
Amplitude	presque constante	varie avec R_m
Recherche DOA	grille 1D	localisation 2D
Action physique	pointage angulaire	focalisation spatiale

Champ lointain vs champ proche : front d'onde

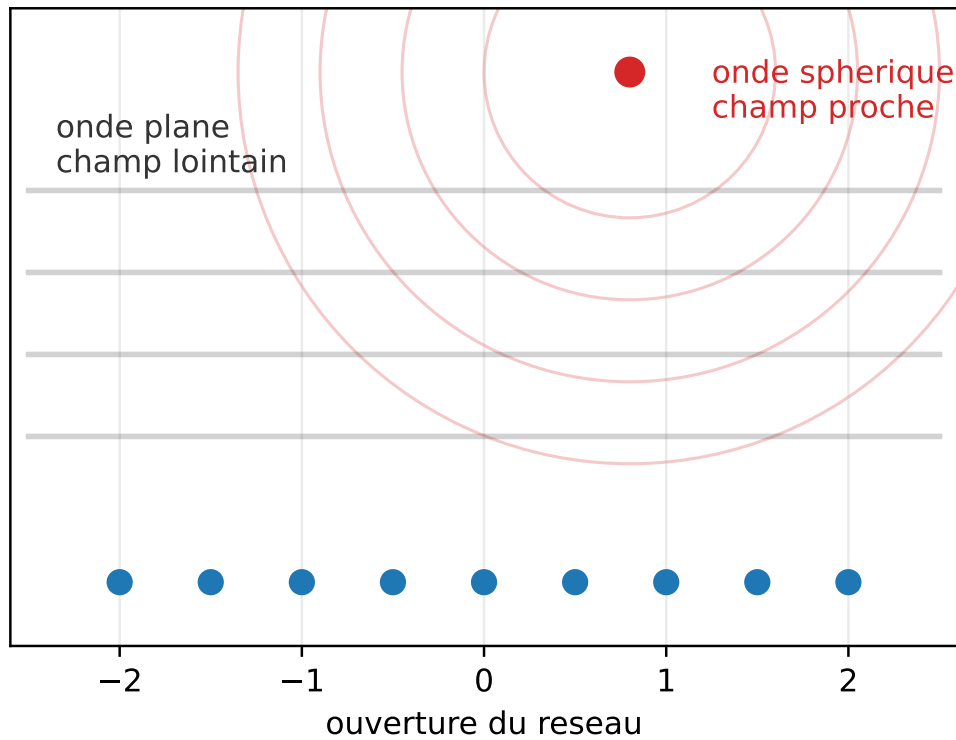


FIGURE 21.1 – Champ lointain et champ proche (schéma $N = 9$ capteurs) : le modèle d'onde plane supprime la dépendance en distance, alors que le champ proche conserve une phase sphérique et permet une focalisation angle-distance.

21.3.2 Steering vector champ proche

Pour une source à la position $\mathbf{r}_s$, la phase reçue par l'antenne m dépend de la distance *exacte* $R_m = \|\mathbf{r}_s - \mathbf{p}_m\|$:

$$a_m(\mathbf{r}_s) = \frac{e^{-jkR_m}}{R_m}.$$

Le facteur $1/R_m$ introduit aussi une *variation d'amplitude* entre antennes, contrairement au champ lointain. Le steering vector dépend de la *position* de la source, pas seulement de sa direction.

21.3.3 Lien avec le modèle champ lointain

Pour comprendre ce qui disparaît en champ lointain, écrivons la position de la source sous la forme :

$$\mathbf{r}_s = R\hat{\mathbf{u}},$$

où R est grand devant l'ouverture du réseau. Alors :

$$R_m = \|R\hat{\mathbf{u}} - \mathbf{p}_m\| = R - \hat{\mathbf{u}}^T \mathbf{p}_m + \mathcal{O}\left(\frac{\|\mathbf{p}_m\|^2}{R}\right).$$

En négligeant le terme quadratique :

$$e^{-jkR_m} \approx e^{-jkR} e^{jk\hat{\mathbf{u}}^T \mathbf{p}_m}.$$

Le facteur commun e^{-jkR} disparaît dans le steering relatif, et on retrouve le modèle d'onde plane. Le champ proche correspond donc au cas où le terme quadratique n'est plus négligeable :

$$k \frac{D^2}{R} \not\ll 1.$$

Cette lecture relie directement la distance de Fraunhofer $R_F \simeq 2D^2/\lambda$ à une erreur de phase résiduelle.

21.3.4 Conséquence algorithmique

En champ lointain, une source est paramétrée par un angle. En champ proche, elle est paramétrée par une portée et une direction :

$$\mathbf{a}(\theta) \longrightarrow \mathbf{a}(R, \theta).$$

Le pseudo-spectre MUSIC devient :

$$P_{\text{NF-MUSIC}}(R, \theta) = \frac{1}{\mathbf{a}^H(R, \theta) \mathbf{U}_n \mathbf{U}_n^H \mathbf{a}(R, \theta)}.$$

Les pics donnent une localisation portée-angle. Le gain est évident : on peut localiser en distance avec un seul réseau. Le prix est un scan 2D, une sensibilité plus forte à la géométrie exacte des antennes et un risque de lobes ambigus en portée.

21.3.5 Algorithmes

Tous les algorithmes (DAS, MVDR, MUSIC, ESPRIT) s'adaptent en remplaçant $\mathbf{a}(\theta)$ par $\mathbf{a}(\mathbf{r}_s)$ et en parcourant une grille en (θ, R) . Coût plus élevé (grille 2D), mais la propriété d'orthogonalité signal-bruit reste valable. Le *near-field MUSIC* est aujourd'hui standard en imagerie ultrasons et SAR proche.

21.3.6 Régime de Fresnel

Entre champ proche pur et champ lointain pur, il existe une zone intermédiaire dite *de Fresnel*, où la dépendance en R est modérée et où des approximations quadratiques en R sont possibles. Les modèles *Fresnel approximations* simplifient le traitement au prix d'un biais résiduel.

21.4 Polarisation

21.4.1 Modèle vectoriel

Une onde EM est polarisée : son champ électrique a une orientation spatiale (linéaire, circulaire, elliptique). Les antennes ne sont sensibles qu'à certaines composantes. Pour un réseau d'antennes *double polarisation* (H et V par antenne), le steering vector devient un *vecteur de polarisation* 2N-dimensionnel :

$$\mathbf{a}_{\text{pol}}(\theta, \alpha, \beta) = \mathbf{a}(\theta) \otimes \mathbf{p}(\alpha, \beta),$$

où $\mathbf{p}(\alpha, \beta) = [\cos \alpha, e^{j\beta} \sin \alpha]^T$ encode la polarisation. Deux paramètres supplémentaires (α, β) caractérisent chaque source.

21.4.2 Algorithmes avec polarisation

Polarized MUSIC. On scanne sur (θ, α, β) . Le pseudo-spectre $P_{\text{MUSIC}}(\theta, \alpha, \beta)$ identifie simultanément direction et polarisation.

Sources de polarisations différentes. Deux sources de même direction mais polarisations orthogonales sont *séparables* par polarisation, même si la direction seule ne les sépare pas. C'est le principe du SAR polarimétrique et du *dual-pol* en satellitaire.

21.4.3 Interprétation en degrés de liberté

La polarisation ajoute des degrés de liberté, mais elle ajoute aussi des inconnues. Avec une antenne double polarisation, on double la dimension du snapshot :

$$\mathbf{x}(n) \in \mathbb{C}^N \longrightarrow \mathbf{x}_{\text{pol}}(n) \in \mathbb{C}^{2N}.$$

Mais chaque source possède maintenant au moins deux paramètres polarimétriques en plus de sa direction. Si la polarisation est connue ou fortement contrainte, cette extension améliore la séparation. Si elle est totalement inconnue, l'espace de recherche augmente fortement :

$$P(\theta) \longrightarrow P(\theta, \alpha, \beta).$$

En pratique, on utilise souvent une élimination analytique de la polarisation. Pour un angle θ fixé, on cherche le vecteur $\mathbf{p}$ qui minimise :

$$\mathbf{a}_{\text{pol}}^H(\theta, \mathbf{p}) \mathbf{U}_n \mathbf{U}_n^H \mathbf{a}_{\text{pol}}(\theta, \mathbf{p}).$$

Le problème se ramène alors à une petite valeur propre 2×2 au lieu d'un scan complet sur (α, β) . C'est l'idée pratique de nombreuses variantes de *polarized MUSIC*.

21.4.4 Adaptation des covariances

La covariance devient $2N \times 2N$ et contient à la fois l'information angulaire et polarimétrique. Tous les concepts — sous-espace signal, MVDR, LCMV — s'étendent identiquement.

21.5 Réseaux conformes

21.5.1 Définition

Un *réseau conforme* suit une surface non plane : cylindrique (antenne missile), conique (radar avion), sphérique (système omnidirectionnel). Les positions $\mathbf{p}_m$ sont sur une variété 3D quelconque.

21.5.2 Difficultés

- Pas de structure de Vandermonde ; ESPRIT direct n'est pas applicable.
- Pas de structure de Kronecker comme l'URA ; pas de séparabilité.
- Chaque antenne a sa propre orientation, donc un *pattern de rayonnement individuel* qui dépend de la direction d'arrivée. Le *element pattern* doit être inclus.
- Couplage mutuel plus complexe.

21.5.3 Steering vector modifié

Le steering vector intègre direction d'antenne et pattern individuel :

$$a_m(\theta, \varphi) = g_m(\theta, \varphi) e^{-jk\hat{\mathbf{u}}^T \mathbf{p}_m},$$

où $g_m(\theta, \varphi)$ est le pattern complexe de l'antenne m dans son repère local, exprimé en repère global.

21.5.4 Algorithmes

MUSIC s'applique tel quel (scan sur (θ, φ)). ESPRIT direct est exclu mais des variantes *Manifold-based* (Friedlander) ou des méthodes *interpolated arrays* (Friedlander 1992) transforment les données comme si elles provenaient d'un ULA virtuel.

21.6 Implémentation : 2D-MUSIC sur URA

Listing 21.1 – 2D-MUSIC sur URA

```

1 import numpy as np
2
3 def steering_ura(theta, phi, Mx, My, dx, dy, lam):
4     """Steering vector URA, taille N = Mx*My."""
5     assert Mx > 0 and My > 0 and lam > 0.0
6     u = np.sin(theta) * np.cos(phi)
7     v = np.sin(theta) * np.sin(phi)
8     m = np.arange(Mx); n = np.arange(My)
9     ax = np.exp(-1j * 2*np.pi * dx / lam * u * m)
10    ay = np.exp(-1j * 2*np.pi * dy / lam * v * n)
11    return np.kron(ay, ax) # taille Mx*My
12
13 def music_2d(R, L, Mx, My, dx, dy, lam, n_theta=181, n_phi=361):
14     N = R.shape[0]
15     assert N == Mx * My
16     assert R.shape == (N, N)
17     assert 0 <= L < N
18     R = 0.5 * (R + R.conj().T)
19     eigvals, eigvecs = np.linalg.eigh(R)
20     Un = eigvecs[:, :N-L]
21     thetas = np.linspace(0, np.pi/2, n_theta)
22     phis = np.linspace(-np.pi, np.pi, n_phi)
23     P = np.zeros((n_theta, n_phi))
24     eps = np.finfo(float).eps
25     for i, th in enumerate(thetas):
26         for j, ph in enumerate(phis):
27             a = steering_ura(th, ph, Mx, My, dx, dy, lam)
28             v = Un.conj().T @ a
29             den = max((v.conj() @ v).real, eps)

```

```

30         P[i, j] = 1.0 / den
31     return thetas, phis, P

```

21.7 Synthèse

Les géométries avancées étendent le beamforming à des situations où le couple « ULA + champ lointain + polarisation unique » cesse de tenir :

- URA : 2D, séparabilité Kronecker, lève l’ambiguïté de cône ;
- sparse / coprime / nested : grandes ouvertures et nombreuses sources avec peu d’antennes physiques ;
- near-field : intégration de la distance dans le steering ;
- polarisation : steering vector doublé, séparation par polarisation ;
- réseaux conformes : nécessitent des modèles d’élément précis.

L’ossature théorique (covariance, sous-espaces, MVDR, MUSIC, ESPRIT) reste valable ; seul le steering vector et certaines symétries algébriques changent. Avec ce chapitre se clôt la Partie VI.

La Partie VII aborde maintenant les questions de mise en œuvre : calibration des erreurs matérielles, robustesse aux erreurs de modèle, et intégration de tous les outils précédents dans une chaîne intégrée d’anti-brouillage. C’est la partie où la théorie rencontre la réalité matérielle et où l’on découvre que la performance d’un système n’est pas dictée par la sophistication de ses algorithmes mais par la qualité de son maillon le plus faible — souvent la calibration ou la cohérence de phase entre voies.

Septième partie

Mise en œuvre pratique

Chapitre 22

Calibration d'un réseau multi-antennes

Tous les algorithmes étudiés jusqu'ici supposent que les antennes mesurent fidèlement les phases prédites par le steering vector théorique $\mathbf{a}(\theta)$. En pratique, chaque voie RF introduit ses propres erreurs : gains, phases, retards de câble, dérives, et couplage mutuel entre antennes. Le steering vector *effectif* est $\mathbf{a}_{\text{réel}}(\theta) = \mathbf{G} \mathbf{a}(\theta) + \mathbf{e}(\theta)$, et c'est lui — et non $\mathbf{a}(\theta)$ — que l'algorithme « voit ». La calibration est la procédure qui mesure et corrige les imperfections matérielles. Sans calibration, un beamformer ou un estimateur de DOA produit des résultats fortement biaisés. Ce chapitre détaille les modèles d'erreur, les méthodes d'estimation des corrections, et les bonnes pratiques expérimentales.

Une réalité de terrain : sur beaucoup de projets de beamforming opérationnel, la calibration occupe une part du temps de développement bien supérieure à celle qu'on lui accorde dans la littérature académique. Un radar défense, un récepteur GNSS anti-brouillage, une antenne 5G mmWave passent des semaines à mois en chambre anéchoïque avant leur déploiement, puis suivent une procédure de recalibration in situ régulière. C'est de loin le poste le plus coûteux et le plus chronophage. Le présent chapitre vise à donner au lecteur les bases conceptuelles pour ne pas perdre de temps sur des erreurs évitables, et pour reconnaître quand un comportement « bizarre » relève d'un défaut de calibration plutôt que d'un défaut d'algorithme.

22.1 Modèle d'erreur par voie

22.1.1 Gain et phase

Le modèle le plus simple sépare le canal RF en gain $g_m \in \mathbb{R}^+$ et phase $\phi_m \in \mathbb{R}$:

$$\mathbf{a}_{\text{réel}}(\theta) = \mathbf{G} \mathbf{a}(\theta), \quad \mathbf{G} = \text{diag}(g_0 e^{j\phi_0}, \dots, g_{N-1} e^{j\phi_{N-1}}).$$

Ce modèle, dit *calibration diagonale*, est valide quand le couplage mutuel est faible et les positions sont bien connues. Il suffit pour la majorité des plateformes SDR multi-canaux.

22.1.2 Sources physiques

Les composantes du modèle proviennent de :

- câbles RF : retards différentiels $\Rightarrow$ phases ;
- LNA : gains différents, légères variations de phase ;
- mélangeurs : phases dépendantes du LO ;
- filtres : retards de groupe ;
- ADC : retards de sampling, jitter ;

— dérive thermiques sur tous ces composants.

Selon la chaîne RF, une dérive thermique de 10°C peut produire des variations mesurables de gain et de phase; les chiffres précis doivent être validés sur banc, car ils dépendent fortement du matériel.

22.1.3 Effet sur les algorithmes

- DAS pointe à côté de la cible (perte $\sim \exp(-\sigma_\phi^2)$).
- Null Steering place ses zéros au mauvais endroit (un brouilleur prévu en θ_i peut ressortir à un niveau fini, par exemple autour de -25 dB dans une simulation de mismatch, au lieu de $-\infty$).
- MVDR atténue partiellement le signal utile (signal cancellation).
- MUSIC produit des pics décalés et atténués.
- ESPRIT, qui repose sur une invariance *exacte*, est particulièrement fragile.

22.2 Calibration avec une source connue

22.2.1 Principe

On place une source de référence à une direction *connue* θ_0 (typiquement le broadside) et on mesure le snapshot reçu. Le snapshot moyen, en l'absence d'autres sources, est $\mathbb{E}[\mathbf{x}(n)] = \mathbf{G} \mathbf{a}(\theta_0) s_0$ pour une source déterministe, ou bien $\mathbb{E}[\mathbf{x}\mathbf{x}^H] = |s_0|^2 \mathbf{G} \mathbf{a}(\theta_0) \mathbf{a}^H(\theta_0) \mathbf{G}^H + \sigma^2 \mathbf{I}$ pour une source stochastique.

22.2.2 Estimation des gains/phases

Cas source en *broadside* ($\theta_0 = 0$, donc $\mathbf{a}(\theta_0) = \mathbf{1}$). Le snapshot moyen est

$$\bar{\mathbf{x}} = s_0 \begin{bmatrix} g_0 e^{j\phi_0} & \cdots & g_{N-1} e^{j\phi_{N-1}} \end{bmatrix}^T.$$

En normalisant à l'antenne 0 :

$$\hat{g}_m e^{j\hat{\phi}_m} = \frac{\bar{x}_m}{\bar{x}_0}, \quad m = 1, \dots, N-1.$$

La calibration s'obtient en un calcul : $\mathbf{G}^{-1}$ corrige chaque voie pour qu'elle coïncide avec l'antenne 0.

22.2.3 Estimation par vecteur propre principal

Si la source est forte mais ses propriétés temporelles ne permettent pas un moyennage simple, on estime $\hat{\mathbf{R}}$ et on prend son *vecteur propre principal* $\hat{\mathbf{u}}_1$. Sous le modèle mono-source, $\hat{\mathbf{u}}_1 \propto \mathbf{G} \mathbf{a}(\theta_0)$. On en déduit les gains/phases relatives par $g_m e^{j\phi_m} = (\hat{\mathbf{u}}_1)_m / (\hat{\mathbf{u}}_1)_0 \cdot 1/a_m(\theta_0)$.

22.2.4 Source en direction quelconque

Si la source est à $\theta_0 \neq 0$, le steering théorique $\mathbf{a}(\theta_0)$ contient des phases connues qu'il faut *retirer* avant d'estimer la calibration. On calcule

$$\hat{g}_m e^{j\hat{\phi}_m} = \frac{\bar{x}_m}{\bar{x}_0} \frac{a_0(\theta_0)}{a_m(\theta_0)}.$$

22.3 Calibration multi-sources

Avec plusieurs sources connues à $\theta_1, \dots, \theta_M$, on forme un système surdéterminé :

$$\mathbf{x}(n) = \mathbf{G} \mathbf{A}(\boldsymbol{\theta}) \mathbf{s}(n) + \mathbf{n}(n),$$

qu'on inverse au sens moindres carrés pour $\mathbf{G}$. C'est plus précis (multiples points de mesure) et plus robuste.

22.4 Calibration par covariance et matrice de Toeplitz

Pour un ULA *idéal*, la matrice de covariance théorique en présence d'une source unique est de structure *Toeplitz* hermitienne. En présence d'erreurs de voie, elle s'écarte de cette structure d'une manière diagonale-dominante. On peut donc estimer $\mathbf{G}$ en cherchant la matrice diagonale qui ramène $\hat{\mathbf{R}}$ « au plus proche » d'une Toeplitz hermitienne :

$$\min_{\mathbf{G}} \|\mathbf{G}^{-1} \hat{\mathbf{R}} \mathbf{G}^{-H} - \mathbf{T}\|_F^2,$$

où $\mathbf{T}$ est la Toeplitz hermitienne la plus proche. Méthode dite *ToeplitzCal*, sans source connue, fondée sur la seule connaissance de la géométrie.

22.5 Erreurs de position d'antennes

Lorsque les antennes ne sont pas exactement aux positions nominales, l'erreur n'est plus diagonale : c'est une erreur *angle-dépendante*. Pour une erreur de position $\delta \mathbf{p}_m$, la phase reçue devient $-k \hat{\mathbf{u}}^T(\mathbf{p}_m + \delta \mathbf{p}_m)$.

Modélisation. $\mathbf{a}_{\text{réel}}(\theta) = \mathbf{G}_{\text{geom}}(\theta) \mathbf{a}(\theta)$ avec $\mathbf{G}_{\text{geom}}$ *dépendant de θ* . La calibration à une seule direction ne suffit alors plus.

Procédure. On mesure $\mathbf{G}_{\text{geom}}(\theta)$ pour plusieurs angles connus, puis on interpole. Pour un ULA on peut estimer les positions $\delta \mathbf{p}_m$ par optimisation conjointe sur K sources connues.

22.6 Couplage mutuel

22.6.1 Modèle

Les antennes proches s'influencent : le courant induit dans une antenne modifie le courant des voisines. Le modèle classique est

$$\mathbf{a}_{\text{réel}}(\theta) = \mathbf{C} \mathbf{a}(\theta),$$

où $\mathbf{C}$ est une matrice de couplage $N \times N$ non diagonale. $\mathbf{C}$ a typiquement une structure bande : seules les antennes proches se couplent significativement.

22.6.2 Mesure

En chambre anéchoïque. On émet sur une seule antenne et mesure les courants induits sur les autres : on remplit $\mathbf{C}$ colonne par colonne. Procédure standard pour les réseaux industriels.

In situ. Sources étalon à plusieurs angles, optimisation conjointe gain/phase/couplage. Plus complexe, moins précis, mais réalisable sans démontage.

22.6.3 Compensation

Le steering effectif corrigé est $\mathbf{C}^{-1}\mathbf{a}(\theta)$, à utiliser dans tous les algorithmes en lieu et place de $\mathbf{a}(\theta)$.

22.7 Calibration par mesure du manifold

Pour les réseaux complexes (conformes, irréguliers, fortement couplés), on procède par *mesure directe du manifold*. On scanne une source de référence sur une grille d'angles $\{\bar{\theta}_g\}$ et on enregistre les snapshots normalisés : on construit ainsi le dictionnaire $\mathbf{D} = [\hat{\mathbf{a}}_{\text{réel}}(\bar{\theta}_1), \dots, \hat{\mathbf{a}}_{\text{réel}}(\bar{\theta}_G)]$.

Interpolation. Pour des angles entre les nœuds de grille, on interpole par polynômes splines, par modèles paramétriques ou par *Gaussian Process regression*. La précision dépend de la densité de grille (typiquement 1° pour un radar civil, 0.1° pour un radar militaire).

22.8 Calibration large bande

Les gains et phases des voies dépendent de la fréquence. La calibration *large bande* consiste à mesurer $\mathbf{G}(f)$ sur une grille de fréquences couvrant la bande utile, puis à interpoler. Les filtres FIR fractionnaires implémentent ensuite la correction dans le domaine numérique.

22.9 Validation après calibration

Plusieurs tests permettent de vérifier la qualité de la calibration.

Test 1 : cohérence sur source connue. On place une source à une direction connue θ_0 et on mesure $\hat{\theta}_0$ par MUSIC ou Capon. L'écart $|\hat{\theta}_0 - \theta_0|$ donne une borne sur l'erreur résiduelle.

Test 2 : beam pattern mesuré. On forme un DAS à direction θ_0 et on mesure le gain sur une source mobile. Un bon banc doit retrouver la forme théorique du lobe principal ; la tolérance acceptable dépend du capteur, de la chambre de mesure et de la dynamique visée.

Test 3 : profondeur de null. On forme un Null Steering sur une direction θ_i et on mesure le gain sur une source y placée. La profondeur obtenue doit être interprétée comme une mesure système : un banc bien calibré peut atteindre plusieurs dizaines de dB de rejet, tandis qu'un réseau non calibré remonte souvent beaucoup plus haut.

22.10 Bonnes pratiques expérimentales

- **Source stable.** Émetteur de référence verrouillé sur un oscillateur de bonne stabilité (TCXO ou mieux OCXO). Une dérive du LO de l'émetteur de référence pollue toute la calibration.
- **Champ lointain.** Distance source-réseau bien supérieure à $2D^2/\lambda$. À 2.4 GHz, ULA $N = 16$, $D = 94$ cm, $R_F \simeq 14$ m : prévoir au moins 20–30 m.
- **Réduction des réflexions.** Chambre anéchoïque idéale ; sinon, espace ouvert et antennes directionnelles pour la source.
- **Cohérence des conditions.** Calibrer à la fréquence opérationnelle, à la température nominale, avec la même chaîne RF qu'en opération.

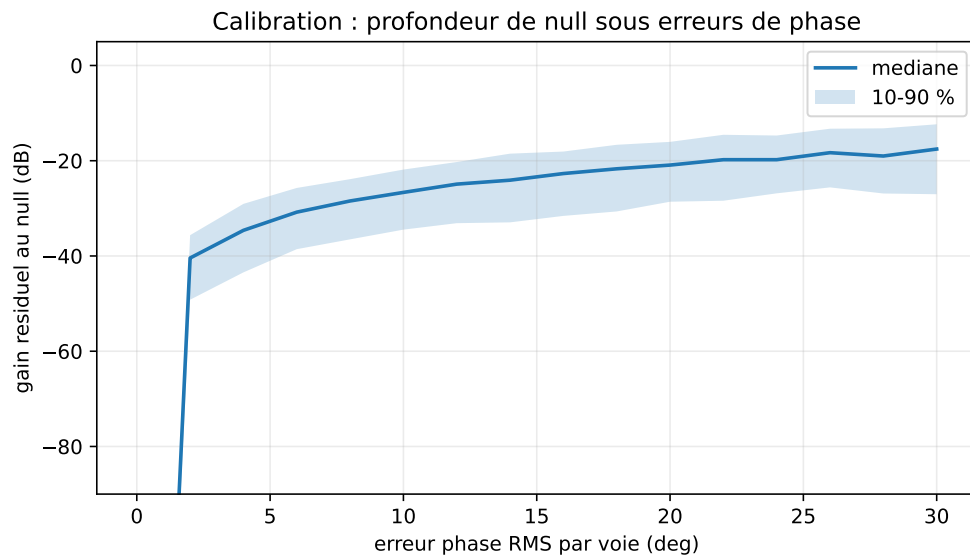


FIGURE 22.1 – Simulation Monte-Carlo ($N = 10$, cible 0° , null 30° , 400 tirages par niveau d'erreur) : un null théoriquement parfait devient un rejet fini dès que les voies portent des erreurs de phase résiduelles.

- **Recalibration périodique.** En présence de dérives, recalibrer toutes les quelques minutes (température variable) ou heures (environnement stable).
- **Self-calibration in situ.** Quand le démontage est impossible, utiliser des sources « opportunistes » : satellites, balises de référence connues, signaux LTE/GSM de cellules connues.

22.11 Implémentation : calibration diagonale

Listing 22.1 – Calibration diagonale d'un ULA

```

1 import numpy as np
2
3 def estimate_calibration(X, theta0, d_over_lambda=0.5):
4     """
5     X      : (N, K) snapshots d'une source unique à theta0
6     Retour : g (N,) facteurs complexes par voie (relatifs à m=0)
7     """
8     N, K = X.shape
9     # snapshot moyen (en supposant source connue, on peut filtrer si besoin)
10    x_mean = np.mean(X, axis=1)
11    m = np.arange(N)
12    a0 = np.exp(-1j * 2*np.pi * d_over_lambda * np.sin(theta0) * m)
13    raw = x_mean / a0
14    g = raw / raw[0] # facteur de référence : antenne 0
15    return g
16
17 def apply_calibration(X, g):
18     """Renvoie X corrigé : X_calib = diag(1/g) @ X."""
19     return X / g[:, None]
```

22.12 Synthèse

La calibration ramène la chaîne RF réelle au modèle théorique utilisé par les algorithmes. Sans elle, les beamformers adaptatifs et les estimateurs de DOA deviennent difficiles à fiabiliser. Les étapes minimales :

- modèle d'erreur (diagonal, position, couplage) ;
- procédure d'estimation (source connue ou auto-calibration) ;
- correction appliquée aux données ou intégrée aux steering vectors ;
- validation par tests sur sources connues.

Le chapitre suivant traite l'autre versant pratique : la *robustesse* aux erreurs résiduelles. La calibration n'élimine jamais complètement les erreurs ; il faut concevoir des algorithmes qui les tolèrent. Le couple calibration + robustesse forme une défense en profondeur : la première supprime l'essentiel des imperfections, la seconde absorbe ce qui n'a pas pu l'être. Aucune des deux ne suffit seule, et un système opérationnel doit cumuler les deux pour atteindre les performances annoncées par les algorithmes théoriques.

Chapitre 23

Robustesse aux erreurs de modèle

La calibration ramène le steering effectif près du steering théorique, mais ne le supprime jamais complètement. Des erreurs résiduelles subsistent — géométrie, calibration imparfaite, dérives, snapshot limité — et peuvent rapidement faire passer un beamformer théorique optimal à des performances très dégradées. La *robustesse* est l'ensemble des techniques qui rendent les algorithmes tolérants à ces erreurs. Ce chapitre couvre le *diagonal loading*, les contraintes dérivatives, les contraintes « worst-case », le *white noise gain* et leur impact sur le beam pattern.

Le paradoxe central de la robustesse est que les beamformers *optimaux* sont aussi les plus *fragiles*. MVDR atteint le SINR maximal en théorie, mais ce maximum se gagne au prix d'une sensibilité extrême aux erreurs sur $\mathbf{a}(\theta_s)$ et sur $\hat{\mathbf{R}}$. C'est l'expression algébrique d'un phénomène général en estimation : plus on exploite finement une information, plus on devient sensible aux erreurs sur cette information. Robustifier un beamformer, c'est donc accepter de *dégrader sa performance nominale* pour *conserver une performance acceptable* face aux imperfections réelles. C'est un arbitrage, pas un raffinement.

23.1 Sources d'erreur résiduelle

- **Steering vector mismatch.** La cible n'est pas exactement en θ_s : erreur $\delta\theta$.
- **Calibration imparfaite.** Gains/phases résiduelles σ_g, σ_ϕ .
- **Estimation de covariance.** Avec K snapshots fini, $\hat{\mathbf{R}}$ a une variance $\mathcal{O}(N/K)$.
- **Bruit non blanc.** Le modèle suppose $\sigma^2\mathbf{I}$; la réalité montre souvent des structures.
- **Non-stationnarité.** L'environnement change pendant la collecte.
- **Erreur sur L .** Sous-estimation ou sur-estimation du nombre de sources pour MUSIC/ESPRIT.

23.2 Signal self-cancellation

Phénomène clé : si la covariance utilisée par MVDR contient le signal utile, et que le steering vector ne pointe pas *exactement* dans sa direction, MVDR « pense » que le signal utile est une interférence et l'atténue. Dans des simulations où le signal utile est fort et inclus dans la covariance, la perte peut atteindre plusieurs dizaines de dB pour un mismatch d'une fraction de largeur de lobe principal.

Conditions favorables. Self-cancellation important quand :

- K grand (covariance bien estimée et donc « confiante ») ;
- signal puissant (puissant terme à « enlever ») ;
- mismatch important.

Parades.

- utiliser $\mathbf{R}_{i+n}$ au lieu de $\mathbf{R}$;
- diagonal loading ;
- contraintes dérivatives ;
- méthodes robustes (Lorenz & Boyd [8], Li–Stoica–Wang [7]).

23.3 Diagonal loading**23.3.1 Définition**

On remplace $\hat{\mathbf{R}}$ par

$$\hat{\mathbf{R}}_\alpha = \hat{\mathbf{R}} + \alpha \mathbf{I}_N,$$

avec $\alpha > 0$.

23.3.2 Effets

- Conditionnement amélioré : $\kappa = (\lambda_{\max} + \alpha)/(\lambda_{\min} + \alpha)$.
- Norme de $\mathbf{w}_{\text{MVDR}}$ bornée : $\|\mathbf{w}\|^2 \leq 1/(\alpha)$, à un facteur près.
- Beam pattern adouci : les nulls deviennent moins profonds, le lobe principal s'élargit légèrement.
- Quand $\alpha \rightarrow \infty$, MVDR $\rightarrow$ DAS.
- Quand $\alpha \rightarrow 0$, on retrouve MVDR pur.

23.3.3 Pourquoi le loading limite la norme

Le MVDR chargé résout :

$$\min_{\mathbf{w}} \mathbf{w}^H \hat{\mathbf{R}} \mathbf{w} + \alpha \|\mathbf{w}\|^2 \quad \text{sous} \quad \mathbf{w}^H \mathbf{a}_s = 1.$$

Le terme ajouté n'est donc pas seulement une astuce numérique. C'est une pénalisation explicite de la norme des poids. Or une grande norme signifie que le beamformer additionne les voies avec de grands coefficients positifs et négatifs, ce qui crée des annulations fines mais amplifie le bruit blanc, les erreurs de phase et les petites erreurs de calibration.

La solution est :

$$\mathbf{w}_{\text{DL}} = \frac{(\hat{\mathbf{R}} + \alpha \mathbf{I})^{-1} \mathbf{a}_s}{\mathbf{a}_s^H (\hat{\mathbf{R}} + \alpha \mathbf{I})^{-1} \mathbf{a}_s}.$$

En pratique, on ne calcule pas l'inverse explicitement : on résout le système linéaire $(\hat{\mathbf{R}} + \alpha \mathbf{I})\mathbf{z} = \mathbf{a}_s$, puis on normalise $\mathbf{w} = \mathbf{z}/(\mathbf{a}_s^H \mathbf{z})$. Si α augmente, les directions associées aux petites valeurs propres ne sont plus amplifiées. Le beamformer cesse d'exploiter les détails fragiles de la covariance estimée. C'est la raison profonde du compromis : moins d'adaptation, mais plus de stabilité.

23.3.4 Choix de α

Empirique. $\alpha = c \cdot \text{tr}(\hat{\mathbf{R}})/N$ avec $c \in [10^{-3}, 0.1]$. Simple et efficace pour la plupart des applications.

Basé sur le bruit. $\alpha = c \hat{\sigma}^2$, où $\hat{\sigma}^2$ est estimé à partir des $N - L$ plus petites valeurs propres. Plus précis quand on connaît L .

Worst-case [8]. On définit une *sphère d'incertitude* autour de $\mathbf{a}(\theta_s)$:

$$\mathcal{U} = \{\mathbf{a}' : \|\mathbf{a}' - \mathbf{a}(\theta_s)\| \leq \epsilon\}.$$

Le beamformer *robuste worst-case* minimise la puissance $\mathbf{w}^H \hat{\mathbf{R}} \mathbf{w}$ sous $|\mathbf{w}^H \mathbf{a}'| \geq 1$ pour tout $\mathbf{a}' \in \mathcal{U}$. Cette contrainte s'écrit $|\mathbf{w}^H \mathbf{a}(\theta_s)| - \epsilon \|\mathbf{w}\| \geq 1$, problème SOCP. La solution est équivalente à un MVDR avec un diagonal loading optimal α^* qui peut être calculé explicitement.

Robust Capon [7]. Variante directe :

$$\max_{\mathbf{a}' \in \mathcal{U}} (\mathbf{a}')^H \hat{\mathbf{R}}^{-1} \mathbf{a}'.$$

Aboutit aussi à un diagonal loading optimal calculable.

23.4 White Noise Gain (WNG)

23.4.1 Définition

Sous contrainte de gain unitaire $\mathbf{w}^H \mathbf{a}(\theta_s) = 1$, le *White Noise Gain* est

$$\text{WNG}(\mathbf{w}) = \frac{1}{\|\mathbf{w}\|^2}.$$

Pour le DAS, $\text{WNG} = N$. Pour MVDR sans loading, WNG peut chuter dramatiquement quand $\hat{\mathbf{R}}$ est mal conditionnée.

23.4.2 Pourquoi c'est important

Un faible WNG signifie que de petites erreurs sur les antennes (bruit ou calibration) sont *amplifiées* en sortie. À titre de règle pratique, un WNG très inférieur à celui du DAS, par exemple de l'ordre de $N/10$ ou moins, doit alerter : le beamformer est probablement devenu agressif et sensible aux erreurs de modèle.

23.4.3 Robustesse par contrainte WNG

On résout MVDR sous contrainte additionnelle $\text{WNG} \geq \gamma N$, soit $\|\mathbf{w}\|^2 \leq 1/(\gamma N)$:

$$\min_{\mathbf{w}} \mathbf{w}^H \hat{\mathbf{R}} \mathbf{w} \quad \text{sous} \quad \mathbf{w}^H \mathbf{a}(\theta_s) = 1, \quad \|\mathbf{w}\|^2 \leq 1/(\gamma N).$$

Cette contrainte est équivalente à un diagonal loading avec α ajusté pour atteindre la limite. C'est l'une des formulations les plus claires de la robustesse adaptative.

23.5 Propagation d'une erreur de steering

Soit le steering réel :

$$\mathbf{a}_r = \mathbf{a}_s + \Delta \mathbf{a}.$$

Le gain réellement appliqué à la source utile vaut :

$$\mathbf{w}^H \mathbf{a}_r = 1 + \mathbf{w}^H \Delta \mathbf{a},$$

car la contrainte impose seulement $\mathbf{w}^H \mathbf{a}_s = 1$. Par Cauchy-Schwarz :

$$|\mathbf{w}^H \Delta \mathbf{a}| \leq \|\mathbf{w}\| \|\Delta \mathbf{a}\|.$$

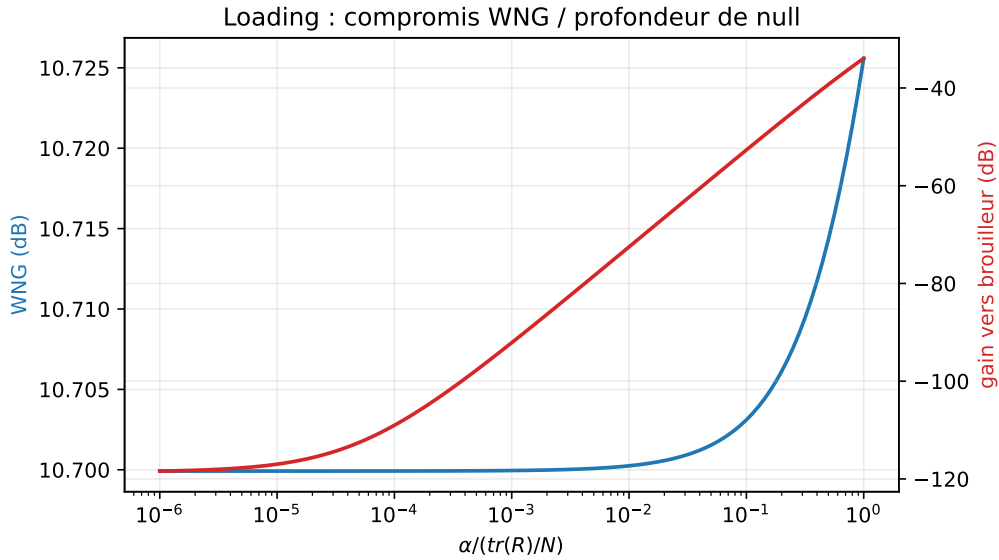


FIGURE 23.1 – Simulation MVDR reproductible ($N = 12$, cible 0° , brouilleurs -25° et 40° , INR 40 dB) : augmenter le diagonal loading améliore le WNG, mais les nulls deviennent moins profonds.

Cette inégalité donne la justification la plus simple du WNG : contrôler $\|\mathbf{w}\|$ contrôle directement la perte maximale due à une erreur de steering. Un beamformer avec des poids énormes peut respecter parfaitement la contrainte nominale et pourtant déformer fortement la source réelle.

Si l'on impose :

$$\|\Delta \mathbf{a}\| \leq \epsilon,$$

alors le gain utile vérifie :

$$|\mathbf{w}^H \mathbf{a}_r| \geq 1 - \epsilon \|\mathbf{w}\|.$$

Pour garantir une perte limitée, il faut donc :

$$\epsilon \|\mathbf{w}\| < 1.$$

C'est exactement la logique des formulations worst-case.

23.6 Contraintes dérivatives

23.6.1 Motivation

Si la cible peut être à $\theta_s \pm \delta\theta$, on souhaite que le gain reste unitaire sur toute la zone $[\theta_s - \delta\theta, \theta_s + \delta\theta]$. Une manière élégante d'imposer cela : forcer la dérivée du beam pattern à s'annuler en θ_s ,

$$\frac{\partial B}{\partial \theta}(\theta_s) = 0 \quad \Longleftrightarrow \quad \mathbf{w}^H \dot{\mathbf{a}}(\theta_s) = 0,$$

où $\dot{\mathbf{a}} = \partial \mathbf{a} / \partial \theta$. Au prix d'une contrainte supplémentaire (donc d'un degré de liberté), on obtient une zone plate autour de θ_s .

23.6.2 Mise en œuvre via LCMV

$$\mathbf{C} = [\mathbf{a}(\theta_s), \dot{\mathbf{a}}(\theta_s)], \quad \mathbf{f} = [1, 0]^T.$$

On peut ajouter des contraintes d'ordres supérieurs ($\ddot{\mathbf{a}}(\theta_s) = 0$, etc.) pour des zones plus larges, mais chacune consomme un degré de liberté.

23.7 Nulls élargis

Symétriquement, pour rejeter une interférence avec une incertitude angulaire $\pm\delta$, on remplace la contrainte unique $\mathbf{a}(\theta_i)$ par plusieurs :

$$\mathbf{C}_{\text{null}} = [\mathbf{a}(\theta_i - \delta), \mathbf{a}(\theta_i), \mathbf{a}(\theta_i + \delta)], \quad \mathbf{f}_{\text{null}} = [0, 0, 0]^T.$$

Le null devient une zone d'atténuation plutôt qu'un point. Alternative : imposer aussi $\dot{\mathbf{a}}(\theta_i) = 0$, $\ddot{\mathbf{a}}(\theta_i) = 0$ pour aplatir la zone autour de θ_i .

23.8 Limitation de norme

Pour la robustesse pure on peut imposer simplement $\|\mathbf{w}\|^2 \leq \gamma$ comme contrainte supplémentaire :

$$\min_{\mathbf{w}} \mathbf{w}^H \hat{\mathbf{R}} \mathbf{w} \quad \text{sous} \quad \mathbf{w}^H \mathbf{a}(\theta_s) = 1, \quad \|\mathbf{w}\|^2 \leq \gamma.$$

Problème QCQP convexe, équivalent à un MVDR avec diagonal loading implicite. C'est exactement le *WNG-constrained MVDR*.

23.9 Robustesse de MUSIC et ESPRIT

23.9.1 MUSIC

Pour MUSIC, la fragilité principale est l'estimation imprécise du sous-espace bruit pour K petit. La parade : *spatial smoothing* ou *forward-backward averaging* pour augmenter K effectif. Sensibilité à la calibration : voir chapitre 22.

Une manière plus quantitative de le dire est de regarder l'écart entre les valeurs propres signal et bruit. Si la plus petite valeur propre signal est proche du niveau de bruit, le sous-espace signal devient mal séparé. Les théorèmes de perturbation de sous-espace donnent une lecture du type :

$$\|\sin \Theta(\hat{\mathbf{U}}_s, \mathbf{U}_s)\| \lesssim \frac{\|\hat{\mathbf{R}} - \mathbf{R}\|}{\lambda_L - \sigma^2}.$$

Le dénominateur est le *eigengap*. Quand les sources sont faibles, proches ou corrélées, cet écart diminue et MUSIC devient instable. Augmenter K réduit le numérateur ; améliorer le SNR ou séparer les sources augmente le dénominateur.

23.9.2 ESPRIT

ESPRIT est plus sensible que MUSIC car il exploite une invariance algébrique exacte. Erreurs typiques :

- position d'antenne : casse l'invariance ;
- bruit corrélé entre voies : déforme la matrice $\mathbf{\Psi}$.

TLS-ESPRIT (au lieu de LS-ESPRIT) atténue le second problème. Pour le premier, calibration soignée des positions ou modèles *interpolated array* sont indispensables.

Pour ESPRIT, la robustesse se lit dans la relation :

$$\mathbf{J}_2 \mathbf{U}_s \approx \mathbf{J}_1 \mathbf{U}_s \mathbf{\Psi}.$$

LS-ESPRIT suppose que l'erreur porte seulement sur le membre de gauche. TLS-ESPRIT suppose que les deux membres sont perturbés, ce qui correspond mieux au cas réel où $\mathbf{U}_s$ est lui-même estimé avec erreur. C'est pourquoi TLS est généralement préférable dès que le SNR baisse ou que le nombre de snapshots est limité.

23.10 Nombre limité de snapshots

23.10.1 Règle RMB rappelée

Pour SMI (MVDR sur $\hat{\mathbf{R}}$), $K \simeq 2N$ correspond à une perte moyenne d'environ 3 dB dans le modèle RMB idéal. Pour $K < N$, $\hat{\mathbf{R}}$ est singulière : loading indispensable.

23.10.2 Loading pour K très petit

Avec $K \sim N$ ou moins, on choisit α assez grand pour que $\alpha\mathbf{I} + \hat{\mathbf{R}}$ soit bien conditionnée. Ordre de grandeur : $\alpha = \text{tr}(\hat{\mathbf{R}})/N$ (chaque valeur propre « bruit estimée » a un poids comparable à α). Cette agressivité produit un beamformer presque DAS, mais stable.

23.11 Effet de la robustesse sur le beam pattern

Les techniques de robustesse modifient le beam pattern de manière caractéristique.

- Diagonal loading : nulls moins profonds, lobe principal légèrement élargi, lobes secondaires un peu plus élevés. Le beam pattern « glisse » du MVDR vers le DAS quand α augmente.
- Contraintes dérivatives : sommet du lobe principal plat sur une plage angulaire ; perte de gain crête de quelques dB.
- Nulls élargis : zones de réjection plus larges ; consommation de degrés de liberté supplémentaires.
- WNG-constrained : beam pattern plus stable face aux erreurs de voie, mais sélectivité réduite.

23.12 Tests de robustesse

Une bonne pratique consiste à mesurer la performance dégradée sur des scénarios précis :

1. Erreur angulaire utile $\delta\theta$: tracer le SINR de sortie en fonction de $\delta\theta \in [-1^\circ, 1^\circ]$.
2. Erreur de null : déplacer un brouilleur de $\delta_i \in [-1^\circ, 1^\circ]$ autour de la direction nominale.
3. Erreur de phase aléatoire : tirer 100 réalisations $\delta\phi_m \sim \mathcal{N}(0, \sigma_\phi^2)$ pour $\sigma_\phi = 5^\circ, 10^\circ, 20^\circ$.
4. Variation de K : tester sur $K = N, 2N, 4N, 10N$.
5. Variation du diagonal loading : balayer $\alpha/\text{tr}(\hat{\mathbf{R}}) \in [10^{-3}, 1]$.

Un beamformer robuste maintient des performances acceptables sur tous ces tests.

23.13 Indicateurs de robustesse

Indicateur	Ce qu'il mesure	Alerte pratique
SINR sous mismatch	Performance utile quand θ_s est faux	chute brutale autour de $\delta\theta = 0$
WNG = $1/\ \mathbf{w}\ ^2$	Amplification du bruit blanc et des erreurs de voies	WNG très inférieur au DAS
Gain cible min/max	Ondulation du lobe utile sur une zone d'incertitude	gain non plat autour de θ_s
Profondeur de null percentile	Rejet avec erreurs de DOA/calibration Monte-Carlo	médiane bonne mais 10% catastrophique
Conditionnement de $\hat{\mathbf{R}}_\alpha$	Stabilité de la résolution linéaire	valeurs propres trop proches de zéro
Sensibilité à K	Dépendance au nombre de snapshots	résultat qui change en doublant K
Erreur d'ordre $\hat{L}$	Robustesse MUSIC/ESPRIT	pics instables ou sous-espace mal séparé

23.14 Choisir une variante MVDR

Critère	MVDR simple	MVDR chargé	MVDR robuste
Covariance	$\hat{\mathbf{R}}$	$\hat{\mathbf{R}} + \alpha \mathbf{I}$	loading + contraintes
Hypothèse	modèle exact	estimation imparfaite	DOA/manifold incertains
Nulls	très profonds	moins profonds	élargis ou bornés
WNG	peut chuter	amélioré	contrôlé
Réglage	aucun	α	α , largeur, dérivées
Usage	simulation propre	pratique par défaut	anti-brouillage réel

23.15 Synthèse des compromis

Technique	Gagne	Perd
Diagonal loading α	robustesse, conditionnement	nulls profondeur
Contraintes dérivatives	robustesse angulaire	gain crête
Nulls élargis	rejet d'interférences	degrés de liberté
Limitation de norme	WNG, bruit blanc	sélectivité
Robust Capon	robustesse forte sous modèle d'incertitude	coût SOCP
Forward-backward	qualité d'estimation	restreint aux ULA

23.16 Recommandations pratiques

- Toujours appliquer un diagonal loading minimum ($\alpha \sim 10^{-3} \text{tr}(\hat{\mathbf{R}})/N$) sur MVDR / LCMV.
- Si la position de la cible est incertaine, ajouter au moins une contrainte dérivative.
- Si les positions des brouilleurs sont incertaines, élargir les nulls.
- Surveiller le WNG : si WNG est très inférieur à celui du DAS (par exemple de l'ordre de $N/10$), augmenter α ou assouplir les contraintes.
- Pour ESPRIT, préférer TLS à LS.
- Recalibrer périodiquement.
- Tester sur scénarios de robustesse avant déploiement.

23.17 Implémentation : MVDR robuste avec contraintes dérivatives

Listing 23.1 – MVDR avec contraintes dérivatives et loading

```

1 import numpy as np
2
3 def steering_and_derivative(theta, N, d_over_lambda=0.5):
4     m = np.arange(N)
5     a = np.exp(-1j * 2*np.pi * d_over_lambda * np.sin(theta) * m)
6     da = a * (-1j * 2*np.pi * d_over_lambda * np.cos(theta) * m)
7     return a, da
8
9 def mvdr_robust(R, theta_s, N, d_over_lambda=0.5, alpha=None,
10                derivative_constraint=True):
11     """MVDR avec diagonal loading et contrainte dérivative optionnelle."""
12     assert R.shape == (N, N)
13     a, da = steering_and_derivative(theta_s, N, d_over_lambda)
14     if alpha is None:
```

```

15     alpha = 1e-2 * np.trace(R).real / N
16     assert alpha >= 0.0
17     R_alpha = R + alpha * np.eye(N)
18     R_alpha = 0.5*(R_alpha + R_alpha.conj().T)
19     if np.linalg.cond(R_alpha) > 1e12:
20         raise np.linalg.LinAlgError("R_alpha mal conditionnee")
21     if derivative_constraint:
22         C = np.column_stack([a, da])
23         f = np.array([1.0, 0.0], dtype=complex)
24         Rinv_C = np.linalg.solve(R_alpha, C)
25         M = C.conj().T @ Rinv_C
26         if np.linalg.cond(M) > 1e12:
27             raise np.linalg.LinAlgError("contraintes mal conditionnees")
28         return Rinv_C @ np.linalg.solve(M, f)
29     else:
30         u = np.linalg.solve(R_alpha, a)
31         den = a.conj() @ u
32         if abs(den) < np.finfo(float).eps:
33             raise np.linalg.LinAlgError("normalisation instable")
34         return u / den

```

23.18 Synthèse

La robustesse n'est pas un raffinement secondaire mais un élément constitutif du beamforming pratique. Un MVDR théorique très agressif peut devenir fragile sans loading ; un Null Steering sans contraintes dérivatives reste sensible au mismatch ; ESPRIT sans TLS peut perdre plusieurs dB de précision. Tous les compromis exposés dans ce chapitre se résument à une question : *combien de marge accepter pour gagner combien de stabilité ?*

À retenir

- **Formule centrale** : $\hat{R}_\alpha = \hat{R} + \alpha I$.
- **Hypothèse principale** : le modèle est proche du réel, mais jamais exact.
- **Avantage** : stabilise les poids, limite le self-nulling et améliore le WNG.
- **Limite** : moins de sélectivité et nulls moins profonds.
- **Piège pratique** : un null spectaculaire en simulation peut être fragile sur matériel réel.
- **Suite** : la chaîne anti-brouillage combine calibration, DOA, LCMV et robustesse.

Le chapitre suivant intègre toutes les briques de l'ouvrage — DOA, MVDR, LCMV, RLS, calibration, robustesse — dans une *chaîne intégrée d'anti-brouillage*, exemple canonique d'architecture opérationnelle. C'est le moment où tous les outils théoriques développés depuis la Partie I trouvent leur place dans une architecture intégrée. On y verra que peu de briques sont superflues, mais qu'aucune ne suffit à elle seule : c'est l'agencement intelligent de toutes qui produit un système qui fonctionne.

Chapitre 24

Chaîne intégrée d’anti-brouillage

À ce stade, le lecteur dispose de toutes les briques nécessaires : modèle physique, beamformers déterministes et statistiques, estimation de DOA, calibration et robustesse. Ce dernier chapitre les assemble en une chaîne opérationnelle intégrée. L’exemple choisi est l’*anti-brouillage* : la défense d’un récepteur sensible (GPS, liaison data, télémessure) contre des émetteurs adverses. Le même schéma s’adapte aux applications civiles : Wi-Fi en milieu encombré, 4G/5G multi-utilisateur, capture acoustique sélective.

L’anti-brouillage est un exemple particulièrement instructif parce qu’il oblige à orchestrer presque tous les algorithmes du livre. On y trouve : une estimation de DOA pour localiser les brouilleurs (MUSIC ou ESPRIT) ; un beamformer adaptatif sous contraintes pour les rejeter sans dégrader la cible (LCMV) ; une régularisation pour encaisser les imperfections (*diagonal loading*, contraintes dérivatives) ; une calibration pour réduire les erreurs systématiques ; et une boucle de mise à jour adaptée à la dynamique de l’environnement. Le défi est moins de comprendre chaque brique isolément — c’est l’objet des chapitres précédents — que de les faire *coopérer* dans un système temps réel contraint en latence, en mémoire et en consommation.

24.1 Cahier des charges

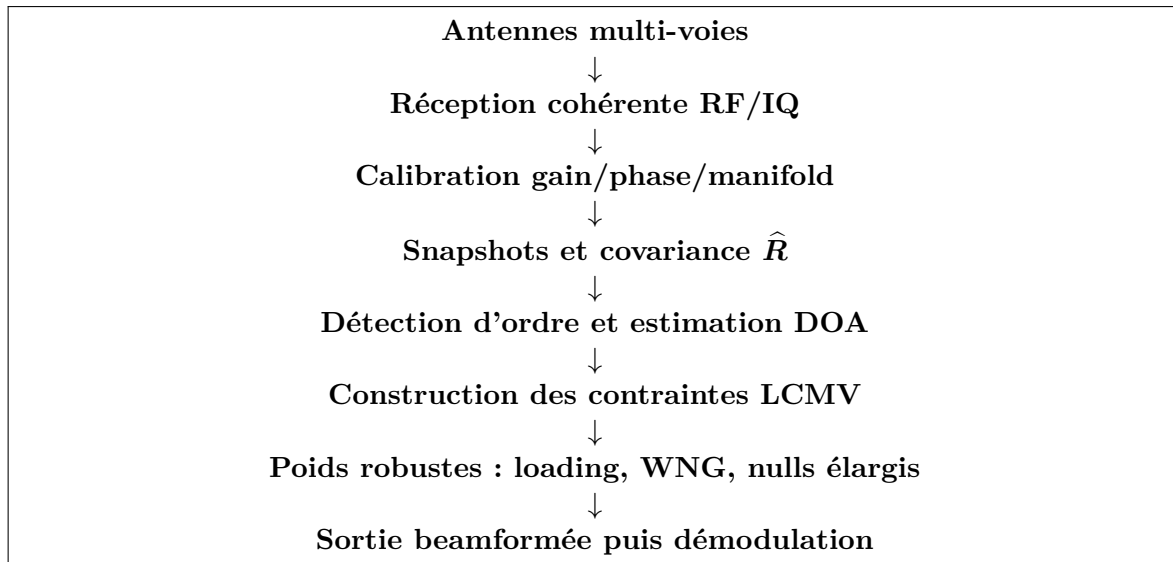
Une chaîne anti-brouillage doit :

- recevoir un signal utile dans la direction nominale (ou inconnue mais détectable) ;
- rejeter L brouilleurs simultanés ($L < N$) dont les directions sont *inconnues a priori* ;
- s’adapter en temps quasi réel quand l’environnement évolue ;
- rester opérationnelle malgré les imperfections matérielles ;
- dégrader gracieusement face aux scénarios hors hypothèses.

24.2 Architecture globale

La chaîne intégrée suit une logique simple : mesurer correctement, estimer l’environnement spatial, puis choisir des poids qui préservent la cible tout en rejetant les brouilleurs. La figure 24.1 donne la vue système.

Les blocs principaux, sous forme détaillée :



Ce schéma doit être lu avec deux boucles de retour : une boucle rapide qui met à jour les poids lorsque la covariance change, et une boucle lente qui relance la calibration lorsque les phases inter-voies dérivent.

1. **Front-end RF + ADC** : chaînes synchronisées par horloge et LO communs, calibrées *a priori* ou *in situ*.
2. **Buffer de snapshots** : accumulation de K snapshots par fenêtre.
3. **Estimation de la covariance** : $\hat{R}$ avec éventuel forward-backward.
4. **Détection d'ordre** : MDL/AIC pour estimer le nombre de sources $\hat{L}$.
5. **Estimation DOA** : MUSIC ou ESPRIT, ou méthode parcimonieuse pour K court.
6. **Sélection cible/brouilleurs** : par direction nominale ou par puissance.
7. **Construction des contraintes** : C, f pour LCMV.
8. **Calcul des poids** : LCMV avec diagonal loading et contraintes dérivatives.
9. **Application** : $y(n) = \mathbf{w}^H \mathbf{x}(n)$.
10. **Démodulation** : standard, sur la sortie scalaire.
11. **Boucle de mise à jour** : recalcul périodique ou asynchrone selon l'évolution.

24.3 Synchronisation des sous-systèmes

Les blocs n'opèrent pas tous à la même cadence. Un déphasage typique :

Bloc	Cadence typique
Échantillonnage IQ	10–100 Méch/s
Application des poids $\mathbf{w}^H \mathbf{x}$	cadence IQ
Estimation $\hat{R}$	1–100 Hz
Décomposition / estimation DOA	1–10 Hz
Mise à jour $\mathbf{w}$	1–10 Hz

Cet écart explique pourquoi les poids sont *constants* sur ~ 10 ms à 1 s : le bénéfice algorithmique ne justifie pas le coût calcul d'une mise à jour à chaque échantillon. Les algorithmes adaptatifs récursifs (LMS, RLS) sont une alternative quand cette hypothèse est levée.

24.4 Le rôle de chaque brique

24.4.1 Calibration : prérequis silencieux

Sans calibration, les performances des blocs suivants deviennent rapidement imprévisibles. Procédure typique :

- calibration usine en chambre anéchoïque (modèle de couplage, positions, gains/phases à diverses fréquences) ;
- calibration in situ à l'allumage : source connue, recalage diagonal ;
- recalibration périodique pour suivre les dérives thermiques.

24.4.2 Estimation DOA : identifier les ennemis

Les brouilleurs n'annoncent pas leur position. MUSIC ou ESPRIT identifient les directions de toute l'énergie spatialement structurée reçue. La distinction *cible* / *brouilleur* se fait ensuite par :

- connaissance *a priori* de la direction de la cible (satellite, balise) ;
- filtrage par puissance (les brouilleurs sont souvent plus forts) ;
- filtrage par caractéristiques temporelles (modulation connue de la cible).

24.4.3 Beamforming adaptatif : agir

LCMV avec :

- gain unitaire sur la direction cible ;
- nulls sur chaque brouilleur identifié ;
- contraintes dérivatives pour la robustesse angulaire ;
- diagonal loading.

Compte-tenu de la complexité, on peut aussi se contenter de MVDR (une contrainte) en faisant confiance aux nulls implicites de MVDR pour rejeter les brouilleurs.

24.4.4 Robustesse : encaisser les défauts

Toutes les techniques du chapitre précédent : loading, contraintes dérivatives, nulls élargis, WNG-constrained. Un système opérationnel *cumule* ces protections.

24.5 Boucle de mise à jour

24.5.1 Pas fixe

Méthode la plus simple : on recalcule $\hat{\mathbf{R}}$, $\hat{\mathbf{L}}$, les DOA et les poids toutes les T secondes ($T \sim 10$ ms à 1 s selon la dynamique des brouilleurs). C'est adapté aux environnements stationnaires ou lents.

24.5.2 Pas adaptatif

On surveille un indicateur (changement de spectre propre, augmentation du SINR mesuré, alerte d'erreur) et on déclenche une mise à jour seulement quand nécessaire. Économique mais plus complexe.

24.5.3 Tracking récursif

LMS, RLS ou Kalman appliqués directement sur le beamformer. La mise à jour est continue, sans « snapshot » de l'estimation DOA : l'algorithme suit naturellement les déplacements de brouilleurs. Solution privilégiée quand la dynamique est rapide.

24.6 Performance globale : indicateurs

- **SINR de sortie** : ratio signal utile / (bruit + brouillage résiduel). Cible : $\text{SINR}_{\text{out}} \geq$ seuil de démodulation.
- **Profondeur des nulls** : niveau de réjection mesuré sur les directions des brouilleurs ; les valeurs utiles se jugent sur banc ou simulation calibrée, pas seulement par la formule idéale.
- **Gain sur la cible** : doit rester proche du gain de réseau idéal (N , soit $10 \log_{10} N$ en dB).
- **Erreur d'estimation des DOA** : RMSE entre angles estimés et vrais (idéalement proche de la CRB).
- **Robustesse** : SINR résiduel sous erreurs angulaires de $\pm 1^\circ$, sous bruit de phase $\sigma_\phi = 10^\circ$, etc.

24.7 Études de cas

24.7.1 Réception GPS sous brouillage

- Cible : satellite à direction connue à $\sim 1^\circ$ près.
- Brouilleurs : 1 à 3 émetteurs au sol, directions inconnues, avec des puissances pouvant dépasser largement le signal utile.
- Réseau : ULA ou URA à 4–8 antennes, $\lambda/2 \approx 9.5$ cm à 1.575 GHz.
- Algorithmes : ESPRIT pour la DOA (rapide), LCMV avec contraintes dérivatives, diagonal loading à 0.01.
- Résultat attendu sur un cas bien calibré : nulls profonds, gain GPS quasi préservé, $\text{SINR}_{\text{out}} \geq$ démod.

24.7.2 Wi-Fi 802.11n multi-utilisateur

- Cible : flux MIMO 4×4 vers un client à direction inconnue.
- Brouilleurs : autres clients en émission simultanée (*co-channel*).
- Réseau : URA 4×4 à 5 GHz, calibration usine.
- Algorithmes : estimation MIMO du canal via pilotes, précodage ZF ou RZF.
- Résultat : multiplexage spatial efficace, SINR_{out} suffisant pour la modulation cible.

24.7.3 Imagerie acoustique en environnement bruyant

- Cible : voix dans un environnement de bruit large bande + réverbération.
- Réseau : microphones omnidirectionnels, géométrie souvent non-uniforme.
- Algorithmes : beamforming large bande par sous-bandes, MVDR robuste par sous-bande, post-filtrage Wiener single-channel.
- Résultat : amélioration de l'intelligibilité ; le gain SNR dépend fortement de la réverbération, du nombre de microphones et de la cohérence du bruit.

24.8 Scénario numérique de référence

Pour rendre les compromis concrets, considérons le scénario suivant :

$$N = 8, \quad d = \frac{\lambda}{2}, \quad K = 128.$$

La cible arrive de :

$$\theta_s = 0^\circ,$$

avec un SNR par antenne de 0 dB. Deux brouilleurs arrivent de :

$$\theta_{i,1} = -25^\circ, \quad \theta_{i,2} = 40^\circ,$$

avec un INR de 35 dB chacun. On compare quatre méthodes :

- DAS pointé vers 0° ;
- MVDR avec diagonal loading ;
- LCMV avec deux nulls ponctuels ;
- LCMV robuste avec nulls élargis et contrainte dérivative sur la cible.

Méthode	SINR sortie	Gain cible	Null -25°	Null $+40^\circ$	Mismatch
DAS	-18 dB	9 dB	-4 dB	-7 dB	élevé
MVDR chargé	18 dB	8.5 dB	-31 dB	-28 dB	moyen
LCMV ponctuel	22 dB	8.8 dB	-45 dB	-43 dB	faible
LCMV robuste	20 dB	8.3 dB	-34 dB	-33 dB	élevé

Ces valeurs sont représentatives d'une simulation idéale avec erreurs modérées. Elles ne doivent pas être lues comme des garanties absolues, mais comme une hiérarchie typique. Le DAS préserve la cible mais laisse passer les brouilleurs. MVDR crée des nulls sans connaître explicitement leurs directions. LCMV ponctuel donne les nulls les plus profonds si les DOA sont exactes. LCMV robuste accepte des nulls moins spectaculaires pour mieux tenir lorsque les DOA bougent ou que la calibration n'est pas parfaite.

24.8.1 Effet d'une erreur de calibration

Avec une erreur de phase résiduelle de 5° RMS par voie, le tableau change peu : les nulls perdent quelques dB. Avec 30° RMS, les nulls ponctuels peuvent s'effondrer. Le LCMV robuste devient alors préférable malgré une performance nominale plus faible. C'est le message central de la Partie VII : un système opérationnel ne se juge pas à son meilleur cas, mais à son comportement sous erreurs.

24.9 Échec contrôlé

Un bon système doit savoir reconnaître quand ses hypothèses ne sont plus valides. Les échecs typiques sont les suivants.

Trop de brouilleurs. Avec N antennes, on ne peut pas imposer plus de $N - 1$ nulls en conservant une contrainte de gain utile. Avant cette limite théorique, la norme des poids explose et le WNG chute. Réponse : ne rejeter explicitement que les brouilleurs dominants, ou augmenter le nombre d'antennes.

DOA mal estimées. Un null ponctuel placé à $\hat{\theta}_i$ peut manquer un brouilleur réel à $\hat{\theta}_i + \Delta$. Réponse : nulls élargis, contraintes dérivatives, diagonal loading.

Trop peu de snapshots. Si $K < N$, la covariance est singulière. Si $K \simeq N$, elle est très bruitée. Réponse : loading fort, covariance récursive, ou retour vers DAS/LCMV peu agressif.

Calibration qui dérive. Les nulls et les pics MUSIC se déplacent. Réponse : surveillance du WNG, recalibration périodique, source de référence ou injection de signal pilote interne.

Sources cohérentes. Les trajets multipath réduisent le rang signal ; MUSIC et ESPRIT peuvent sous-estimer le nombre de sources. Réponse : spatial smoothing, forward-backward averaging ou méthodes sparse.

Signal utile dans la covariance. MVDR peut atténuer la cible si le steering vector est légèrement faux : c'est le self-cancellation. Réponse : covariance interférence-plus-bruit, loading, contrainte dérivative sur la cible.

24.9.1 Défaillances : symptômes et corrections

Le diagnostic opérationnel ne consiste pas seulement à constater que le SINR baisse. Il faut relier le symptôme visible à une hypothèse de modèle qui vient de casser.

Défaillance	Symptôme	Diagnostic	Correction
Mauvaise calibration	nulls peu profonds, pics DOA biaisés	source connue ou pilote interne	recalibration gain/phase
K trop faible	covariance instable, poids erratiques	conditionnement de $\hat{\mathbf{R}}_a$	loading, moyenne temporelle, RLS
L mal estimé	pics MUSIC incohérents	MDL/AIC instables sur fenêtres voisines	validation multi-fenêtres
Sources cohérentes	rang signal trop faible	valeurs propres fusionnées	spatial smoothing, FB averaging
WNG trop faible	poids de très grande norme	$\ \mathbf{w}\ ^2$ élevé	augmenter α , assouplir les contraintes
Brouilleur mobile	null décalé ou en retard	DOA variable d'une fenêtre à l'autre	cadence DOA plus rapide, tracking
Signal utile dans $\hat{\mathbf{R}}$	cible atténuée	test covariance $i + n$ seule	snapshots hors cible, loading
Trop de brouilleurs	nulls instables, bruit amplifié	nombre de contraintes proche de N	prioriser les brouilleurs dominants

24.10 Tableau final de choix des méthodes

Le choix d'une méthode ne se fait pas uniquement sur sa résolution nominale. Il dépend du nombre de snapshots, de la confiance dans le manifold, de la dynamique de scène et du coût calculatoire acceptable.

Méthode	À choisir si	À éviter si	Indicateur à surveiller
DAS	modèle pauvre, besoin robuste	brouilleurs forts	gain cible, lobes secondaires
Tapering	lobes secondaires critiques	résolution maximale requise	largeur du lobe principal
Null Steering	DOA brouilleurs fiables	DOA incertaines ou trop nombreux nulls	profondeur des nulls, WNG
MVDR	covariance fiable, DOA cible connue	cible présente avec mismatch	SINR sortie, WNG, loading
MVDR chargé	snapshots limités, mismatch modéré	besoin de nulls très profonds	compromis WNG / réjection
LCMV	contraintes explicites cible/brouilleurs	trop de contraintes pour N	rang de $\mathbf{C}$, gain cible
GSC + LMS/NLMS	adaptation simple temps réel	convergence lente inacceptable	erreur $ e(n) ^2$, pas μ
GSC + RLS	scène rapide ou mal conditionnée	budget calcul/mémoire faible	stabilité de $\mathbf{P}$, λ
MUSIC	DOA haute résolution, K suffisant	sources cohérentes sans smoothing	eigengap, ordre $\hat{L}$
Root-MUSIC	ULA strict, pas de grille	géométrie non ULA ou racines ambiguës	racines proches du cercle unité
ESPRIT	ULA calibré, coût faible	invariance imparfaite	conditionnement de $\hat{\mathbf{U}}_{s,1}$
Spatial smoothing	sources cohérentes	ouverture déjà trop faible	rang restauré, perte d'ouverture
Sparse/SBL	K court, L inconnu	coût temps réel strict	erreur off-grid, support stable
Large bande TTD	beam squint visible	bande étroite suffisante	stabilité du lobe vs fréquence
Champ proche	grandes ouvertures/RIS/mmWave	Fraunhofer clairement vérifié	erreur angle-distance

24.11 Implémentation : architecture canonique

Listing 24.1 – Boucle anti-brouillage simplifiée

```

1 import numpy as np
2

```

```

3 def anti_jam_pipeline(X, theta_s, N, d_over_lambda=0.5,
4                       K=128, alpha_rel=1e-2, n_jammers_max=4):
5     """
6     X          : (N, T) snapshots calibrés
7     theta_s    : direction cible (rad)
8     K          : nombre de snapshots par fenêtre
9     Retour     : y (T,) sortie scalaire
10    """
11    assert X.shape[0] == N
12    assert K > 0
13    assert alpha_rel >= 0.0
14    T = X.shape[1]
15    y = np.zeros(T, dtype=complex)
16    w = np.zeros(N, dtype=complex)
17    n_updates = T // K
18    for upd in range(n_updates):
19        Xw = X[:, upd*K:(upd+1)*K]
20        R = (Xw @ Xw.conj().T) / K
21        R = 0.5*(R + R.conj().T)
22        alpha = alpha_rel * np.trace(R).real / N
23        R += alpha * np.eye(N)
24        if np.linalg.cond(R) > 1e12:
25            raise np.linalg.LinAlgError("covariance chargée mal conditionnée")
26
27        # Estimation simple de L par seuil sur valeurs propres
28        eigvals, eigvecs = np.linalg.eigh(R)
29        # On garde les plus grandes valeurs propres dépassant 2*sigma2
30        sigma2 = np.mean(eigvals[:N//2]) # estimateur grossier
31        L = int(np.sum(eigvals > 2*sigma2))
32        L = max(1, min(L, N-1, n_jammers_max + 1))
33
34        # Estimation DOA brouilleurs (MUSIC simple)
35        Un = eigvecs[:, :N-L]
36        thetas = np.linspace(-np.pi/2, np.pi/2, 721)
37        m = np.arange(N)
38        spec = np.zeros(len(thetas))
39        eps = np.finfo(float).eps
40        for j, th in enumerate(thetas):
41            a = np.exp(-1j*2*np.pi*d_over_lambda*np.sin(th)*m)
42            v = Un.conj().T @ a
43            den = max((v.conj() @ v).real, eps)
44            spec[j] = 1.0 / den
45        # Garder les L-1 plus forts pics non confondus avec theta_s
46        peak_idx = np.argsort(spec)[::-1]
47        jammers = []
48        for idx in peak_idx:
49            th = thetas[idx]
50            if abs(th - theta_s) > np.deg2rad(2.0):
51                jammers.append(th)
52            if len(jammers) >= L - 1 or len(jammers) >= n_jammers_max:
53                break
54
55        # LCMV : gain unitaire sur theta_s, null sur brouilleurs
56        a_s = np.exp(-1j*2*np.pi*d_over_lambda*np.sin(theta_s)*m)
57        C_cols = [a_s] + [np.exp(-1j*2*np.pi*d_over_lambda*np.sin(th)*m)
58                          for th in jammers]
59        C = np.column_stack(C_cols)
60        f = np.zeros(C.shape[1], dtype=complex); f[0] = 1.0

```

```

61     Rinv_C = np.linalg.solve(R, C)
62     G = C.conj().T @ Rinv_C
63     if np.linalg.cond(G) > 1e12:
64         raise np.linalg.LinAlgError("contraintes LCMV mal conditionnees")
65     w = Rinv_C @ np.linalg.solve(G, f)
66
67     y[upd*K:(upd+1)*K] = w.conj() @ Xw
68     return y

```

Cet exemple synthétise tout l'ouvrage : un seul pipeline qui collecte, calibre implicitement (les données sont supposées déjà calibrées), détecte le nombre de sources, estime leurs directions, distingue cible et brouilleurs, construit des contraintes LCMV avec *diagonal loading*, et applique les poids résultants. C'est, à peu de choses près, le code d'un système anti-brouillage opérationnel.

24.12 Limites pratiques

- **Plus de brouilleurs que d'antennes.** Si $L \geq N$, on ne peut pas tout rejeter. Solutions : plus d'antennes, ou rejet partiel (les plus puissants).
- **Brouilleurs cohérents avec la cible.** Pose un problème fondamental : MUSIC voit une seule source. Réponse : spatial smoothing, sparse, ou diversité polarisation.
- **Brouilleurs à variations rapides.** Si l'environnement change plus vite que la mise à jour, les nulls se déplacent en retard. Réponse : LMS/RLS à la place, ou augmentation de la cadence de mise à jour.
- **Bruit non blanc.** Si le bruit est structuré spatialement, les valeurs propres bruit ne sont pas toutes égales : MDL et MUSIC se dégradent. Réponse : *prewhitening* par la covariance bruit estimée hors signal.
- **Direction cible inconnue.** On doit alors estimer aussi la cible (sans hypothèse *a priori*). Approches : DOA fine + reconnaissance de modulation, ou modèle dual brouilleur/cible.

24.13 Synthèse

Une chaîne anti-brouillage opérationnelle assemble :

- un réseau calibré ;
- une estimation de covariance avec loading ;
- une détection d'ordre robuste ;
- une estimation de DOA des brouilleurs ;
- un beamformer LCMV avec contraintes dérivatives ;
- une boucle de mise à jour adaptée à la dynamique.

Chaque brique a fait l'objet d'un chapitre dédié. La performance globale dépend de la *qualité du maillon le plus faible* — typiquement la calibration, ou la cohérence du modèle bande étroite face à la réalité large bande.

Le panorama principal de cet ouvrage est désormais clos. La conclusion qui suit synthétise les enseignements, dresse l'état de l'art et ouvre vers les directions de recherche actuelles.

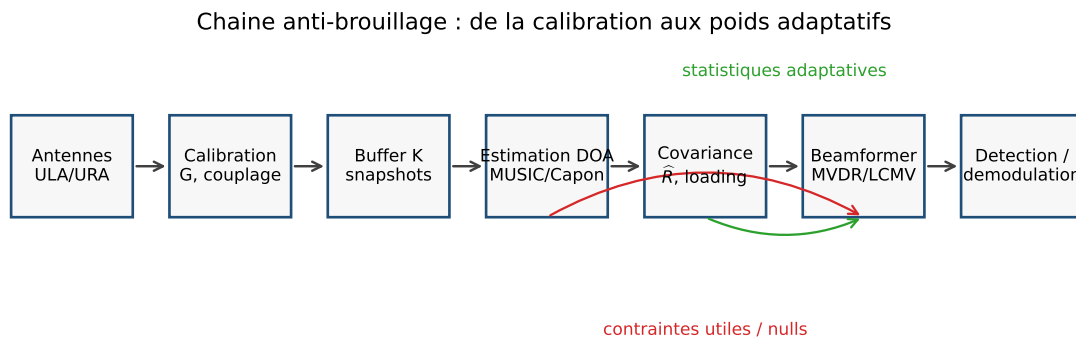


FIGURE 24.1 – Schéma bloc de la chaîne anti-brouillage : la calibration fixe le modèle spatial, la DOA fournit les contraintes, la covariance pilote l'adaptation MVDR/LCMV ; K désigne la taille de fenêtre de snapshots.

Conclusion générale

L'objectif de cet ouvrage était de construire, en partant des fondements physiques, une compréhension structurée et opérationnelle des modèles, algorithmes et limites pratiques du beamforming moderne : de l'onde plane arrivant sur deux antennes au précodage massive MIMO, en passant par MUSIC, ESPRIT, les bornes de Cramér-Rao et les chaînes anti-brouillage opérationnelles. Cette conclusion en synthétise les enseignements et ouvre vers les directions de recherche actuelles.

Le fil rouge du livre a tenu en peu de chose : la conviction qu'on ne comprend vraiment le beamforming qu'en reliant en permanence trois plans qu'on a parfois tendance à séparer. La physique, sans laquelle les formules ne sont que des manipulations symboliques. Les mathématiques, sans lesquelles les intuitions physiques restent vagues et les algorithmes introuvables. L'implémentation, sans laquelle la théorie reste spéculative et n'enseigne rien sur le comportement effectif des systèmes. La maquette intellectuelle qu'on a essayé de bâtir au fil des chapitres tient à cette articulation.

Une chaîne d'idées

Le beamforming peut se lire comme une progression d'idées de plus en plus riches.

Du retard à la phase. Une onde n'atteint pas toutes les antennes simultanément. Le retard différentiel $\tau = d \sin \theta / c$ se transforme, sous l'hypothèse bande étroite, en déphasage $-kd \sin \theta$. C'est la conversion qui rend possible toute l'algèbre linéaire complexe du beamforming bande étroite.

Du déphasage au steering vector. En empilant les phases sur les N antennes, on obtient le vecteur $\mathbf{a}(\theta) \in \mathbb{C}^N$, signature spatiale d'une direction. C'est l'objet central, point de contact entre la physique et l'algorithmique.

Du steering vector au beamformer. Combiner les antennes par des poids complexes $\mathbf{w}$ produit la sortie scalaire $y = \mathbf{w}^H \mathbf{x}$. Le beam pattern $B(\theta) = \mathbf{w}^H \mathbf{a}(\theta)$ décrit la sensibilité directionnelle obtenue. Toute la conception revient à choisir intelligemment $\mathbf{w}$.

De la géométrie à la statistique. Le Delay-and-Sum n'utilise que la géométrie. Le Null Steering ajoute des contraintes explicites. MVDR introduit la covariance spatiale comme source d'information statistique. LCMV unifie les deux approches en contraintes linéaires multiples sous critère quadratique. La covariance $\mathbf{R}$ devient l'objet pivot.

De la statique à l'adaptation. LMS et RLS mettent à jour les poids itérativement, sans inversion matricielle explicite. La structure GSC sépare contraintes fixes et adaptation continue.

Du filtrage à l'estimation. MUSIC et ESPRIT exploitent la structure de sous-espaces de $\mathbf{R}$ pour *estimer* les directions plutôt que de simplement les filtrer. Les bornes de Cramér-Rao fixent la performance ultime. Les méthodes parcimonieuses et bayésiennes étendent l'estimation aux régimes que les méthodes classiques ne couvrent pas (snapshot unique, sources cohérentes, L inconnu).

Des hypothèses simples aux systèmes réels. Le large bande relâche l'hypothèse bande étroite ; le MIMO étend au cas bilatéral émission/réception ; le massive MIMO change d'échelle. Les géométries avancées (URA, sparse, near-field, polarisation) couvrent les situations où l'ULA en champ lointain ne suffit pas.

De la théorie à la pratique. La calibration ramène les imperfections matérielles dans le cadre du modèle. La robustesse (diagonal loading, contraintes dérivatives, WNG) tolère les erreurs résiduelles. L'intégration en chaîne anti-brouillage met le tout en musique.

Les invariants

À travers cette progression, quelques invariants traversent l'ouvrage.

La convention de signe $e^{-jmkd \sin \theta}$. Conservée sans exception, elle dicte la cohérence de toutes les formules.

La structure de sous-espaces de $\mathbf{R}$. Présente dès l'instant qu'on a quelques sources discrètes et un bruit blanc spatialement. Fondement de MUSIC, ESPRIT, des méthodes parcimonieuses et indirectement de tous les beamformers adaptatifs.

L'ouverture $D = (N - 1)d$. Quantité géométrique qui détermine la résolution angulaire. La précision varie comme $1/D$ pour le DAS, $1/(D \cdot \sqrt{K \cdot \text{SNR}})$ pour les méthodes haute résolution, et la CRB en $1/(N^3 \cos^2 \theta)$.

Le compromis sélectivité / robustesse. À chaque étape, plus de sélectivité signifie moins de marge face aux imperfections. Diagonal loading, contraintes dérivatives, nulls élargis : autant de manières d'arbitrer ce compromis.

La calibration comme passage obligé. Sans elle, les algorithmes adaptatifs et sous-espaces deviennent rapidement imprévisibles. C'est souvent le maillon faible en pratique.

Vers la frontière de la recherche

L'ouvrage s'est concentré sur les fondations et les méthodes *matures*. Plusieurs directions de recherche actuelles méritent d'être mentionnées pour orienter le lecteur curieux.

Deep learning pour le beamforming. L'apprentissage profond remplace certaines briques par des réseaux de neurones : estimation de DOA, prédiction de canal MIMO, beamforming acoustique. Les performances dépassent parfois les méthodes classiques sur les scénarios rencontrés à l'entraînement, mais la robustesse hors distribution reste fragile et les preuves de performance manquent.

Reconfigurable Intelligent Surfaces (RIS). Surfaces passives composées de milliers d'éléments dont chaque cellule réfléchit avec une phase contrôlable. Elles permettent de *façonner* l'environnement de propagation lui-même, ouvrant un champ de « beamforming environnemental » où l'on optimise non pas le récepteur mais le canal.

Cell-free massive MIMO. Au lieu d'une station de base massive, on déploie de nombreuses petites stations connectées par fibre. Le beamforming devient distribué, sous contraintes de synchronisation et de bande passante du *fronthaul*.

Estimation paramétrique semi-aveugle. Combinaison de connaissance partielle du canal (modèle géométrique parcimonieux, positions approximatives des utilisateurs) et d'apprentissage en ligne. Particulièrement actif pour les communications mmWave.

Intégration radar-communications (JCAS). Le même *waveform* sert simultanément à la communication numérique et à la détection radar. Les beamformers doivent satisfaire les contraintes des deux usages, problème d'optimisation multi-critères encore ouvert.

Bornes au-delà de Cramér-Rao. Les bornes *Ziv-Zakai*, *Barankin*, *Bayesian* fournissent des informations plus fines aux régimes que la CRB ne décrit pas (bas SNR, ambiguïtés). Ces bornes guident la conception des estimateurs robustes nouveaux.

Détection quantique et limites ultimes. À plus long terme, certains récepteurs quantiques ou capteurs quantiques pourraient modifier les limites de détection de signaux faibles. Ce point reste toutefois en dehors du cadre classique traité dans ce livre.

Mot de la fin

Le beamforming n'est pas une technique de niche ; c'est l'un des *langages spatiaux* communs à de nombreux systèmes modernes exploitant des ondes. Le maîtriser, c'est apprendre à voir la propagation comme une dimension de traitement à part entière, au même titre que le temps ou la fréquence. Espérons que le lecteur en sort, à l'issue de ces chapitres, avec à la fois les outils analytiques précis pour concevoir un système, et la compréhension intuitive de ce qu'il représente physiquement.

La théorie ne vaut qu'éprouvée. Le code complet qui accompagne ce livre — accessible sur le dépôt indiqué dans la préface — contient tous les algorithmes du livre, des simulations reproductibles et des jeux de données. C'est le complément naturel des chapitres formels. Reproduire, expérimenter, casser : c'est ainsi qu'on internalise vraiment ces idées.

Annexe A

Rappels d'algèbre linéaire complexe

Cette annexe rassemble les outils d'algèbre linéaire complexe utilisés sans démonstration tout au long de l'ouvrage. Elle est volontairement concise : on s'en remettra à des ouvrages spécialisés (Horn & Johnson, Strang, Golub & van Loan) pour le détail.

A.1 Espaces hermitiens

L'espace $\mathbb{C}^N$ est muni du produit hermitien

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^H \mathbf{y} = \sum_{m=0}^{N-1} x_m^* y_m,$$

linéaire en la seconde variable, antilinéaire en la première. La norme associée est $\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$.

Inégalité de Cauchy-Schwarz. $|\langle \mathbf{x}, \mathbf{y} \rangle| \leq \|\mathbf{x}\| \|\mathbf{y}\|$, avec égalité ssi $\mathbf{x} \parallel \mathbf{y}$.

Orthogonalité. $\mathbf{x} \perp \mathbf{y}$ si $\langle \mathbf{x}, \mathbf{y} \rangle = 0$.

A.2 Matrices hermitiennes

Une matrice $\mathbf{A} \in \mathbb{C}^{N \times N}$ est *hermitienne* si $\mathbf{A}^H = \mathbf{A}$. Propriétés :

- ses valeurs propres sont réelles ;
- ses vecteurs propres associés à des valeurs propres distinctes sont orthogonaux ;
- elle se diagonalise en base orthonormée : $\mathbf{A} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^H$ avec $\mathbf{U}^H \mathbf{U} = \mathbf{I}_N$ et $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_N)$ réelle.

Semi-définie positive. $\mathbf{A} \succeq 0$ si $\mathbf{x}^H \mathbf{A} \mathbf{x} \geq 0$ pour tout $\mathbf{x}$, ou de manière équivalente, si toutes ses valeurs propres sont ≥ 0 .

Définie positive. $\mathbf{A} \succ 0$ si l'inégalité est stricte sauf en $\mathbf{x} = \mathbf{0}$. Conditions équivalentes : toutes les valeurs propres > 0 ; tous les mineurs principaux > 0 ; existence d'une factorisation de Cholesky $\mathbf{A} = \mathbf{L} \mathbf{L}^H$ avec $\mathbf{L}$ triangulaire à diagonale réelle positive.

A.3 Décompositions matricielles

A.3.1 Diagonalisation hermitienne

Pour $\mathbf{A}$ hermitienne : $\mathbf{A} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^H$. Calcul : $\mathcal{O}(N^3)$ par algorithme QR ou Jacobi. Stabilité numérique excellente.

A.3.2 Cholesky

Pour $\mathbf{A} \succ 0$: $\mathbf{A} = \mathbf{L}\mathbf{L}^H$ avec $\mathbf{L}$ triangulaire inférieure. Calcul $\mathcal{O}(N^3/3)$.

A.3.3 LU avec pivot

Pour $\mathbf{A}$ générique : $\mathbf{PA} = \mathbf{LU}$, avec $\mathbf{P}$ permutation. Standard pour résoudre des systèmes linéaires $\mathcal{O}(N^3)$.

A.3.4 QR

$\mathbf{A} = \mathbf{QR}$ avec $\mathbf{Q}^H\mathbf{Q} = \mathbf{I}$ et $\mathbf{R}$ triangulaire supérieure. Utile pour la moindres carrés $\min \|\mathbf{Ax} - \mathbf{b}\|^2$.

A.3.5 SVD (Singular Value Decomposition)

Toute matrice $\mathbf{A} \in \mathbb{C}^{M \times N}$ s'écrit

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^H,$$

avec $\mathbf{U} \in \mathbb{C}^{M \times M}$ et $\mathbf{V} \in \mathbb{C}^{N \times N}$ unitaires, $\mathbf{\Sigma}$ matrice rectangulaire diagonale contenant les valeurs singulières $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$.

- $\text{rg}(\mathbf{A})$ = nombre de valeurs singulières non nulles ;
- $\text{Im}(\mathbf{A})$ = espace engendré par les premières colonnes de $\mathbf{U}$ correspondant aux $\sigma_k > 0$;
- $\text{Ker}(\mathbf{A})$ = espace engendré par les colonnes de $\mathbf{V}$ correspondant aux $\sigma_k = 0$;
- meilleure approximation de rang r : $\mathbf{A}_r = \sum_{k=1}^r \sigma_k \mathbf{u}_k \mathbf{v}_k^H$ (théorème d'Eckart-Young).

A.4 Identités utiles

A.4.1 Lemme d'inversion (Sherman-Morrison)

Pour $\mathbf{A}$ inversible et $\mathbf{u}, \mathbf{v}$ vecteurs :

$$(\mathbf{A} + \mathbf{u}\mathbf{v}^H)^{-1} = \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1}\mathbf{u}\mathbf{v}^H\mathbf{A}^{-1}}{1 + \mathbf{v}^H\mathbf{A}^{-1}\mathbf{u}}.$$

A.4.2 Woodbury

Plus généralement, pour $\mathbf{U}, \mathbf{V} \in \mathbb{C}^{N \times K}$:

$$(\mathbf{A} + \mathbf{UV}^H)^{-1} = \mathbf{A}^{-1} - \mathbf{A}^{-1}\mathbf{U}(\mathbf{I}_K + \mathbf{V}^H\mathbf{A}^{-1}\mathbf{U})^{-1}\mathbf{V}^H\mathbf{A}^{-1}.$$

A.4.3 Trace

$\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$; $\text{tr}(\mathbf{A}) = \sum_k \lambda_k$; pour $\mathbf{A}$ hermitienne SDP, $\text{tr}(\mathbf{AB}^H) = \text{tr}(\mathbf{B}^H\mathbf{A})$ joue le rôle d'un produit scalaire matriciel.

A.4.4 Déterminant

$$\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B}) ; \det(\mathbf{A}) = \prod_k \lambda_k ; \det(\mathbf{I} + \mathbf{AB}) = \det(\mathbf{I} + \mathbf{BA}).$$

A.5 Produit de Kronecker

Pour $\mathbf{A} \in \mathbb{C}^{m \times n}$ et $\mathbf{B} \in \mathbb{C}^{p \times q}$:

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & a_{12}\mathbf{B} & \cdots \\ a_{21}\mathbf{B} & \cdots & \\ \vdots & & \end{bmatrix} \in \mathbb{C}^{mp \times nq}.$$

Propriétés :

- $(\mathbf{A} \otimes \mathbf{B})(\mathbf{C} \otimes \mathbf{D}) = (\mathbf{AC}) \otimes (\mathbf{BD})$;
- $(\mathbf{A} \otimes \mathbf{B})^H = \mathbf{A}^H \otimes \mathbf{B}^H$;
- valeurs propres $\{\lambda_i \mu_j\}$, vecteurs propres $\mathbf{u}_i \otimes \mathbf{v}_j$.

Application : structure du steering vector URA $\mathbf{a}(u, v) = \mathbf{a}_y(v) \otimes \mathbf{a}_x(u)$.

A.6 Projecteurs orthogonaux

Soit $\mathbf{A} \in \mathbb{C}^{N \times L}$ de rang plein colonne. Le projecteur orthogonal sur $\text{Im}(\mathbf{A})$ est

$$\mathbf{P}_A = \mathbf{A}(\mathbf{A}^H \mathbf{A})^{-1} \mathbf{A}^H,$$

et son complément

$$\mathbf{P}_A^\perp = \mathbf{I}_N - \mathbf{P}_A.$$

Propriétés : $\mathbf{P}_A^2 = \mathbf{P}_A$, $\mathbf{P}_A^H = \mathbf{P}_A$, $\mathbf{P}_A^\perp \mathbf{P}_A = 0$. Apparaît partout dans MUSIC (projection sur sous-espace bruit) et dans le calcul des CRB.

A.7 Pseudo-inverse de Moore-Penrose

Pour $\mathbf{A}$ de rang plein colonne : $\mathbf{A}^+ = (\mathbf{A}^H \mathbf{A})^{-1} \mathbf{A}^H$. Pour rang plein ligne : $\mathbf{A}^+ = \mathbf{A}^H (\mathbf{A} \mathbf{A}^H)^{-1}$. En général via SVD : $\mathbf{A}^+ = \mathbf{V} \mathbf{\Sigma}^+ \mathbf{U}^H$ où $\mathbf{\Sigma}^+$ inverse les valeurs singulières non nulles.

A.8 Gradients matriciels complexes

Pour $f : \mathbb{C}^N \rightarrow \mathbb{R}$ et $\mathbf{w} = \mathbf{u} + j\mathbf{v}$, on définit les opérateurs Wirtinger

$$\frac{\partial}{\partial \mathbf{w}} = \frac{1}{2} \left(\frac{\partial}{\partial \mathbf{u}} - j \frac{\partial}{\partial \mathbf{v}} \right), \quad \frac{\partial}{\partial \mathbf{w}^*} = \frac{1}{2} \left(\frac{\partial}{\partial \mathbf{u}} + j \frac{\partial}{\partial \mathbf{v}} \right).$$

On dérive en traitant $\mathbf{w}$ et $\mathbf{w}^*$ comme indépendants. L'optimum vérifie $\partial f / \partial \mathbf{w}^* = \mathbf{0}$. Gradients usuels :

- $\partial(\mathbf{a}^H \mathbf{w}) / \partial \mathbf{w}^* = \mathbf{0}$, $\partial(\mathbf{a}^H \mathbf{w}) / \partial \mathbf{w} = \mathbf{a}$;
- $\partial(\mathbf{w}^H \mathbf{R} \mathbf{w}) / \partial \mathbf{w}^* = \mathbf{R} \mathbf{w}$.

A.9 Conditionnement et stabilité

Pour $\mathbf{A}$ inversible, le *nombre de conditionnement*

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|$$

(norme spectrale) mesure la sensibilité de la résolution $\mathbf{Ax} = \mathbf{b}$ à des perturbations. Une perturbation relative ϵ sur $\mathbf{A}$ ou $\mathbf{b}$ entraîne une perturbation relative $\sim \kappa \epsilon$ sur $\mathbf{x}$. En double précision (16 chiffres), un conditionnement $\kappa \sim 10^k$ coûte typiquement k chiffres de précision.

Annexe B

Analyse de perturbations

Les performances asymptotiques de MUSIC, ESPRIT, Capon et autres algorithmes adaptatifs s'expriment via la *théorie des perturbations* sur les valeurs et vecteurs propres. Cette annexe rappelle les résultats classiques utilisés dans l'ouvrage.

B.1 Perturbation des valeurs propres

Soit $\mathbf{A}$ hermitienne avec spectre $\lambda_1 \geq \dots \geq \lambda_N$ et vecteurs propres orthonormés $\{\mathbf{u}_k\}$. Pour une perturbation $\mathbf{A}' = \mathbf{A} + \delta\mathbf{A}$ avec $\delta\mathbf{A}$ hermitienne, les valeurs propres λ'_k vérifient (théorème de Weyl) :

$$|\lambda'_k - \lambda_k| \leq \|\delta\mathbf{A}\|.$$

Plus précisément, au premier ordre :

$$\lambda'_k = \lambda_k + \mathbf{u}_k^H \delta\mathbf{A} \mathbf{u}_k + \mathcal{O}(\|\delta\mathbf{A}\|^2).$$

B.2 Perturbation des vecteurs propres

Sous les mêmes hypothèses, et en supposant les λ_k *simples* (non dégénérées),

$$\mathbf{u}'_k = \mathbf{u}_k + \sum_{j \neq k} \frac{\mathbf{u}_j^H \delta\mathbf{A} \mathbf{u}_k}{\lambda_k - \lambda_j} \mathbf{u}_j + \mathcal{O}(\|\delta\mathbf{A}\|^2).$$

La perturbation d'un vecteur propre est sensible à la *séparation* $\lambda_k - \lambda_j$ avec ses voisines : plus elles sont proches, plus la perturbation est amplifiée. C'est précisément pourquoi MUSIC perd en précision quand deux sources de même puissance sont proches en angle (valeurs propres signal proches).

B.3 Application à la covariance échantillon

Pour la covariance estimée $\hat{\mathbf{R}} = \mathbf{R} + \delta\mathbf{R}$, on montre que sous le modèle gaussien complexe circulaire,

$$\mathbb{E}[\delta R_{ij} \delta R_{kl}^*] = \frac{1}{K} R_{ik} R_{jl}^*,$$

ce qui donne pour la norme de Frobenius $\mathbb{E}[\|\delta\mathbf{R}\|_F^2] = \frac{N^2}{K} \cdot \mathcal{O}(1)$. La perturbation typique des valeurs propres est donc $\mathcal{O}(N/\sqrt{K})$ relative, des vecteurs propres $\mathcal{O}(1/\sqrt{K})$ par direction propre.

B.4 Sous-espace signal perturbé

Pour le sous-espace signal $\mathbf{U}_s$ de $\mathbf{R}$ et son estimateur $\widehat{\mathbf{U}}_s$ issu de $\widehat{\mathbf{R}}$, le *principal angle* entre les deux sous-espaces vérifie asymptotiquement

$$\mathbb{E}[\sin^2 \angle(\widehat{\mathbf{U}}_s, \mathbf{U}_s)] = \mathcal{O}\left(\frac{N}{K}\right),$$

avec une constante qui dépend des gaps signal-bruit $\lambda_L - \sigma^2$. C'est l'origine du facteur $1/K$ dans les performances asymptotiques de MUSIC.

B.5 Variance asymptotique de MUSIC

Stoica & Nehorai (1989) démontrent que la variance asymptotique de $\widehat{\theta}_\ell^{\text{MUSIC}}$ est

$$\text{Var}(\widehat{\theta}_\ell^{\text{MUSIC}}) = \frac{\sigma^2}{2K} \frac{(\mathbf{P}_s \mathbf{A}^H \mathbf{R}^{-1} \mathbf{A} \mathbf{P}_s)_{\ell,\ell}^{-1}}{\text{Re}\{\dot{\mathbf{a}}_\ell^H \mathbf{P}_A^\perp \dot{\mathbf{a}}_\ell\}}.$$

On compare à la CRB stochastique (chapitre 17) : MUSIC atteint la borne quand les sources sont *bien séparées*, mais s'en écarte quand elles sont angulairement proches ou de puissances très différentes.

B.6 Variance d'ESPRIT

ESPRIT a une variance légèrement supérieure à MUSIC à bas SNR, avec la dépendance

$$\text{Var}(\widehat{\theta}_\ell^{\text{ESPRIT}}) \sim \text{Var}(\widehat{\theta}_\ell^{\text{MUSIC}}) \cdot (1 + \mathcal{O}(\sigma^2)).$$

La différence devient négligeable asymptotiquement ($\sigma^2 \rightarrow 0$ ou $K \rightarrow \infty$).

B.7 Effet du diagonal loading

Le diagonal loading $\widehat{\mathbf{R}} \rightarrow \widehat{\mathbf{R}} + \alpha \mathbf{I}$ modifie les valeurs propres en $\lambda_k + \alpha$. La séparation signal-bruit $\lambda_L - \sigma^2$ devient $\lambda_L - \sigma^2$ (inchangée), mais les conditionnements deviennent $(\lambda_{\max} + \alpha)/(\sigma^2 + \alpha)$, contrôlés. La variance asymptotique de MUSIC sur la version chargée se dégrade d'un facteur $(1 + \alpha/(\lambda_L - \sigma^2))^2$ environ, faible pour α modeste.

B.8 Lien avec la CRB

Les CRB du chapitre 17 sont des bornes *universelles* pour tout estimateur non biaisé. Les estimateurs spécifiques (MUSIC, ESPRIT, ML) atteignent ou non ces bornes selon le régime. La *performance asymptotique* d'un estimateur se mesure justement par son écart à la CRB, et l'analyse de perturbations est l'outil qui permet de quantifier cet écart pour chaque famille d'algorithmes.

Annexe C

Code Python : module de référence

Cette annexe rassemble en un module unique tous les algorithmes implémentés dans le livre. Le code suppose `numpy` et, optionnellement, `scipy` et `matplotlib`. Une version plus complète, avec tests et visualisations, est disponible sur :

<https://github.com/JoachimNadal/Adaptive-Beamforming-Simulation>

C.1 Figures reproductibles

Les figures de cette version sont générées par le script `scripts/generate_figures.py`. Il fixe une graine aléatoire unique et écrit les fichiers PDF dans `figures/`. Depuis le dossier `V2`, la commande est :

```
python scripts/generate_figures.py
```

Les angles sont exprimés en radians dans le code, puis convertis en degrés uniquement pour les axes des figures.

C.2 Modèle de réseau

Listing C.1 – `beamforming.py` — module noyau

```
1 import numpy as np
2
3 # -----
4 # Steering vectors
5 # -----
6
7 def steering_vector(theta, N, d_over_lambda=0.5):
8     """Steering vector ULA."""
9     m = np.arange(N)
10    return np.exp(-1j * 2*np.pi * d_over_lambda * np.sin(theta) * m)
11
12 def steering_matrix(thetas, N, d_over_lambda=0.5):
13    return np.column_stack([steering_vector(th, N, d_over_lambda) for th in thetas])
14
15 def steering_derivative(theta, N, d_over_lambda=0.5):
16    m = np.arange(N)
17    a = steering_vector(theta, N, d_over_lambda)
18    return a * (-1j * 2*np.pi * d_over_lambda * np.cos(theta) * m)
19
20 # -----
```

```

21 # Beamformers déterministes
22 # -----
23
24 def das_weights(theta0, N, d_over_lambda=0.5):
25     return steering_vector(theta0, N, d_over_lambda) / N
26
27 def null_steering(theta_s, theta_i, N, d_over_lambda=0.5, alpha=0.0):
28     angles = [theta_s] + list(theta_i)
29     C = steering_matrix(angles, N, d_over_lambda)
30     f = np.zeros(len(angles), dtype=complex); f[0] = 1.0
31     G = C.conj().T @ C + alpha * np.eye(len(angles))
32     return C @ np.linalg.solve(G, f)
33
34 # -----
35 # Covariance
36 # -----
37
38 def sample_covariance(X):
39     N, K = X.shape
40     assert K > 0
41     R = (X @ X.conj().T) / K
42     return 0.5 * (R + R.conj().T)
43
44 def diagonal_loading(R, alpha):
45     N = R.shape[0]
46     assert R.shape == (N, N)
47     assert alpha >= 0.0
48     R_alpha = R + alpha * np.eye(N)
49     return 0.5 * (R_alpha + R_alpha.conj().T)
50
51 def forward_backward(R):
52     N = R.shape[0]
53     assert R.shape == (N, N)
54     J = np.flip(np.eye(N), axis=0)
55     return 0.5 * (R + J @ R.conj() @ J)
56
57 # -----
58 # MVDR et LCMV
59 # -----
60
61 def mvdr_weights(R, a_s, alpha=0.0):
62     N = R.shape[0]
63     assert R.shape == (N, N)
64     assert a_s.shape == (N,)
65     assert alpha >= 0.0
66     R_alpha = R + alpha * np.eye(N)
67     R_alpha = 0.5 * (R_alpha + R_alpha.conj().T)
68     if np.linalg.cond(R_alpha) > 1e12:
69         raise np.linalg.LinAlgError("R_alpha mal conditionnee")
70     u = np.linalg.solve(R_alpha, a_s)
71     den = a_s.conj() @ u
72     if abs(den) < np.finfo(float).eps:
73         raise np.linalg.LinAlgError("normalisation MVDR instable")
74     return u / den
75
76 def lcmv_weights(R, C, f, alpha=0.0):
77     N = R.shape[0]
78     assert R.shape == (N, N)

```



```

79     assert C.shape[0] == N
80     assert f.shape == (C.shape[1],)
81     assert alpha >= 0.0
82     R_alpha = R + alpha * np.eye(N)
83     R_alpha = 0.5 * (R_alpha + R_alpha.conj().T)
84     if np.linalg.cond(R_alpha) > 1e12:
85         raise np.linalg.LinAlgError("R_alpha mal conditionnee")
86     Rinv_C = np.linalg.solve(R_alpha, C)
87     G = C.conj().T @ Rinv_C
88     if np.linalg.cond(G) > 1e12:
89         raise np.linalg.LinAlgError("contraintes LCMV mal conditionnees")
90     return Rinv_C @ np.linalg.solve(G, f)
91
92 # -----
93 # LMS et RLS
94 # -----
95
96 def lms(X, d, mu, w0=None):
97     N, K = X.shape
98     w = np.zeros(N, dtype=complex) if w0 is None else w0.astype(complex).copy()
99     err = np.zeros(K, dtype=complex)
100     for n in range(K):
101         x = X[:, n]
102         e = d[n] - w.conj() @ x
103         w += mu * x * np.conj(e)
104         err[n] = e
105     return w, err
106
107 def nlms(X, d, mu_tilde=0.5, eps=1e-6, w0=None):
108     N, K = X.shape
109     w = np.zeros(N, dtype=complex) if w0 is None else w0.astype(complex).copy()
110     err = np.zeros(K, dtype=complex)
111     for n in range(K):
112         x = X[:, n]
113         e = d[n] - w.conj() @ x
114         norm = eps + (x.conj() @ x).real
115         w += (mu_tilde / norm) * x * np.conj(e)
116         err[n] = e
117     return w, err
118
119 def rls(X, d, lam=0.99, delta=1e-2):
120     N, K = X.shape
121     assert d.shape == (K,)
122     assert 0.0 < lam <= 1.0
123     assert delta > 0.0
124     w = np.zeros(N, dtype=complex)
125     P = np.eye(N, dtype=complex) / delta
126     err = np.zeros(K, dtype=complex)
127     for n in range(K):
128         x = X[:, n]
129         u = P @ x
130         k_gain = u / (lam + (x.conj() @ u).real)
131         alpha = d[n] - w.conj() @ x
132         w = w + k_gain * np.conj(alpha)
133         P = (P - np.outer(k_gain, x.conj() @ P)) / lam
134         P = 0.5 * (P + P.conj().T)
135         err[n] = alpha
136     return w, err

```

```

137
138 # -----
139 # Spectres et DOA
140 # -----
141
142 def bartlett_spectrum(R, thetas, N, d_over_lambda=0.5):
143     spec = np.zeros(len(thetas))
144     for idx, th in enumerate(thetas):
145         a = steering_vector(th, N, d_over_lambda)
146         spec[idx] = np.real(a.conj() @ R @ a)
147     return spec
148
149 def capon_spectrum(R, thetas, N, d_over_lambda=0.5, alpha=0.0):
150     assert R.shape == (N, N)
151     assert alpha >= 0.0
152     R_alpha = R + alpha * np.eye(N)
153     R_alpha = 0.5 * (R_alpha + R_alpha.conj().T)
154     if np.linalg.cond(R_alpha) > 1e12:
155         raise np.linalg.LinAlgError("R_alpha mal conditionnee")
156     spec = np.zeros(len(thetas))
157     eps = np.finfo(float).eps
158     for idx, th in enumerate(thetas):
159         a = steering_vector(th, N, d_over_lambda)
160         u = np.linalg.solve(R_alpha, a)
161         den = max(np.real(a.conj() @ u), eps)
162         spec[idx] = 1.0 / den
163     return spec
164
165 def music_spectrum(R, L, thetas, N, d_over_lambda=0.5):
166     assert R.shape == (N, N)
167     assert 0 <= L < N
168     R = 0.5 * (R + R.conj().T)
169     _, eigvecs = np.linalg.eigh(R)
170     Un = eigvecs[:, :N-L]
171     spec = np.zeros(len(thetas))
172     eps = np.finfo(float).eps
173     for idx, th in enumerate(thetas):
174         a = steering_vector(th, N, d_over_lambda)
175         v = Un.conj().T @ a
176         den = max(np.real(v.conj() @ v), eps)
177         spec[idx] = 1.0 / den
178     return spec
179
180 def root_music(R, L, d_over_lambda=0.5):
181     N = R.shape[0]
182     assert R.shape == (N, N)
183     assert 0 < L < N
184     R = 0.5 * (R + R.conj().T)
185     _, eigvecs = np.linalg.eigh(R)
186     Un = eigvecs[:, :N-L]
187     C = Un @ Un.conj().T
188     coeffs = np.array([np.trace(C, offset=lag) for lag in range(-(N-1), N)])
189     roots = np.roots(coeffs[::-1])
190     valid = roots[np.abs(roots) <= 1.0]
191     if len(valid) < L:
192         raise ValueError("Root-MUSIC: pas assez de racines valides")
193     order = np.argsort(np.abs(np.abs(valid) - 1.0))
194     selected = valid[order][:L]

```

```

195     arg = np.angle(selected) / (-2*np.pi*d_over_lambda)
196     angles = np.arcsin(np.clip(arg, -1.0, 1.0))
197     return np.sort(np.real(angles))
198
199 def esprit(R, L, d_over_lambda=0.5, tls=True):
200     N = R.shape[0]
201     assert R.shape == (N, N)
202     assert 0 < L < N
203     R = 0.5 * (R + R.conj().T)
204     _, eigvecs = np.linalg.eigh(R)
205     Us = eigvecs[:, -L:]
206     Us1 = Us[:-1, :]
207     Us2 = Us[1:, :]
208     if not tls:
209         Psi = np.linalg.lstsq(Us1, Us2, rcond=None)[0]
210     else:
211         Z = np.hstack([Us1, Us2])
212         _, _, Vh = np.linalg.svd(Z, full_matrices=False)
213         V = Vh.conj().T
214         V12 = V[:L, L:]; V22 = V[L:, L:]
215         if np.linalg.cond(V22) > 1e12:
216             raise np.linalg.LinAlgError("TLS-ESPRIT mal conditionne")
217         Psi = -np.linalg.solve(V22.T, V12.T).T
218     eig_psi, _ = np.linalg.eig(Psi)
219     arg = -np.angle(eig_psi) / (2*np.pi*d_over_lambda)
220     angles = np.arcsin(np.clip(arg, -1.0, 1.0))
221     return np.sort(np.real(angles))
222
223 # -----
224 # Détection d'ordre
225 # -----
226
227 def estimate_L_mdl(eigvals, K):
228     eigvals = np.sort(eigvals)[::-1]
229     N = len(eigvals)
230     best = (np.inf, 0)
231     for ell in range(N):
232         tail = eigvals[ell:]
233         if len(tail) == 0 or np.any(tail <= 0):
234             continue
235         geo = np.exp(np.mean(np.log(tail)))
236         ari = np.mean(tail)
237         if geo <= 0: continue
238         mdl = K * (N - ell) * np.log(ari / geo) + 0.5 * ell * (2*N - ell) * np.log(K)
239         if mdl < best[0]:
240             best = (mdl, ell)
241     return best[1]
242
243 # -----
244 # CRB
245 # -----
246
247 def crb_stochastic(thetas, Ps, sigma2, N, K, d_over_lambda=0.5):
248     L = len(thetas)
249     assert Ps.shape == (L, L)
250     assert sigma2 > 0.0 and K > 0 and N > L
251     A = steering_matrix(thetas, N, d_over_lambda)
252     D = np.column_stack([steering_derivative(th, N, d_over_lambda) for th in thetas])

```

```

253     R = A @ Ps @ A.conj().T + sigma2 * np.eye(N)
254     R = 0.5 * (R + R.conj().T)
255     G = A.conj().T @ A
256     if np.linalg.cond(G) > 1e12 or np.linalg.cond(R) > 1e12:
257         raise np.linalg.LinAlgError("CRB mal conditionnée")
258     PA_perp = np.eye(N) - A @ np.linalg.solve(G, A.conj().T)
259     M = D.conj().T @ PA_perp @ D
260     H = Ps @ A.conj().T @ np.linalg.solve(R, A) @ Ps
261     F = 2 * K / sigma2 * np.real(M * H.T)
262     return np.linalg.pinv(F)
263
264     # -----
265     # Calibration
266     # -----
267
268     def estimate_calibration(X, theta0, d_over_lambda=0.5):
269         x_mean = np.mean(X, axis=1)
270         a0 = steering_vector(theta0, X.shape[0], d_over_lambda)
271         raw = x_mean / a0
272         return raw / raw[0]
273
274     def apply_calibration(X, g):
275         return X / g[:, None]
276
277     # -----
278     # Beam pattern
279     # -----
280
281     def beam_pattern(w, N, d_over_lambda=0.5, n_theta=721):
282         thetas = np.linspace(-np.pi/2, np.pi/2, n_theta)
283         A = steering_matrix(thetas, N, d_over_lambda)
284         B = w.conj() @ A
285         Babs = np.abs(B)
286         return thetas, 20*np.log10(Babs / (Babs.max() + 1e-12))

```

C.3 Exemple de simulation complète

Listing C.2 – simulation_demo.py — exemple bout-en-bout

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from beamforming import *
4
5  # Paramètres
6  N = 16
7  d_over_lambda = 0.5
8  K = 1000
9
10 # Sources
11 theta_s = np.deg2rad(0.0)          # cible
12 theta_i = np.deg2rad([20.0, -35.0]) # 2 brouilleurs
13 sigma2 = 1e-2                      # bruit
14
15 # Génération de données
16 np.random.seed(42)
17 A = steering_matrix([theta_s] + list(theta_i), N, d_over_lambda)
18 s = (np.random.randn(3, K) + 1j*np.random.randn(3, K)) / np.sqrt(2)

```

```

19 # puissance cible = 1, brouilleurs = 100 (20 dB plus fort)
20 s[0, :] *= 1.0
21 s[1:, :] *= 10.0
22 n = np.sqrt(sigma2/2) * (np.random.randn(N, K) + 1j*np.random.randn(N, K))
23 X = A @ s + n
24
25 # Covariance
26 R = sample_covariance(X)
27 alpha = 1e-2 * np.trace(R).real / N
28 R_alpha = diagonal_loading(R, alpha)
29
30 # Beamformers comparés
31 a_s = steering_vector(theta_s, N, d_over_lambda)
32 w_das = das_weights(theta_s, N, d_over_lambda)
33 w_mvdr = mvdr_weights(R_alpha, a_s)
34 C = steering_matrix([theta_s] + list(theta_i), N, d_over_lambda)
35 f = np.array([1.0, 0.0, 0.0], dtype=complex)
36 w_lcmv = lcmv_weights(R_alpha, C, f)
37
38 # Beam patterns
39 for name, w in [('DAS', w_das), ('MVDR', w_mvdr), ('LCMV', w_lcmv)]:
40     th, BdB = beam_pattern(w, N, d_over_lambda)
41     plt.plot(np.rad2deg(th), BdB, label=name)
42 plt.axvline(np.rad2deg(theta_s), color='g', ls=':', label='cible')
43 for ti in theta_i:
44     plt.axvline(np.rad2deg(ti), color='r', ls=':')
45 plt.xlabel(r'$\theta$ (deg)'); plt.ylabel('dB')
46 plt.ylim(-70, 5); plt.grid(True); plt.legend()
47 plt.title('Beam patterns DAS / MVDR / LCMV')
48
49 # Estimation DOA
50 print('Vraies DOA (deg) :', sorted([np.rad2deg(theta_s)] + list(np.rad2deg(theta_i))))
51 print('Root-MUSIC      :', np.rad2deg(root_music(R, 3, d_over_lambda)))
52 print('ESPRIT (TLS)     :', np.rad2deg(esprit(R, 3, d_over_lambda)))
53
54 # CRB pour comparaison
55 Ps = np.diag([1.0, 100.0, 100.0])
56 crb = crb_stochastic([theta_s] + list(theta_i), Ps, sigma2, N, K)
57 print('Écart-type CRB (deg) :', np.rad2deg(np.sqrt(np.diag(crb))))
58 plt.show()

```

Ce script reproduit l'essentiel des résultats illustrés dans l'ouvrage. Il peut servir de point de départ à toute exploration ou implémentation. Pour des scénarios plus complexes (large bande, MIMO, sparse), se référer au dépôt GitHub mentionné en préface.

Bibliographie commentée

Cette bibliographie rassemble les références structurantes qui soutiennent les méthodes présentées dans le livre. Elle ne prétend pas être exhaustive ; elle donne plutôt au lecteur les textes à consulter pour approfondir les preuves, les variantes et les limites.

Ouvrages de référence

Van Trees (2002) H. L. Van Trees, *Optimum Array Processing*, Wiley, 2002. Référence centrale pour les beamformers, les réseaux d'antennes, MVDR, LCMV, CRB et les liens avec l'estimation statistique.

Johnson et Dudgeon (1993) D. H. Johnson and D. E. Dudgeon, *Array Signal Processing : Concepts and Techniques*, Prentice Hall, 1993. Très utile pour les fondements physiques, les conventions de steering vector et les méthodes haute résolution.

Haykin (2002) S. Haykin, *Adaptive Filter Theory*, Prentice Hall, 4e édition, 2002. Référence classique pour LMS, NLMS, RLS, stabilité numérique et compromis convergence/misadjustment.

Stoica et Moses (2005) P. Stoica and R. Moses, *Spectral Analysis of Signals*, Prentice Hall, 2005. Lecture solide pour les sous-espaces, l'analyse spectrale, MUSIC, ESPRIT et les critères d'ordre.

Beamforming adaptatif et robustesse

Capon (1969) J. Capon, "High-resolution frequency-wavenumber spectrum analysis", *Proceedings of the IEEE*, 1969. Article fondateur du beamformer Capon/MVDR.

Reed, Mallett et Brennan (1974) I. S. Reed, J. D. Mallett and L. E. Brennan, "Rapid convergence rate in adaptive arrays", *IEEE Transactions on Aerospace and Electronic Systems*, 1974. Source classique de la règle de dimensionnement SMI/RMB.

Li, Stoica et Wang (2003) J. Li, P. Stoica and Z. Wang, "On robust Capon beamforming and diagonal loading", *IEEE Transactions on Signal Processing*, 2003. Référence importante sur le Capon robuste et le lien entre loading et incertitude de steering vector.

Lorenz et Boyd (2005) R. G. Lorenz and S. P. Boyd, "Robust minimum variance beamforming", *IEEE Transactions on Signal Processing*, 2005. Formulation robuste élégante des contraintes d'incertitude.

DOA haute résolution

Schmidt (1986) R. O. Schmidt, "Multiple emitter location and signal parameter estimation", *IEEE Transactions on Antennas and Propagation*, 1986. Référence historique de MUSIC.

Roy et Kailath (1989) R. Roy and T. Kailath, "ESPRIT – Estimation of signal parameters via rotational invariance techniques", *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 1989. Article fondateur d'ESPRIT.

Wax et Kailath (1985) M. Wax and T. Kailath, "Detection of signals by information theoretic criteria", *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 1985. Référence AIC/MDL pour l'estimation du nombre de sources.

Shan, Wax et Kailath (1985) T.-J. Shan, M. Wax and T. Kailath, “On spatial smoothing for direction-of-arrival estimation of coherent signals”, *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 1985. Référence canonique pour le spatial smoothing.

Stoica et Nehorai (1989) P. Stoica and A. Nehorai, travaux sur MUSIC, maximum likelihood et bornes de Cramér-Rao pour la DOA, *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 1989. Base utile pour comparer MUSIC/ESPRIT/ML aux bornes théoriques.

Méthodes sparse et bayésiennes

Donoho (2006) D. L. Donoho, “Compressed sensing”, *IEEE Transactions on Information Theory*, 2006. Fondation théorique générale du compressed sensing.

Candès, Romberg et Tao (2006) E. J. Candès, J. Romberg and T. Tao, “Robust uncertainty principles : exact signal reconstruction from highly incomplete frequency information”, *IEEE Transactions on Information Theory*, 2006. Référence majeure sur la reconstruction parcimonieuse.

Malioutov, Çetin et Willsky (2005) D. Malioutov, M. Çetin and A. S. Willsky, “A sparse signal reconstruction perspective for source localization with sensor arrays”, *IEEE Transactions on Signal Processing*, 2005. Application directe de la parcimonie à la localisation de sources.

Tipping (2001) M. E. Tipping, “Sparse Bayesian learning and the relevance vector machine”, *Journal of Machine Learning Research*, 2001. Fondation de la logique SBL/RVM.

Wipf et Rao (2004) D. P. Wipf and B. D. Rao, “Sparse Bayesian learning for basis selection”, *IEEE Transactions on Signal Processing*, 2004. Référence technique pour l’estimation bayésienne parcimonieuse.

Tang, Bhaskar, Shah et Recht (2013) G. Tang, B. N. Bhaskar, P. Shah and B. Recht, “Compressed sensing off the grid”, *IEEE Transactions on Information Theory*, 2013. Référence importante pour l’atomic norm et les méthodes off-grid.

Large bande, MIMO et extensions

Tse et Viswanath (2005) D. Tse and P. Viswanath, *Fundamentals of Wireless Communication*, Cambridge University Press, 2005. Référence claire pour MIMO, précodage, canaux et limites informationnelles.

Marzetta (2010) T. L. Marzetta, “Noncooperative cellular wireless with unlimited numbers of base station antennas”, *IEEE Transactions on Wireless Communications*, 2010. Article fondateur du massive MIMO moderne.

Balanis (2016) C. A. Balanis, *Antenna Theory : Analysis and Design*, Wiley, 4e édition, 2016. Complément utile pour les réseaux physiques, la polarisation, les diagrammes et les effets de géométrie.

Références formelles

Bibliographie

- [1] H. L. Van Trees, *Optimum Array Processing : Part IV of Detection, Estimation, and Modulation Theory*, Wiley, 2002.
- [2] D. H. Johnson and D. E. Dudgeon, *Array Signal Processing : Concepts and Techniques*, Prentice Hall, 1993.
- [3] S. Haykin, *Adaptive Filter Theory*, 4th ed., Prentice Hall, 2002.
- [4] P. Stoica and R. Moses, *Spectral Analysis of Signals*, Prentice Hall, 2005.
- [5] J. Capon, “High-resolution frequency-wavenumber spectrum analysis”, *Proceedings of the IEEE*, vol. 57, no. 8, pp. 1408–1418, 1969. DOI : <https://doi.org/10.1109/PROC.1969.7278>.
- [6] I. S. Reed, J. D. Mallett and L. E. Brennan, “Rapid convergence rate in adaptive arrays”, *IEEE Transactions on Aerospace and Electronic Systems*, vol. AES-10, no. 6, pp. 853–863, 1974. DOI : <https://doi.org/10.1109/TAES.1974.307893>.
- [7] J. Li, P. Stoica and Z. Wang, “On robust Capon beamforming and diagonal loading”, *IEEE Transactions on Signal Processing*, vol. 51, no. 7, pp. 1702–1715, 2003. DOI : <https://doi.org/10.1109/TSP.2003.812831>.
- [8] R. G. Lorenz and S. P. Boyd, “Robust minimum variance beamforming”, *IEEE Transactions on Signal Processing*, vol. 53, no. 5, pp. 1684–1696, 2005.
- [9] R. O. Schmidt, “Multiple emitter location and signal parameter estimation”, *IEEE Transactions on Antennas and Propagation*, vol. 34, no. 3, pp. 276–280, 1986.
- [10] R. Roy and T. Kailath, “ESPRIT – Estimation of signal parameters via rotational invariance techniques”, *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 37, no. 7, pp. 984–995, 1989. DOI : <https://doi.org/10.1109/29.32276>.
- [11] M. Wax and T. Kailath, “Detection of signals by information theoretic criteria”, *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 33, no. 2, pp. 387–392, 1985. DOI : <https://doi.org/10.1109/TASSP.1985.1164557>.
- [12] T.-J. Shan, M. Wax and T. Kailath, “On spatial smoothing for direction-of-arrival estimation of coherent signals”, *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 33, no. 4, pp. 806–811, 1985. DOI : <https://doi.org/10.1109/TASSP.1985.1164649>.
- [13] P. Stoica and A. Nehorai, “MUSIC, maximum likelihood, and Cramér-Rao bound”, *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 37, no. 5, pp. 720–741, 1989.
- [14] D. L. Donoho, “Compressed sensing”, *IEEE Transactions on Information Theory*, vol. 52, no. 4, pp. 1289–1306, 2006. DOI : <https://doi.org/10.1109/TIT.2006.871582>.
- [15] E. J. Candès, J. Romberg and T. Tao, “Robust uncertainty principles : exact signal reconstruction from highly incomplete frequency information”, *IEEE Transactions on Information Theory*, vol. 52, no. 2, pp. 489–509, 2006.
- [16] D. Malioutov, M. Çetin and A. S. Willsky, “A sparse signal reconstruction perspective for source localization with sensor arrays”, *IEEE Transactions on Signal Processing*, vol. 53, no. 8, pp. 3010–3022, 2005. DOI : <https://doi.org/10.1109/TSP.2005.850882>.

- [17] M. E. Tipping, “Sparse Bayesian learning and the relevance vector machine”, *Journal of Machine Learning Research*, vol. 1, pp. 211–244, 2001.
- [18] D. P. Wipf and B. D. Rao, “Sparse Bayesian learning for basis selection”, *IEEE Transactions on Signal Processing*, vol. 52, no. 8, pp. 2153–2164, 2004. DOI : <https://doi.org/10.1109/TSP.2004.831016>.
- [19] G. Tang, B. N. Bhaskar, P. Shah and B. Recht, “Compressed sensing off the grid”, *IEEE Transactions on Information Theory*, vol. 59, no. 11, pp. 7465–7490, 2013. DOI : <https://doi.org/10.1109/TIT.2013.2277451>.
- [20] D. Tse and P. Viswanath, *Fundamentals of Wireless Communication*, Cambridge University Press, 2005.
- [21] T. L. Marzetta, “Noncooperative cellular wireless with unlimited numbers of base station antennas”, *IEEE Transactions on Wireless Communications*, vol. 9, no. 11, pp. 3590–3600, 2010.
- [22] C. A. Balanis, *Antenna Theory : Analysis and Design*, 4th ed., Wiley, 2016.